

ZMotion Basic 编程手册

Version 3.1.5

ZBasic 是 ZMotion 运动控制器所使用的 Basic 编程语言。

ZBasic 提供所有标准程序语法：变量、数组、条件判断、循环及数学运算。此外，扩展的 Basic 指令和函数提供广泛的运动控制功能，例如单轴运动，多轴的同步和异步运动，同时还有对数字和模拟 I/O 的控制。

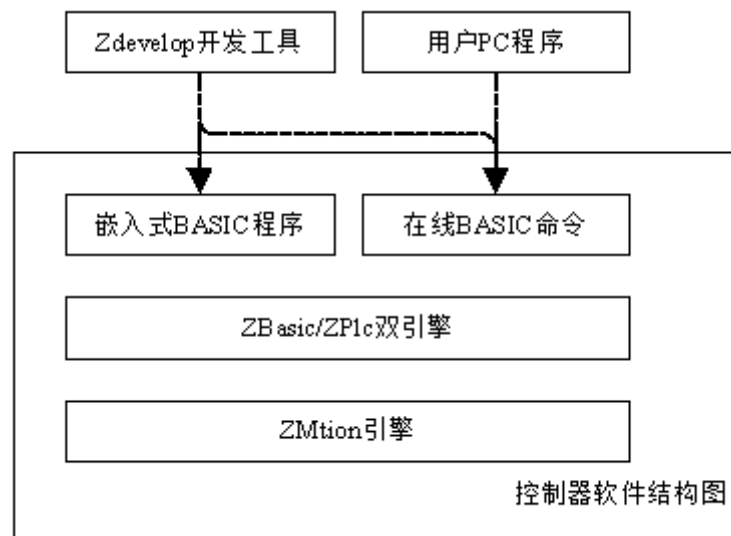
ZBasic 支持自定义 SUB 过程，可以把一些通用的功能编写为自定义 SUB 过程，方便程序编写和修改；ZBasic 同时支持 G 代码形式的 SUB 过程。

ZBasic 支持全局的变量，数组和 SUB 过程；文件模块变量，数组，和 SUB 过程，以及局部变量（LOCAL）。

ZBasic 具有实时多任务特性，多个 ZBasic 程序可以同时构建并同时运行，使得复杂的应用变得简单易行。

ZBasic 支持中断程序，例如掉电中断，通过掉电中断可以使得掉电的状态得到恢复。

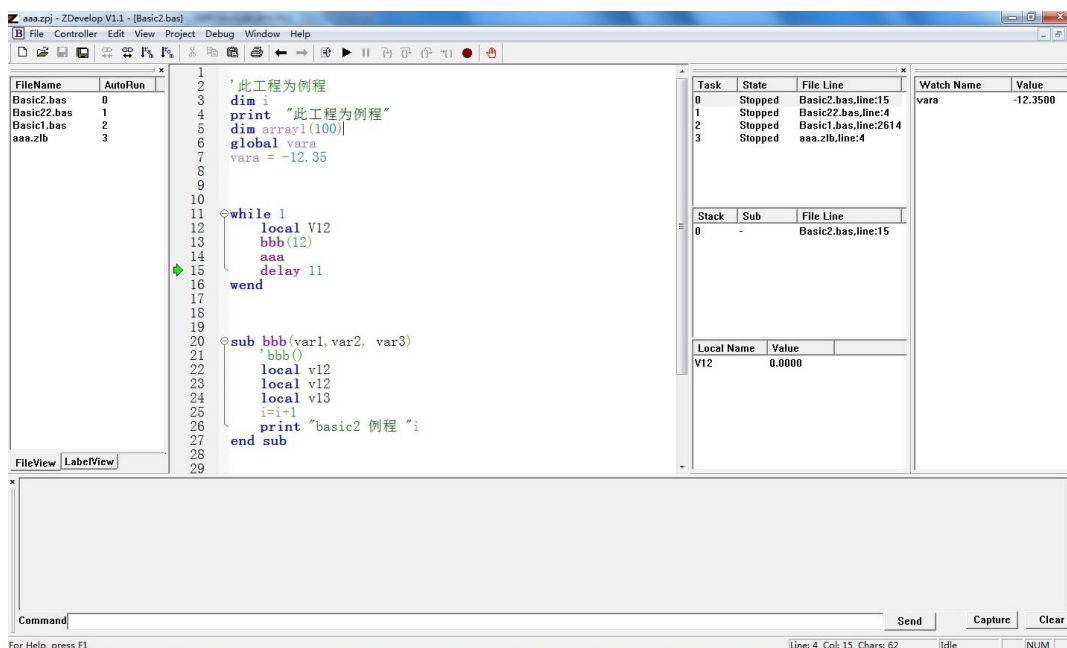
通过 PC 在线发送 BASIC 命令也可以实现同样的效果，控制器的 BASIC 程序和 PC 在线 BASIC 命令可以同时运行。



第一章 ZBASIC 编程入门

编写和调试 ZBASIC 程序需要 ZDevelop 开发软件, ZMC 运动控制器(或仿真器), PC 在线命令发送需要 zmotion.dll 动态库。

ZDevelop 开发软件使用



新建项目简单步骤:

- 1, 点击“文件(File)”菜单-“新建项目”, 点击“保存”保存项目。
- 2, 点击“文件(File)”菜单-“新建文件”(或者直接点击“新建(New)”按钮)建立 Bas 程序文件, 写入程序代码, 点击“保存”, 选择与项目文件相同的目录保存。
- 3, 点击“项目(Project)”-“增加到项目”, 选择刚才保存的 Bas 文件。
- 4, 双击窗口左边文件列表对应 Bas 文件的右侧“自动运行 (AutoRun)”选项, 填入数字 0。
- 5, 连接到控制器或仿真器, 点击“下载到 RAM”菜单, 此时会下载刚才的程序到控制器并自动运行。RAM 下载掉电不保存, ROM 下载掉电保存。



不建立项目的时候, 只有 Bas 文件无法下载到控制器。



自动运行的数字 0 表示任务编号, 以任务 0 运行程序。



打开程序时, 选择打开项目, 只打开 Bas 文件无法下载到控制器。

程序是什么？

程序是由序列组成的，告诉计算机如何完成一个具体的任务。程序是软件开发人员根据用户需求开发的、用程序设计语言描述的适合计算机执行的指令（语句）序列。

一个程序应该包括以下两方面的内容。

1、对数据的描述。在程序中要指定数据的类型和数据的组织形式，即数据结构。（参见：DIM GLOBAL CONST 等变量定义语句描述。）

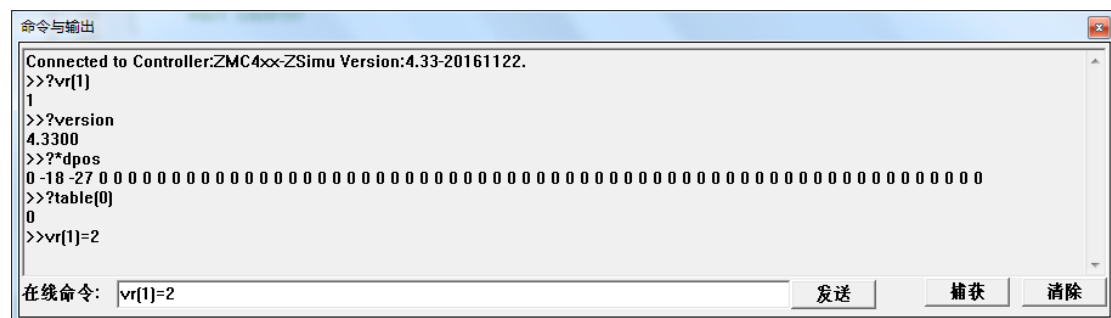
2、对操作的描述。即操作步骤，也就是算法。

为编写算法，我们一般要用到三种描述方式：顺序、选择、循环。

文档查看说明

指令中曲线图片是通过示波器获取的，示波器在“视图”-“示波器”。

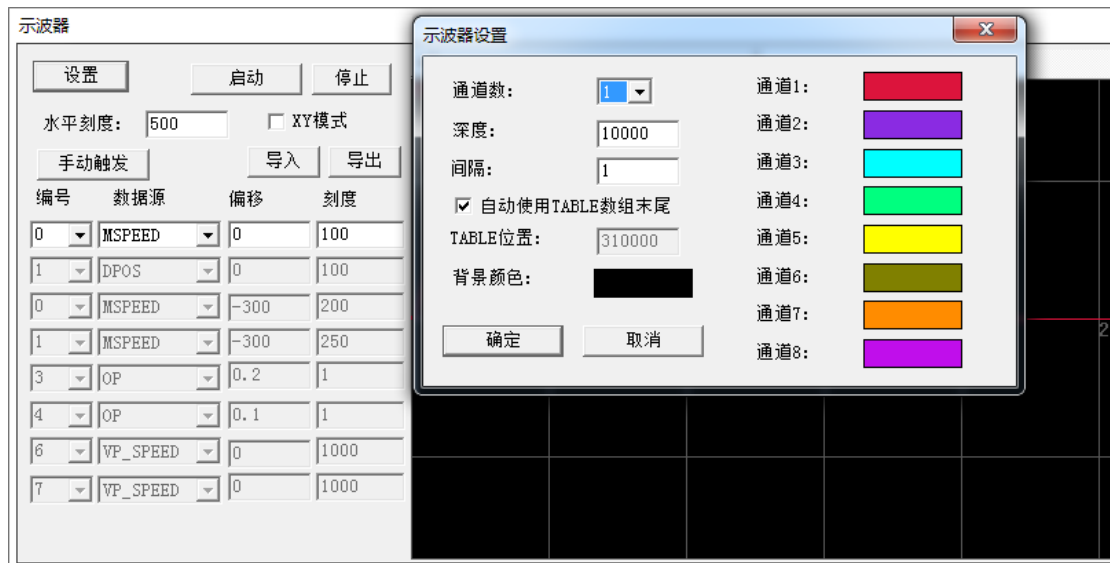
指令例程中“>>”表示 zdevelop 在线命令输入指令。



如非特殊说明，指令例程中 SPEED、DPOS、ACCEL、MOVE 等参数单位都是 UNITS。

示波器使用

示波器是 zdevelop2.61 以上版本支持的功能，在“视图”-“示波器”打开。



示波器设置

通道数：要采样的数据个数

深度：总共采样的次数

间隔：单位为系统周期，servo_period 查看，与控制器固件版本有关，一般默认 1ms

TABLE 位置：设置抓取数据的存放位置

右侧可以修改通道对应颜色

编号：选择轴号、IO 编号等

数据源：选择抓取的数据类型

偏移：上下偏移

刻度：竖轴刻度

水平刻度：横轴刻度

XY 模式：切换成 X 平面显示

示波器工作

设置好参数，点击启动，此时示波器已启动，但是未触发，也无法抓取数据。

可选择手动触发，也可以在程序里使用 TRIGGER 指令自动触发。

顺序

在没有条件和循环的情况下，程序总是从上往下运动。

功能块 1

功能块 2

如上，功能块 1 先执行，然后是功能块 2.

当设置自动运行时，文件缺省都是从文件开始顺序往下执行的。

BASIC 编程下，程序从上往下扫描一次。

PLC 编程下，程序从上往下周期扫描。

选择

根据执行条件的不同，选择不同的语句执行。

主要的选择语句有：IF THEN，ON GOTO ， ON GOSUB 等。

```
dim aa
aa=1
if aa= 0 then
    语句 1
elseif aa=1 then
    语句 2
else
    语句 3
endif
```

循环

程序重复执行，则称为循环。

主要的循环语句有：FOR NEXT，WHILE WEND ， REPEAT UNTIL 等。

```
dim a
a=0

for i = 1 to 10 step 1
    a = a+1
    print a
next
```

多任务

如果多个程序能够互不干扰的同时运行，则称为多任务，一个任务是指一个正在运动的程序。（参见 RUN 等指令描述）

每个任务都有唯一的任务编号，此编号并没有任何优先级关系、顺序关系，仅仅只是一个标识。

```
runtask 1, taskA
runtask 2, taskB
end

taskA:
print "atask":ticks
delay(1000)
goto taskA
```

```
taskB:
print "btask":ticks
delay(1000)
goto taskB
```

变量、数组、SUB 定义

变量有 LOCAL 局部变量 (local)，文件模块变量 (dim)，全局变量 (global) 三种。
数组、SUB 有文件模块数组 (dim)，全局数组 (global) 两种。
变量可以不经定义直接赋值，此时默认为文件模块变量。

数组、SUB 必须先定义再使用。

SUB 定义格式：

```
SUB 函数名()
.....代码块
END SUB
```

建议把变量数组定义语句放自动运行文件的最前面，保证被执行到。

建议把 SUB 过程放在主函数 end 后面，在主函数内调用到时才进入 SUB。

寄存器

控制器寄存器主要有 TABLE、MODBUS 寄存器、VR 寄存器。

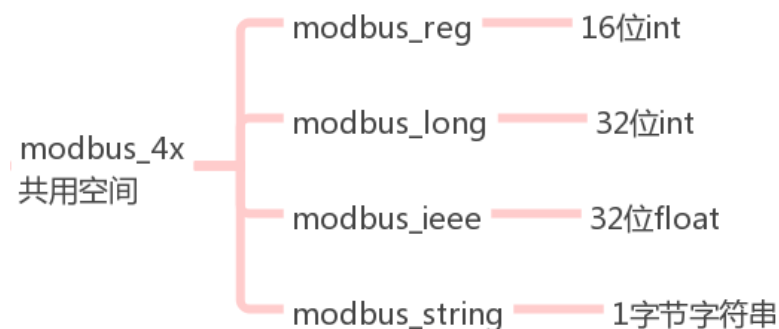
其中 TABLE 是控制器自带的一个超大数组；

VR 寄存器具有掉电存储功能，可无限次读写；

MODBUS 寄存器符合 MODBUS 标准通讯协议，分为位寄存器和字寄存器两类。

位寄存器：MODBUS_BIT，触摸屏一般称为 MODBUS_0X，布尔型

字寄存器：MODBUS_REG、MODBUS_LONG、MODBUS_IEEE、MODBUS_STRING，触摸屏一般叫 MODBUS_4X，类型如下图。



控制器中字寄存器占用同一个存储空间，其中一个 LONG 占用两个 REG，使用注意错开寄存器编号。

Modbus_long(0) 占用 modbus_reg(0) 与 modbus_reg(1) 两个 reg 地址。

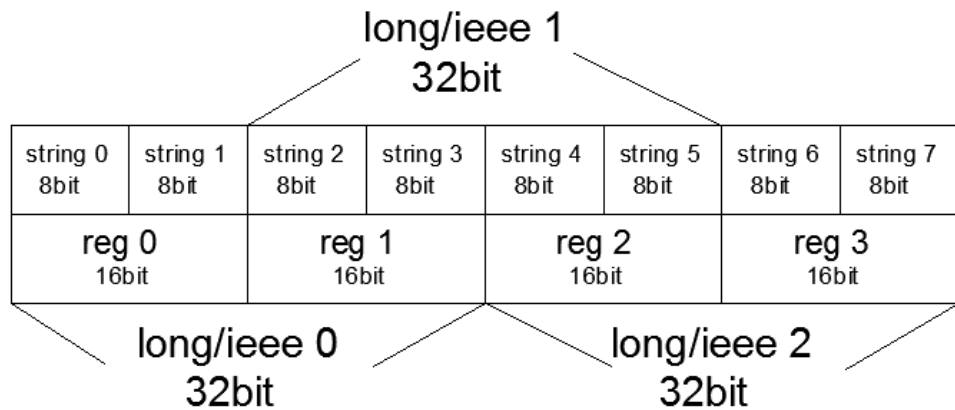
Modbus_long(1) 占用 modbus_reg(1) 与 modbus_reg(2) 两个 reg 地址。

Modbus_ieee(0) 占用 modbus_reg(0) 与 modbus_reg(1) 两个 reg 地址。

Modbus_ieee(1) 占用 modbus_reg(1) 与 modbus_reg(2) 两个 reg 地址。

所以要注意 modbus_reg, modbus_long, modbus_ieee 地址不能重叠。

4X 空间示意图



```
MODBUS_REG(0)=0           ' 初始化置 0
MODBUS_REG(1)=0
MODBUS_LONG(0)=70000      ' long 赋值 70000，超出 reg 范围
?MODBUS_REG(0),MODBUS_REG(1) ' 打印出 reg(0)为 4464，reg(1)为 1
                             ' long(0)=reg(1)*2^16+reg(0)
```

数据类型

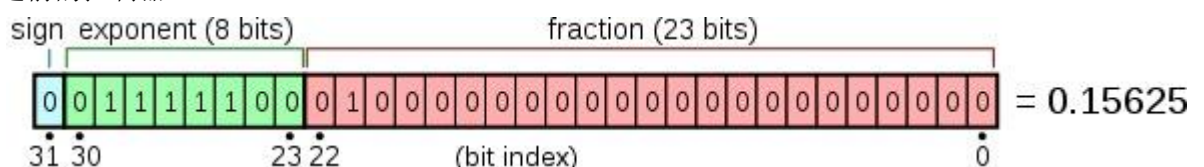
自定义变量的数据类型属于动态类型。

将整数赋值给变量时，变量就是整型；将浮点数赋值给变量，变量就是浮点型。

自定义数组的数据类型分为单精度浮点数和双精度浮点数，参照下文浮点数说明。

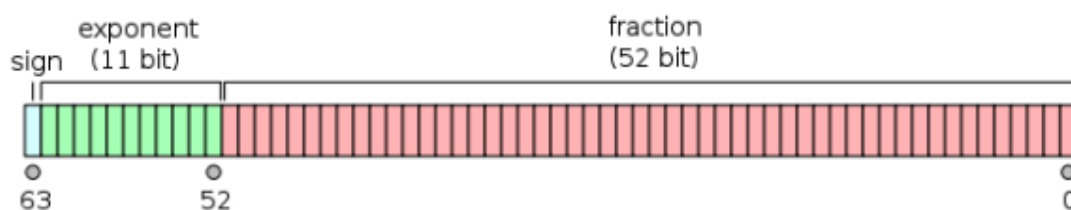
单精度浮点数 32 位，如下图。

存储数据属于单精度浮点数的有：VR、MODBUS_IEEE、TABLE 及自定义数组（ZMC3 系列及之前的控制器）。



双精度浮点数 64 位，如下图。

存储数据属于单精度浮点数的有：TABLE 及自定义数组（ZMC4 系列及之后的控制器）。



常用寄存器数据类型表

寄存器类型	数据类型	取值范围
MODBUS_BIT	布尔型	0 或 1
MODBUS_REG	16 位整型	-32768 到 32767
MODBUS_LONG	32 位整型	-2147483648 到 2147483647
VR_INT		
MODBUS_IEEE	32 位浮点型	-3.4028235E+38 到 -1.401298E-45
VR		
TABLE 及数组 (ZMC3 系列及之前)		
TABLE 及数组 (ZMC4 系列及之后)	64 位浮点型	1.7E-308 到 1.7E+308
VRSTRING	字符	一个字符占一个 VR
MODBUS_STRING	字符	一个字符占 8 位

数据操作

不同类型数据之间的操作，会产生下列问题：

数据丢失

浮点型向整型转换时会丢失小数部分

VR=10.314

MODBUS_REG(0)=0

MODBUS_REG(0)=VR(0)

?MODBUS_REG(0) ' 结果为 10

强制转换

整型存储到浮点型寄存器后会变成浮点型，再使用整型操作数据可能会不对。

常见使用问题：

获取日期时，不要使用单精度浮点型存储，因为日期格式是 8 位的，而单精度浮点有效位只有 6 位。建议直接使用 32 位整型 MODBUS_LONG 来存储。

字符串

部分参数必须用字符串类型常量或变量，各种字符串可以通过“+”合并，对字符串的单个字节的操作需要利用数组来进行。

字符串相关指令列表：

DIM	定义的数组可以直接作为字符串使用，每个元素表示一个字节。
“ ”	双引号直接定义字符串常量。
CHR	把 ASCII 转成一个字符串常量，此字符串只有一个字节。
MODBUS_STRING	标准 MODBUS 协议定义字符串，每个 16 位寄存器存储 2 个字节。
VRSTRING	VR 列表作为字符串使用，一个 VR 存储一个字节。
+	操作符，两个字符串合并。
VAL	数字字符串转换为数值。
TOSTR	数值转换为字符串。
STRCOMP	字符串比较函数。
DMCPY	数组拷贝函数，可以用于拷贝字符串。
HEX	返回为十六进制格式，只能用于打印。
DATE\$	按 DD:MM:YYYY 格式返回 DATE 设置日期。
DAY\$	返回本日的星期英文名。
TIME\$	按 24 小时的格式 hh:mm:ss 返回当前时间。

运动指令

运动指令用来实现点位运动、直线插补、圆弧插补、凸轮运动等。
在运动前要设置好轴参数。

运动缓冲

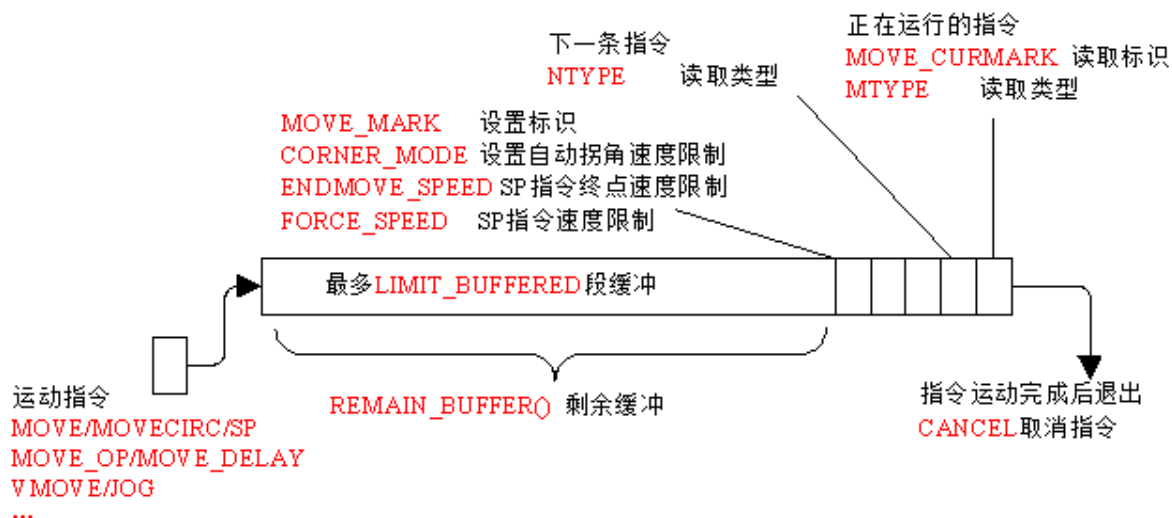
ZMotion 运动控制器具有多级的缓冲，当前有运动指令正在执行时，后面调用的运动指令会填入缓冲，避免程序的阻塞。当多级运动缓冲区全部被塞满时，如果程序运行有更多的运动指令时程序也会被阻塞。

运动指令主要指直线、圆弧等各种插补运动指令以及 MOVE 前缀的一些指令。

使用带 SP 指令时，FORCE_SPEED 和 ENDMOVE_SPEED，STRATMOVE_SPEED 会随 sp 运动指令写入运动缓存区。

可以用 MOVE_OP 把 IO 操作指令填入缓冲，这样在运动指令之间自动加入 IO 输出。

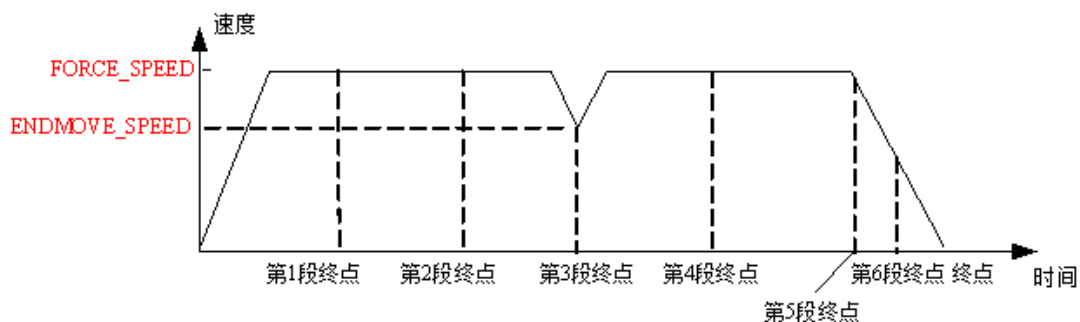
可以用 MOVE_DELAY 把延时指令填入缓冲，这样在运动指令之间自动延时。



连续插补

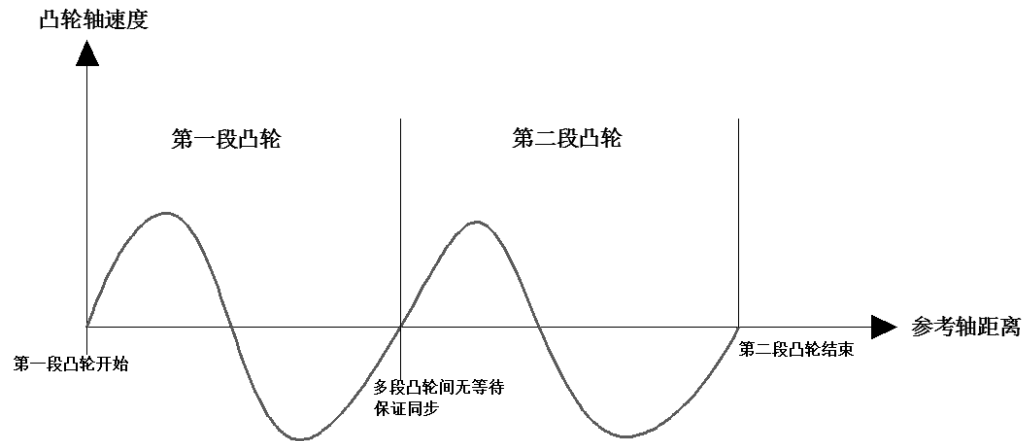
设置 MERGE=ON 以后，相同主轴的插补运动会自动被连续起来，而且 SP 指令可以手动设置运动速度和结束速度。

参见：MERGE，SP，CORNER_MODE，ENDMOVE_SPEED，FORCE_SPEED

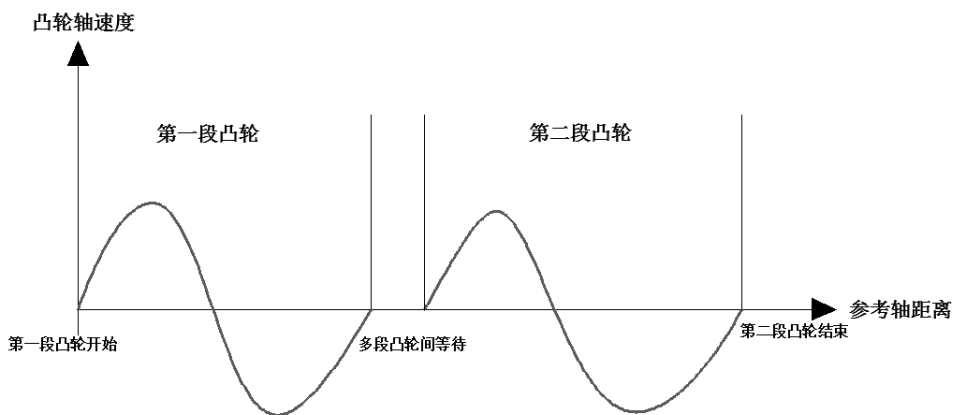


连续凸轮

多段凸轮指令被存入运动缓冲区后，下一段凸轮会在前一段凸轮完成后立刻开始，保证同步。



若凸轮指令没有被存入运动缓冲区，多段凸轮间会有等待时间，不连续。



参见：CAM，CAMBOX

硬件比较输出

硬件比较输出是通过比较轴是否到达设定位置，来操作输出口动作，目前只有 4 系列的脉冲轴与编码器轴支持此功能，需要最新固件。具体查看第七章 HW_SWITCH 指令。

电机轴与编码器轴

电机：主动运行，根据控制器发出的脉冲来运动，一般分为伺服电机和步进电机。发送的脉冲数根据 $DPOS * UNITS$ 确定。

编码器：被动运行，跟随电机转动，产生脉冲，反馈给驱动器(伺服)/控制器(步进)，接收的脉冲数与发送的脉冲数对比，判断运动是否准确。控制器接收的脉冲数根据 ENCODER 指令确定。

轴类型：一般电机轴 ATYPE=1，编码器轴 ATYPE=3/6，具体查看 ATYPE 指令。

轴接线：电机和编码器接线查看控制器硬件手册确定。

轴号：

脉冲控制器

电机轴号是其连线的 DB 轴接口的编号，印在外壳上，形如 Axis0, Axis1...（没有 DB 接口的查看控制器硬件手册确定）



而同一 DB 轴接口的编码器轴号在 ZDevelop 中查看控制器状态确定(或者硬件手册)，如下图，电机轴号是轴 0，对应的编码器轴号是轴 6，电机轴 1 对应编码器轴 7，依次往下。

下图中，假设电机和编码器都连接在 Axis0 接口，那么电机轴号 0，编码器轴号 6；假设电机连接在 Axis1 接口，编码器连接在 Axis2 接口，那么电机轴号 1，编码器轴号 8。



总线控制器

总线控制器包括总线轴和脉冲轴，总线轴通过 AXIS_ADDRESS 指令来映射总线连接的驱动器设备的轴号；脉冲轴和脉冲控制器一致。

修改电机运动方向

- 脉冲轴：1. [invert_step](#) 指令选择模式
2. [step_ratio](#) 指令将分母设为负值
3. 驱动器修改方向
- 总线轴：1. [step_ratio](#) 指令将分母设为负值
2. 驱动器修改转向

参数

参数分轴参数，任务参数，系统参数等，参数可读可写（除少数只读参数外）。
在运动前要设置好轴参数。

参数有自动保存和非自动保存两类。

自动保存的参数修改后自动存储，重新上电不会恢复为缺省值，相关指令有：
[IP_ADDRESS](#)、[APP_PASS](#) 和 [LOCK](#) 等密码指令、[CANIO_ADDRESS](#)

非自动保存的参数上电后恢复为缺省值，要重新修改，比如 [setcom](#) 设置串口参数，每次上电后都要再设置一次，所以 [setcom](#) 指令要放在程序开头。

掉电存储

控制器具有电池保护寄存器 VR 和多个存储 Flash 块 (ZDevelop 在线命令? [FLASH_SECTES](#) 参数可以查看)，可以用于保存掉电的数据。

ONPOWER 掉电中断可以用于触发记录掉电时的位置到 VR。

使用 [setcom](#) 指令可以把 [VR](#) 与 [modbus_reg](#) 寄存器匹配起来，指令参数 variable 需为 0。



VR 是电池维持的，可以无限次的读写，但是长时间不开机可能数据丢失，机器关键参数建议存储到 Flash。



Flash 是有写寿命限制，不可以无限次的擦写，经常改写的的数据建议写入 VR。

设置程序上电时运行

程序必须存储在 ROM(掉电才能保存),并且对应自动运行的文件设置自动运行(autorun)任务号,参见 ZDevelop 工具软件的介绍。

EtherCat 总线使用

必须是控制器支持 EtherCAT 才可以使用相关的指令。

与 Rtex 总线使用一套程序指令,但是功能有所区别,详细请看第十二章总线指令说明。

基本概念:

node 节点编号——根据 EtherCAT 设备接线顺序排列,编号从 0 开始到设备个数-1。

驱动器编号——根据 EtherCAT 设备接线顺序排列,编号从 0 开始到 ecat 驱动器个数-1,只在 AXIS_ADDRESS 配置时有作用,即只计算驱动器设备,其他扩展 IO 等设备不计入编号。

轴号——控制器对轴的编号,从 0 开始到连接总轴数-1,通过 AXIS_ADDRESS 指令可以将轴号映射到任何一个连接的驱动器设备。

一般过程:

- 1, 使用 SLOT_SCAN 扫描设备,判断 RETURN 是否正确,未连接设备时不会报错。
- 2, 通过 NODE_INFO/ NODE_AXIS_COUNT 等对设备类型、信息等进行判断。
- 3, 依次设置 AXIS_ADDRESS, , ATYPE, DRIVE_PROFILE, DRIVE_IO 等。
- 4, SLOT_START 启动设备

EtherCAT 相关指令

SLOT_SCAN	扫描设备。
SLOT_START	总线启动
SLOT_STOP	总线停止
ATYPE	轴类型 65-ecat 周期位置控制
AXIS_ADDRESS	轴地址配置
WDOG	轴统一使能控制
SERVO_PERIOD	刷新率
AXIS_ENABLE	轴使能
SDO_WRITE	SDO 操作
SDO_READ	SDO 操作
SDO_WRITE_AXIS	驱动器 SDO 操作
SDO_READ_AXIS	驱动器 SDO 操作
?*ETHERCAT	总线信息输出
NODE_COUNT	设备个数
NODE_STATUS	设备状态
NODE_AXIS_COUNT	设备带驱动个数
NODE_IO	设备 IO 编号

NODE_AIO	设备 AIO 编号
NODE_INFO	设备信息读取
NODE_PROFILE	设备 profile 设置，选择 PDO 报文内容
DRIVE_FE	驱动误差
DRIVE_FE_LIMIT	驱动误差设置
DRIVE_STATUS	驱动状态字
DRIVE_TORQUE	驱动器力矩反馈
DRIVE_MODE	驱动器运动模式
DRIVE_PROFILE	驱动器 profile 设置，选择 PDO 报文内容
DRIVE_CONTROLWORD	驱动器控制字
DRIVE_CW_MODE	驱动器控制字操作模式
DRIVE_IO	远程 IO 编号设置
AXISSTATUS	轴状态

Rtex 总线使用

必须是控制器支持 Rtex 才可以使用相关的指令。

接线时控制器 RX—驱动器 TX，控制器 TX—RX。

Rtex 驱动器的控制字会自动设置，需要手动设置时请先设置 DRIVE_CW_MODE。

与 EtherCAT 总线使用一套程序指令，但是功能有所区别，详细请看第十二章总线指令说明。

基本概念：

node 节点编号——根据 Rtex 设备接线顺序排列，编号从 0 开始到设备个数-1。

驱动器编号——根据 Rtex 设备接线顺序排列，编号从 0 开始到 Rtex 驱动器个数-1，只在 AXIS_ADDRESS 配置时有作用，即只计算驱动器设备，其他扩展 IO 等设备不计入编号。

轴号——控制器对轴的编号，从 0 开始到连接总轴数-1，通过 AXIS_ADDRESS 指令可以将轴号映射到任何一个连接的驱动器设备。

一般过程：

- 1, 使用 SLOT_SCAN 扫描设备, 判断 RETURN 是否正确，未连接设备时会报错。
- 2, 通过 NODE_INFO/ NODE_AXIS_COUNT 等对设备类型、信息等进行判断。
- 3, 依次设置 AIXS_ADDRESS, , ATYPE, DRIVE_PROFILE, DRIVE_IO 等。
- 4, SLOT_START 启动设备

Rtex 相关指令

SLOT_SCAN	扫描设备，未连接设备会报错。
SLOT_START	总线启动
SLOT_STOP	总线停止
ATYPE	轴类型 50-Rtex 周期位置控制

AXIS_ADDRESS	轴地址配置
WDOG	轴统一使能控制
SERVO_PERIOD	刷新率
AXIS_ENABLE	轴使能
?*Rtex	总线信息输出
NODE_COUNT	设备个数
NODE_STATUS	设备状态
NODE_AXIS_COUNT	设备带驱动个数
NODE_IO	设备 IO 编号
NODE_AIO	设备 AIO 编号
NODE_INFO	设备信息读取
DRIVE_STATUS	驱动状态字
DRIVE_TORQUE	驱动器力矩反馈
DRIVE_PROFILE	驱动器 profile 设置
DRIVE_CONTROLWORD	驱动器控制字,DRIVE_CW_MODE 设为 1 时才可以修改
DRIVE_CW_MODE	驱动器控制字操作模式。自动/手动
DRIVE_IO	远程 IO 编号设置
DRIVE_CLEAR	驱动器清除报警，没有错误时使用控制器会警告
DRIVE_READ	驱动器读参数
DRIVE_WRITE	驱动器写参数
AXISSTATUS	轴状态

程序调试

ZDevelop 连接控制器后，程序下载到 RAM，从菜单选择调试/启动调试，请保证 PC 程序文件和控制器的程序文件一致，否则可能导致光标位置错误；只能在控制器 UNLOCK 时进行程序调试。



调试机器要注意安全！请务必在机器中设计有效的安全保护装置，并在软件中加入出错处理程序，否则所造成的损失，正运动技术公司没有义务或责任对此负责。

连接人机界面

控制器支持 MODBUS-RTU 协议或 MODBUS-TCP 协议连接人机界面。

使用 ADDRESS 参数设置协议站号，缺省 1。串口连接时配置好对应串口号，以太网连接时配置人机界面的 PLC 端 IP 为控制器 IP（仿真器可以为本机 IP 127.0.0.1），端口号 502。

设备属性

名称: MODBUS TCP/IP (zero-based addressing)

☐ HMI ☒ PLC

所在位置: 本机 [设置...]

PLC 类型: MODBUS TCP/IP (zero-based addressing)
V.1.50, MODBUS_TCP/IP_ZERO_BASED.sl

接口类型: 以太网

IP: 192.168.0.11, 端口号=502 [设置...]

☐ 使用 UDP (User Datagram Protocol)

PLC 预设站号: 1

☐ 预设站号使用站号变量

☐ 使用广播命令

PLC 地址整段间隔 (words): 5

最大读取字数 (words): 120

最大写入字数 (words): 120

事件序列

☐ 启用

[确定] [取消]

威纶触摸屏以太网设置图

常见问题

程序问题:



当程序运动出错后，ZDevelop 软件会显示出出错信息，如果出错信息没有看到，可以通过命令行输入 `?*task` 再次查看出错信息，双击出错信息可以自动切换到程序出错位置。下表中未找到对应代码的，请查看《错误码列表》文档。

问题	可能原因
2043:Unknown function is met	不认识的标识符，控制器不支持的功能
stop of error:2049: Line not ended.	1, 部分命令必须占用一整行。 2, GSUB 调用不需要括号()
stop of error:2033: Unknown label is met.	1, 未定义的变量或数组。 2, 未定义的 SUB 过程 3, 有定义数组，但是定义的语句没有执行到，可能应文件没有设置自动运行。
2048:Function can only be used expression	函数必须返回值，不用在一行的开头地方。
2064:Param few	函数参数过少

2063:Param too many	函数参数过多
2072:Need = sign	未写=号
2060:Syntax format error	指令语法错误
error:1010	重复暂停
error:1011	没有运动，无法暂停

电机不动

驱动器原因：

驱动器的出厂设置一般没有反转 I0 电平，会导致驱动器限位报警，要根据驱动器手册设置限位电平反转。比如松下伺服要设置 pr4.01, pr4.02 的参数分别为 010101h (65793)，020202h (131586)。

对应参数	出厂设置值 (10 进制)	位置控制/ 全闭环控制	
		信号名称	逻辑 *1)
Pr4.00	00323232h (3289650)	S1-MON5	常开 (NO)
Pr4.01	00818181h (8487297)	POT	常闭 (NC)
Pr4.02	00828282h (8553090)	NOT	常闭 (NC)

信号名称	记号	设定值	
		常开 (NO)	常闭 (NC)
无效	—	00h	设定不可
正方向驱动禁止输入	POT	01h	81h
负方向驱动禁止输入	NOT	02h	82h

其他品牌驱动器请根据相关驱动器手册操作。

程序可能原因：

1. UNITS 设置过小，电机运动极为缓慢。肉眼无法分辨。
2. 电机处于异常状态（限位、报警...），无法运动，判断 AXISSTATUS。
3. 电机接线错误，发出脉冲无法正确传递。
4. 轴使能 OP 口未打开（伺服电机才需要打开）。
5. 程序处理使得电机无法运动，下空程序确认。
6. 驱动器报警。

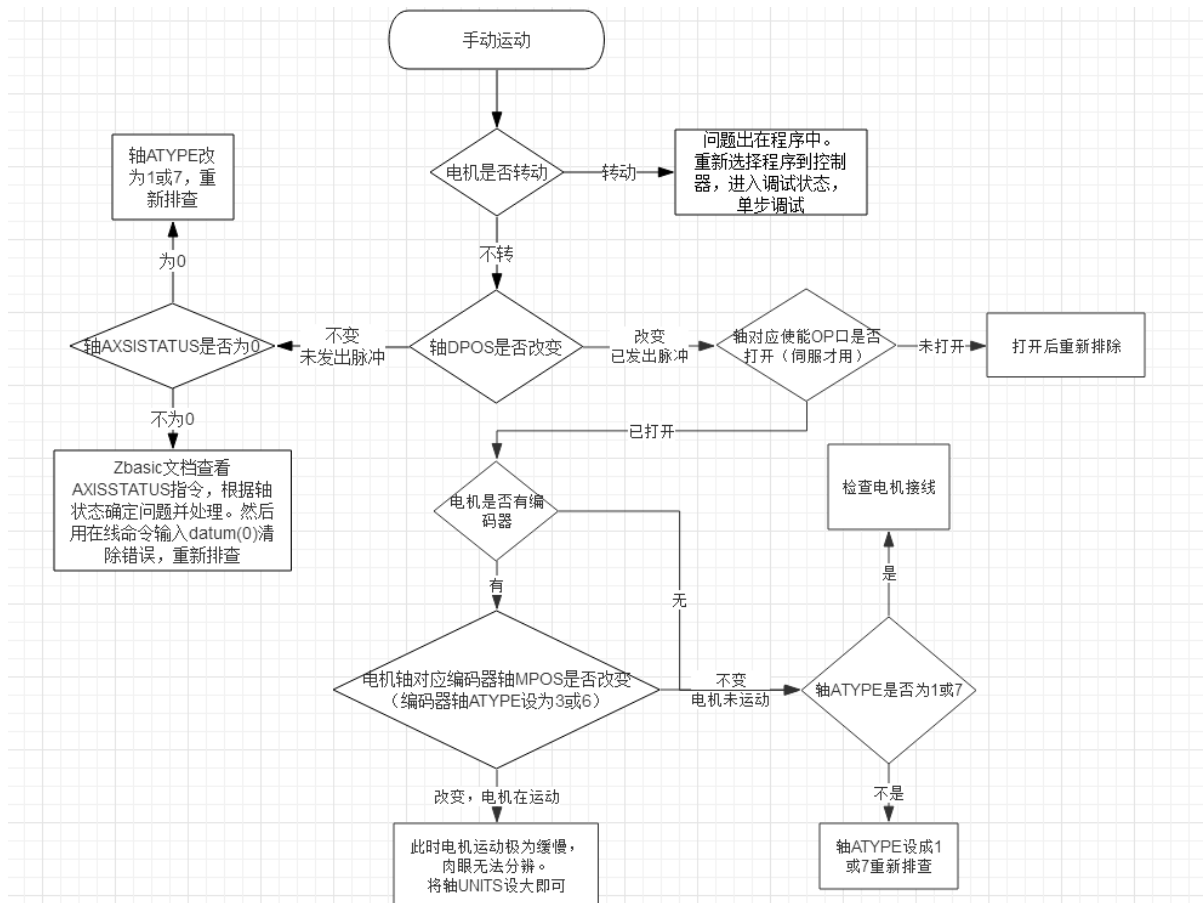
7 及以下原因为总线轴才会出现

7. 总线扫描、开启失败，打印返回值确认。
8. 未打开 wdog 总使能、axis_enable 轴使能指令。
9. 驱动器状态设置错误，具体查看驱动器手册。

排查步骤：

请做以下处理来排查问题：

1. 使用 ZDevelop 软件进行问题排查。
2. 关闭除 ZDevelop 的其他连接控制器的软件、程序等，避免外部因素影响。
3. 使用 ZDevelop 下载一个空程序到控制器，避免内部因素影响。
4. 打开 ZDevelop 的“视图”-“手动运动”和“视图”-“轴参数”，便于操作和查看。
5. **脉冲轴**按照下方步骤进行排查。



电机只能单向运动

可能原因：

1. 电机处于限位状态，查看 AXISSTATUS 确认。
2. 电机控制模式不对，INVERT_STEP 设置为相应模式（双脉冲或脉冲+方向）。
3. 电机接线问题，确认接线。

第二章 运动指令

当前运动指令正在执行时，后续调用的运动指令会自动缓冲起来，ZMotion 运动控制器每个轴可以支持最多达 512 级运动缓冲（不同型号控制器缓冲个数有区别），当缓冲全部占用时，后续调用的运动指令会阻塞当前任务，直到缓冲有空位。

每个运动指令都带有一个 MOVE_MARK 参数，通过 MOVE_CURMARK 可以知道当前缓冲运行到哪一个了。

MOVE 等单轴运动指令采用 SPEED 等各自单轴的轴参数。

MOVE 等多轴插补运动指令采用 base 主轴 SPEED 等轴参数作为矢量合成速度，但其有相应的 SP 指令，SP 指令可以为每个运动指定不同的速度等参数 (FORCE_SPEED, STARTMOVE_SPEED, ENDMOVE_SPEED)，参见*SP。

轴参数 MERGE 用来设置多轴插补指令中间是否减速到零，MERGE=ON 时不减速，此时轴参数 CORNER_MODE 可以设置多轴插补之间是否自动减速到必要的速度。

ZMotion 运动控制器支持运动暂停，参见 MOVE_PAUSE。

ZMotion 运动控制器支持运动叠加，参见 ADDAX。

BASE -- 轴选择

类型	运动指令
描述	选择要设置参数、参与运动的轴。 缺省值依次为：0, 1, 2... 程序中在下一条 BASE 指令被执行前，都以上一条 BASE 指令选择轴 每个任务拥有独立的轴列表，会记住任务中 BASE 选择的轴，用于不同机台的控制。 当插补运动的时候，第一个轴的运动参数作为插补参数。 见例一 如果在 BASE 指令中没有列出所有的轴，BASE 指令自动将剩余轴顺序排列在后面。见例二
语法	BASE(axis<,second axis><,third axis>...) axis 第一个轴 second axis 下一个轴 ... 参数最多为控制器支持的轴数，参照对应控制器硬件手册
适用控制器	通用
例子	例一 BASE(0, 1, 2, 3) ' 轴列表选择为：0, 1, 2, 3 SPEED=100, 10, 20, 30 ' 此时轴 0, 1, 2, 3 设置了对应速度，但是插补 ' 运动时，只有主轴速度 100 起作用 MOVE(100, 100, 100, 100) ' 轴 0, 1, 2, 3 联合插补运动，合成运动的速

	度 ' 为 100，各轴速度为分速度
例二	
BASE (1)	' 轴列表选择为：1
MOVE (100, 100, 100)	' 轴 1, 2, 3 做插补运动
例三	
BASE (0, 2, 5)	' 轴列表选择为：0, 2, 5
MOVE (100, 100, 100)	' 轴 0, 2, 5 做插补运动

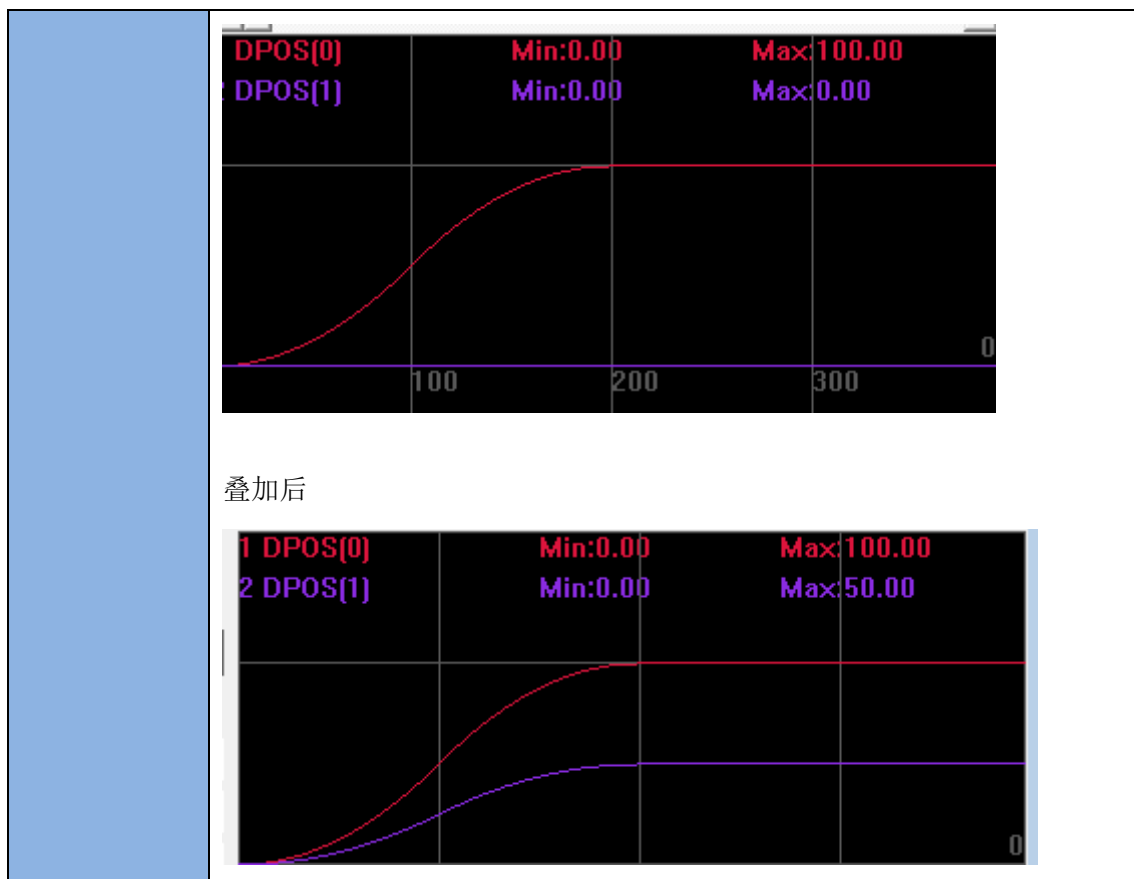
AXIS -- 临时轴选择

类型	辅助指令
描述	临时修改一个运动指令或轴参数到一个指定轴上去执行。 对轴参数，可以 AXIS 可以省掉。
语法	AXIS (expression) expression 临时修正的新轴号，执行完后，轴选择仍以 BASE 指令为准
适用控制器	通用
例子	例一 BASE (0) MOVE (1000) AXIS (1) ' 此时强制轴 1 运动 1000units MOVE (100) ' 轴 0 运动 100units 例二 BASE (1) UNITS AXIS (0)=100 ' 强制指定轴 0 UNITS 设为 100 ' 可简写为 UNITS (0)=100 UNITS=10 ' 轴 1 UNITS 设为 10
相关指令	BASE

ADDAX -- 运动叠加

类型	单轴运动指令
----	--------

描述	运动叠加，把一个轴的运动叠加到另一个轴。 ADDAX 指令叠加的是脉冲个数，而不是设置的 units 单位。 转换关系：叠加轴运动距离*叠加轴 UNITS/被叠加轴 UNITS=被叠加轴运动距离。 假设轴 A 的 UNITS 是 100，轴 B 的 UNITS 是 50, 叠加轴运动 100 把轴 A 的运动叠加到轴 B，此时轴 A 显示运动了 100，轴 B 运动了 100*100/50=200。 把轴 B 的运动叠加到轴 A，此时轴 B 显示运动了 100，轴 A 运动了 100*50/100=50。 轴之间不能相互同时叠加，A 叠加到 B 后，B 不能再叠加到 A。 叠加时速度从被叠加轴开始变化，加减速按照叠加轴加减速及两轴 units 比例确定。 ADDAX 在 轴 MTYPE 为 FRAME，或 REFRAME 的时候不起作用。 在轴 A TYPE 为纯编码器类型的时候，不起作用。												
语法	ADDAX(axis)												
适用控制器	通用												
例子	BASE(0,1) ATYPE=1,1 UNITS=100,200 ' 轴 0 UNITS 设 100，轴 1 UNITS 设 200 SPEED=1000,1000 ' 速度设 1000 ACCEL=10000,10000 ' 加速度 10000 DECEL=10000,10000 ' 减速度 10000 ADDAX(0)AXIS(1) ' 轴 0 的运动叠加到轴 1，按脉冲个数叠加 DPOS=0,0 ' 设置位置为 0,0 TRIGGER ' 自动触发示波器 MOVE(100) ' 轴 0 运动 100，此时轴 1 运动 100*100/200=50 ' 要考虑到两轴 UNITS 的转换 WAIT IDLE ' 等待运行完 ADDAX(-1)AXIS(1) ' 取消叠加 叠加前 <table><tr><th>编号</th><th>数据源</th><th>偏移</th><th>刻度</th></tr><tr><td>0</td><td>DPOS</td><td>0</td><td>100</td></tr><tr><td>1</td><td>DPOS</td><td>0</td><td>100</td></tr></table>	编号	数据源	偏移	刻度	0	DPOS	0	100	1	DPOS	0	100
编号	数据源	偏移	刻度										
0	DPOS	0	100										
1	DPOS	0	100										



DEFPOS -- 位置偏移

类型	坐标指令
描述	设置当前位置为一个新的绝对位置值，不会对已运行/进入缓冲区的运动产生影响。
语法	DEFPOS(pos1 [,pos2[, pos3[, pos4.....]]]) pos1: 绝对位置，采用 unit 定义单位。 pos2: 下一个轴绝对位置，采用 unit 定义单位。
适用控制器	通用
例子	<pre> BASE(0,1) ' 选择轴 0，轴 1 ATYPE=1,1 UNITS=100,100 ' UNITS 设为 100 DPOS=0,0 ' 把 DPOS 清 0 MOVE(100,100) ' 轴 0 和轴 1 运动 100 WIAT IDLE ?DPOS(0),DPOS(1) ' 此时 DPOS 都为 100 DEFPOS(0,10) ' 设置当前位置 ?DPOS(0),DPOS(1) ' 此时 DPOS 为 0，10 </pre>

	与 OFFPOS 相对改变不同，DEFPOS 是改变为绝对位置 BASE (0, 1) ’ 选择轴 0，轴 1 DPOS=100, 100 ’ 设置当前位置为 100, 100 ?DPOS (0), DPOS (1) ’ 打印确认 DEFPOS (10, 20) ’ 设置当前位置为 10, 20 ?DPOS (0), DPOS (1) ’ 当前位置为 10, 20 DEFPOS (10, 20) ’ 多次调用 DEFPOS DEFPOS (10, 20) ?DPOS (0), DPOS (1) ’ 此时当前位置仍为 10, 20 OFFPOS=10, 20 ’ 多次调用 OFFPOS OFFPOS=10, 20 ?DPOS (0), DPOS (1) ’ 此时当前位置变为 30, 60 (10+10+10, 20+20+20)
相关指令	DPOS ， OFFPOS

CANCEL -- 单轴停止

类型	单轴运动指令								
描述	BASE 轴减速停止，如果轴参与插补，也停止插补运动。 Mode0~2 减速度按 FASTDEC 和 DECEL 中最大的值。 CANCEL 后要调用绝对位置运动，必须先 WAIT IDLE 等待停止完成。								
语法	CANCEL (mode) 参数： mode <table border="1"> <tr> <td>0 (缺省)</td><td>取消当前运动</td></tr> <tr> <td>1</td><td>取消缓冲的运动</td></tr> <tr> <td>2</td><td>取消当前运动和缓冲运动。</td></tr> <tr> <td>3</td><td>立即中断脉冲发送。</td></tr> </table>	0 (缺省)	取消当前运动	1	取消缓冲的运动	2	取消当前运动和缓冲运动。	3	立即中断脉冲发送。
0 (缺省)	取消当前运动								
1	取消缓冲的运动								
2	取消当前运动和缓冲运动。								
3	立即中断脉冲发送。								
适用控制器	通用								
例子	例一 BASE (0) ATYPE=1 UNITS=100 SPEED=100 ACCEL=1000 DECEL=1000 ’ 减速度设为 1000 FASTDEC=10000 ’ 快减减速度设为 10000								

	MOVE (1000)	' 当前运动
	MOVE (-1000)	' 缓冲运动
	CANCEL (1)	' 此时轴只执行完 MOVE (1000)，减速度为 10000
	例二	
	BASE (0)	
	ATYPE=1	
	DPOS=0	
	SPEED=100	
	MOVE (10000)	' 当前运动 10000
	DELAY (2000)	' 延时 2 秒
	CANCEL (3)	' 此时直接切断脉冲发送，轴立即停止。
	例三	
	BASE (0, 1)	
	ATYPE=1, 1	
	SPEED=1000	
	MOVE (1000, 1000)	' 插补运动
	DELAY (1000)	' 延时 1 秒
	CANCEL (2) AXIS (1)	' 停止轴 1，轴 1 参与了插补，插补运动也停止
相关指令	RAPIDSTOP , DECEL , FASTDEC	

RAPIDSTOP -- 全部轴停止

类型	多轴运动指令								
描述	所有轴立即停止，如果轴参与插补，也停止插补运动。 Mode0~2 减速度按 FASTDEC 和 DECEL 中最大的值。 RAPIDSTOP 后要调用绝对位置运动，必须先 WAIT IDLE 等待停止完成。								
语法	RAPIDSTOP (mode) 参数: mode <table border="1"> <tr> <td>0 (缺省)</td><td>取消当前运动</td></tr> <tr> <td>1</td><td>取消缓冲的运动</td></tr> <tr> <td>2</td><td>取消当前运动和缓冲运动。</td></tr> <tr> <td>3</td><td>立即中断脉冲发送。</td></tr> </table>	0 (缺省)	取消当前运动	1	取消缓冲的运动	2	取消当前运动和缓冲运动。	3	立即中断脉冲发送。
0 (缺省)	取消当前运动								
1	取消缓冲的运动								
2	取消当前运动和缓冲运动。								
3	立即中断脉冲发送。								
适用控制器	通用								

例子	例一 <pre> BASE (0, 1, 2) ATYPE=1, 1, 1 UNITS=100 SPEED=100 ' 插补合成运动速度 100 ACCEL=1000 DECEL=1000 ' 减速度设为 1000 FASTDEC=10000 ' 快减减速度设为 10000 MOVE (1000, 1000, 1000) ' 当前运动 MOVE (-1000, -1000, -1000) ' 缓冲运动 RAPIDSTOP (1) ' 此时轴只执行完当前运动，减速度为 10000 </pre> 例二 <pre> BASE (0, 1) ATYPE=1, 1 DPOS=0, 0 SPEED=100 MOVE (10000, 10000) ' 插补运动 10000 DELAY (2000) ' 延时 2 秒 RAPIDSTOP (3) ' 此时直接切断脉冲发送，轴立即停止。 </pre>
相关指令	CANCEL , DECEL , FASTDEC

FORWARD -- 正向运动

类型	单轴运动指令
描述	BASE 选择轴正向运动 必须 CANCEL 后才能切换 REVERSE。
语法	Forward [axis(轴)]
适用控制器	通用
例子	例一 <pre> Base (0) FORWARD ' 轴 0 持续正向运动 WAIT UNTIL IN(1)=ON ' 等待输入 1 有效 CANCEL (2) </pre> 例二 <pre> FORWARD AXIS(1) ' 轴 1 正向运动 </pre>

	WAIT UNTIL IN(1)=ON ' 等待输入 1 有效 CANCEL(2) AXIS(1)
相关指令	REVERSE , VMOVE

REVERSE -- 负向运动

类型	单轴运动指令
描述	BASE 选择轴负向运动 必须 CANCEL 后才能切换 FORWARD。
语法	reverse [axis(轴)]
适用控制器	通用
例子	例一 Base(0) REVERSE ' 轴 0 持续负向运动 WAIT UNTIL IN(1)=ON ' 等待输入 1 有效 CANCEL(2) 例二 REVERSE AXIS(1) ' 轴 1 负向运动 WAIT UNTIL IN(1)=ON ' 等待输入 1 有效 CANCEL(2) AXIS(1)
相关指令	FORWARD , VMOVE

VMOVE -- 持续运动

类型	单轴运动指令
描述	连续往一个方向运动。 当前面的 VMOVE 运动没有停止时，此 VMOVE 指令会自动替换前面的 VMOVE 指令并修改方向，因此无需 CANCEL 前面的 VMOVE 指令。
语法	VMOVE(dir1) 参数: dir1 -1 负向运动 1 正向运动
适用控制器	通用

例子	BASE (0) ATYPE=1 SPEED=100 VMOVE (-1) '持续负向运动 WAIT UNTIL IN(1)=ON '等待输入 1 有效 VMOVE (1) '持续正向运动
相关指令	FORWARD , REVERSE

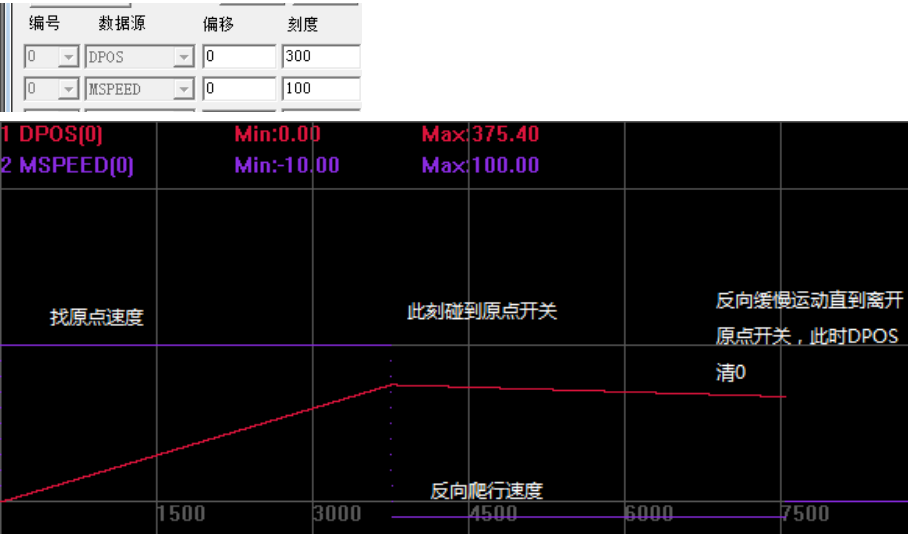
DATUM -- 回零

类型	单轴运动指令										
描述	找原点运动。 原点开关的输入点由参数 DATUM_IN 决定，正负限位开关通过 FWD_IN 和 REV_IN 设置。 ZMC 控制器为 0 触发有效，输入为 OFF 状态时，表示到达原点/限位，常开类型信号需要采用 INVERT_IN 反转电平。 ECI 控制器为 1 触发有效，输入为 ON 状态时，表示到达原点/限位，常闭类型信号需要采用 INVERT_IN 反转电平。 Z 信号回零必须配置为带 Z 信号 ATYPE。 总线控制器使用控制器找原点模式完成后，需要手动清 0 MPOS。										
语法	DATUM (mode), DATUM(21, mode2) 参数： mode 找原点模式，加 10 表示碰到限位后反找，不会碰到限位停止，例如：13=3+限位反找，用于原点在正中间的情况。 <table border="1"> <thead> <tr> <th>值</th><th>描述</th></tr> </thead> <tbody> <tr> <td>0</td><td>清除所有轴的错误状态。</td></tr> <tr> <td>1</td><td> 轴以 CREEP 速度正向运行直到 Z 信号出现。 碰到限位开关会直接停止。 DPOS 值重置为 0 同时纠正 MPOS。 </td></tr> <tr> <td>2</td><td> 轴以 CREEP 速度反向运行直到 Z 信号出现。 碰到限位开关会直接停止。 DPOS 值重置为 0 同时纠正 MPOS。 </td></tr> <tr> <td>3</td><td> 轴以 SPEED 速度正向运行，直到碰到原点开关。然后轴以 CREEP 速度反向运动直到离开原点开关。 </td></tr> </tbody> </table>	值	描述	0	清除所有轴的错误状态。	1	轴以 CREEP 速度正向运行直到 Z 信号出现。 碰到限位开关会直接停止。 DPOS 值重置为 0 同时纠正 MPOS。	2	轴以 CREEP 速度反向运行直到 Z 信号出现。 碰到限位开关会直接停止。 DPOS 值重置为 0 同时纠正 MPOS。	3	轴以 SPEED 速度正向运行，直到碰到原点开关。然后轴以 CREEP 速度反向运动直到离开原点开关。
值	描述										
0	清除所有轴的错误状态。										
1	轴以 CREEP 速度正向运行直到 Z 信号出现。 碰到限位开关会直接停止。 DPOS 值重置为 0 同时纠正 MPOS。										
2	轴以 CREEP 速度反向运行直到 Z 信号出现。 碰到限位开关会直接停止。 DPOS 值重置为 0 同时纠正 MPOS。										
3	轴以 SPEED 速度正向运行，直到碰到原点开关。然后轴以 CREEP 速度反向运动直到离开原点开关。										

		找原点阶段碰到正限位开关会直接停止。 爬行阶段碰到负限位开关会直接停止。 DPOS 值重置为 0 同时纠正 MPOS
	4	轴以 SPEED 速度反向运行，直到碰到原点开关。然后轴以 CREEP 速度正向运动直到离开原点开关。 找原点阶段碰到负限位开关会直接停止。 爬行阶段碰到正限位开关会直接停止。 DPOS 值重置为 0 同时纠正 MPOS。
	5	轴以 SPEED 速度正向运行，直到碰到原点开关。然后轴以 CREEP 速度反向运动直到离开原点开关，然后再继续以爬行速度反转直到碰到 Z 信号。 碰到限位开关会直接停止。 DPOS 值重置为 0 同时纠正 MPOS。
	6	轴以 SPEED 速度反向运行，直到碰到原点开关。然后轴以 CREEP 速度正向运动直到离开原点开关，然后再继续以爬行速度正转直到碰到 Z 信号。 碰到限位开关会直接停止。 DPOS 值重置为 0 同时纠正 MPOS。
	8	轴以 SPEED 速度正向运行，直到碰到原点开关。 碰到限位开关会直接停止。
	9	轴以 SPEED 速度反向运行，直到碰到原点开关。 碰到限位开关会直接停止。
	21	使用 Ethercat 驱动器回零功能，此时 mode2 有效。 设置驱动器回零方式（6098h），缺省 0 表示使用驱动器当前的回零方式。 会使用轴的 SPEED, CREEP, ACCEL, DECEL，乘以 UNITS 后自动设置驱动器的 6099h, 609Ah 动作时序： 6098 回零方式→6099 速度→609A 加速度→6060 切换当前模式
Mode2: mode=21 时有效，缺省 0，非 0 时设置到驱动器回零方式，根据驱动器手册数据字典 6098h 设置值。		
适用控制器	通用	
例子	例一 直接找原点 Base(0) Speed = 100 ' 找原点速度 Creep = 10 ' 反向爬行速度 Datum_in=5 ' 输入 IN5 作为原点开关 Invert_in(5,on) ' 反转 IN5 电平信号，常开信号进行反转(ZMC 控制器) TRIGGER ' 自动触发示波器 datum(3) ' 轴 0, 先以 100units/s 正向回零，找到原点后以	

10units/s
'直到离开原点，同时 DPOS 清 0

运动轨迹与速度曲线



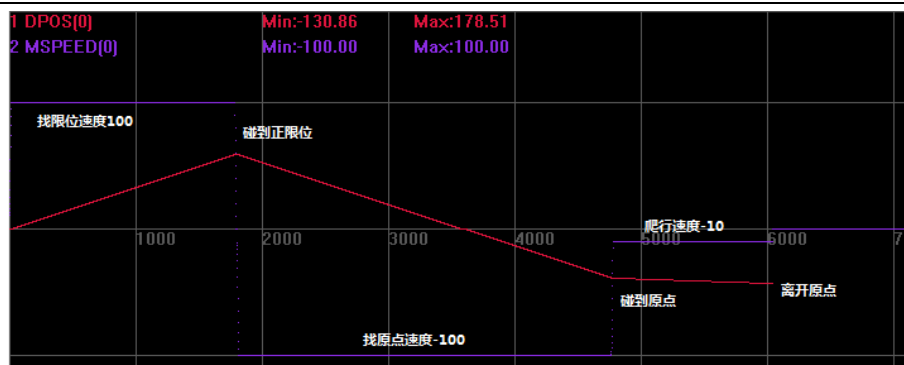
为了直观体现，所以爬行过程比较长，实际应用中，爬行过程很短。

例二 限位反找原点

Base (0)
Speed = 100 ' 找原点速度
Creep = 10 ' 反向爬行速度
Datum_in=5 ' 输入 IN5 作为原点开关
Fwd_in=6 ' 输入 IN6 作为正限位开关
Invert_in (5, on) ' 反转电平信号，一般常闭的信号才进行反转
Invert_in (6, on)
Trigger ' 自动触发示波器
datum (13) ' 轴 0, 先以 100units/s 正向运动，碰到正限位反向运行，仍
 ' 以 100units/s 找原点，找到原点后以 10units/s 直到离开原
 ' 点，同时 DPOS 清 0

运动轨迹与速度曲线





例三 EtherCAT 总线回零(松下 A6B 伺服)

先根据第十三章 [EtherCAT 初始化例程](#) 正常使能电机

SPEED=100 ' 找零速度*UNITS 后自动写入 6099

CREEP=10 ' 爬行速度*UNITS 后自动写入 6099

ACCEL=1000 ' 加减速*UNITS 后自动写入 609A

DECEL=1000

DATUM(21, 0) ' 按驱动器当前回零模式开始回零
' 此时按驱动器信号判断，而不是按
' 控制器信号判断

WHILE 1

TABLE(0)=DRIVE_STATUS ' 读取状态字判断回零状态

IF READ_BIT2(10, TABLE(0)) THEN ' 根据下图确定

IF READ_BIT2(12, TABLE(0)) THEN

?'回零完成"

ENDIF

ENDIF

WEND

ENDIF

驱动器厂家的回零过程可能不同，具体根据驱动器手册确定

bit13、bit12、bit10 的值的组合参照下表。

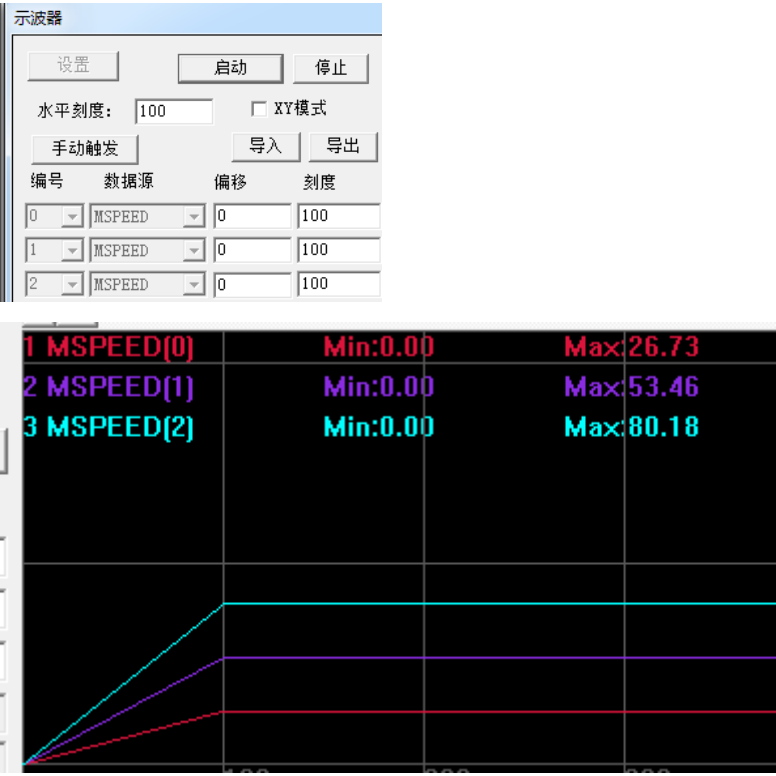
bit 13	bit 12 *2)	bit 10	Definition
0	0	0	原点复位动作中
0	0	1	原点复位动作中断，或者未开始
0	1	0	原点复位动作完成，但是未达到目标位置
0	1	1	原点复位动作正常完成
1	0	0	检出原点复位异常还在动作中
1	0	1	检出原点复位异常，停止状态

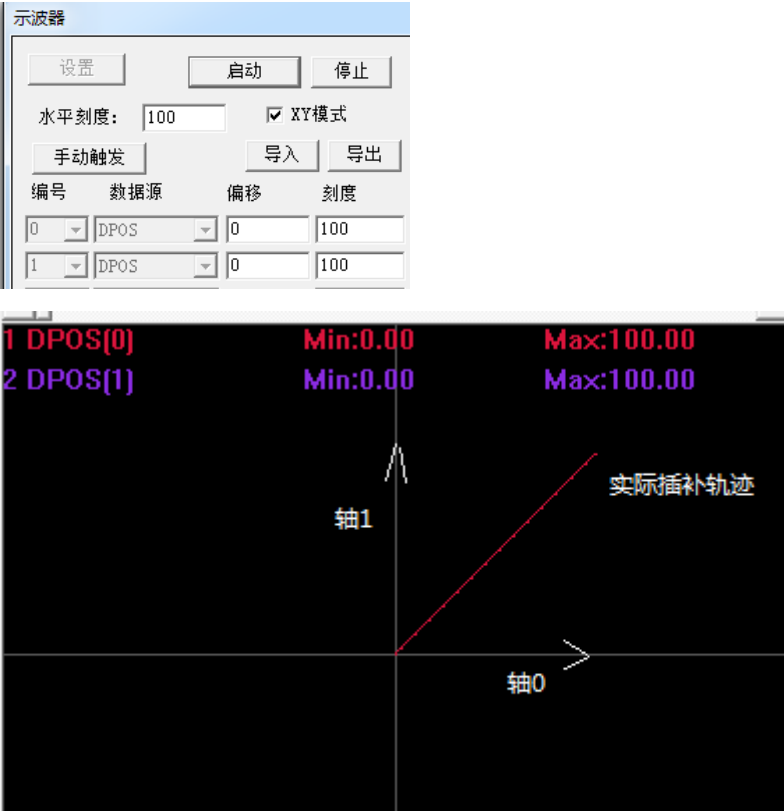
例四 Rtex 总线回零(松下 A6B 伺服)

	先根据第十三章 Rtex 初始化例程 正常使能电机	
	SPEED=100	’ 找零速度
	ACCEL=1000	’ 加减速
	DECEL=1000	
	DATUM (21, \$11)	’ 按驱动器当前回零模式开始回零
		’ 此时按驱动器信号判断，而不是按
		’ 控制器信号判断
	根据驱动器手册确定回零模式	
	[有] 初始化 模式	11h Z 相
		12h HOME ↑ *2)
		13h HOME ↓ *3)
		14h POT ↑ *2)
		15h POT ↓ *3)
		16h NOT ↑ *2)
		17h NOT ↓ *3)
		18h EXT1 ↑ *2)
		19h EXT1 ↓ *3)
		1Ah EXT2 ↑ *2)
		1Bh EXT2 ↓ *3)
		1Ch EXT3 ↑ *2)
		1Dh EXT3 ↓ *3)
相关指令	DATUM IN , INVERT IN	

MOVE —— 直线运动

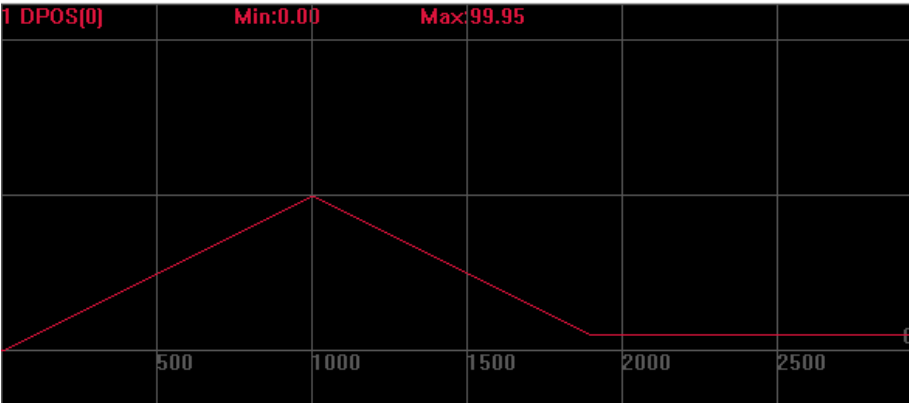
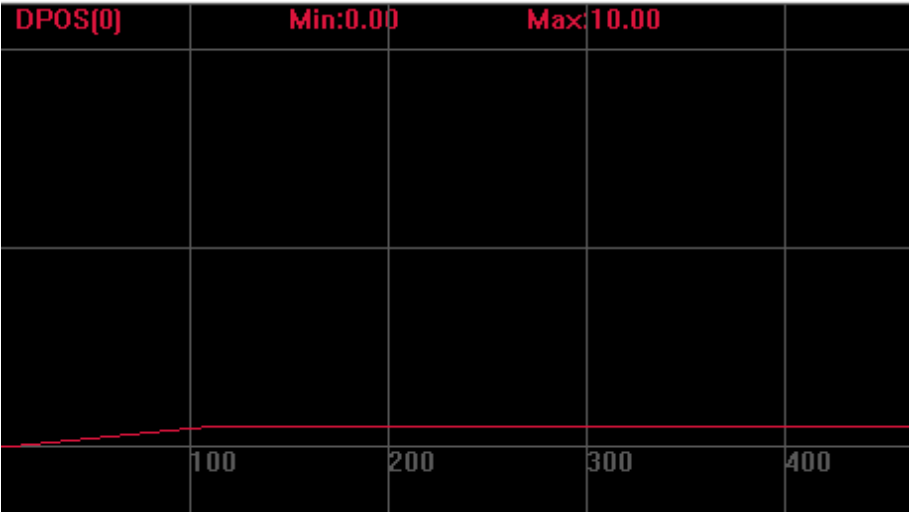
类型	多轴运动指令
描述	直线插补运动，相对运动一段距离。 插补运动时只有主轴速度参数有效，主轴是 BASE 的第一个轴，运动参照这个轴的参数。 自定义速度的连续插补运动可以使用 SP 后缀的指令，见*SP 描述。 插补运动距离 $X = \sqrt{X_0^2 + X_1^2 + X_2^2 + \dots + X_n^2}$ 运动时间 $T = X / \text{主轴 SPEED}$
语法	MOVE(distance1 [,distance2 [,distance3 [,distance4...]]) 参数： distance1 第一个轴运动距离 distance2 下一个轴运动距离

适用控制器	通用
例子	例一 <pre> Base(0, 1, 2,) ' 主轴为轴 0 Units=100, 100, 100 ' 脉冲当量设置 Speed=100, 10, 1000 ' 只有主轴速度 100 起作用，作为合成运动的速度 Accel=1000, 1000, 1000 Decel=1000, 1000, 1000 Dpos = 0, 0, 0 Trigger ' 自动触发示波器 move(500, 1000, 1500) ' 轴 0, 1, 2, 3 直线插补相对距离。 Wait idle ' 等待运动停止 Print *dpos ' 打印结果，500, 1000, 1500 </pre> 插补运动各轴实际速度为主轴速度的分速度  例二 <pre> Base(0, 1) Units=100, 100, 100 ' 脉冲当量设置 Speed=100, 100 Accel=1000, 1000, 1000 Decel=1000, 1000, 1000 Dpos=0, 0 </pre>

	Trigger ’ 自动触发示波器 Move(100,100) 插补轨迹 
相关指令	MOVEABS , *SP

MOVEMODIFY -- 修改运动位置

类型	运动指令
描述	修改上一个运动的目标位置。 前面没有运动时与 MOVEABS 效果一样，但不会进入运动缓冲。见例一 需要 WAIT 指令才能正确使用。见例二 连续插补时使用 MOVEMODIFY 会破坏速度连续性。 MOVEMODIFY 同时对多轴运动时不一定为直线插补。
语法	MOVEMODIFY (distance1 [,distance2 [,distance3 [,distance4...]]]) distance1 第一个轴运动距离 distance2 下一个轴运动距离 目前只支持单轴修改。

适用控制器	通用																
例子	例一 BASE (0) UNITS=100' 脉冲当量设置 DPOS=0 SPEED=100' 速度设置 ACCEL=10000' 加速度设置 DECEL=10000 TRIGGER' 自动触发示波器 MOVEABS (100) MOVEABS (10)' 此时轴会先运动到 100，在往回运动到 10 <table><tr><th>编号</th><th>数据源</th><th>偏移</th><th>刻度</th></tr><tr><td>0</td><td>DPOS</td><td>0</td><td>100</td></tr></table> MOVEMODIFY (100) MOVEMODIFY (10)' 此时轴会直接运动到 10 位置,MOVEMODIFY 不进入运动缓冲。 <table><tr><th>编号</th><th>数据源</th><th>偏移</th><th>刻度</th></tr><tr><td>0</td><td>DPOS</td><td>0</td><td>100</td></tr></table>	编号	数据源	偏移	刻度	0	DPOS	0	100	编号	数据源	偏移	刻度	0	DPOS	0	100
编号	数据源	偏移	刻度														
0	DPOS	0	100														
编号	数据源	偏移	刻度														
0	DPOS	0	100														

例二

BASE (0)

UNITS=100 '脉冲当量设置

DEFPOS (0)

SPEED=100 '速度设置

ACCEL=10000 '加速度设置

DECEL=10000

TRIGGER '自动触发示波器

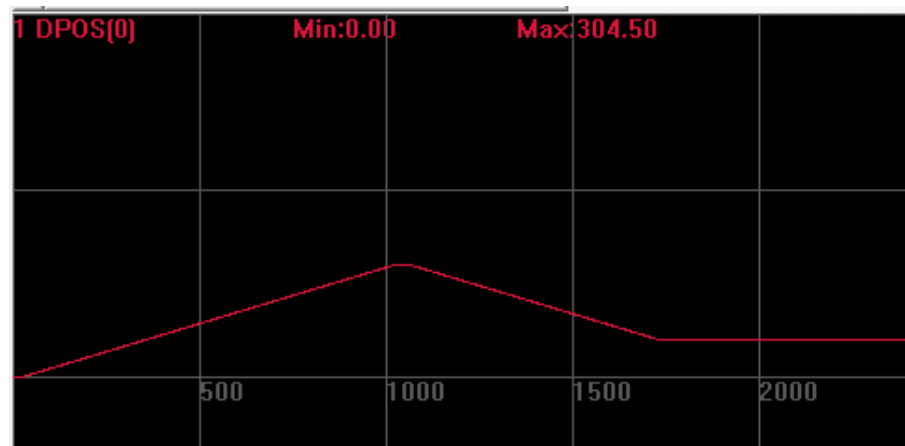
MOVEABS (500)

WAIT UNTIL DPOS >=300 '等待运动到 300 时，才修改终点位置

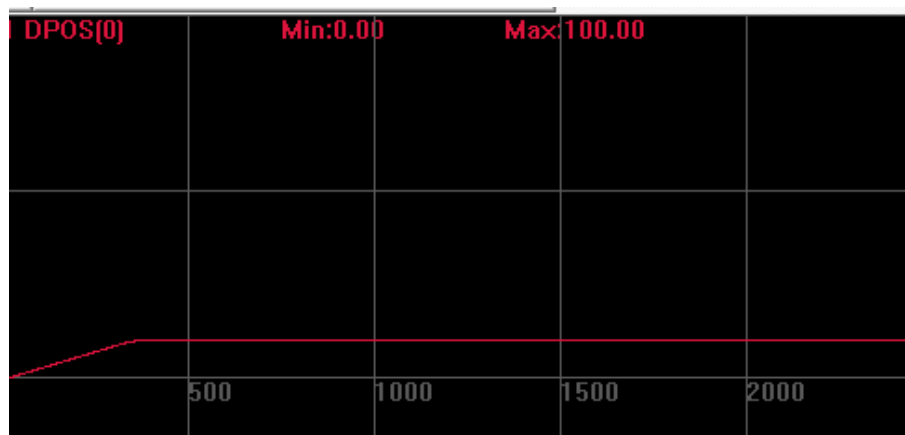
MOVEMODIFY (100) '修改目标位置为 100，此时会减速停止后再反向运动。

使用 WAIT

编号	数据源	偏移	刻度
0	DPOS	0	500

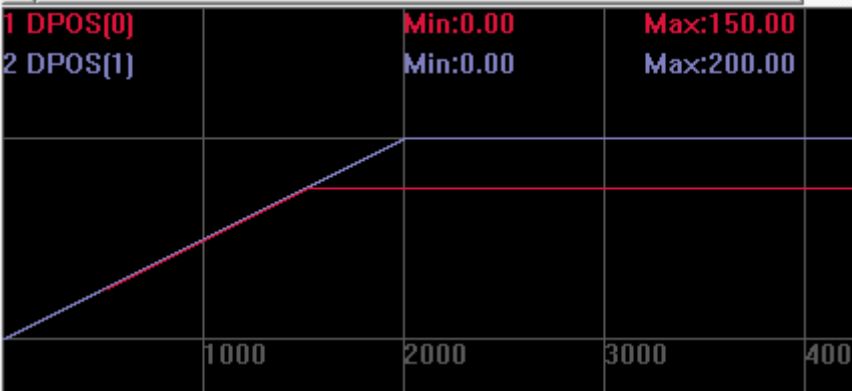


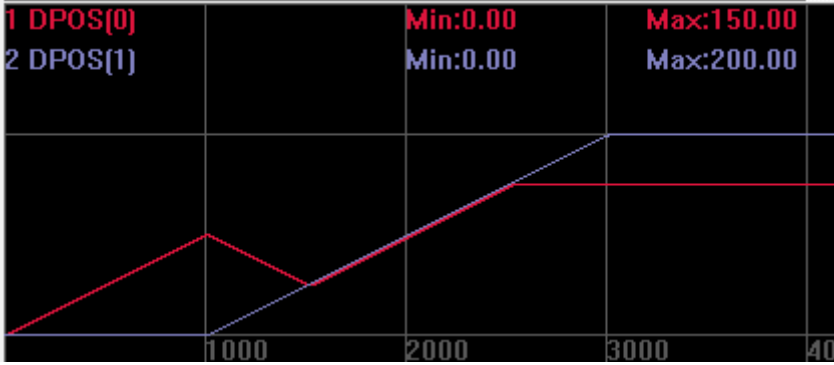
不使用 WAIT，直接运动到 100



相关指令	MOVEMODIFY2
------	-----------------------------

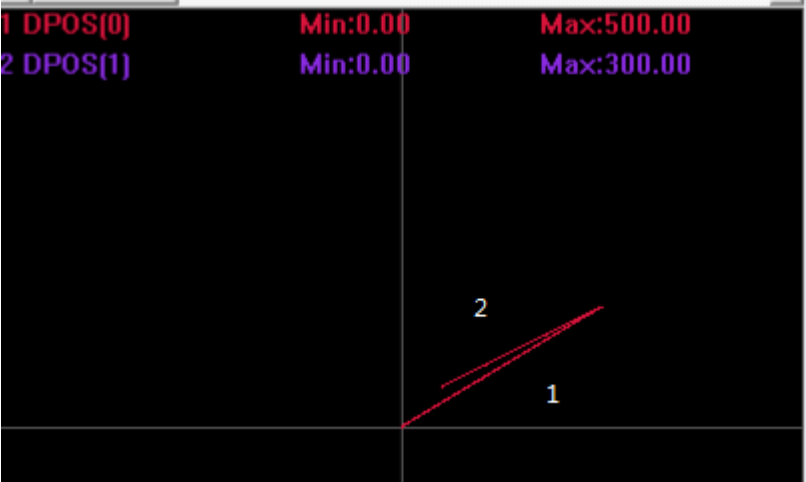
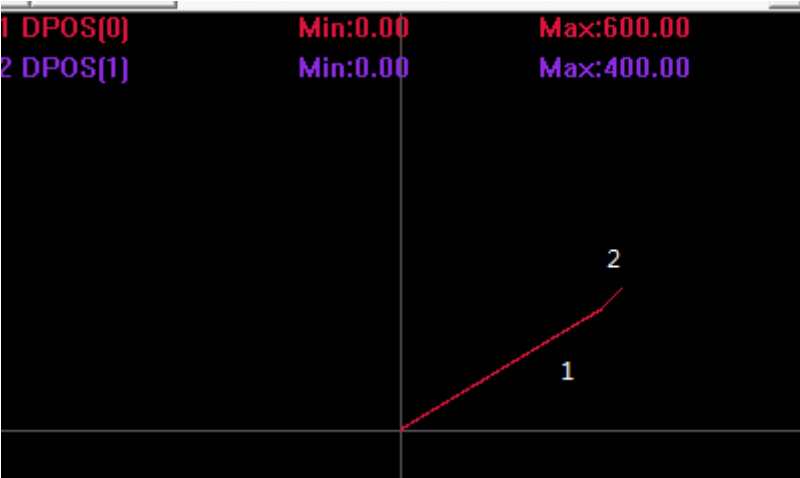
MOVEMODIFY2 -- 运动新位置

类型	运动指令
描述	强制停止原来的运动，接着原来的速度定位到新目标位置。 前面没有运动时与 MOVEABS 效果一样，但不会进入运动缓冲。见例一 需要 WAIT 指令才能正确使用。见例二 连续插补时使用 MOVEMODIFY2 会破坏速度连续性。 MOVEMODIFY2 同时对多轴运动时不一定为直线插补。
语法	MOVEMODIFY2 (abspos1, abspos2, [...]) abspos1 BASE 第一个轴的新目标位置 abspos2 BASE 第二个轴的新目标位置 3X 系列 20161209 以上固件支持。 4 系列 20170509 以上固件支持。
适用控制器	特殊固件
例子	base(0) dpos=0, 0 speed = 100, 100 例一 move(200) MOVEMODIFY2(50, 200) '取消 move200，强制定位新的位置 50, 200 move(100) 

	例二 <pre> move(200) wait until dpos(0)>=100 ' 等待轴 0 运行过 100 MOVEMODIFY2(50,200) move(100) </pre> 
相关指令	MOVEMODIFY

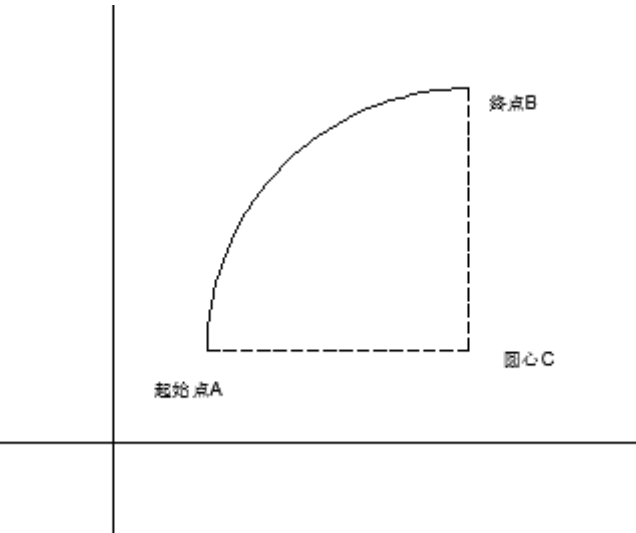
MOVEABS -- 直线运动-绝对

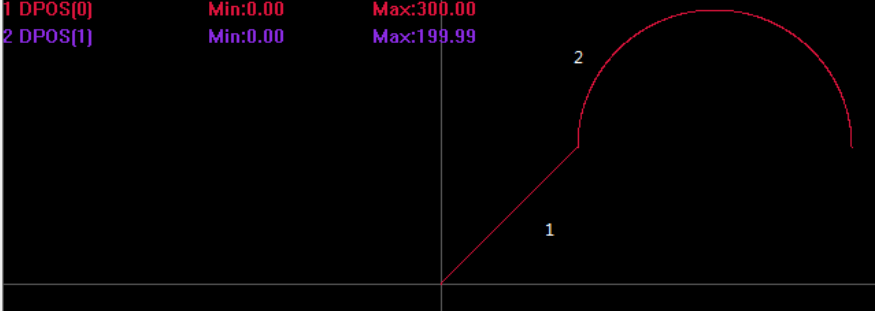
类型	多轴运动指令
描述	直线插补运动，绝对运动到指定坐标。 自定义速度的连续插补运动可以使用 SP 后缀的指令，见*SP 描述。
语法	MOVEABS(position1[, position2[, position3[, position4...]]]) 参数： position1 第一个轴运动坐标 position2 下一个轴运动坐标
适用控制器	通用
例子	<pre> BASE(0,1) UNITS=100,100 DPOS=0,0 SPEED=100,100 ACCEL=10000,10000 DECEL=10000,10000 TRIGGER ' 自动触发示波器 MOVEABS(500,300) ' 轴 0 运动到 500，轴 1 运动到 300，插补运动 MOVEABS(100,100) ' 轴 0 往回运动到 100，轴 1 往回运动到 100 </pre> 插补轨迹

	示波器 设置 启动 停止 水平刻度: 100 ☒ XY模式 手动触发 导入 导出 <table><tr><th>编号</th><th>数据源</th><th>偏移</th><th>刻度</th></tr><tr><td>0</td><td>DPOS</td><td>0</td><td>300</td></tr><tr><td>1</td><td>DPOS</td><td>0</td><td>300</td></tr></table> 1 DPOS[0] Min:0.00 Max:500.00 2 DPOS[1] Min:0.00 Max:300.00 	编号	数据源	偏移	刻度	0	DPOS	0	300	1	DPOS	0	300
	编号	数据源	偏移	刻度									
	0	DPOS	0	300									
1	DPOS	0	300										
如果使用 MOVE 相对运动 插补轨迹 1 DPOS[0] Min:0.00 Max:600.00 2 DPOS[1] Min:0.00 Max:400.00 													
相关指令 MOVE, *SP													

MOVECIRC -- 圆心画弧

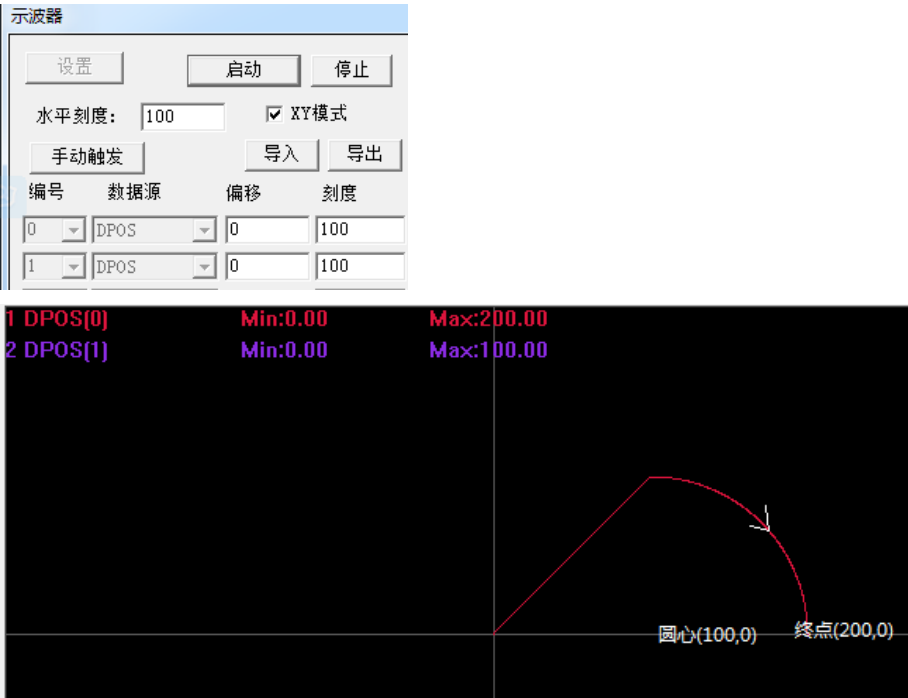
类型	多轴运动指令
描述	两轴圆弧插补，圆心画弧，相对运动。

	BASE 第一轴和第二轴进行圆弧插补，相对移动方式，当终点距离为 0 时为整圆。 自定义速度的连续插补运动可以使用 SP 后缀的指令，见*SP 描述。 使用时需获得圆心和圆弧终点相对于起始点坐标。 坐标位置请确保正确，否则实际运动轨迹会错误。  假设起始点 A 坐标为 (100,100)，圆心 C 坐标为 (400,100)，终点 B 坐标为 (400,400)。 圆心 C 相对于起始点 A 的坐标为 (300,0)，终点 B 相对于起始点 A 的坐标为 (300,300)。
语法	MOVECIRC(end1, end2, centre1, centre2, direction) 参数： - end1 终点第一个轴运动坐标，相对于起始点。 - end2 终点第二个轴运动坐标，相对于起始点。 - centre1 圆心第一个轴运动坐标，相对于起始点。 - centre2 圆心第二个轴运动坐标，相对于起始点。 - direction 0-逆时针，1-顺时针
适用控制器	通用
例子	BASE (0, 1) UNITS=100, 100 DPOS=0, 0 SPEED=100, 100 ACCEL=10000, 10000 DECEL=10000, 10000 TRIGGER '自动触发示波器 MOVE (100, 100) '先运动 100, 100 位置 MOVECIRC (200, 0, 100, 0, 1) '半径 100 顺时针画半圆，终点坐标 (300, 100) 插补轨迹

	示波器 设置 启动 停止 水平刻度: 100 ☒ XY模式 手动触发 导入 导出 编号 数据源 偏移 刻度 0 DPOS 0 150 1 DPOS 0 150 1 DPOS[0] Min:0.00 Max:300.00 2 DPOS[1] Min:0.00 Max:199.99  MOVECIRC (0, 0, 100, 0, 0) ' 半径 100 逆时针画圆 插补轨迹 1 DPOS[0] Min:0.00 Max:200.00 2 DPOS[1] Min:99.99 Max:99.99  相关指令 MOVECIRCABS, MOVECIRC2, *SP
--	--

MOVECIRCABS -- 圆心画弧-绝对

类型	多轴运动指令
描述	两轴圆弧插补，圆心画弧，绝对运动。 BASE 第一轴和第二轴进行圆弧插补，绝对移动方式。 无法画整圆，整圆建议使用 MOVECIRC 相对圆弧。 自定义速度的连续插补运动可以使用 SP 后缀的指令，见*SP 描述。
语法	MOVECIRCABS(end1, end2, centre1, centre2, direction)

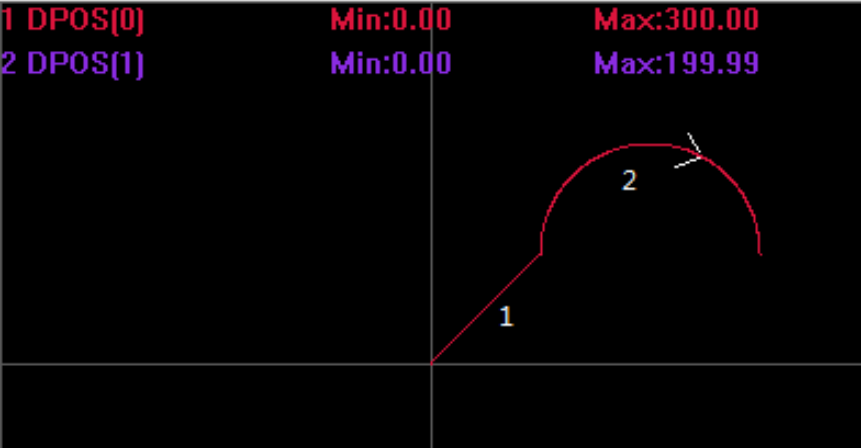
	参数： end1 终点第一个轴运动坐标，绝对位置 end2 终点第二个轴运动坐标，绝对位置 centre1 圆心第一个轴运动坐标，绝对位置 centre2 圆心第二个轴运动坐标，绝对位置 direction 0-逆时针，1-顺时针 坐标位置请确保正确，否则实际运动轨迹会错误。
适用控制器	通用
例子	BASE(0,1) UNITS=100,100 DPOS=0,0 SPEED=100,100 ACCEL=10000,10000 DECEL=10000,10000 TRIGGER '自动触发示波器 MOVE(100,100) '先运动 100,100 位置 MOVECIRCABS(200,0,100,0,1) '半径 100 顺时针画 1/4 圆，终点坐标 (200,0) 插补轨迹 
相关指令	MOVECIRC ， MOVECIRC2ABS ， *SP

MOVECIRC2 -- 三点画弧

类型	多轴运动指令
描述	两轴圆弧插补，三点画弧，相对运动。 BASE 第一轴和第二轴进行圆弧插补，相对移动方式。相对移动方式，相对起始点的距离。 自定义速度的连续插补运动可以使用 SP 后缀的指令，见*SP 描述。 注意：不能用此指令进行整圆插补运动，整圆使用 MOVECIRC 相对圆弧，或连续两条此类指令。
语法	MOVECIRC2(mid1, mid2, end1, end2) 参数： - mid1 中间点第一个轴坐标，相对起始点距离 - mid2 中间点第二个轴坐标，相对起始点距离 - end1 结束点第一个轴坐标，相对起始点距离 - end2 结束点第二个轴坐标，相对起始点距离 坐标位置请确保正确，否则实际运动轨迹会错误。
适用控制器	通用
例子	BASE (0, 1) UNITS=100, 100 DPOS=0, 0 SPEED=100, 100 ACCEL=10000, 10000 DECEL=10000, 10000 DPOS=0, 0 SPEED=100, 100 TRIGGER '自动触发示波器 MOVE (100, 100) '运动 100, 100 位置 MOVECIRC2 (100, 100, 200, 0) '三点画半圆，相对坐标 插补轨迹 

相关指令	MOVECIRC2ABS , MOVECIRC , *SP

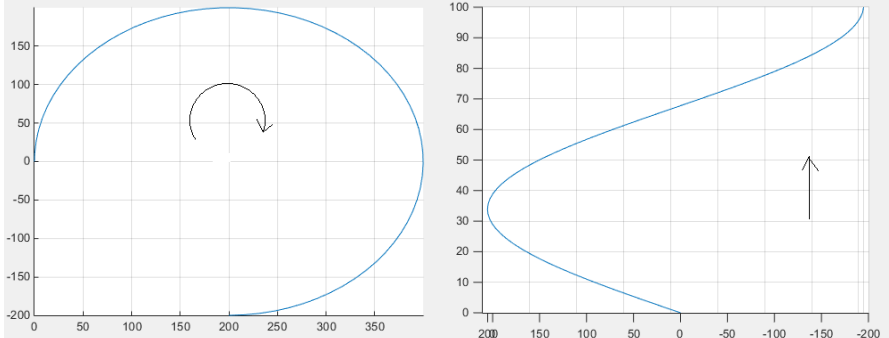
MOVECIRC2ABS -- 三点画弧-绝对

类型	多轴运动指令
描述	圆弧插补，三点画弧，绝对运动。 BASE 第一轴和第二轴进行圆弧插补，绝对移动方式。自定义速度的连续插补运动可以使用 SP 后缀的指令，见*SP 描述。 注意：不能用此指令进行整圆插补运动，整圆使用 MOVECIRC 相对圆弧，或连续两条此类指令。
语法	MOVECIRC2ABS(mid1, mid2, end1, end2) 参数： - mid1 中间点第一个轴坐标，绝对位置 - mid2 中间点第二个轴坐标，绝对位置 - end1 结束点第一个轴坐标，绝对位置 - end2 结束点第二个轴坐标，绝对位置 坐标位置请确保正确，否则实际运动轨迹会错误。
适用控制器	通用
例子	BASE (0, 1) UNITS=100, 100 DPOS=0, 0 SPEED=100, 100 ACCEL=10000, 10000 DECEL=10000, 10000 TRIGGER '自动触发示波器 MOVE (100, 100) '先运动 100, 100 位置 MOVECIRC2ABS (200, 200, 300, 100) '三点画半圆，绝对坐标插补轨迹 

相关指令	MOVECIRC2 , MOVECIRCABS , *SP

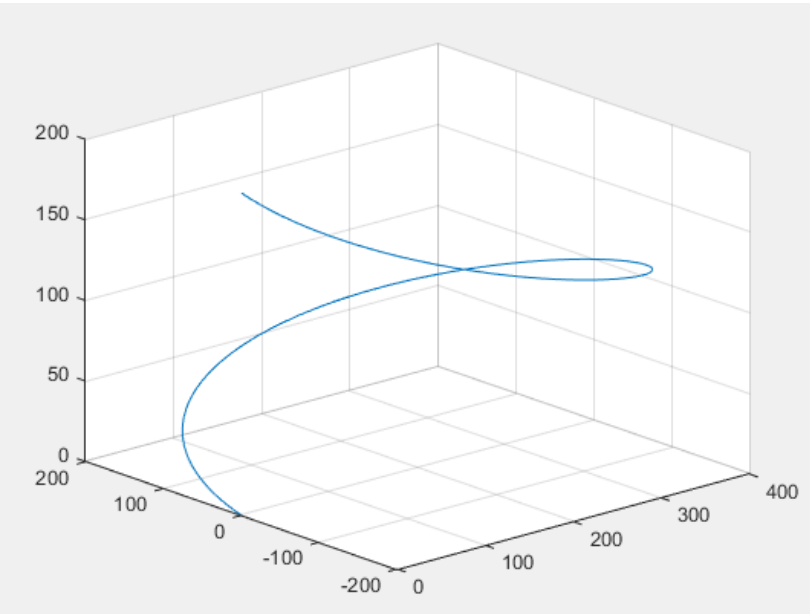
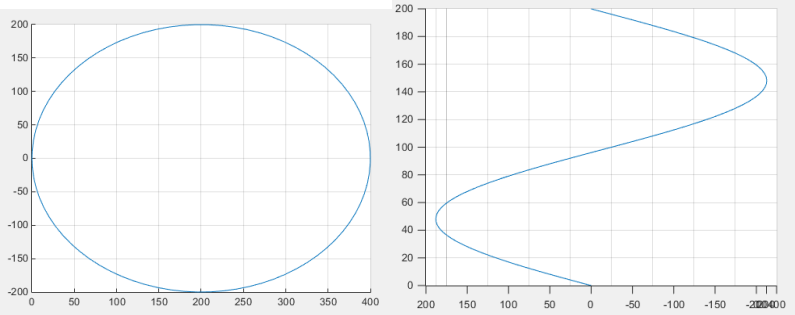
MHELICAL -- 圆心螺旋

类型	多轴运动指令						
描述	螺旋插补，圆心画弧，相对运动。 BASE 第一轴和第二轴进行圆弧插补，第三轴进行螺旋，相对于起始点。自定义速度的连续插补运动可以使用 SP 后缀的指令，见*SP 描述。 可完整螺旋一圈，从 Z 方向看为一整圆。						
语法	MHELICAL (end1, end2, centre1, centre2, direction, distance3, [mode]) 参数： - end1 结束点第一个轴运动坐标，相对于起始点 - end2 结束点第二个轴运动坐标，相对于起始点 - centre1 圆心第一个轴运动坐标，相对于起始点 - centre2 圆心第二个轴运动坐标，相对于起始点 - direction 0-逆时针，1-顺时针 - distance3 第三个轴运动距离。 - mode 第三轴的速度计算： <table border="1"> <tr> <th>值</th><th>描述</th></tr> <tr> <td>0 (缺省)</td><td>第三轴参与插补速度计算。</td></tr> <tr> <td>1</td><td>第三轴不参与插补速度计算。</td></tr> </table> 坐标位置请确保正确，否则实际运动轨迹会错误。	值	描述	0 (缺省)	第三轴参与插补速度计算。	1	第三轴不参与插补速度计算。
值	描述						
0 (缺省)	第三轴参与插补速度计算。						
1	第三轴不参与插补速度计算。						
适用控制器	通用						
例子	<pre> BASE (0, 1, 2) UNITS=100, 100, 100 DPOS=0, 0, 0 SPEED=100 ' 主轴速度 ACCEL=10000 ' 主轴加速度 DECEL=10000 MHELICAL (400, 0, 200, 0, 1, 100) ' 原点作为起点，中心 (200, 0)，终点 (200, -200)，顺时针，Z 轴参与速度计算，运动 100 </pre> 插补轨迹						

	
相关指令	MHELICAL2 , MHELICALABS , *SP

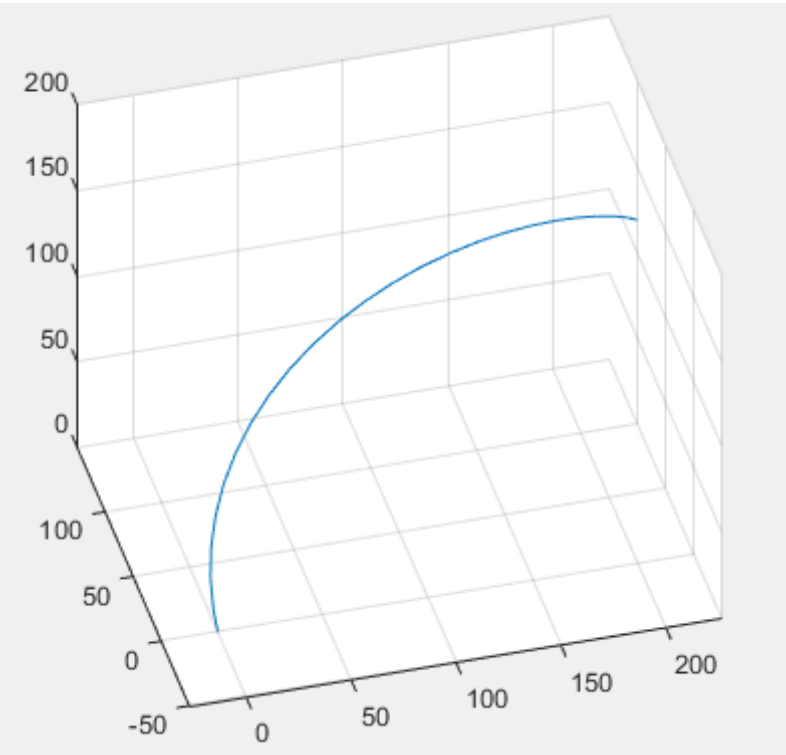
MHELICALABS -- 圆心螺旋-绝对

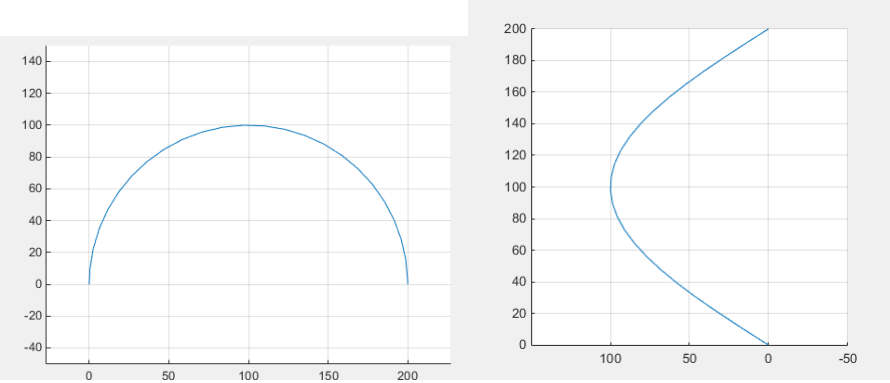
类型	多轴运动指令						
描述	螺旋插补，圆心画弧，绝对运动。 BASE 第一轴和第二轴进行圆弧插补，第三轴进行螺旋，绝对移动方式。 自定义速度的连续插补运动可以使用 SP 后缀的指令，见*SP 描述。 可完整螺旋一圈，从 Z 方向看为一整圆。						
语法	MHELICALABS (end1,end2,centre1,centre2,direction,distance3,[mode]) 参数： - end1 第一个轴运动坐标 - end2 第二个轴运动坐标 - centre1 第一个轴运动圆心 - centre2 第二个轴运动圆心 - direction 0-逆时针，1-顺时针 - distance3 第三个轴运动坐标 - mode 第三轴的速度计算： <table border="1"> <tr> <th>值</th><th>描述</th></tr> <tr> <td>0(缺省)</td><td>第三轴参与速度计算。</td></tr> <tr> <td>1</td><td>第三轴不参与速度计算。</td></tr> </table> 坐标位置请确保正确，否则实际运动轨迹会错误。	值	描述	0(缺省)	第三轴参与速度计算。	1	第三轴不参与速度计算。
值	描述						
0(缺省)	第三轴参与速度计算。						
1	第三轴不参与速度计算。						
适用控制器	通用						
例子	<pre> BASE(0,1,2) UNITS=100,100,100 DPOS=0,0,0 SPEED=100 ' 主轴速度 ACCEL=10000 ' 主轴加速度 DECEL=10000 MHELICALABS(0,0,200,0,1,100) ' 起点原点，中心(200,0)，终点(0,0)， </pre>						

	顺时针，Z 轴参与速度计算，运动 200 插补轨迹  
相关指令	MHELICAL , MHELICAL2ABS , *SP

MHELICAL2 -- 三点螺旋

类型	多轴运动指令
描述	螺旋插补，三点画弧，相对运动。 BASE 第一轴和第二轴进行圆弧插补，第三轴进行螺旋，相对移动方式。 自定义速度的连续插补运动可以使用 SP 后缀的指令，见*SP 描述。 无法螺旋一圈，需要螺旋一圈请使用 MHELICAL 或 MHELICALABS。
语法	MHELICAL2(mid1, mid2, end1, end2, distance3, [mode]) 参数： - mid1 中间点第一个轴坐标，相对起始点距离 - mid2 中间点第二个轴坐标，相对起始点距离 - end1 结束点第一个轴坐标，相对起始点距离 - end2 结束点第二个轴坐标，相对起始点距离

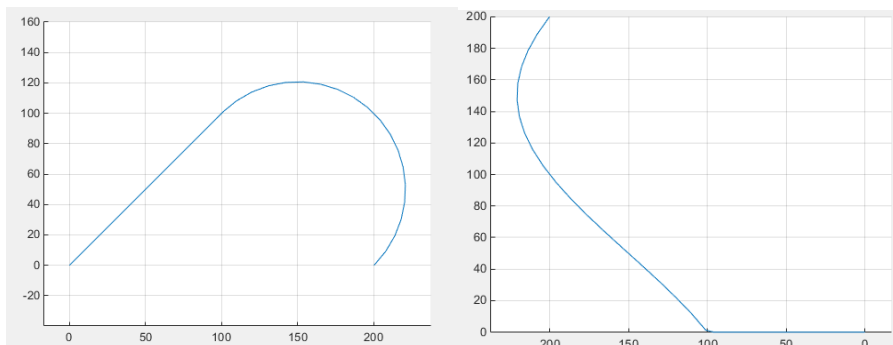
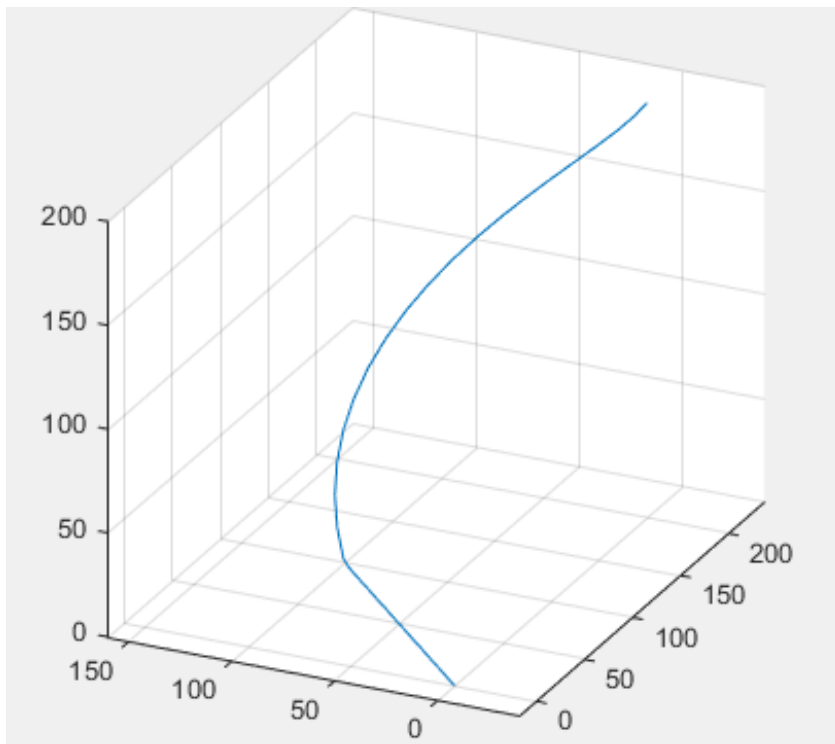
	distance3 第三个轴运动距离，相对起始点距离 mode 第三轴的速度计算： <table border="1"> <tr> <th>值</th><th>描述</th></tr> <tr> <td>0(缺省)</td><td>第三轴参与速度计算。</td></tr> <tr> <td>1</td><td>第三轴不参与速度计算。</td></tr> </table> 坐标位置请确保正确，否则实际运动轨迹会错误。	值	描述	0(缺省)	第三轴参与速度计算。	1	第三轴不参与速度计算。
值	描述						
0(缺省)	第三轴参与速度计算。						
1	第三轴不参与速度计算。						
适用控制器	通用						
例子	BASE(0, 1, 2) UNITS=100, 100, 100 DPOS=0, 0, 0 SPEED=100 ' 主轴速度 ACCEL=10000 ' 主轴加速度 DECEL=10000 MHELICAL2(100, 100, 200, 0, 200) ' 起点原点，中间点(100, 100)，终点(200, 0)，Z 轴参与速度计算，运动 200 插补轨迹 						

	
相关指令	MHELICAL2ABS , MHELICAL , *SP

MHELICAL2ABS -- 三点螺旋-绝对

类型	多轴运动指令						
描述	螺旋插补，三点画弧，绝对运动。 BASE 第一轴和第二轴进行圆弧插补，第三轴进行螺旋，绝对移动方式。 自定义速度的连续插补运动可以使用 SP 后缀的指令，见*SP 描述。 无法螺旋一圈，需要螺旋一圈请使用 MHELICAL 或 MHELICALABS。						
语法	MHELICAL2ABS(mid1, mid2, end1, end2, distance3, [mode]) 参数： - mid1 中间点第一个轴坐标。 - mid2 中间点第二个轴坐标。 - end1 结束点第一个轴坐标。 - end2 结束点第二个轴坐标。 - distance3 第三个轴结束点坐标，勘误：20150306 以前的版本此参数有问题，建议使用 MHELICAL2 相对指令。 - mode 第三轴的速度计算： <table border="1"> <tr> <th>值</th><th>描述</th></tr> <tr> <td>0(缺省)</td><td>第三轴参与速度计算。</td></tr> <tr> <td>1</td><td>第三轴不参与速度计算。</td></tr> </table> 坐标位置请确保正确，否则实际运动轨迹会错误。	值	描述	0(缺省)	第三轴参与速度计算。	1	第三轴不参与速度计算。
值	描述						
0(缺省)	第三轴参与速度计算。						
1	第三轴不参与速度计算。						
适用控制器	通用						
例子	<pre>BASE(0, 1, 2) UNITS=100, 100, 100 DPOS=0, 0, 0 SPEED=100 ' 主轴速度</pre>						

ACCEL=10000 ' 主轴加速度
 DECEL=10000
 MOVE(100,100) ' 运动到 (100,100)
MHELICAL2ABS(200,100,200,0,200) ' 起点 (100,100) ， 中间点 (200,100)， 结束点 (200,0)， Z 轴参与速度计算， 运动 200 插补轨迹



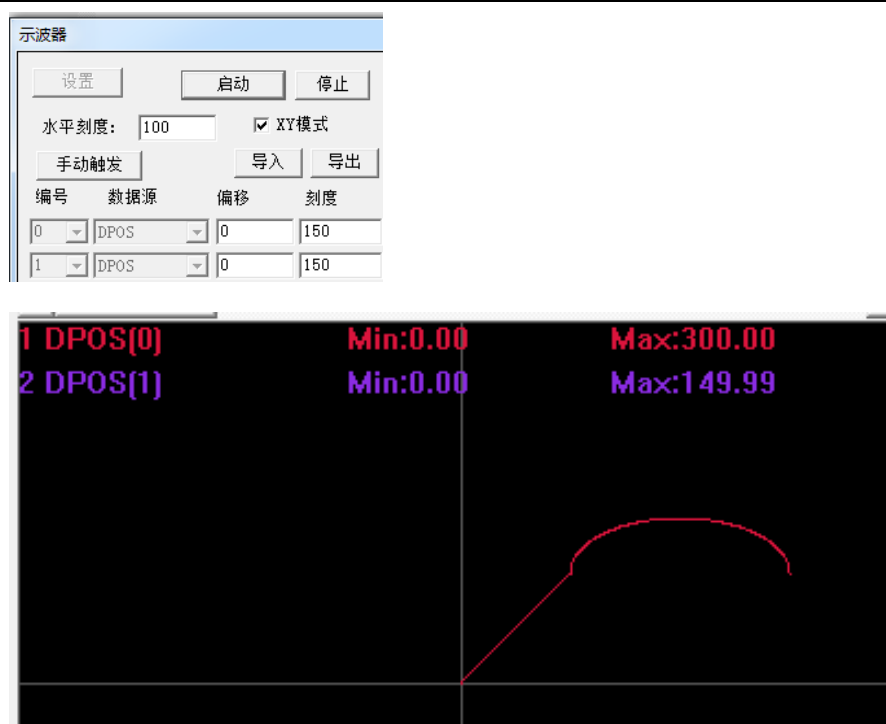
相关指令

[MHELICAL2](#)，[MHELICALABS](#)，[*SP](#)

MECLIPSE -- 椭圆

类型	多轴运动指令
描述	椭圆插补，中心画弧，相对运动，可选螺旋。

	BASE 第一轴和第二轴进行椭圆插补，相对移动方式，可选第三个轴同步螺旋。 自定义速度的连续插补运动可以使用 SP 后缀的指令，见*SP 描述。 可画整椭圆。 只能画长轴（短轴）与 X 平行或垂直的椭圆。						
语法	MECLIPSE (end1, end2, centre1, centre2, direction, adis, bdis[, end3]) 参数： end1 终点第一个轴运动坐标，相对于起始点。 end2 终点第二个轴运动坐标，相对于起始点。 centre1 中心第一个轴运动坐标，相对于起始点。 centre2 中心第二个轴运动坐标，相对于起始点。 direction 0-逆时针，1-顺时针 <table border="1"> <thead> <tr> <th>值</th><th>描述</th></tr> </thead> <tbody> <tr> <td>0</td><td>逆时针</td></tr> <tr> <td>1</td><td>顺时针</td></tr> </tbody> </table> adis 第一轴的椭圆半径，半长轴或者半短轴都可。 bdis 第二轴的椭圆半径，半长轴或者半短轴都可，AB 相等时自动为圆弧或螺旋。 end3 第三个轴的运动距离，需要螺旋时填入。 坐标位置请确保正确，否则实际运动轨迹会错误。	值	描述	0	逆时针	1	顺时针
值	描述						
0	逆时针						
1	顺时针						
适用控制器	通用						
例子	例一 不进行螺旋 BASE (0, 1, 2) UNITS=100, 100, 100 DPOS=0, 0, 0 SPEED=100 ' 主轴速度 ACCEL=10000 ' 主轴加速度 DECEL=10000 TRIGGER ' 自动触发示波器 MOVE (100, 100) MECLIPSE (200, 0, 100, 0, 1, 100, 50) ' 中心 (200, 100)，终点 (300, 100)，半短轴 50，半长轴 100，画半椭圆圆弧，不进行螺旋。 插补轨迹						



例二 进行螺旋

BASE (0, 1, 2)

UNITS=100, 100, 100

DPOS=0, 0, 0

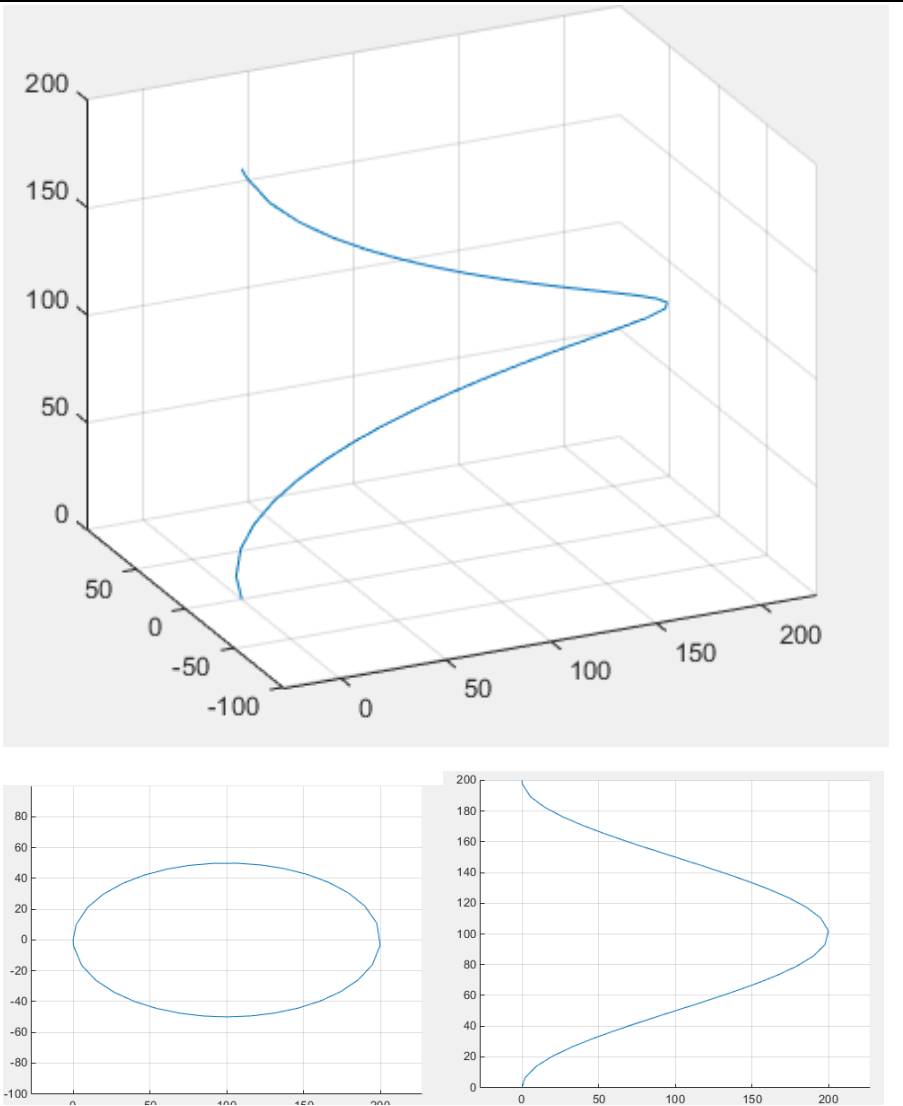
SPEED=100 ' 主轴速度

ACCEL=10000 ' 主轴加速度

DECEL=10000

MECLIPSE (0, 0, 100, 0, 1, 100, 50, 200) ' 中心 (100, 0)， 终点 (0, 0)， 半短轴 50， 半长轴 100， 画整椭圆， 同时螺旋。

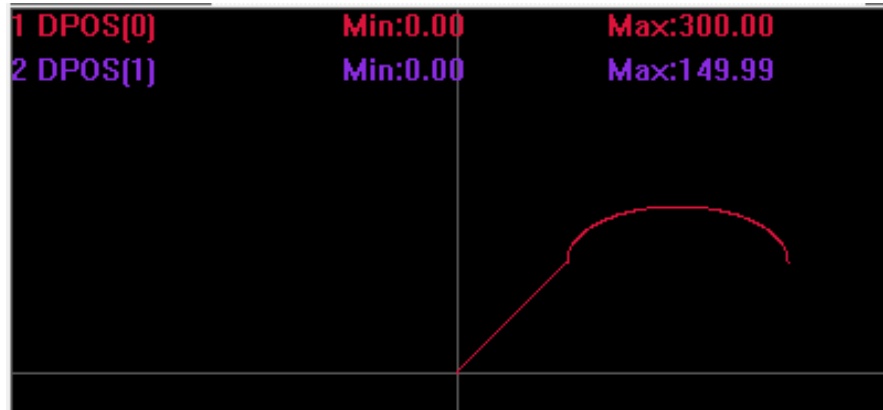
插补轨迹

	
相关指令	MECLIPSEABS , *SP

MECLIPSEABS —— 椭圆-绝对

类型	多轴运动指令
描述	椭圆插补，中心画弧，绝对运动，可选螺旋。 BASE 第一轴和第二轴进行椭圆插补，绝对移动方式，可选第三个轴同步螺旋。 自定义速度的连续插补运动可以使用 SP 后缀的指令，见*SP 描述。 可画整椭圆。
语法	MECLIPSEABS(end1, end2, centre1, centre2, direction, adis, bdis[, end3])

	参数: end1 终点第一个轴运动坐标 end2 终点第二个轴运动坐标 centre1 中心第一个轴运动坐标。 centre2 中心第二个轴运动坐标。 direction 0-逆时针, 1-顺时针 <table border="1"> <tr> <th>值</th><th>描述</th></tr> <tr> <td>0</td><td>逆时针</td></tr> <tr> <td>1</td><td>顺时针</td></tr> </table> adis 第一轴的椭圆半径, 半长轴或者半短轴都可。 bdis 第二轴的椭圆半径, 半长轴或者半短轴都可, AB 相等时自动为圆弧或螺旋。 end3 第三个轴的运动坐标, 需要螺旋时填入。 坐标位置请确保正确, 否则实际运动轨迹会错误。	值	描述	0	逆时针	1	顺时针
值	描述						
0	逆时针						
1	顺时针						
适用控制器	通用						
例子	例一 不进行螺旋 BASE (0, 1, 2) UNITS=100, 100, 100 DPOS=0, 0, 0 SPEED=100 ' 主轴速度 ACCEL=10000 ' 主轴加速度 DECEL=10000 TRIGGER ' 自动触发示波器 MOVE (100, 100) MECLIPSEABS (300, 100, 200, 100, 1, 100, 50) ' 中心 (200, 100), 终点 (300, 100), 半短轴 50, 半长轴 100, 画半椭圆圆弧, 不进行螺旋。 插补轨迹  示波器界面截图显示：设置、启动、停止按钮；水平刻度：100，XY模式复选框；手动触发、导入、导出按钮；下方表格显示编号0和1，数据源均为DPOS，偏移均为0，刻度均为150。						



例二 进行螺旋

BASE(0, 1, 2)

UNITS=100, 100, 100

DPOS=0, 0, 0

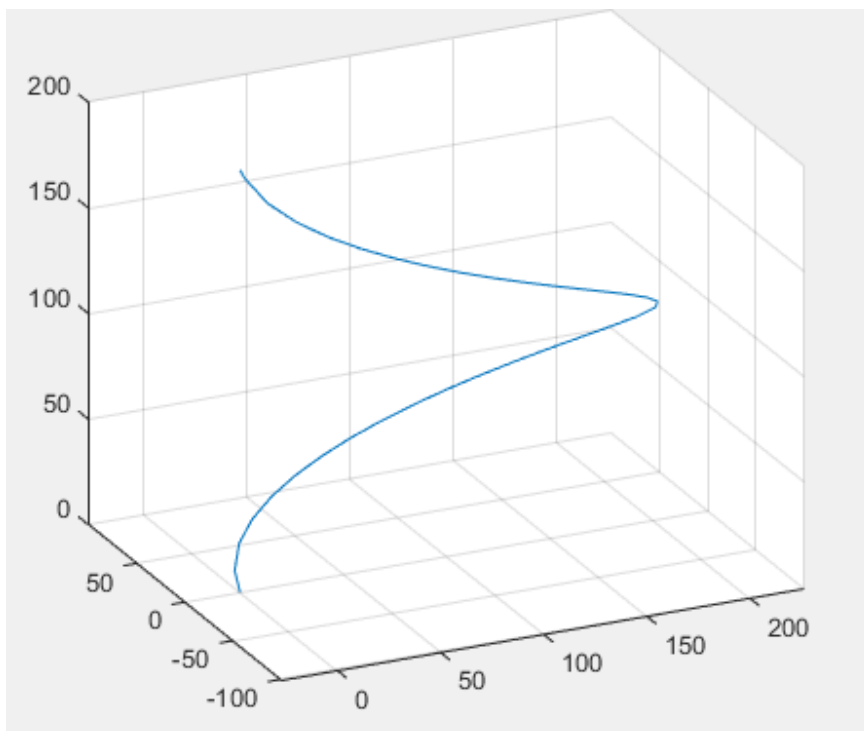
SPEED=100 ' 主轴速度

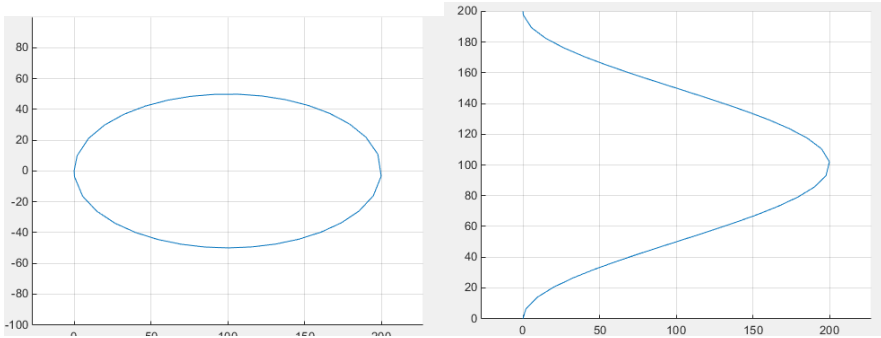
ACCEL=10000 ' 主轴加速度

DECEL=10000

MECLIPSEABS(0, 0, 100, 0, 1, 100, 50, 200) ' 中心(100, 0)，终点(0, 0)，半短轴 50，半长轴 100，画整椭圆，同时螺旋。

插补轨迹



	
相关指令	MECLIPSE , *SP

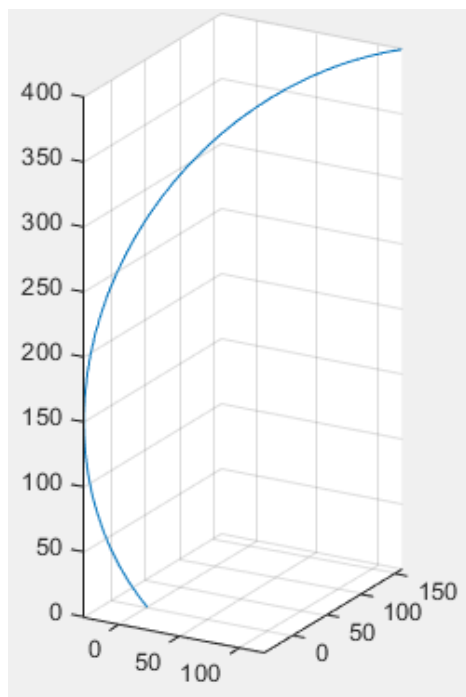
MSPHERICAL -- 空间圆弧

类型	多轴运动指令										
描述	空间圆弧插补运动，相对移动方式，可选螺旋。 自定义速度的连续插补运动可以使用 SP 后缀的指令，见*SP 描述。										
语法	MSPHERICAL (end1, end2, end3, centre1, centre2, centre3, mode[, distance4] [, distance5]) 参数： - end1 第 1 个轴运动距离参数 1 - end2 第 2 个轴运动距离参数 1 - end3 第 3 个轴运动距离参数 1 - centre1 第 1 个轴运动距离参数 2 - centre2 第 2 个轴运动距离参数 2 - centre3 第 3 个轴运动距离参数 2 - mode 指定前面参数的意义 <table border="1" data-bbox="459 1525 1252 2004"> <thead> <tr> <th>值</th><th>描述</th></tr> </thead> <tbody> <tr> <td>0</td><td>当前点，中间点，终点三点定圆弧。 距离参数 1 为终点距离，距离参数 2 为中间点距离。</td></tr> <tr> <td>1</td><td>当前点，圆心，终点定圆弧。 走最短的圆弧。 距离参数 1 为终点距离，距离参数 2 为圆心的距离。</td></tr> <tr> <td>2</td><td>当前点，中间点，终点三点定圆。 距离参数 1 为终点距离，距离参数 2 为中间点距离。</td></tr> <tr> <td>3</td><td>当前点，圆心，终点定圆。 先走最短的圆弧，再继续走完整圆。 距离参数 1 为终点距离，距离参数 2 为圆心的距离。</td></tr> </tbody> </table>	值	描述	0	当前点，中间点，终点三点定圆弧。 距离参数 1 为终点距离，距离参数 2 为中间点距离。	1	当前点，圆心，终点定圆弧。 走最短的圆弧。 距离参数 1 为终点距离，距离参数 2 为圆心的距离。	2	当前点，中间点，终点三点定圆。 距离参数 1 为终点距离，距离参数 2 为中间点距离。	3	当前点，圆心，终点定圆。 先走最短的圆弧，再继续走完整圆。 距离参数 1 为终点距离，距离参数 2 为圆心的距离。
值	描述										
0	当前点，中间点，终点三点定圆弧。 距离参数 1 为终点距离，距离参数 2 为中间点距离。										
1	当前点，圆心，终点定圆弧。 走最短的圆弧。 距离参数 1 为终点距离，距离参数 2 为圆心的距离。										
2	当前点，中间点，终点三点定圆。 距离参数 1 为终点距离，距离参数 2 为中间点距离。										
3	当前点，圆心，终点定圆。 先走最短的圆弧，再继续走完整圆。 距离参数 1 为终点距离，距离参数 2 为圆心的距离。										

	Distane4 第四轴螺旋的功能，指定第 4 轴的相对距离，此轴不参与速度计算。 Distane5 第五轴螺旋的功能，指定第 5 轴的相对距离，此轴不参与速度计算。 坐标位置请确保正确，否则实际运动轨迹会错误。
适用控制器	通用
例子	BASE (0, 1, 2) UNITS=100, 100, 100 DPOS=0, 0, 0 SPEED=100 ’ 主轴速度 ACCEL=10000 ’ 主轴加速度 DECEL=10000 使用圆心为 (120, 160, 150)，半径 250 的圆演示 4 种 mode 的运动轨迹 mode 0 MSPHERICAL (120, 160, 400, 240, 320, 300, 0) ’ 终点 (120, 160, 400)，中间点 (240, 320, 300)，模式 0，三点画弧插补轨迹

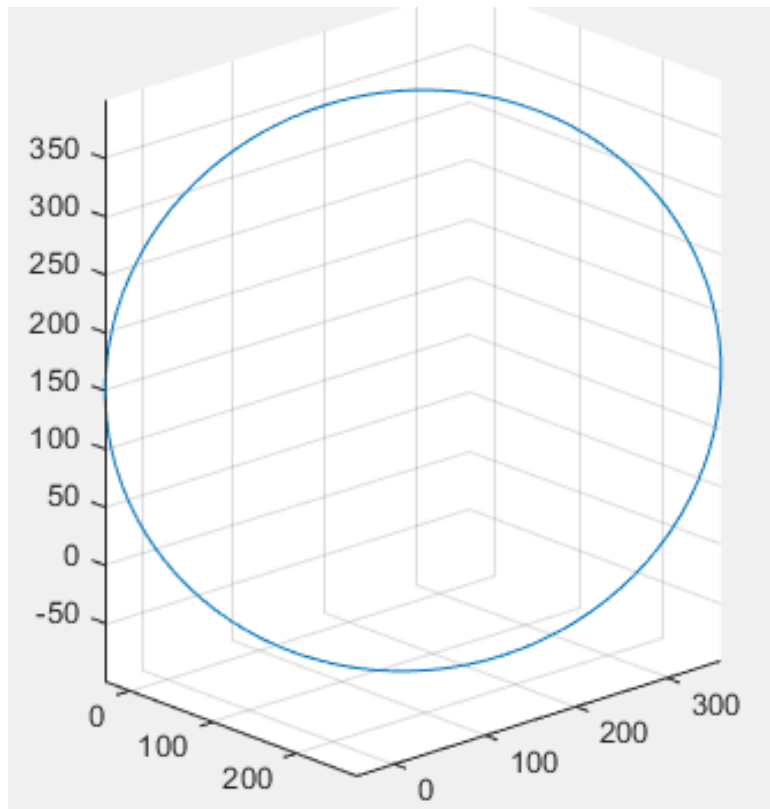
mode 1

MSPHERICAL (120, 160, 400, 120, 160, 150, 1) ' 终点 (120, 160, 400) , 圆心 (120, 160, 150), 模式 1, 走最短的圆弧
插补轨迹



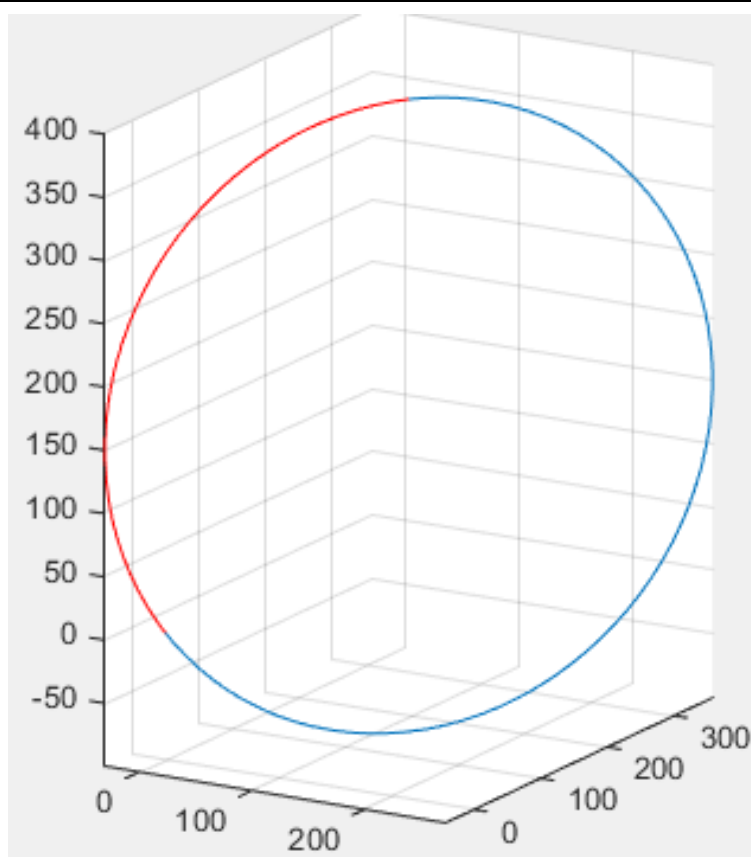
mode 2

MSPHERICAL (120, 160, 400, 240, 320, 300, 2) ' 终点 (120, 160, 400) , 中间点 (240, 320, 300), 模式 2, 三点画圆
插补轨迹



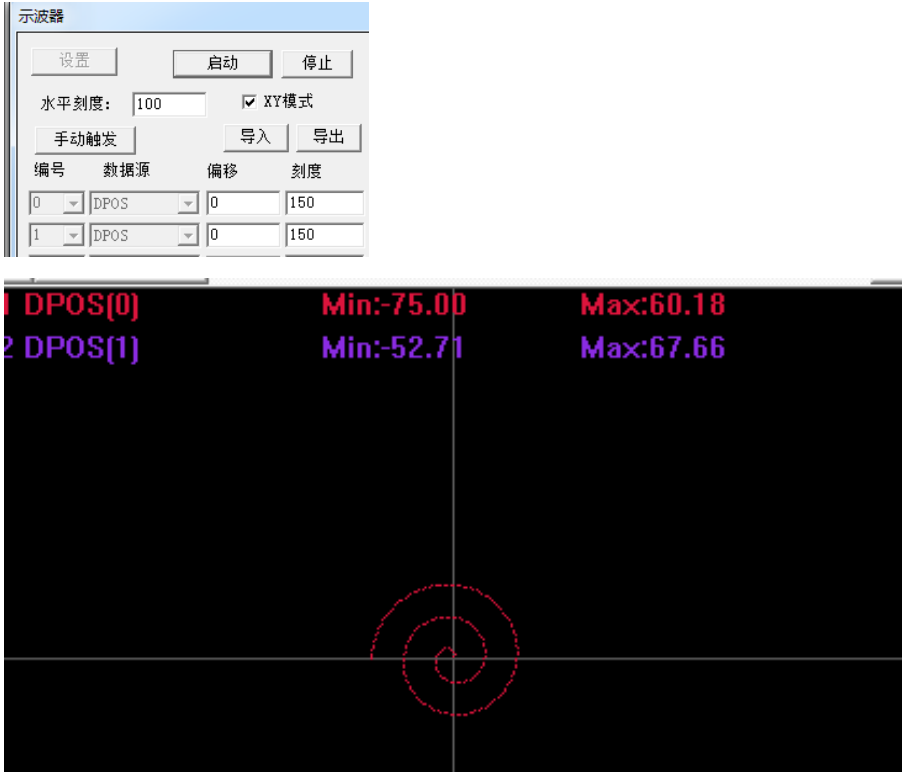
mode 3

MSPHERICAL (120, 160, 400, 120, 160, 150, 3) ' 终点 (120, 160, 400) , 圆 心 (120, 160, 150), 模式 3, 先走最短的圆弧(红色部分), 再走完整圆插补轨迹



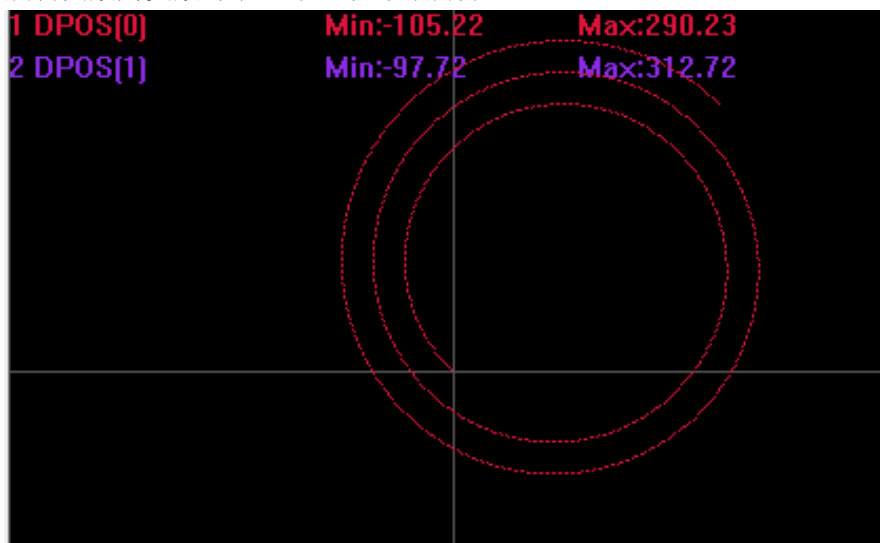
MOVESPIRAL -- 渐开线圆弧

类型	多轴运动指令
描述	渐开线圆弧插补运动，相对移动方式，可选螺旋。 当前点和圆心距离确定起始半径，当起始半径 0 时角度无法确定，直接从 0 角度开始。见例一 自定义速度的连续插补运动可以使用 SP 后缀的指令，见*SP 描述。
语法	MOVESPIRAL (centre1, centre2, circles, pitch [, distance3] [, distance4]) 参数： - centre1: 圆心的第 1 轴相对距离 - centre2: 圆心的第 2 轴相对距离 - circles: 要旋转的圈数，可以为小数圈，负数表示顺时针，每圈终点位置为起点和圆心连线上的一点，见例二 - pitch: 每圈的扩散距离，可以为负。 - distance3 第 3 轴螺旋的功能，指定第 3 轴的相对距离，此轴不参与速度计算。 - distance4 第 4 轴螺旋的功能，指定第 4 轴的相对距离，此轴不参与速度计算。

适用控制器	通用
例子	BASE (0, 1, 2) UNITS=100, 100, 100 DPOS=0, 0, 0 SPEED=100 ' 主轴速度 ACCEL=10000 ' 主轴加速度 DECEL=10000 TRIGGER ' 自动触发示波器 例一 从起点扩散 MOVESPIRAL (0, 0, 2.5, 30) ' 此时以起始位置为中心，逆时针旋转 2.5 圈，每圈扩散 30。 插补轨迹  例二 不螺旋 MOVESPIRAL (100, 100, 2.5, 30) ' 起始半径 100，以(100,100)为圆心，逆时针旋转 2.5 圈，每圈向外扩散 30。 插补轨迹



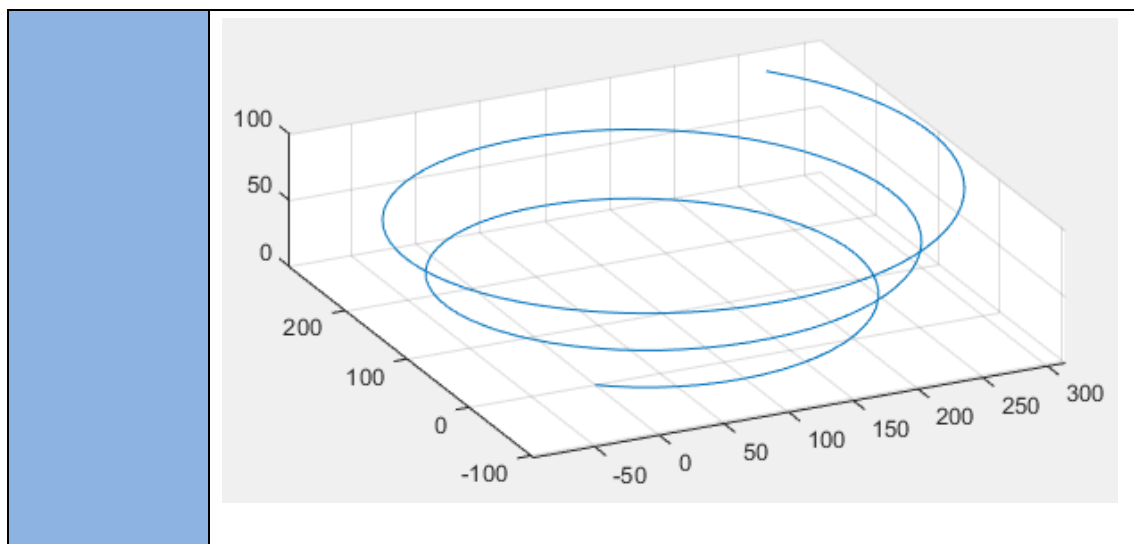
旋转圈数为负数时 (-2.5)，顺时针旋转



例三 螺旋

MOVESPIRAL (100, 100, 2.5, 30, 100) ' 起始半径 100，以 (100, 100) 为圆心，逆时针旋转 2.5 圈，每圈向外扩散 30，同时 Z 轴向上运动到 100。

插补轨迹



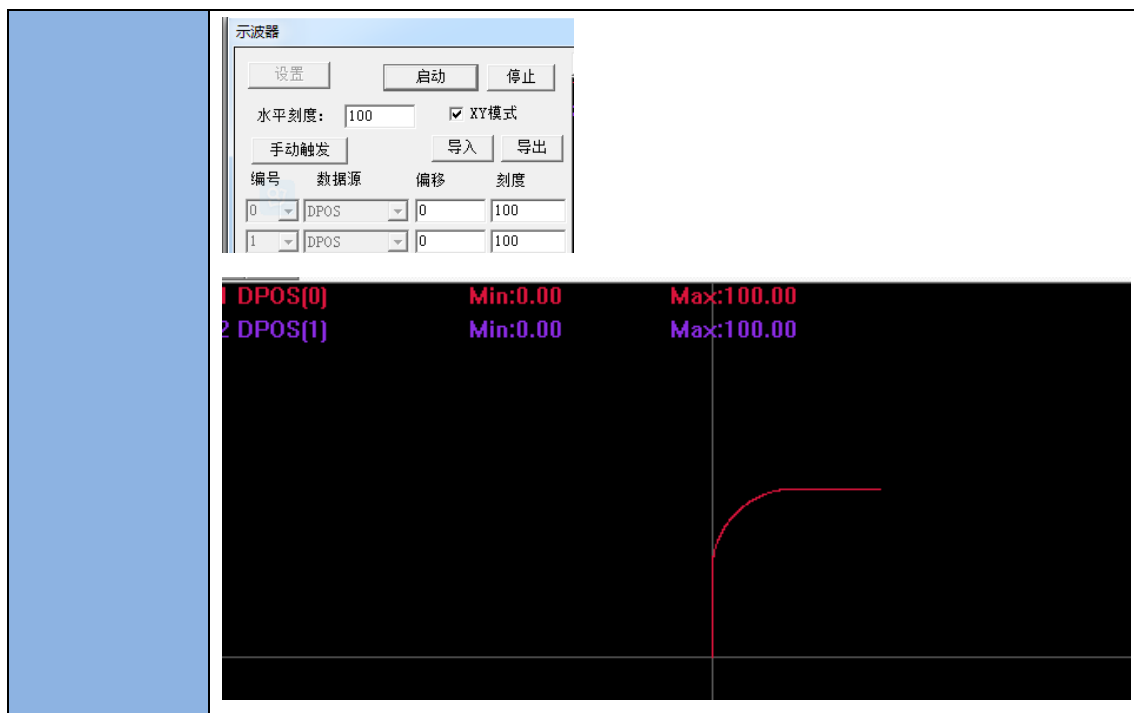
MOVESPLINE/MOVESPLINEABS -- 样条插补

类型	特殊运动指令										
描述	样条插补运动，相对/绝对运动。 样条运动的控制点位置提前写入 TABLE 数组。 此运动没有 SP 指令，自定义速度的连续插补运动通过 CORNER_MODE 的 BIT8 来设置。										
语法	MOVESPLINE (axes,mode ,dtendcontrol4, dtcontrol2, dtcontrol3) 参数： <table border="0"> <tr> <td>axes</td><td>插补的轴个数</td></tr> <tr> <td>mode</td><td>模式，0 - 使用 3 阶贝塞尔样条.</td></tr> <tr> <td>dtendcontrol4</td><td>第 4 个控制点存储 table 索引，对贝塞尔曲线就是终点。</td></tr> <tr> <td>dtcontrol2</td><td>第 2 个控制点存储 table 索引</td></tr> <tr> <td>dtcontrol3</td><td>第 3 个控制点存储 table 索引</td></tr> </table> 对贝塞尔曲线，第 1 个控制点就是当前位置。	axes	插补的轴个数	mode	模式，0 - 使用 3 阶贝塞尔样条.	dtendcontrol4	第 4 个控制点存储 table 索引，对贝塞尔曲线就是终点。	dtcontrol2	第 2 个控制点存储 table 索引	dtcontrol3	第 3 个控制点存储 table 索引
axes	插补的轴个数										
mode	模式，0 - 使用 3 阶贝塞尔样条.										
dtendcontrol4	第 4 个控制点存储 table 索引，对贝塞尔曲线就是终点。										
dtcontrol2	第 2 个控制点存储 table 索引										
dtcontrol3	第 3 个控制点存储 table 索引										
适用控制器	4 系列 170507 以上固件支持 306x 161208 以上固件支持										
例子	<pre>BASE(0,1) trigger CORNER_MODE=2 + 256 '设置 SP 运动，使用 FORCE_SPEED FORCE_SPEED=100 table(0, 100, 100) 'table(0)(1)存储第二个控制点，相对起点距离 table(10, 200, -100) 'table(10)(11)存储第三个控制点，相对起点距</pre>										

	离 table(20, 300, 0) ' table(20) (21) 存储第四个控制点, 终点距离 MOVESPLINE(2, 0, 20, 0, 10) ' 2 轴相对样条插补
相关指令	CORNER_MODE

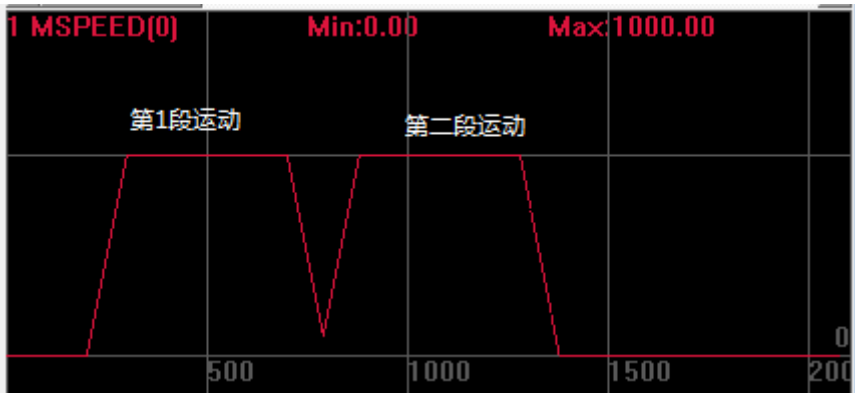
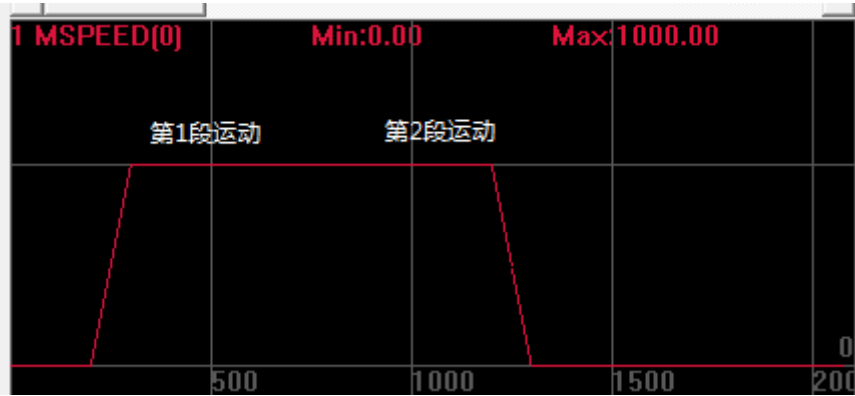
MOVESMOOTH -- 倒圆角

类型	多轴运动指令 此指令为早期开发指令，有轴数限制，建议使用 CORNER_MODE 指令，功能更多。
描述	空间直线倒圆运动。 根据下一个直线运动的绝对坐标在拐角自动插入圆弧，加入圆弧后会使得运动的终点与直线的终点不一致，拐角过大时不会插入圆弧，当距离不够时会自动减小半径。 自定义速度的连续插补运动可以使用 SP 后缀的指令，见*SP 描述。
语法	MOVESMOOTH (end1, end2, end3, next1, next2, next3, radius) 参数： end1 第 1 个轴运动绝对坐标 end2 第 2 个轴运动绝对坐标 end3 第 3 个轴运动绝对坐标 next1 第 1 个轴下一个直线运动绝对坐标 next2 第 2 个轴下一个直线运动绝对坐标 next3 第 3 个轴下一个直线运动绝对坐标 radius 插入圆弧的半径，当过大的时候自动缩小。
适用控制器	通用
例子	BASE(0, 1, 2) UNITS=100, 100, 100 DPOS=0, 0, 0 SPEED=100 ' 主轴速度 ACCEL=10000 ' 主轴加速度 DECEL=10000 TRIGGER ' 自动触发示波器 MOVESMOOTH (0, 100, 0, 100, 100, 0, 50) ' 加入圆弧后，实际运动到了 (50, 100, 0) MOVEABS(100, 100, 0) 插补轨迹



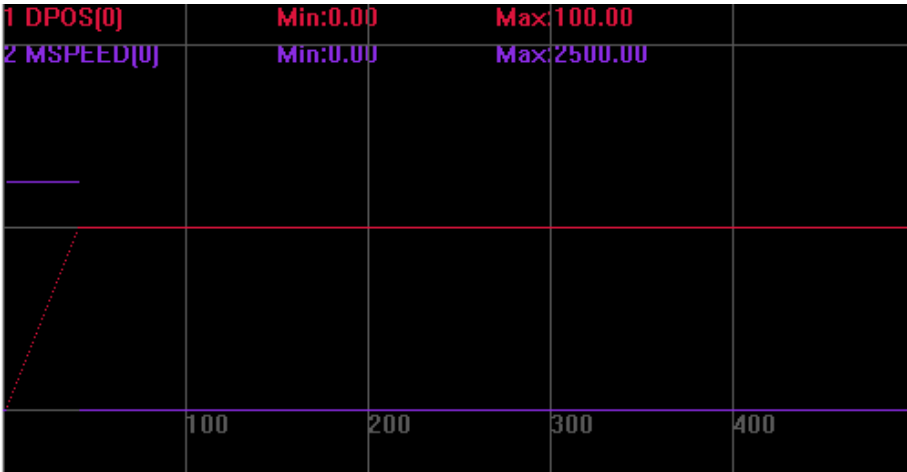
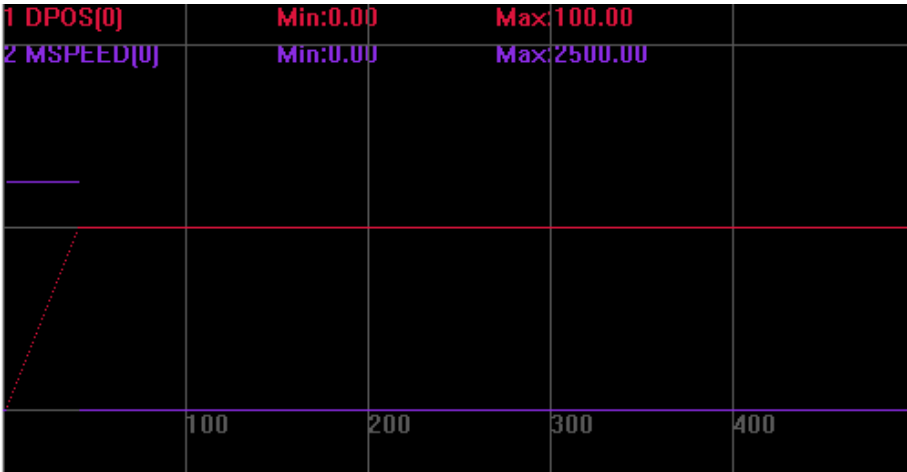
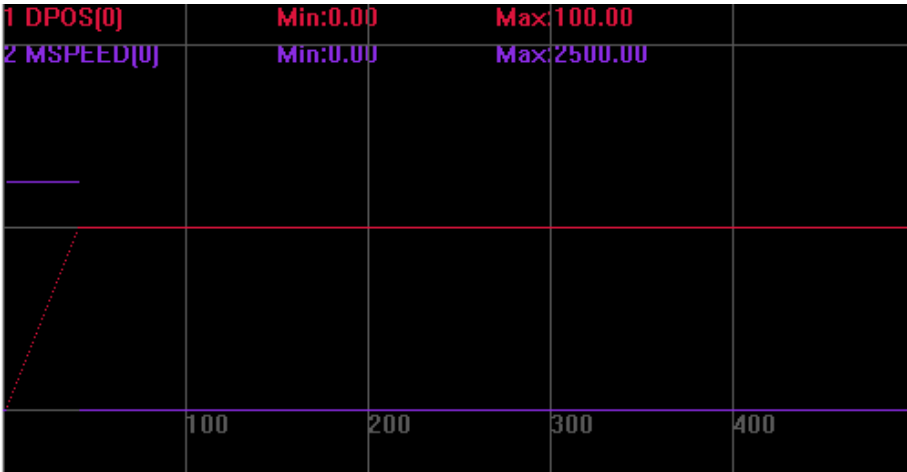
MOVELIMIT -- 速度限制

类型	特殊运动指令
描述	在当前的运动末尾位置增加速度限制，用于强制拐角减速。
语法	MOVELIMIT(limitspeed) limitspeed 限制到的速度
适用控制器	通用
例子	<pre> BASE(0) UNITS=100 DPOS=0 SPEED=1000 ' 轴速度 ACCEL=10000 ' 轴加速度 DECEL=10000 MERGE=ON ' 开启连续插补，多段运动间速度连续 TRIGGER ' 自动触发示波器 MOVE(500) MOVELIMIT(100) ' 在两个运动的中间降速到 100. MOVE(500) 插补速度 用 MOVELIMIT 限制速度时 </pre>

	 不限制速度时 
	相关指令 CORNER MODE

MOVE_PT -- 单位时间距离

类型	特殊运动指令
描述	在一段时间内驱动电机运动设置的距离。 一般是 PC 每个周期计算好对应的坐标，然后传给控制器。 不要在极短时间运动大距离，脉冲频率会过高，电机堵转，可以分解成小段，重复发送。 运动时的速度=(DIS/TICKS)*1000 units/s
语法	MOVE_PT (TICKS, DIS1, DIS2...) 参数： TICKS 时间的长度。Ticks 会自己不断减 1，1TICKS 大约 1ms DIS1 运动的距离
适用控制器	通用

例子	BASE (0)																				
	Units=100																				
	Dpos = 0																				
	Speed=100																				
	Accel=10000																				
	Decel=10000																				
	TRIGGER	' 自动触发示波器																			
	For i=0 to 9																				
	MOVE_PT (4, 10)	' 在 4tick 内, 运动 10units, 速度=2500 units/s																			
	next																				
	print *dpos	' 打印结果, 100																			
	<table><tr><th>编号</th><th>数据源</th><th>偏移</th><th>刻度</th></tr><tr><td>0</td><td>DPOS</td><td>0</td><td>100</td></tr><tr><td>0</td><td>MSPEED</td><td>0</td><td>2000</td></tr></table>				编号	数据源	偏移	刻度	0	DPOS	0	100	0	MSPEED	0	2000					
编号	数据源	偏移	刻度																		
0	DPOS	0	100																		
0	MSPEED	0	2000																		
	<table><tr><td>1 DPOS[0]</td><td>Min:0.00</td><td>Max:100.00</td><td></td></tr><tr><td>2 MSPEED[0]</td><td>Min:0.00</td><td>Max:2500.00</td><td></td></tr><tr><td colspan="4"></td></tr><tr><td></td><td>100</td><td>200</td><td>300</td><td>400</td></tr></table>				1 DPOS[0]	Min:0.00	Max:100.00		2 MSPEED[0]	Min:0.00	Max:2500.00							100	200	300	400
1 DPOS[0]	Min:0.00	Max:100.00																			
2 MSPEED[0]	Min:0.00	Max:2500.00																			
																					
	100	200	300	400																	
相关指令	MOVE_PTABS																				

MOVE_PTABS -- 单位时间距离绝对

类型	特殊运动指令
描述	在一段时间内驱动电机运动到某个位置。 一般是 PC 每个周期计算好对应的坐标, 然后传给控制器。 不要在极短时间运动大距离, 脉冲频率会过高, 电机堵转, 可以分解成小段, 重复发送。 运动时的速度=(DIS/TICKS)*1000 units/s
语法	<pre> MOVE_PTABS (TICKS, DIS1, DIS2...) </pre> 参数: TICKS 时间的长度。Ticks 会自己不断减 1, 1TICKS 大约 1ms DIS1 运动的绝对位置

适用控制器	通用
例子	<pre>Base(0,1) DPOS=0,0 MOVE_PTABS (3, 20,20) ' 3tick 时间内运动到到 20,20 print *dpos ' 打印结果, 20,20</pre>
相关指令	MOVE_PT

MOVE_PAUSE -- 运动暂停

类型	特殊运动指令								
描述	BASE 轴运动暂停。 只有在单轴或多轴插补运动时有效，多轴联动时一起暂停。 可以通过 AXISSTATUS 来查询是否有暂停。 当轴已经暂停或不在运动中时，调用这个指令会有警告输出，但不影响程序运行。								
语法	MOVE_PAUSE (mode) 参数： mode 暂停方式： <table border="1"> <tr> <td>0（缺省）</td><td>暂停当前运动。</td></tr> <tr> <td>1</td><td>在当前运动完成后正准备执行下一条运动指令时暂停。</td></tr> <tr> <td>2</td><td>在当前运动完成后正准备执行下一条运动指令时，并且两条指令的 MARK 标识不一样时暂停。 这个模式可以用于一个动作由多个指令来实现时，可以在一整个动作完成后暂停。</td></tr> <tr> <td>3</td><td>强制暂停，IDLE 模式下也可以进入暂停状态。 20170513 固件版本增加此功能</td></tr> </table>	0（缺省）	暂停当前运动。	1	在当前运动完成后正准备执行下一条运动指令时暂停。	2	在当前运动完成后正准备执行下一条运动指令时，并且两条指令的 MARK 标识不一样时暂停。 这个模式可以用于一个动作由多个指令来实现时，可以在一整个动作完成后暂停。	3	强制暂停，IDLE 模式下也可以进入暂停状态。 20170513 固件版本增加此功能
0（缺省）	暂停当前运动。								
1	在当前运动完成后正准备执行下一条运动指令时暂停。								
2	在当前运动完成后正准备执行下一条运动指令时，并且两条指令的 MARK 标识不一样时暂停。 这个模式可以用于一个动作由多个指令来实现时，可以在一整个动作完成后暂停。								
3	强制暂停，IDLE 模式下也可以进入暂停状态。 20170513 固件版本增加此功能								
适用控制器	通用								
例子	<pre>BASE(0) DPOS=0 SPEED=100 例一 mode 0 MOVE(1000) ' 当前运动 MOVEABS(-100) ' 缓冲运动 MOVE_PAUSE(0) ' 模式 0, 暂停当前运动 ?DPOS(0) ' 打印结果, 0.02</pre>								

	’此时当前运动只运行了极短时间，扫描到 MOVE_PASEME 时直接暂停。 例二 mode 1 MOVE(1000) ’ 当前运动 MOVEABS(-100) ’ 缓冲运动 MOVE_PAUSE(1) ’ 模式 1，先完成当前运动再暂停 ?DPOS(0) ’ 打印结果，1000 ’此时运行完当前运动，DPOS 为 1000 例三 mode 2 MOVE_MARK=1 ’ 标号手动设为 1 MOVE(200) ’ 当前运动 MOVE_MARK=1 ’ 标号设为与上一个运动一样 MOVEABS(-100) ’ 缓冲运动 MOVEABS(100) ’ 不设置运动标号，自动加一 MOVE_PAUSE(2) ’ 模式 2，先完成当前运动，然后直到下一条运动指令的标号与当前标号不一致时再暂停 DELAY(3000) ’ 等待暂停后 ?DPOS(0) ’ 打印结果，-100 ’此时运行完当前运动，下一条运动的标号被手动设置成相同的，继续执行，直到第 3 条运动标号不一致，暂停。
相关指令	MOVE_MARK ， MOVE_RESUME ， AXISSTATUS

MOVE_RESUME -- 运动恢复

类型	特殊运动指令
描述	当 BASE 轴暂停时，从暂停处继续运动。 可以通过 AXISSTATUS 来查询是否有暂停。
语法	MOVE_RESUME
适用控制器	通用
例子	BASE(0) DPOS=0 SPEED=100 MOVE(100) ’ 当前运动 MOVE(100) ’ 缓冲运动 MOVE_PAUSE(1) ’ 当前运动完成后暂停 ?DPOS(0) ’ 打印结果，100 DELAY(1000) MOVE_RESUME ’ 继续运行 WAIT IDLE ?DPOS(0) ’ 打印结果，200

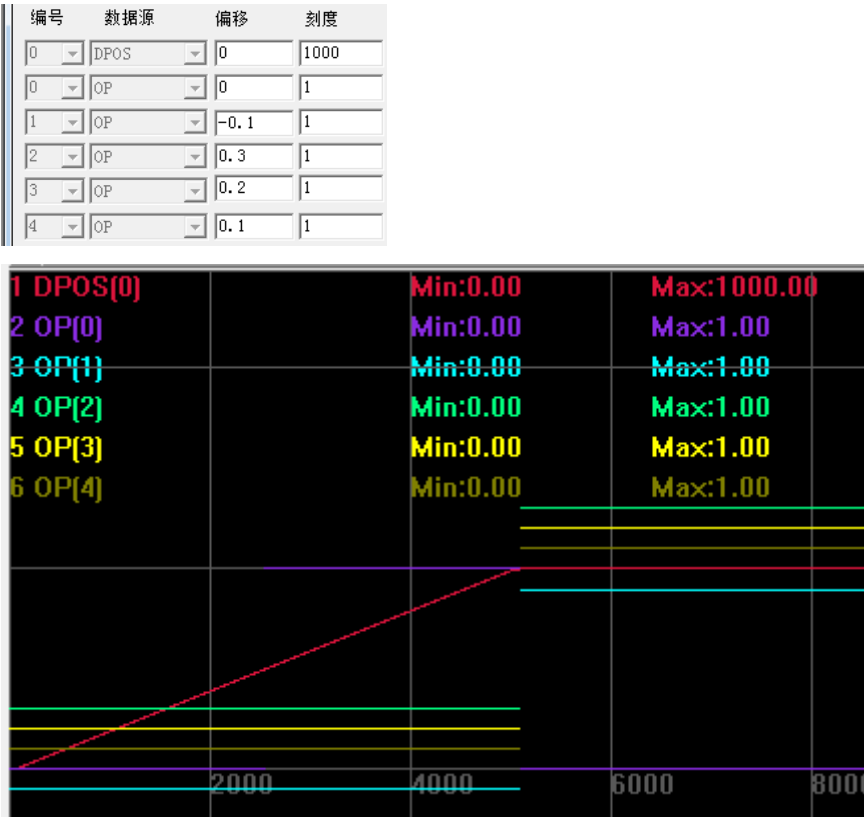
相关指令	MOVE_PAUSE , AXISSTATUS

MOVE_OP -- 缓冲输出

类型	特殊运动指令
描述	BASE 轴运动缓冲加入一个输出口操作。 这个指令 LOAD 执行时不做任何运动，只操作输出口。语法同 OP 指令。 普通模式，误差在一个扫描周期，所有控制器都可使用。 精准位置输出模式，误差 1 个脉冲，4 系列产品，20170421 以上版本支持。 1: 只有支持硬件比较输出的 OP 口支持此功能。 2: 每个生效的 MOVE_OP 需要间隔 1 周期时间才能继续生效，此间隔内新的 MOVE_OP 自动使用普通方式；，超过此间隔，新 MOVE_OP 可继续生效。连续的 MOVE_OP，因为没有间隔时间，只有第一个生效。 3: MOVE_OP 精准功能基于 BASE 主轴，多个轴插补时，与 BASE 主轴 ATYPE 类型不一样的从轴，无法保证精准位置输出。
语法	MOVE_OP ([ionum],value) 或 MOVE_OP (ionum1, ionum2,value[,mask]) 参数: ionum 输出编号, 0-, 没有这个参数时输出 0-31。 value 输出状态, 多个输出口操作时按位来指明多个口状态。 ionum1 要操作的第一个输出通道 ionum2 要操作的最后一个输出通道 mask 按位来设置值, 指定哪些 IO 需要操作, 不填时第一个通道到最后一个通道都操作
适用控制器	通用
例子	普通模式使用 <pre> BASE (0) UNITS=100 DPOS=0 SPEED=200 ACCEL=10000 DECEL=10000 TRIGGER ' 自动触发示波器 MOVE (500) MOVE_OP (0, ON) ' 等待上条运动完成后, OUT0 口输出信号 MOVE (500) </pre>

MOVE_OP (0, OFF) '等待上条运动完成后, OUT0 口关闭信号
MOVE_OP (1, 4, 15) 'OUT1-4 口输出信号, 15 对应二进制 1111

轨迹曲线, 为方便查看, 进行了竖直偏移



精准位置输出模式

例一

? "ZSET:", [SYSTEM ZSET](#)

BASE (0)

ATYPE=1

merge=1

MOVE (100)

MOVE_OP (0, 1) '精准生效

MOVE (100) '超过 2MS 时间, 下个 MOVE_OP 可继续精准生效

MOVE_OP (0, 0) '精准生效

MOVE (100)

例二

20170505 以上固件版本支持

[SYSTEM ZSET](#) = 3+16 'BIT4 支持编码器精确位置

BASE (0)

ATYPE= 4 '轴使用脉冲加编码器类型, 必须实际有接编码器线

DPOS=0

	<pre> MPOS=0 units=1000 SPEED= 1000 ACCEL = 10000 merge=1 trigger opnum = 0 MOVEOP_DELAY = 1.5 '实际输出时间延迟 1.5ms OP(opnum, 0) '初始化 OP HW_TIMER(2, 10000, 5000, 1, 0, opnum) '输出变为 on 后, 5000us 后 切换为 off MOVE(200) MOVE_OP(opnum, 1) '输出 on 后靠 HW_TIMER 来延时关闭. MOVE(100) MOVE_OP(opnum, 1) MOVE(100) </pre>
相关指令	OP , MOVE_OP2 精确位置输出模式 SYSTEM_ZSET , AXIS_ZSET

MOVE_OP2 -- 缓冲输出 2

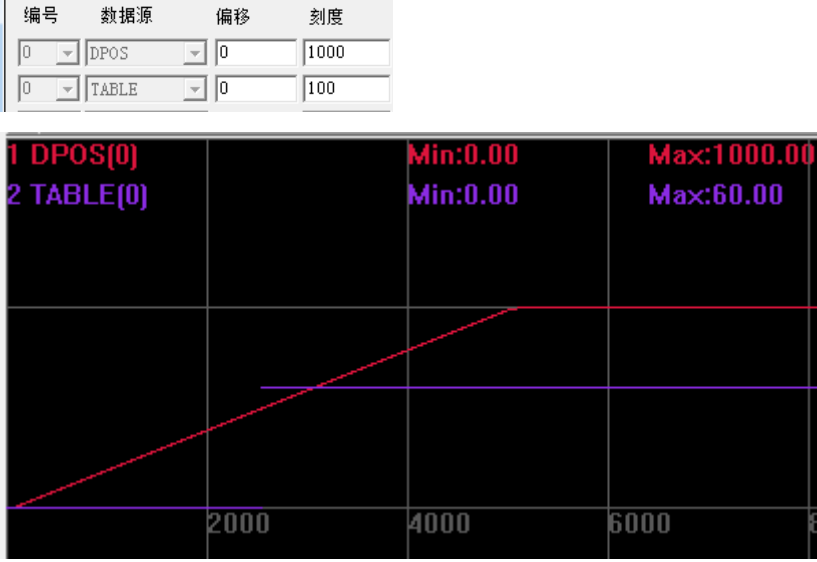
类型	特殊运动指令
描述	BASE 轴运动缓冲加入一个输出口操作，指定时间后输出状态翻转。 这个指令 LOAD 执行时不做任何运动，只操作输出口。 单个轴同一时间只支持一个脉冲输出，第二个 MOVE_OP2 指令会自动关闭前面指令的脉冲。
语法	<pre> MOVE_OP2(ionum, state, offtimems) </pre> 参数: ionum 输出编号, 0-, 没有这个参数时输出 0-31。 State 输出状态 offtimems ms 时间后翻转, 以产生脉冲 输出的效果。
适用控制器	通用
例子	<pre> BASE(0) UNITS=100 DPOS=0 SPEED=200 ACCEL=10000 </pre>

DECEL=10000

OP (0, OFF)

MOVE_TABLE -- 缓冲 Table

类型	特殊运动指令
描述	BASE 轴运动缓冲加入一个 TABLE。 这个指令 LOAD 执行时不做任何运动，只修改 TABLE。此指令的 MTYPE 与 MOVE_OP 一致。
语法	MOVE_TABLE(tablenum, value) 参数: tablenum TABLE 编号 Value 要修改的值
适用控制器	通用
例子	BASE(0) UNITS=100 DPOS=0 SPEED=200

	<pre> ACCEL=10000 DECEL=10000 TABLE(0)=0 ' 先令 TABLE(0)=0 ?TABLE(0) ' 打印确认 TABLE(0) 的值 ' 打印结果, 0 TRIGGER ' 自动触发示波器 MOVE(500) MOVE_TABLE(0, 60) ' 等待运动完成后, TABLE(0) 赋值 60 MOVE(500) WAIT IDLE ?TABLE(0) ' 打印修改后 TABLE(0) 的值 ' 打印结果, 60 </pre> 轨迹曲线 
相关指令	TABLE

MOVE_PARA -- 缓冲参数

类型	特殊运动指令
描述	BASE 轴运动缓冲修改参数。 这个指令 LOAD 执行时不做任何运动, 只修改参数。此指令的 MTYPE 与 MOVE_OP 一致。
语法	<pre>MOVE_PARA (PARAMNAME, INDEX, VALUE)</pre> PARAMNAME: 参数名, 必须是?*SET 里面的非只读参数

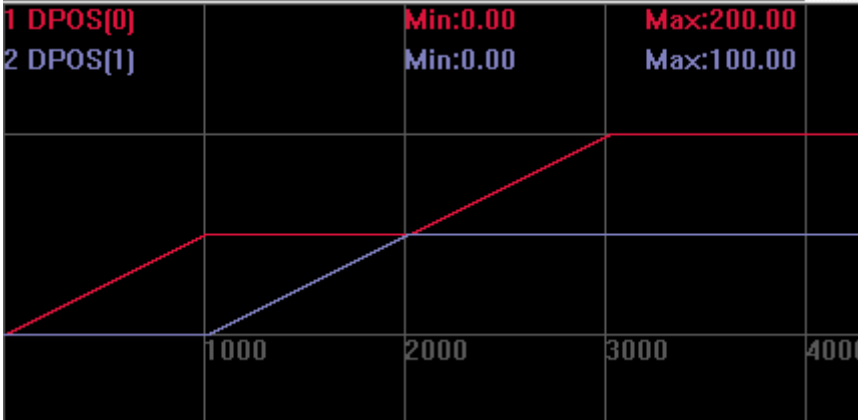
	INDEX: 参数编号 VALUE: 参数值
适用控制器	20170503 以上固件

MOVE_PWM -- 缓冲 PWM

类型	特殊运动指令
描述	BASE 轴运动缓冲操作 PWM。 这个指令 LOAD 执行时不做任何运动,只操作 PWM。此指令的 MTYPE 与 MOVE_OP 一致。
语法	MOVE_PWM(PWMINDEX, duty[, freq]) PWMINDEX: pwm 编号 duty : 占空比, 指有效电平占整个周期的比例。 0-1, 设置 0 时关闭 PWM。 一个周期中先输出有效电平, 再输出无效电平。 freq: 频率, 缺省为 1k, 硬件最大为 1M, 软件最大为 2K
适用控制器	20170503 以上固件
例程	RAPIDSTOP WAIT IDLE trigger TICKS=0 base(0) speed = 1000 move(10) MOVE_PWM(0, 0.111, 2000) ' 轴 0 运行到 10 时, 操作 PWM0 MOVE_DELAY(111) MOVE_PWM(0, 0.333) MOVE_DELAY(111) MOVE_PWM(0, 0.555, 3000) move(100) WHILE NOT IDLE MOVE_PWM(0, 0, 1000) ' 关闭 PWM ? -TICKS, PWM_FREQ(0), PWM_DUTY(0) WA 10

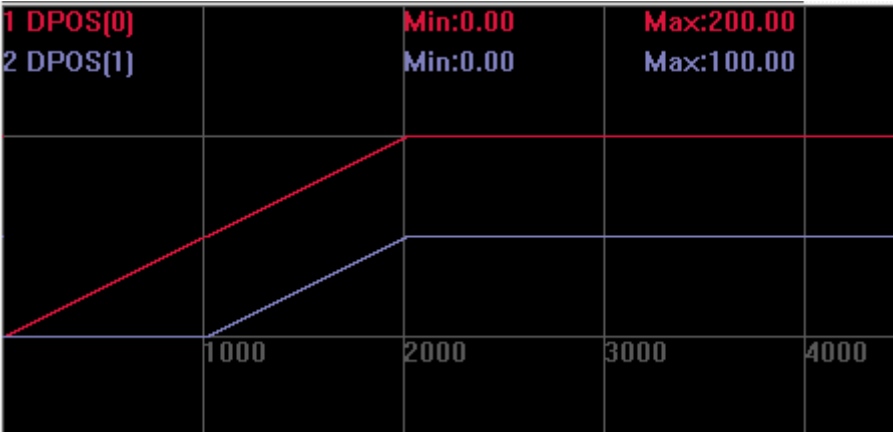
	WEND
相关指令	PWM_DUTY , PWM_FREQ

MOVE_SYNMOVE -- 缓冲触发其他轴

类型	特殊运动指令
描述	BASE 轴运动缓冲触发其他轴运动，当前轴等待。 此指令的 MTYPE 与 MOVE_DELAY 一致。
语法	MOVE_SYNMOVE (AXISNUM, DIS[, IFSP]) AXISNUM: 需要同步运动的轴号 DIS: 相对运动距离 IFSP: 是否使用 SP 运动，缺省不使用
适用控制器	20170503 以上固件
例程	RAPIDSTOP WAIT IDLE trigger TICKS=0 base(0) dpos=0, 0 speed = 100, 100 move(100) MOVE_SYNMOVE(1, 100, 0) ' 轴 0 运行到 100 时，轴 1 开始运动 100 move(100) ' 待轴同步完成，轴 0 再继续 

相关指令 [MOVE_ASYNCMOVE](#)

MOVE_ASYNCMOVE -- 缓冲触发其他轴 2

类型	特殊运动指令
描述	BASE 轴运动缓冲触发其他轴，当前轴不等待。 此指令的 MTYPE 与 MOVE_OP 一致。
语法	MOVE_ASYNCMOVE (AXISNUM, DIS[, IFSP]) AXISNUM: 需要同步运动的轴号 DIS: 相对运动距离 IFSP: 是否使用 SP 运动，缺省不使用
适用控制器	20170503 以上固件
例程	RAPIDSTOP WAIT IDLE trigger TICKS=0 base(0) dpos=0,0 speed = 100,100 move(100) MOVE_ASYNCMOVE(1, 100, 0) '轴 0 运行到 100 时，轴 1 开始运动 100 move(100) '轴 0 持续运动 
相关指令	MOVE_SYNMOVE

MOVE_TASK -- 缓冲开启任务

类型	特殊运动指令				
描述	BASE 轴运动缓冲加入启动 TASK。 这个指令 LOAD 执行时不做任何运动，只启动任务，此指令的 MTYPE 与 MOVE_OP 一致。 当任务已经启动时，会报错，但不影响程序执行。				
语法	MOVE_TASK(tasknum, label) 参数： <table> <tr> <td>tasknum</td><td>任务号</td></tr> <tr> <td>label</td><td>函数名或标号</td></tr> </table>	tasknum	任务号	label	函数名或标号
tasknum	任务号				
label	函数名或标号				
适用控制器	通用				
例子	<pre> BASE(0) DPOS=0 SPEED=100 MOVE(100) MOVE_TASK(1, task_move) ' 第一个运动完成后，将 task_move 作为任务 1 启动 MOVE(100) Task_move: Print "task_move" end </pre>				
相关指令	RUNTASK ， RUN				

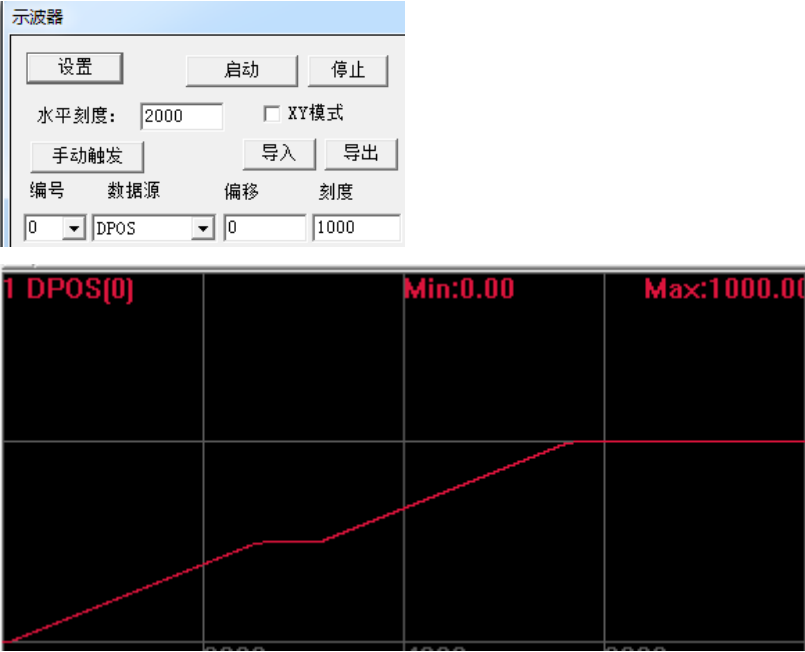
MOVE_AOUT -- 缓冲模拟量输出

类型	特殊运动指令				
描述	BASE 轴运动缓冲加入一个 AOUT 指令。 这个指令 LOAD 执行时不做任何运动，只修改 AOUT 值。此指令的 MTYPE 与 MOVE_OP 一致。				
语法	MOVE_AOUT(danum, value) 参数： <table> <tr> <td>danum</td><td>DA 编号</td></tr> <tr> <td>Value</td><td>要修改的值</td></tr> </table>	danum	DA 编号	Value	要修改的值
danum	DA 编号				
Value	要修改的值				
适用控制器	通用，实际只用在含有 DA 通道的控制器				
例子	BASE(0)				

	UNITS=100 DPOS=0 SPEED=200 ACCEL=10000 DECEL=10000 AOUT (0) =0 ' DA0 通道赋值 0 ?AOUT (0) ' 打印确认 TRIGGER ' 自动触发示波器 MOVE (500) MOVE_AOUT (0, 30.5) ' 第一个运动完成后, 将 DA0 通道赋值 30.5 MOVE (500) WAIT IDLE ?AOUT (0) ' 打印 DA0, 30.5												
	<table><tr><th>编号</th><th>数据源</th><th>偏移</th><th>刻度</th></tr><tr><td>0</td><td>DPOS</td><td>0</td><td>1000</td></tr><tr><td>0</td><td>AOUT</td><td>0</td><td>100</td></tr></table>	编号	数据源	偏移	刻度	0	DPOS	0	1000	0	AOUT	0	100
编号	数据源	偏移	刻度										
0	DPOS	0	1000										
0	AOUT	0	100										
	<table><tr><td>1 DPOS[0]</td><td></td><td>Min:0.00</td><td>Max:1000.00</td></tr><tr><td>2 AOUT[0]</td><td></td><td>Min:0.00</td><td>Max:30.50</td></tr></table>	1 DPOS[0]		Min:0.00	Max:1000.00	2 AOUT[0]		Min:0.00	Max:30.50				
1 DPOS[0]		Min:0.00	Max:1000.00										
2 AOUT[0]		Min:0.00	Max:30.50										
相关指令	AOUT												

MOVE_DELAY -- 缓冲延时

类型	特殊运动指令
描述	BASE 轴运动缓冲加入一个延时。 这个指令 LOAD 执行时不做任何运动，只延时指定时间。 前面的运动指令结束时速度会自动将为 0。
语法	<pre>MOVE_DELAY(timems)</pre> 别名: MOVE_WA(timems) 参数: timems 延时, 毫秒数

适用控制器	通用
例子	BASE (0) UNITS=100 DPOS=0 SPEED=200 ACCEL=10000 DECEL=10000 TRIGGER ’ 自动触发示波器 MOVE (500) MOVE_DELAY (1000) ’ 两个 MOVE 中间等待 1 秒 MOVE (500) 轨迹曲线 
相关指令	DELAY

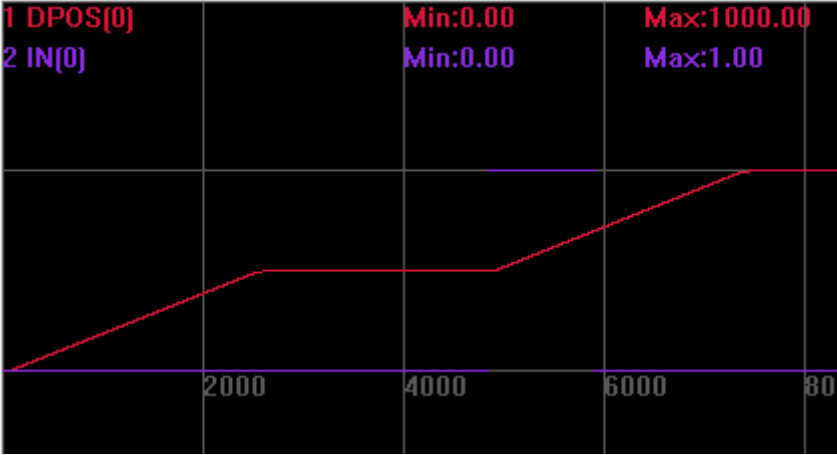
MOVEOP_DELAY -- 缓冲输出延时

类型	特殊运动指令
描述	BASE 轴缓冲信号延时输出。 设置在 BASE 主轴上，当 MOVE_OP 精准功能使用时，可以调整实际触发 OP 操作的时间，毫秒(ms)单位，支持小数，实际最长延时时间 100ms。 设置为负数，可以提前打开 OP，一般步进可以提前 2ms，伺服可以提前 20ms，点胶起胶可以使用。

语法	MOVEOP_DELAY=timems 参数: timems 毫秒数
适用控制器	20170505 以上固件版本
相关指令	MOVE_OP , HW_PSWITCH , HW_PSWITCH2

MOVE_WAIT -- 缓冲等待

类型	特殊运动指令								
描述	BASE 轴运动缓冲加入一个可变的延时。 这个指令 LOAD 执行时不做任何运动，只等待指定的条件满足，前面的运动指令结束时速度会自动降为 0。								
语法	MOVE_WAIT (PARANAME, par anum, eq, value) 参数： PARANAME 选择参数名： DPOS MPOS IN AIN VPSPEED MSPEED MODBUS_REG MODBUS_IEEE MODBUS_BIT NVRAM VECT_BUFFED REMAIN Par anum 参数编号或轴号 Eq : 1 >= -1 《= 对 IN 等 BIT 类型参数无效。 Value：比较值								
适用控制器	固件 150802 以上版本， 或 XPLC160405 以上版本支持。								
例子	BASE (0) UNITS=100 DPOS=0 SPEED=200 ACCEL=10000 DECEL=10000 TRIGGER '自动触发示波器 MOVE (500) MOVE_WAIT (IN, 0, 0, 1) '等待 IN(0)有信号，才执行下一条运动缓冲 MOVE (500) 轨迹曲线 <table><tr><th>编号</th><th>数据源</th><th>偏移</th><th>刻度</th></tr><tr><td>0</td><td>DPOS</td><td>0</td><td>1000</td></tr></table>	编号	数据源	偏移	刻度	0	DPOS	0	1000
编号	数据源	偏移	刻度						
0	DPOS	0	1000						

	
相关指令	WAIT

MOVE_TURNABS -- 旋转台插补

类型	多轴运动指令
描述	旋转的插补运动，保证在旋转台上是直线运动。 旋转功能是指工作平台在与 XY 平行的平面上旋转，旋转的正向与 XY 的正向要一致（右手法则）。 旋转的参数存储在 TABLE 表里面，依次存储 R 轴号，R 轴一圈脉冲数，X 轴号，Y 轴号，X 圆心，Y 圆心。 自定义速度的连续插补运动可以使用 SP 后缀的指令，见*SP 描述。 建议直接使用机械手的旋转台算法，详情查看《正运动机械手指令说明》的 frame11/17。
语法	MOVE_TURNABS(tablenum, position1[, position2[, position3[, position4...]]]) 参数： - tablenum 存储旋转参数的 table 编号， - position1 第一个轴运动坐标 - position2 下一个轴运动坐标
适用控制器	通用
例子	<pre>Base(0, 1, 3) dpos=0, 0, 0, 0 Table(0, 3, 3600, 0, 1, 0, 0) ' 设置旋转台的参数 TRIGGER Move_turnabs(0, 100, 200, 90) ' 直线插补至目标位置 Wait idle ' 等待运动停止</pre>

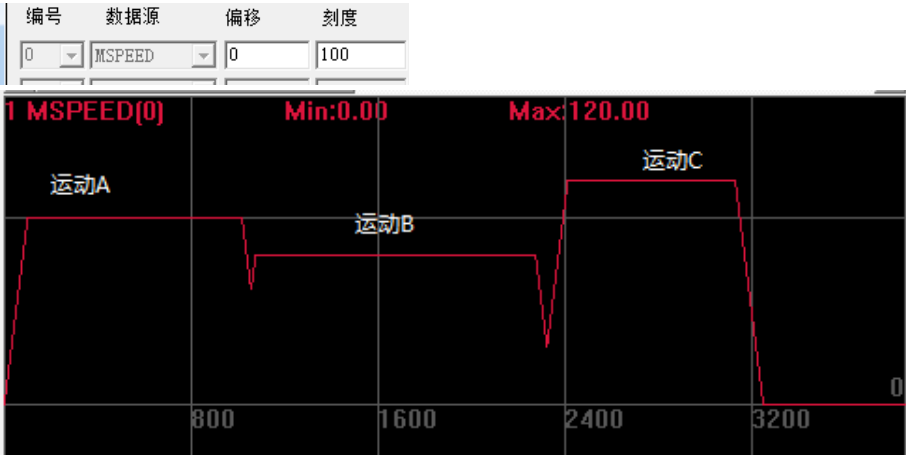
相关指令	MCIRC_TURNABS

MCIRC_TURNABS -- 旋转台插补-绝对

类型	多轴运动指令
描述	旋转的插补运动，保证在旋转台上是圆弧运动。 旋转功能是指工作平台在与 XY 平行的平面上旋转，旋转的正向与 XY 的正向要一致（右手法则）。 旋转的参数存储在 TABLE 表里面，依次存储 R 轴号，R 轴一圈脉冲数，X 轴号，Y 轴号，X 圆心，Y 圆心。 自定义速度的连续插补运动可以使用 SP 后缀的指令，见*SP 描述。
语法	MCIRC_TURNABS(tablenum, refpos1, refpos2, mode, end1, end2 [, dis3, dis4, dis5]) 参数： - tablenum 存储旋转参数的 table 编号 - refpos1 第一个轴参考点，绝对位置 - refpos2 第二个轴参考点，绝对位置 - mode 1-参考点是当前点前面，2-参考点是结束点后面，3-参考点在中间，采用三点定圆的方式。 - end1 第一个轴结束点，绝对位置 - end2 第二个轴结束点，绝对位置 - dis3 螺旋轴的结束位置。
适用控制器	通用
例子	Base(0, 1, 3) Table(0, 3, 3600, 0, 1, 0, 0) ' 设置旋转台的参数 TURN_POSMAKE(0, 100, 200, 5, 10) MCIRC_TURNABS(0, table(10), table(11), 3, 200, 300, 10) ' 圆弧的同时 3 轴旋转。 Wait idle
相关指令	MOVE_TURNABS , TURN_POSMAKE

*SP -- 运动单独速度

类型	多轴运动指令
描述	用于设置每段运动的运行速度，起始速度和终止速度。 多轴运动指令有对应 SP 运动指令，此时可以使用轴参数 FORCE_SPEED 和 ENDMOVE_SPEED , STRATMOVE_SPEED 来设置每个运动的速度和终止速度，当

	不需要每个运动设置速度时，不需要使用 SP 指令。
语法	SP 指令有：MOVESP, MOVEABSSP MOVECIRCSP, MOVECIRABSSP, MHELICALSP, MHELICALABSSP, MECLIPSESP, MECLIPSEABSSP, MSPHERICALSP FORCE_SPEED 和 ENDMOVE_SPEED ,STRATMOVE_SPEED 会随 sp 运动指令写入运动缓存区
适用控制器	通用
例子	例一 <pre> BASE(0) UNITS=100 DPOS=0 ACCEL=1000 DECEL=1000 MERGE=ON ' 打开连续插补 SPEED=100 ' 运行速度 100 FORCE_SPEED=80 ' 限制速度 80 STARTMOVE_SPEED=60 ' 起始速度 60 ENDMOVE_SPEED=30 ' 终止速度 30 TRIGGER ' 自动触发示波器 MOVE(100) ' 运动 A，不使用 SP 限速 MOVESP(100) ' 运动 B，使用 SP 限速 FORCE_SPEED=120 ' 限制速度 120 ENDMOVE_SPEED=30 ' 终止速度 30 MOVESP(100) ' 运动 C，使用 SP 限速 </pre> 速度轨迹  不使用 SP 指令时，运行速度为 SPEED，使用 SP 指令后，运行速度为 FORCE_SPEED。 当 STARTMOVE_SPEED 和 ENDMOVE_SPEED 同时设置，STARTMOVE_SPEED 生效。

	例二 BASE (0, 1) MERGE =ON CORNER_MODE=2+8 ' 启动自动拐角减速与小圆限速 DECEL_ANGLE = 15 * (PI/180) ' 设置开始减速角度 STOP_ANGLE = 45 * (PI/180) ' 设置结束减速角度 FULL_SP_RADIUS = 5 ' 设置小圆限速最大半径 Speed = 1000 , 1000 Startmove_speed = 1000 ' 设置一个较大的起始速度 Endmove_speed = 1000 ' 设置一个较大的结束速度 Force_speed = 50 ' 每段速度受 speed 限制，将 speed 设置为一个较大值即可 TRIGGER ' 自动触发示波器 Movesp(0, 100) ' 该段速度为 50units/s Force_speed = 70 ' 每段速度受 speed 限制，将 speed 设置为一个较大值即可 Movesp(100, 0) ' 该段速度为 70units/s 
相关指令	FORCE_SPEED , ENDMOVE_SPEED , STRATMOVE_SPEED

CAM -- 凸轮表运动

类型	同步运动指令
描述	CAM 指令根据存储在 TABLE 中的数据来决定轴的运动，这些 Table 数据值对应运动轨迹的位置，是相对于运动起始点的绝对位置。

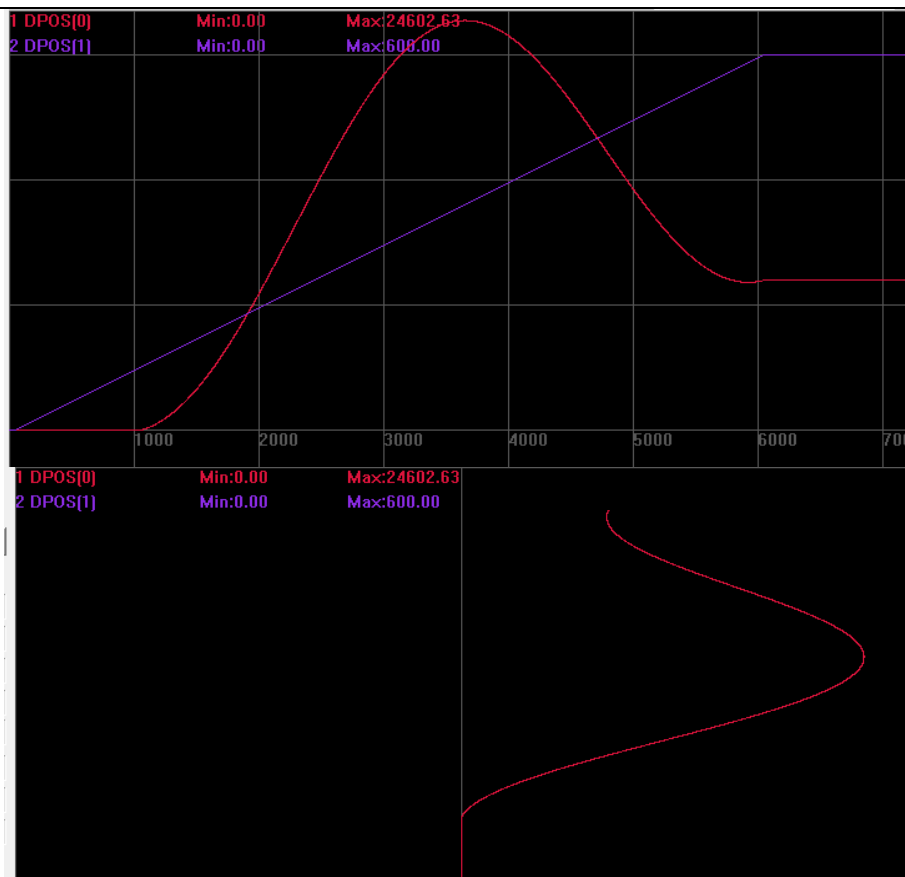
	注：两个或多个 CAM 指令可以同时使用同一段 Table 数据区进行操作。 运动的总时间由设置速度和第四个参数决定，运动的实际速度根据 table 轨迹与时间自动匹配。 Tbale 数据需要手动设置，第一个数据为引导点，建议设为 0。 Table 数据 * table multiplier 这个比例 = 发出的脉冲数。												
语法	CAM(start point, end point, table multiplier, distance) 参数： start point 起始点 TABLE 编号，存储第一个点的位置。 end point 结束点 TABLE 编号 table multiplier 位置乘以这个比例，一般设为脉冲当量值。 distance 参考运动的距离。 总时间=distance / 轴 SPEED												
适用控制器	通用												
例子	BASE (0) UNITS=100 DPOS=0 ACCEL=1000 DECEL=1000 SPEED=100 TABLE (10, 0, 80, 75, 40, 50, 20, 50, 0) ' table 从 10 开始存数据 TRIGGER ' 自动触发示波器 CAM (10, 17, 100, 500) ' 运动轨迹为 table10 到 17, 运动总时间为 500/100=5s 轨迹与速度 <table><tr><th>编号</th><th>数据源</th><th>偏移</th><th>刻度</th></tr><tr><td>0</td><td>DPOS</td><td>0</td><td>100</td></tr><tr><td>0</td><td>MSPEED</td><td>0</td><td>100</td></tr></table> 示波器设置 通道数： 2 深度： 10000 间隔： 5 1 DPOS[0]Min:0.00Max:79.97 2 MSPEED[0]Min:-70.00Max:112.00	编号	数据源	偏移	刻度	0	DPOS	0	100	0	MSPEED	0	100
编号	数据源	偏移	刻度										
0	DPOS	0	100										
0	MSPEED	0	100										

相关指令	CAMBOX

CAMBOX -- 跟随凸轮表运动

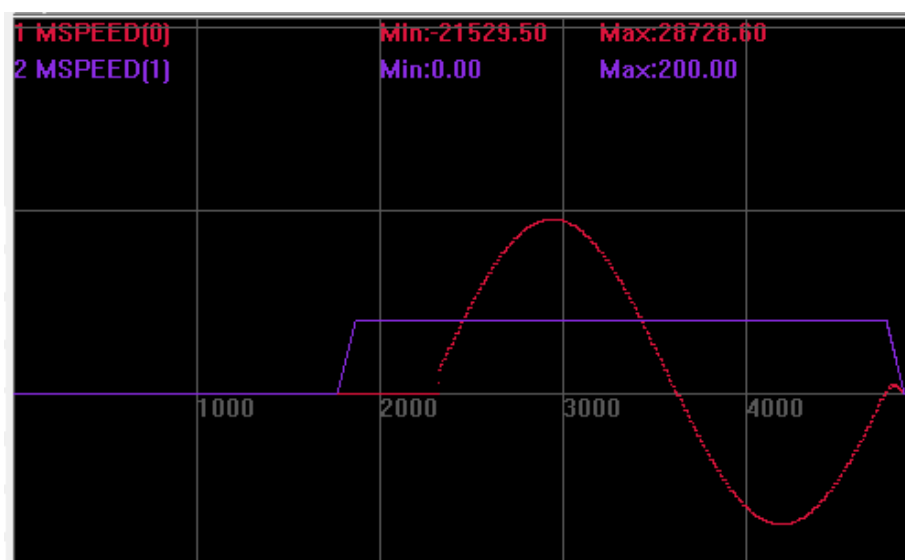
类型	同步运动指令
描述	CAMBOX 指令根据存储在 TABLE 中的数据来决定轴的运动，这些 Table 数据值对应运动轨迹的位置，是相对于运动起始点的距离。跟随轴的运动根据参考轴的运动来进行。 注：两个或多个 CAMBOX 指令可以同时使用同一段 Table 数据区进行操作。 Table 数据 * table multiplier 这个比例=发出的脉冲数 当前轴运动的总时间由参考轴的运动距离和轴速度确定，速度自动匹配。 Tbale 数据需要手动设置，第一个数据为引导点，建议设为 0。 Table 数据 * table multiplier 这个比例 = 发出的脉冲数。
语法	CAMBOX(start_point, end_point, table_multiplier, link_distance , link_axis[, link_options][, link_pos][, link_offpos]) 参数： start_point: 起始点 TABLE 编号，存储第一个点的位置。 end_point: 结束点 TABLE 编号 table_multiplier: 位置乘以这个比例，一般设为脉冲当量值。 link_distance: 参考轴运动的距离 link_axis: 参考轴。 link_options: 与参考轴的连接方式，不同的二进制位代表不同的意义。 Bit 0 当参考轴 MARK 信号事件触发时，当前轴与参考轴开始进行连接运动。 Bit 1 当参考轴运动到设定的绝对位置时，当前轴与参考轴开始连接运动。 Bit 2 自动重复连续双向运行。(通过设置 REP_OPTION=1, 可以取消重复) Bit4 从中间某个位置启动，配合掉电中断实现恢复凸轮。 Bit 5 只有参考轴的正向运动才连接。 Bit8 当参考轴 MARKB 信号事件触发时，当前轴与参考轴开始进行连接运动，需要最新固件支持。 link_pos: 当 link_options 参数设置为 2 时，该参数表示连接开始启动的绝对位置。 Link offpos: 当 link_options 参数 bit4 置为 1 时，该参数表示主轴已经运行完的相对位置。20170428 以上固件支持。

适用控制器	通用												
例子	<pre>ERRSWITCH = 3 base(0,1) ' 选择第 0 轴 atype=1,1 ' 脉冲方式步进或伺服 dpos = 0,0 units = 100,100 ' 脉冲当量, 每 mm100 脉冲 speed = 200,200 accel = 2000,2000 decel = 2000,2000 ' 计算 table 的数据 dim deg, rad, x, stepdeg stepdeg = 2 ' 可以通过这个来修改段数, 段数越多速度越平稳 FOR deg=0 TO 360 STEP stepdeg rad = deg * 2 * PI/360 ' 转换为弧度 x = deg * 25 + 10000 * (1-COS(rad)) TABLE(deg/stepdeg,x) ' 存储 table trace deg/stepdeg,x NEXT deg TRIGGER ' 自动触发示波器 while 1 ' 循环运动 if in(0) = on then ' 输入 0 有效启动运动 dpos = 0,0 CAMbox(0,360/stepdeg, 100, 500, 1,2,100) ' 轴 1 到 100 位 置的时启动 move(600) axis(1) wait until idle and idle(1)' 等待运动停止 delay(100) ' 延时 endif wend end ' 停止当前任务 插补轨迹</pre> <table><tr><th>编号</th><th>数据源</th><th>偏移</th><th>刻度</th></tr><tr><td>0</td><td>DPOS</td><td>0</td><td>10000</td></tr><tr><td>1</td><td>DPOS</td><td>0</td><td>300</td></tr></table>	编号	数据源	偏移	刻度	0	DPOS	0	10000	1	DPOS	0	300
编号	数据源	偏移	刻度										
0	DPOS	0	10000										
1	DPOS	0	300										



速度曲线

编号	数据源	偏移	刻度
0	MSPEED	0	30000
1	MSPEED	0	500

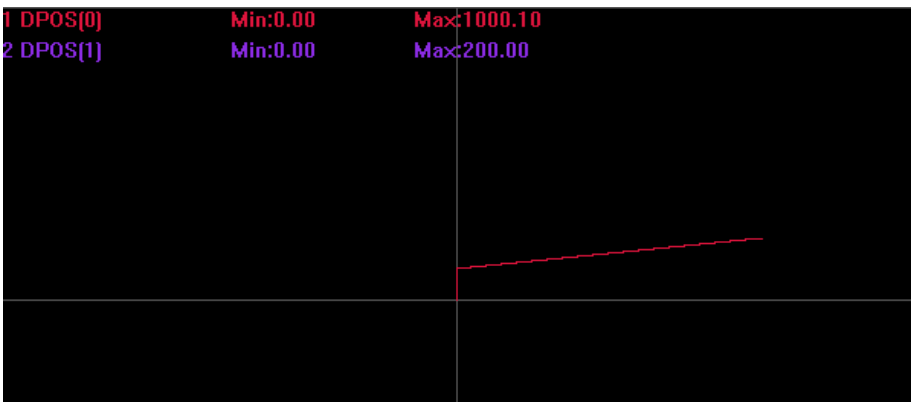


相关指令

[CAM](#)

CONNECT -- 同步运动

类型	同步运动指令												
描述	将当前轴的目标位置与 driving_axis 轴的测量位置通过电子齿轮连接。 连接的是脉冲个数，要考虑不同轴 UNITS 的比例。 假设连接轴 0 的 UNIST 为 100，被连接轴 1 的 UNITS 为 10。 使用 CONNECT 连接，比例 ratio 为 1，当轴 0 运动 s1=100 时，轴 1 运动 =s1*UNITS(0)*ratio/UNITS(1)。 比率可以通过重复调用指令动态变化。 常用于手轮使用。 取消连接时用 CANCEL。												
语法	CONNECT (ratio, driving_axis) 参数： ratio: 比率，可正可负，注意是脉冲个数的比例。 driving_axis: 连接轴的轴号，手轮时为编码器轴。												
适用控制器	通用												
例子	BASE (0, 1) ATYPE=1, 1 UNITS=10, 100 DPOS=0, 0 SPEED=100, 100 TRIGGER ’ 自动触发示波器 MOVE(100) AXIS(1) ’ 轴 1 运动 100，此时轴 0 不动 WAIT IDLE(1) CONNECT (1, 1) AXIS(0) ’ 轴 0 连接到轴 1，比例为 1 MOVE(100) AXIS(1) ’ 轴 1 运动 100，轴 0 运动 1000 合成轨迹（XY 模式） 示波器 设置 启动 停止 水平刻度： 100 ☒ XY模式 手动触发 导入 导出 <table><thead><tr><th>编号</th><th>数据源</th><th>偏移</th><th>刻度</th></tr></thead><tbody><tr><td>0</td><td>DPOS</td><td>0</td><td>100</td></tr><tr><td>1</td><td>DPOS</td><td>0</td><td>500</td></tr></tbody></table>	编号	数据源	偏移	刻度	0	DPOS	0	100	1	DPOS	0	500
编号	数据源	偏移	刻度										
0	DPOS	0	100										
1	DPOS	0	500										

	
相关指令	CLUTCH_RATE , CONNPATH

CONNPATH -- 同步运动 2

类型	同步运动指令
描述	将当前轴的目标位置与 driving_axis 轴的插补矢量长度通过电子齿轮连接。 连接的是脉冲个数，要考虑不同轴 UNITS 的比例。 比率可以通过重复调用指令动态变化。 取消连接时用 CANCEL。
语法	CONNPATH (ratio, driving_axis) 参数： ratio: 比率，可正可负，注意是脉冲个数的比例。 driving_axis: 连接轴的轴号，手轮时为编码器轴。
适用控制器	通用
例子	<pre> BASE(0,1) ATYPE=1,1 UNITS=10,100 SPEED=100,100 TRIGGER ' 自动触发示波器 MOVE(100) AXIS(1) ' 轴 1 运动 100，此时轴 0 不动 WAIT IDLE(1) CONNPATH(-0.5,1) AXIS(0) ' 轴 0 连接到轴 1，比例为-0.5 MOVE(100) AXIS(1) ' 轴 1 运动 100，轴 0 运动-500 合成轨迹(XY 模式) </pre>

	示波器 设置 启动 停止 水平刻度: 100 ☒ XY模式 手动触发 导入 导出 编号 数据源 偏移 刻度 0 DPOS 0 100 1 DPOS 0 500 1 DPOS[0] Min:-500.00 Max:0.00 2 DPOS[1] Min:0.00 Max:200.00 
相关指令	CLUTCH_RATE , , CONNECT

CONNFRAME -- 机械手逆解

类型： 同步运动指令

语法：CONNFRAME (frame, tablenum, viraxis0, viraxis1, [...])

描述：将当前关节坐标系的目标位置与虚拟坐标系的位置关联；CLUTCH_RATE=0 时关节坐标系的运动速度和加速度受 SPEED 等参数的限制。

 当关节轴告警等出错时，此运动会被 CANCEL。

 不要在虚拟轴高速运动中 CANCEL 此运动，会引起轴高速运动中停止。

 此命令 LOAD 时会自动修改虚拟轴的坐标，使得与关节轴一致，因此调用后需要 WAIT LOADED 后才开始运动。

 连接过程中不要修改虚拟轴的坐标，或是调用 DATUM 等可能改坐标的运动，这样会导致关节轴快速运动到新的虚拟位置。

 CONNFRAME 生效后，关节轴的 MTYPE 为 33，此时无法直接运动关节轴，必须通过运行虚拟轴来间接运动关节轴，当要直接移动关节轴时，先调用 CANCEL 取消 CONNFRAME，再运动关节轴。

参数：

frame: 坐标系类型， 1- scara，（如需针对特殊的机械手类型定制，请联系厂

家)。

tablenum: 存储转换参数的 TABLE 位置, frame=1 时, 依次存放: 第一个关节轴长度, 第二个关节轴长度, 第一个关节轴一圈脉冲数, 第二个关节轴一圈脉冲数。

viraxis0: 虚拟坐标系第一个轴

viraxis1: 虚拟坐标系第二个轴

[...] 虚拟坐标系第 N 个轴, 此运动轴可以是实际轴。



当虚拟轴和实际轴都是旋转轴时, 例如末端旋转轴, 虚拟轴和实际轴的脉冲当量注意要一致。

FRAME 机械结构一览表

详细说明请查看《正运动机械手指令说明》文档。

其他特殊的机械结构如需支持请联系厂家。

frame	结构类型
1	标准 SCARA 机械手
101	SCARA+摆动, 虚拟轴定义 4 个
105	SCARA+摆动, 虚拟轴定义 5 个
106	特殊 SCARA
107	特殊 SCARA
108	特殊 5 轴 SCARA
11	旋转台
17	双旋转台
18	偏移旋转台
19	偏移双旋转台
3	码垛机械手
103	码垛变形, 喷涂机械手
5	旋转伸缩机械手
15	XY 滑台
102	2 轴 delta
2	3 轴 delta, R 类型控制器支持
12	4 轴 delta, R 类型控制器支持
13	3 轴垂直蜘蛛手, R 类型控制器支持
25	5 关节机械手
6	6 自由度机械手, R 类型控制器支持
26	特殊 6 自由度
36	特殊 6 自由度

100	XYZ+2 轴手腕，虚拟轴定义 3 个
104	XYZ+2 轴手腕，虚拟轴定义 5 个

CONNREFRAME -- 机械手正解

类型： 同步运动指令

语法：CONNREFRAME (frame, tablenum, axis0, axis1, [...])

描述：将虚拟轴的坐标与关节轴的坐标关联，关节轴运动后，虚拟轴自动走到相应的位置，此指令为 CONNFRAME 的反运动指令。



当虚拟轴 CONNREFRAME 运动 LOAD 时，关节轴的 CONNFRAME 运动会自动被 CANCEL。



当关节轴的 CONNFRAME 运动 LOAD 时，虚拟轴 CONNREFRAME 运动会自动被 CANCEL。

参数： 参见 CONNFRAME 的参数，BASE 轴与参数里面的轴的位置相反。

举例：

```
L1=10          ' 第一个关节轴长度
L2=10          ' 第二个关节轴长度
BASE(0, 1)     ' 假设关节轴号是 0/1
UNITS=10, 10   ' 这里以度为单位
DPOS=0, 0      ' 设置关节轴的位置，此处要根据实际情况来修改。
BASE(6, 7)
ATYPE=0, 0     ' 设置为虚拟轴
TABLE(0, L1, L2, 3600, 3600) ' 参数存储从 TABLE 0 开始存储，电机一圈 360 个
```

脉冲。

```
UNITS=1000, 1000 ' 此脉冲当量要提前设置，中途不能变化
CONNREFRAME(1, 0, 0, 1) ' 6/7 的坐标根据 0/1 来计算
```

' 第三步：运动关节轴

```
BASE(0, 1)
```

```
MOVEABS(90, 0) ' 虚拟轴的坐标变为 0, 20
```

MOVELINK -- 自动凸轮

类型	同步运动指令
描述	此指令用于自定义的凸轮运动，该运动带有可设置的加减速阶段。

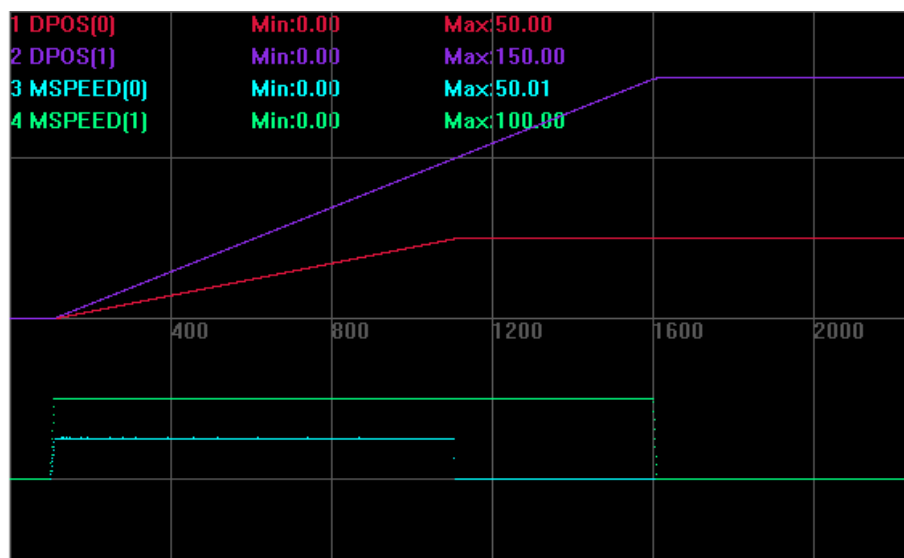
	被连接轴为参考轴，连接轴为跟随轴。 连接轴的距离分成 3 个阶段应用于参考轴的运动。这些部分是加速部分，匀速部分和减速部分。 在加速和减速阶段为了与速度匹配，Link distance（基本轴运动距离）必须是 distance（跟随轴运动距离）的两倍。
语法	MOVELINK (distance, link dist, link acc, link dec, link axis[, link options] [, link pos][, link offpos]) 参数： distance: 从连接开始到结束，跟随轴移动的距离，采用 units 单位。 link dist: 参考轴在连接的整个过程中移动的绝对距离，采用 units 单位。 link acc: 在跟随轴加速阶段，参考轴移动的绝对距离，采用 units 单位。 link dec: 在跟随轴减速阶段，参考轴移动的绝对距离，采用 units 单位。 注：如果参数 3 和参数 4 的和大于第二个参数，他们会被自动按比例减小，使其和值与第二个参数值相等。 link axis: 参考轴的轴号。 link options: 连接模式选项，不同的二进制位代表不同的意义。 1 位 0, 连接精确开始于参考轴上 MARK 事件被触发的时刻。 2 位 1, 连接开始于参考轴到达一个绝对位置时。(见 link pos 参数描述) 4 位 2, 当这位被设置时，MOVELINK 会自动重复执行并且可以反向。(这个模式可以通过设置轴参数 REP_OPTION 的第 1 位为 1 来清除) 8 位 3, 当设置时,采用 S 曲线加减速。20170502 以上固件支持 16 位 4 从中间某个位置启动，配合掉电中断实现恢复跟随。 32 位 5 只有参考轴的正向运动才连接。 256 位 8 连接精确开始于参考轴上 MARKB 事件被触发的时刻，需要最新固件支持。 link pos: 当 link options 参数设置为 2 时，该参数表示基本轴在该绝对位置值时，连接开始。 link offpos: 当 link_options 参数 bit4 置为 1 时，该参数表示主轴已经运行完的相对位置。20170428 以上固件支持。
适用控制器	
例子	轴 0 为跟随轴，轴 1 为参考轴。 例一

BASE (0, 1)
UNITS=100, 100
ATYPE=1, 1
DPOS=0, 0
SPEED=100, 100
ACCEL=2000, 2000
DECEL=2000, 2000
TRIGGER ’ 自动触发示波器
MOVELINK (100, 100, 0, 0, 1) ’ 不设置加减速阶段时，效果与 CONNECT 相同，区别在不需要考虑 UNITS 的不同，且不会有累积误差。此时运动比例 1:1。
MOVE (150) AXIS (1) ’ 轴 1 运动 150，轴 0 跟随轴 1 运动完 100。
插补轨迹与速度曲线

编号	数据源	偏移	刻度
0	DPOS	0	100
1	DPOS	0	100
0	MSPEED	-200	200
1	MSPEED	-200	200



MOVELINK (50, 100, 0, 0, 1) 时



例二 飞剪应用

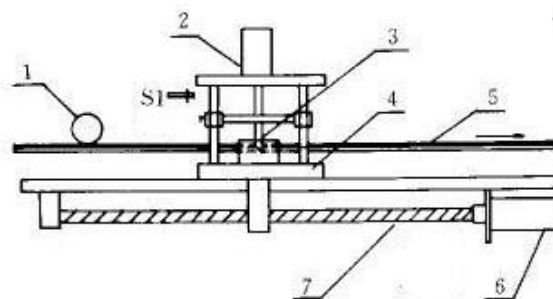


图1 飞剪系统原理图
1—测长轮；2—液压缸；3—刀具；4—工作台；
5—型材；6—伺服电机；7—丝杠

型材持续运动，工作台先静止；直到型材持续运动了某段距离，工作台开始加速；待工作台速度与型材一致，然后开关 S1 工作刀具下剪，剪切完后刀具回升；工作台开始减速，然后退回起始点。重复过程，剪切得到设定长度的型材。

假设要切的型材长度为 50 单位，工作台运行距离 1 个单位，轴 1 为基本轴（型材），轴 0 为跟随轴（工作台），OUT0 口控制刀具，飞剪部分程序如下

```
BASE(0,1)
UNITS=100,100
ATYPE=1,1
DPOS=0,0
SPEED=100,100
ACCEL=2000,2000
DECEL=2000,2000
```

	VMOVE(1) AXIS(1) ' 型材持续运动																				
	TRIGGER ' 自动触发示波器																				
	WHILE 1																				
	MOVELINK(0, 45, 0, 0, 1) ' 型材运动 45 单位前，工作台静止																				
	MOVELINK(0.4, 0.8, 0.8, 0, 1) ' 工作台加速阶段																				
	MOVELINK(0.2, 0.2, 0, 0, 1) ' 速度跟随阶段																				
	MOVE_OP2(0, on, 1000) ' 刀具下剪，1s 后回升(时间要计算好)																				
	MOVELINK(0.4, 0.8, 0, 0.8, 1) ' 工作台减速阶段																				
	MOVELINK(-1, 3.2, 0.5, 0.5, 1) ' 工作台回到起始点																				
	WEND																				
	运动轨迹和速度曲线																				
	<table><tr><th>编号</th><th>数据源</th><th>偏移</th><th>刻度</th></tr><tr><td>0</td><td>DPOS</td><td>0</td><td>5</td></tr><tr><td>1</td><td>DPOS</td><td>0</td><td>100</td></tr><tr><td>0</td><td>MSPEED</td><td>-300</td><td>200</td></tr><tr><td>1</td><td>MSPEED</td><td>-300</td><td>250</td></tr></table>	编号	数据源	偏移	刻度	0	DPOS	0	5	1	DPOS	0	100	0	MSPEED	-300	200	1	MSPEED	-300	250
编号	数据源	偏移	刻度																		
0	DPOS	0	5																		
1	DPOS	0	100																		
0	MSPEED	-300	200																		
1	MSPEED	-300	250																		
	工作台(跟随轴)的运动距离: 0.4(加速)+0.2(跟随)+0.4(减速)=1 单位，然后-1 返回。																				
	型材(参考轴)的运动距离: 45+0.8+0.2+0.8+3.2=50 单位，全程匀速。																				
相关指令	MOVESLINK , MOVELINK MODIFY																				

MOVELINK_MODIFY -- 同步距离修改

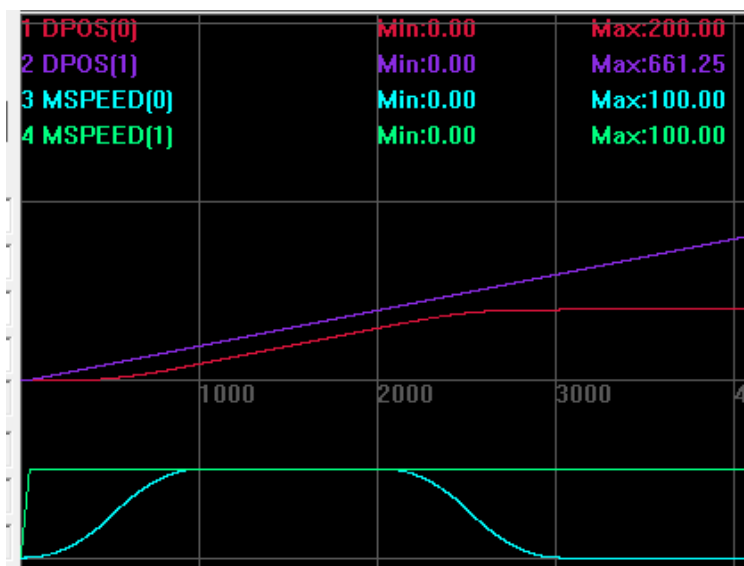
类型	轴参数
描述	相对修改 MOVELINK 指令的同步区长度。

语法	VAR1 = MOVELINK_MODIFY, MOVELINK_MODIFY = expression
适用控制器	20160926 以后固件版本支持。
例子	<pre> REGIST(3) MOVELINK(19000, 20000, 1000, 1000, 1, 16) WAIT LOADED WAIT UNTIL MARK OR IDLE IF MARK THEN MOVELINK_MODIFY = REG_POS + 5000 - ENDMOVE ENDIF </pre>
相关指令	MOVELINK

MOVESLINK -- 自动凸轮 2

类型	同步运动指令
描述	此指令用于自定义的凸轮运动，该运动自动规划中间曲线，不用计算凸轮表。 被连接轴为参考轴，连接轴为跟随轴。 在加速和减速阶段为了与速度匹配，下一条 MOVESLINK 的 start sp 必须与当前 MOVESLINK 的 end sp 相同。
语法	MOVESLINK (distance, link dist, start sp, end sp, link axis[, link options] [, link pos][, link offpos]) [, link options] [, link start] 可选参数不填时，逗号不能省掉，控制器根据参数的位置来判断是什么参数。 distance: 从连接开始到结束，跟随轴移动的距离，采用 units 单位。 link dist: 参考轴在连接的整个过程中移动的绝对距离，采用 units 单位。 start sp: 启动时跟随轴和参考轴的速度比例，units/units 单位，负数表示跟随轴负向运动。 end sp: 结束时跟随轴和参考轴的速度比例，units/units 单位，负数表示跟随轴负向运动。 注：当 start sp = end sp = distance/ link dist 时，匀速运动。 link axis: 参考轴的轴号。 link options: 连接模式选项，不同的二进制位代表不同的意义。 1 位 0，连接精确开始于参考轴上 MARK 事件被触发的时刻。 2 位 1，连接开始于参考轴到达一个绝对位置时。（见 link pos 参数描述） 4 位 2，当这位被设置时，MOVESLINK 会自动重复执行并且可以反向。（这个模式可以通过设置轴参数 REP_OPTION 的第 1 位为 1 来清除） 16 位 4，使用 link offpos 从中间来启动，配合掉电中断实现恢复跟随。 20170428 以上固件支持

	32 位 5 只有参考轴的正向运动才连接。 256 位 8 连接精确开始于参考轴上 MARKB 事件被触发的时刻，需要最新固件支持。 link pos: 当 link options 参数设置为 2 时，该参数表示参考轴在该绝对位置值时，连接开始。 link offpos: 当 link_options 参数 bit4 置为 1 时，该参数表示主轴已经运行完的相对位置。20170428 以上固件支持。																				
适用控制器	通用																				
例子	功能与 MOVELINK 相同，仅是参数设置区别 举例 1: BASE(0, 1) UNITS=100, 100 ATYPE=1, 1 DPOS=0, 0 SPEED=100, 100 ACCEL=2000, 2000 DECEL=2000, 2000 TRIGGER 自动触发示波器 MOVESLINK(50, 100, 0, 1, 1) 轴 0 跟踪轴 1 运动，从速度 0 到速度一致。 MOVESLINK(100, 100, 1, 1, 1) 轴 0 跟踪轴 1 运动，匀速 100units。 MOVESLINK(50, 100, 1, 0, 1) 轴 0 跟踪轴 1 运动，减速到 0。 VMOVE(1) AXIS(1) <table><thead><tr><th>编号</th><th>数据源</th><th>偏移</th><th>刻度</th></tr></thead><tbody><tr><td>0</td><td>DPOS</td><td>0</td><td>500</td></tr><tr><td>1</td><td>DPOS</td><td>0</td><td>500</td></tr><tr><td>0</td><td>MSPEED</td><td>-200</td><td>200</td></tr><tr><td>1</td><td>MSPEED</td><td>-200</td><td>200</td></tr></tbody></table>	编号	数据源	偏移	刻度	0	DPOS	0	500	1	DPOS	0	500	0	MSPEED	-200	200	1	MSPEED	-200	200
编号	数据源	偏移	刻度																		
0	DPOS	0	500																		
1	DPOS	0	500																		
0	MSPEED	-200	200																		
1	MSPEED	-200	200																		



举例 2:

BASE (0, 1)

UNITS=100, 100

ATYPE=1, 1

DPOS=0, 0

SPEED=100, 100

ACCEL=2000, 2000

DECEL=2000, 2000

TRIGGER '自动触发示波器

VMOVE (1) AXIS (1)

WHILE 1

MOVESLINK (0, 45, 0, 0, 1) ' 型材运动 45 单位前，工作台静止

MOVESLINK (0.4, 0.8, 0.8, 0, 1) ' 工作台加速阶段

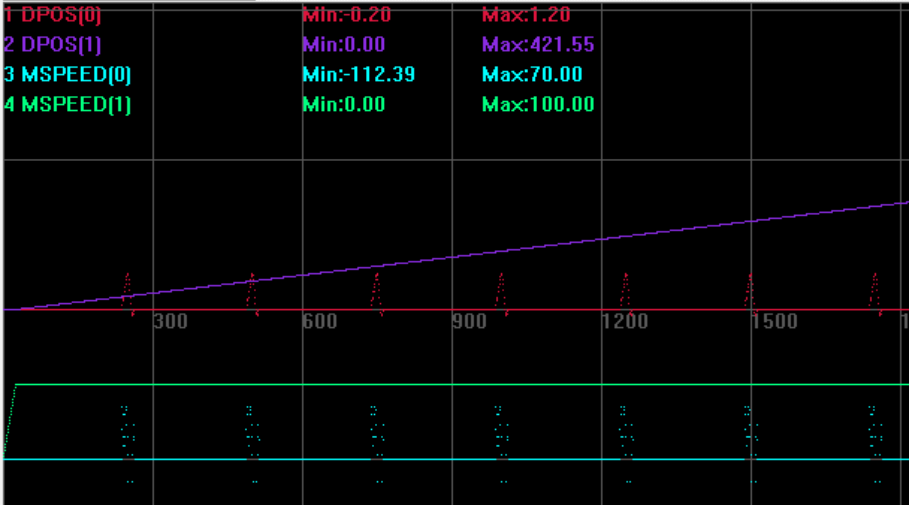
MOVESLINK (0.2, 0.2, 0, 0, 1) ' 速度跟随阶段

MOVESLINK (0.4, 0.8, 0, 0.8, 1) ' 工作台减速阶段

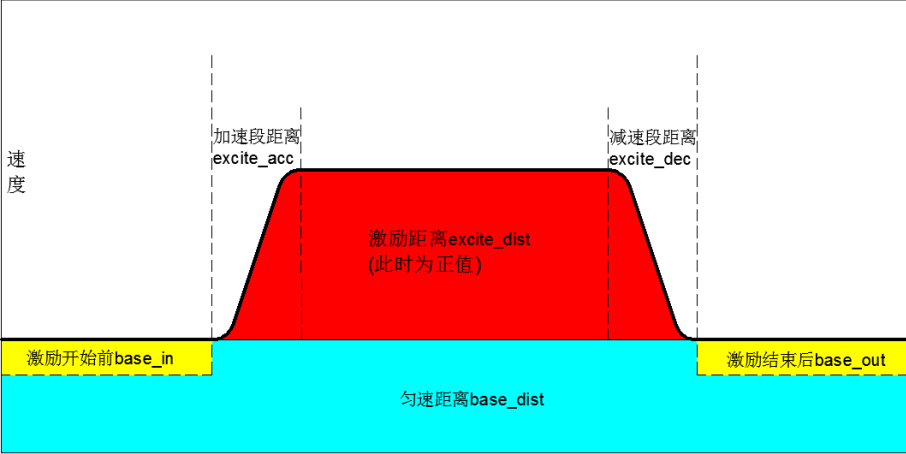
MOVESLINK (-1, 3.2, 0.5, 0.5, 1) ' 工作台回到起始点

WEND

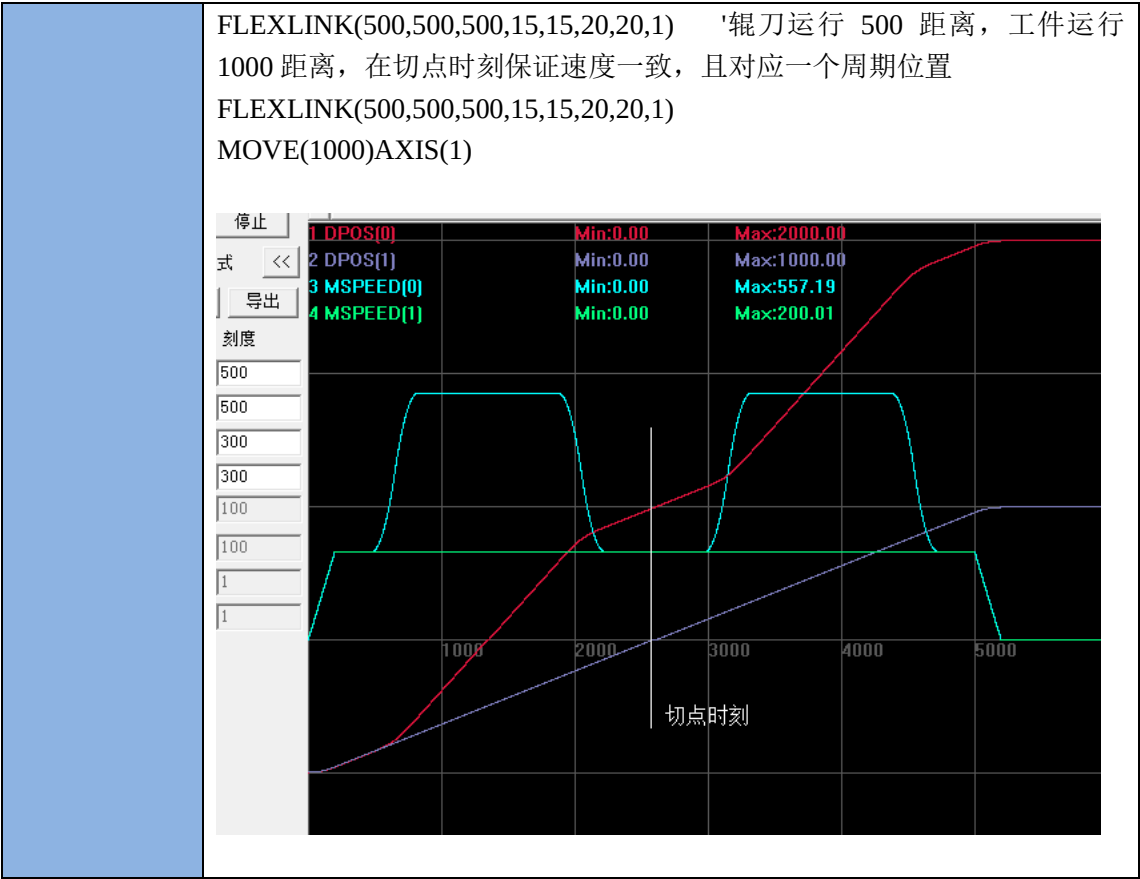
编号	数据源	偏移	刻度
0	DPOS	0	5
1	DPOS	0	500
0	MSPEED	-200	200
1	MSPEED	-200	200

	1 DPOS[0] 2 DPOS[1] 3 MSPEED[0] 4 MSPEED[1] Min:-0.20 Min:0.00 Min:-112.39 Min:0.00 Max:1.20 Max:421.55 Max:70.00 Max:100.00 
相关指令	MOVELINK

FLEXLINK -- 激励运动

类型	同步运动指令
描述	此指令用于轴的激励运动，分为匀速运动和激励运动两部分。 
语法	FLEXLINK(base_dist, excite_dist, link_dist, base_in, base_out, excite_acc, excite_dec, link_axis, link_options, [start_pos], [link_offpos]) 参数 base_dist 跟随轴匀速运动距离 excite_dist 跟随轴激励运动距离，+值表示增加，-值表示减少 跟随轴运动总距离=base_dist+excite_dist link_dist 整个指令过程，跟随轴运动完成，主轴移动的距离

	base_in 激励开始前，跟随轴运动距离占 base_dist 的百分比 base_out 激励完成后，跟随轴剩余运动距离占 base_dist 的百分比 两者相加不要超过 100%，否则 excite_dist 无效 excite_acc 激励过程中，跟随轴加速阶段运动距离占 excite_dist 的百分比，excite_dist 为负值时，为减速阶段。 excite_dec 激励过程中，跟随轴减速阶段运动距离占 excite_dist 的百分比，excite_dist 为负值时，为加速阶段。 以上 4 个参数在 excite_dist 不为 0 时生效 link_axis 主轴轴号 link_options: 与参考轴的连接方式，不同的二进制位代表不同的意义。 Bit 0 当参考轴锁存信号事件 Mark 触发。 Bit 1 当参考轴运动到设定的绝对位置时，当前轴与参考轴开始连接运动。 Bit 2 自动重复连续双向运行。(通过设置 REP_OPTION=1，可以取消重复) Bit 4 凸轮中间启动。 Bit 5 只有参考轴的正向运动才连接。 BIT 8 markb 启动 start_pos: 绝对位置触发。 link_offpos 凸轮中间启动。
适用控制器	4 系列 20170518 以上固件版本支持 3 系列 20161212 以上固件版本支持
例子	例一 轮切应用 假设辊刀切一次周长500 工件长度1000 辊刀恒速，移动距离小于工件长度，此时工件需要有加速过程 实际应用一般是辊刀变速，这里为了方便演示，用工件变速 BASE(0,1) DPOS=0,0 SPEED = 200,200 TRIGGER



REGIST -- 锁存

类型	位置锁存指令
描述	REGIST 指令用来锁存轴的测量反馈位置。 支持编码器轴锁存，4 系列及以上控制器最新固件支持虚拟轴、脉冲轴锁存。 EtherCAT 支持驱动器锁存，此时使用驱动器 IO 点实现锁存，具体模式查看指令语法。 Rtex 只支持控制器锁存。 4 系列及以上控制器支持 4 锁存通道。 锁存输入 432 2 个，432N 4 个，412 8 个。 支持 EtherCAT 驱动器锁存与控制器锁存同时使用，需要有 4 锁存通道功能。 4 个通道指，MARK，MARKB，MARKC，MARKD，通过 REG_INPUTS 指定锁存输入入口对应的锁存通道。 当锁存产生时，轴状态 MARK 会被设置为 ON，同时锁存到的位置会被存储在参数 REG_POS 内。

	每个轴有输入信号 R0,R1, EZ 信号可以使用锁存功能。当使用两个信号锁存时，第二个信号锁存使用 MARKB 和 REG_POSB。 R0,R1 输入一般对应到输入口 0 和 1，详细请查看控制器的硬件手册中通用输入章节。
语法	语法一 REGIST(mode) 参数： mode: 锁存方式 上升下降沿是以控制器内部状态而言，不同类型的输入口可能不一致，需要确认实际锁存的边沿。 脉冲和 Rtex 总线控制器： 1 当 Z 脉冲上升沿时的绝对位置送到 REG_POS 2 当 Z 脉冲下降沿时的绝对位置送到 REG_POS 3 当输入信号 R0 上升沿的绝对位置送到 REG_POS 4 当输入信号 R0 下降沿的绝对位置送到 REG_POS 6 输入信号 R0 上升沿时的绝对位置送到 REG_POS, Z 信号上升沿时的绝对位置送到 REG_POSB 7 输入信号 R0 上升沿时的绝对位置送到 REG_POS, Z 信号下降沿时的绝对位置送到 REG_POSB 8 输入信号 R0 下降沿时的绝对位置送到 REG_POS, Z 信号上升沿时的绝对位置送到 REG_POSB 9 输入信号 R0 下降沿时的绝对位置送到 REG_POS, Z 信号下降沿时的绝对位置送到 REG_POSB 10 输入信号 R0 上升沿时的绝对位置送到 REG_POS, 输入信号 R1 上升沿时的绝对位置送到 REG_POSB 11 输入信号 R0 上升沿时的绝对位置送到 REG_POS, 输入信号 R1 下降沿时的绝对位置送到 REG_POSB 12 输入信号 R0 下降沿时的绝对位置送到 REG_POS, 输入信号 R1 上升沿时的绝对位置送到 REG_POSB 13 输入信号 R0 下降沿时的绝对位置送到 REG_POS, 输入信号 R1 下降沿时的绝对位置送到 REG_POSB 14 以后 150804 以后版本支持，每个锁存通道独立，支持 4 通道锁存。 14 输入信号 R1 上升沿时的绝对位置送到 REG_POSB 15 输入信号 R1 下降沿时的绝对位置送到 REG_POSB 16 Z 信号上升沿时的绝对位置送到 REG_POSB 17 Z 信号下降沿时的绝对位置送到 REG_POSB 18 输入信号 R2 上升沿时的绝对位置送到 REG_POSC 19 输入信号 R2 下降沿时的绝对位置送到 REG_POSC 20 输入信号 R3 上升沿时的绝对位置送到 REG_POSD 21 输入信号 R3 下降沿时的绝对位置送到 REG_POSD

	EtherCAT 总线控制器： R0, R1, Z 脉冲这三种使用驱动器锁存；R2, R3 使用控制器锁存。 语法二 REGIST(100+mode, tableindex, numes) 参数： mode: 锁存方式 Tableindex: 连续锁存的内容存储的 table 位置，第一个 table 元素存储锁存的个数，后面存储锁存的坐标，最多保存个数= numes-1，溢出时循环写入。 Numes: 占用的 table 个数。 通过把模式加 100 来支持连续锁存，锁存结果存储到 TABLE 里面。分别对两个通道进行连续锁存，可以实现上下边沿的连续锁存。 ECI: 20150829 以上固件支持 4 系列控制器：20170523 以上固件支持 100+mode: 只能使用单一通道的 mode，加 100 表示使用连续锁存。 1 当 Z 脉冲上升沿时的绝对位置送到 REG_POS 2 当 Z 脉冲下降沿时的绝对位置送到 REG_POS 3 当输入信号 R0 上升沿的绝对位置送到 REG_POS 4 当输入信号 R0 下降沿的绝对位置送到 REG_POS 14 输入信号 R1 上升沿时的绝对位置送到 REG_POSB 15 输入信号 R1 下降沿时的绝对位置送到 REG_POSB 16 Z 信号上升沿时的绝对位置送到 REG_POSB 17 Z 信号下降沿时的绝对位置送到 REG_POSB 23 当输入信号 R0 上升沿的绝对位置送到 REG_POSB 24 当输入信号 R0 下降沿的绝对位置送到 REG_POSB 33 当输入信号 R0 上升沿的绝对位置送到 REG_POS, 下一次切换下降沿，轮流切换。 34 当输入信号 R0 下降沿的绝对位置送到 REG_POS, 下一次切换上升沿，轮流切换。 35 当输入信号 R1 上升沿的绝对位置送到 REG_POSB, 下一次切换下降沿，轮流切换。 36 当输入信号 R1 下降沿的绝对位置送到 REG_POSB, 下一次切换上升沿，轮流切换。 见例三
适用控制器	有锁存 in 口
例子	举例 1: ’锁存轴 0 的输入信号 R0 上跳沿时的位置，并打印。

```
BASE(0)
ATYPE=3
REGIST(3)
WAIT UNTIL MARK
PRINT REG_POS
```

举例 2:

’ PC 交互位置锁存，一般用于运动抓拍，通过锁存的位置得知抓拍时的实际位置。

```
global g_start
global g_posx, g_posy

while 1
wait until g_start=1 ’等待 PC 发出启动
regist(4) axis(0) ’输入 0 锁存， 24V 变 0V 的时刻
regist(4) axis(1)
wait until mark(0) and mark(1)
g_start=0
g_posx=reg_pos
g_posy=reg_posb
’print g_posx, g_posy
Wend
```

举例 3: 100+mode

```
,
dim num
num=1

base(6)
atype=6
REGIST(100+4, 0, 100) ’自动循环，不需要再写入到 while 循环中
’ table(0) 保存锁存次数，table(1-100) 存储每次锁存的数据。
’ 超过 99 次后，table(0) 清 0，重新从 table(1) 记录数据。
while 1
wait until mark
?REG_POS, TABLE(num), table(0) ’打印
If num=100 then
Num=1
Else
Num=num+1
Endif
wa 1 ’延时 1ms，防抖
wend
```

相关指令	MARK , MARKB , REG_POS , REG_POSB
------	---

ENCODER_RATIO -- 编码器齿轮比

类型	运动设置指令
描述	编码器输入齿轮比, 缺省(1, 1)
语法	ENCODER_RATIO(mpos_count, input_count, [, mode]) 参数: mpos_count 分子, 不要超过 65535 input_count 分母, 不要超过 65535 zmc406 20170502 以后固件支持 Mode 1, AB 1X 模式 2, AB 2X 模式 4, AB 4X 模式 必须先设置 ATYPE, 再调用 mode 设置。
适用控制器	通用
例子	ENCODER_RATIO(4, 1) ' 编码器 4 倍输入。
相关指令	PP_STEP , ENCODER

STEP_RATIO -- 电机齿轮比

类型	运动设置指令
描述	设置步进输出齿轮比, 缺省(1, 1)。 范围 1-65535。 不能设置为负值。 建议不要随便修改, 可以通过修改脉冲当量来达到相同的效果。
语法	STEP_RATIO(output_count, dpos_count) 参数: output_count 分子, 不要超过 65535 input_count 分母, 不要超过 65535
适用控制器	通用
例子	STEP_RATIO (16, 1) ' 16 倍脉冲个数输出, 把脉冲当量乘以 16 也可以达到同样的效果。

BACKLASH -- 反向间隙补偿

类型	运动设置指令
描述	设置轴的反向间隙，扩展轴无效。
语法	BACKLASH(enable [,dist[, speed, acceleration]]) enable 使能与否 dist 距离，UNITS 单位 speed 反向间隙速度 acceleration 反向间隙加速度
适用控制器	通用
例子	BACKLASH(0) ' 关闭反向间隙功能。 BACKLASH(1, 0.1) ' 设置反向间隙 0.1mm

PITCHSET -- 螺距补偿

类型	运动设置指令
描述	设置轴的螺距补偿，扩展轴无效。 每点的补偿脉冲个数存储在 TABLE 表里面。
语法	PITCHSET(enable [, startpos, disone, maxpoint, tablenum]) enable 使能与否 startpos 开始补偿的 MPOS 位置，UNITS 单位，startpos 对应的点不存储 disone 每个点之间的距离，UNITS 单位 maxpoint 补偿的总点数 tablenum 螺距补偿表存储的 TABLE 位置，从 startpos 下一个点开始存储，脉冲数单位
适用控制器	通用
例子	BASE(0) ATYPE=1 UNITS=100 SPEED=100 ACCEL=100 TABLE(0, 10, 20, 30, 40, -10, -20, -30, -40) ' TBALE 存贮螺距补偿值 DPOS=0 PITCHSET(1, 300, 100, 8, 0) ' MPOS=300 时，开始补偿 8 个点，间隔 100

	TRIGGER move(1500)	
	开始补偿点 (MPOS)	补偿值 (脉冲个数)
	300	-
	400	10
	500	20
	600	30
	700	40
	800	-10
	900	-20
	1000	-30
	1100	-40

第三章 程序结构和程序流程

’ -- 注释

类型	特殊字符
描述	后面全部是注释，直到下一行开始。
适用控制器	通用

_ -- 换行

类型	特殊字符
描述	后面换行继续。 在条件判断语句、内存存储、打印输出时尽量不要使用。
适用控制器	通用

CONST -- 常量定义

类型	语法指令
描述	定义符号表示的常数，从而避免直接用数值表示。
语法	<pre>CONST CVARNAME = value</pre> CVARNAME 常量名 value 常量值
适用控制器	通用
例子	例一 <pre>CONST MAX_VALUE = 100000 ' 定义常量 TABLE(0)=MAX_VALUE ' table(0)赋值 10000</pre> 例二 <pre>GLOBAL CONST MAX_AXIS=6 ' 定义总轴数</pre>
相关指令	DIM

DIM -- 变量定义

类型	语法指令
描述	定义文件模块变量，数组。 当变量不定义就赋值时，会自动定义为文件模块变量。 文件模块变量只能在本程序文件内部使用。 数组可以作为字符串使用，一个元素表示一个字节。
语法	<pre>DIM VarName, ArrayName(space)</pre> VarName 变量名称 ArrayName 数组名称 space 数组长度
适用控制器	通用
例子	<pre>DIM ARRAY1(100) ' 定义数组 ARRAY1 DIM VAR1 ' 定义变量 VAR1 VAR2 = 100 ' 赋值语句会自动定义文件模块变量。 ARRAY1 = "asdf" ARRAY1(0, 100, 200, 300) ' 对数组进行连续赋值. ARRAY1(0)=100, ' ARRAY1(1)=200, ARRAY1(3)=300</pre>

相关指令	CONST , LOCAL , GLOBAL
------	--

LOCAL -- 局部定义

类型	语法指令
描述	定义局部变量，局部变量。 局部变量主要用于 SUB 中。 同一个 SUB 的局部变量有个数限制，SUB 过程的输入参数自动转化为局部变量。 不同任务的同一个 SUB 过程调用生成不同的局部变量，同一个任务的 SUB 过程递归调用也生成不同的局部变量。
语法	<pre>SUB subA() LOCAL localname localname 局部变量名 ENDSUB</pre>
适用控制器	通用
例子	<pre>SUB aaa() LOCAL v1 ' 定义局部变量 V1 v1=100 END SUB</pre>
相关指令	DIM , GLOBAL

GLOBAL -- 全局定义

类型	语法指令								
描述	定义全局变量，数组；定义全局 SUB 过程。								
语法	语法 1: GLOBAL VAR1 语法 2: GLOBAL SUB SUB1() 语法 3: GLOBAL CONST CVARNAME = value <table> <tr> <td>VAR1</td><td>变量名称</td></tr> <tr> <td>SUB1</td><td>过程名称</td></tr> <tr> <td>CVARNAME</td><td>常量名称</td></tr> <tr> <td>value</td><td>常量值</td></tr> </table>	VAR1	变量名称	SUB1	过程名称	CVARNAME	常量名称	value	常量值
VAR1	变量名称								
SUB1	过程名称								
CVARNAME	常量名称								
value	常量值								

适用控制器	通用
例子	GLOBAL SUB g_sub2() ' 定义全局过程 g_sub2，可以在任意文件中使用。 GLOBAL CONST g_convar = 100 ' 定义全局使用的常量 GLOBAL g_var2 ' 定义全局变量 g_var2
相关指令	DIM , LOCAL

SUB -- 自定义子函数 SUB

类型	语法指令
描述	用户自定义 SUB 过程，可以前面增加 GLOBAL 描述以定义全局使用的 SUB 过程。
语法	SUB label ([para1] [, para2]...) END SUB 参数 label: 过程名称，不能与现有的关键词冲突。 para1: 过程调用时传入的参数，自动作为 LOCAL 局部变量。 para2: 过程调用时传入的参数，自动作为 LOCAL 局部变量。
适用控制器	通用
例子	SUB sub1() ' 定义过程 SUB1，只能在当前文件中使用。 ?1 ... End sub GLOBAL SUB g_sub2() ' 定义全局过程 g_sub2，可以在任意文件中使用。 ?2 ... End sub GLOBAL SUB g_sub3(para1, para2) ' 定义全局过程 g_sub3，传递两个参数 ?Para1, para2 ... Return para1+para2 ' 函数返回参数相加

	End sub
相关指令	SUB_PARA , SUB_IFPARA

SUB_PARA -- SUB 传递参数

类型	语法指令
描述	选择 SUB 的传入参数。
语法	SUB_PARA(address) 参数 address: 第几个传递参数, 从 0 开始。
适用控制器	通用
例子	SUB AAA(NUM1, NUM2, NUM3) ?SUB_PARA(0) ' 调用 AAA 时, 打印传递的第一个 num1 值 ?SUB_PARA(1) ' 打印传递的第二个 num2 值 ?SUB_PARA(2) ' 打印传递的第三个 num3 值 END SUB
相关指令	SUB , SUB_IFPARA

SUB_IFPARA -- SUB 传参判断

类型	语法指令
描述	判断 SUB 是否传入参数。
语法	SUB_IFPARA(address) -1 传入, 0 未传入 参数 address: 第几个传递参数, 从 0 开始。

适用控制器	通用
例子	<pre> AAA (0, 100) ' 传入 num1, num2 AAA (, 100) ' 只传入 num2 END SUB AAA (NUM1, NUM2) IF SUB_IFPARA (0) THEN ' 调用 AAA 时, 判断 num1 是否传入 ?1 ' 传入打印 1 ELSE ?0 ENDIF END SUB </pre>
相关指令	SUB , SUB PARA

GSUB -- 自定义子函数-G 代码格式

类型	语法指令
描述	用户自定义 GSUB 过程。 可以前面增加 GLOBAL 描述以定义全局使用的 GSUB 过程, GSUB 过程调用的时候采用 G 代码语法, 不要加括号。
语法	<pre> GSUB label ([char1] [, char2]...) End sub </pre> 参数 label: 过程名称, 不能与现有的关键词冲突。 char1: 过程调用时传入的字母参数, 自动作为 LOCAL 局部变量。 char2: 过程调用时传入的字母参数, 自动作为 LOCAL 局部变量。 字母参数只能为单字符
适用控制器	通用
例子	<pre> G01 X100 Y100 Z100 U100 ' 调用 G01 END ' 主程序结束 GLOBAL GSUB G01 (X, Y, Z, U) ' 定义 GSUB 过程 G01 ... End sub </pre>

相关指令	GSUB PARA , GSUB IFPARA

GSUB_PARA -- GSUB 传递参数

类型	语法指令
描述	选择 GSUB 的传入参数。
语法	GSUB_PARA(char) 参数 char: GSUB 定义时传入的字母参数。
适用控制器	通用
例子	GSUB AAA(X, Y, Z) ?GSUB_PARA(X) ' 调用 AAA 时, 打印传递的 X 值 ?GSUB_PARA(Y) ' 打印传递的 Y 值 ?GSUB_PARA(Z) ' 打印传递的 Z 值 END SUB
相关指令	GSUB , GSUB IFPARA

GSUB_IFPARA -- 判断 GSUB 是否传参

类型	语法指令
描述	判断 SUB 是否传入参数。
语法	GSUB_IFPARA(char) -1 传入, 0 未传入 参数 char: GSUB 定义时传入的字母参数。
适用控制器	通用
例子	AAA X0 Y100 ' 传入 X, Y AAA X0 ' 只传入 X END GSUB AAA(X, Y)

	<pre> IF GSUB_IFPARA(Y) THEN ' 调用 AAA 时，判断 Y 是否传入 ?1 ' 传入打印 1 ELSE ?0 ENDIF END SUB </pre>
相关指令	GSUB , GSUB_PARA

: -- 标号

类型	语法结构
描述	用户过程标号，标号可以作为无参数的 SUB 过程来使用。
语法	Label Label: 标号名称，不能与现有的关键词冲突。
适用控制器	通用
例子	<pre> goto label1 END ' 主程序结束 label1: ' 加：定义标号 end </pre>

DMINS -- 数组链表插入

类型	语法指令
描述	数组的链表操作，插入后当前元素以及后面的所有元素往后移动一个位置。 谨慎对超长数组进行操作，特别是数组 TABLE。
语法	<pre> DMINS ArrayName(pos) ArrayName 数组名称 pos 数组索引 </pre>
适用控制器	通用

例子	<pre> DIM aa(6) ' 定义数组 aa FOR i=0 TO 4 ' 赋值 0, 1, 2, 3, 4 aa(i)=i NEXT ?*aa ' 打印数组所有元素 DMINS aa(0) ' 插入元素 0, 原有的所有元素都想后移动一个位置 aa(0) = 10 ' 为插入元素赋值 ?*aa ' 打印插入后的数组所有元素 </pre>
相关指令	DMDEL , DMCPY

DMDEL -- 数组链表删除

类型	语法指令
描述	数组的链表操作；删除数组元素，删除后当前元素后面的所有元素向前移动。 谨慎对超长数组进行操作，特别是数组 TABLE。
语法	<pre> DMDEL ArrayName(pos) ArrayName 数组名称 pos 数组索引 </pre>
适用控制器	通用
例子	<pre> DIM aa(6) ' 定义数组 aa FOR i=0 TO 4 ' 赋值 0, 1, 2, 3, 4 aa(i)=i NEXT ?*aa ' 打印数组所有元素 DMDEL aa(0) ' 删除数组 a 的第 1 个元素 ?*aa ' 打印删除后的数组所有元素 </pre>
相关指令	DMINS , DMCPY

DMCPY -- 数组拷贝

类型	语法指令
描述	数组拷贝，从数组 Src 拷贝到数组 Des。 谨慎对超长数组进行操作，特别是数组 TABLE。
语法	<pre>DMCPY ArrayDes(startpos) , ArraySrc(startpos)[, size] ArrayName 数组名称 startpos 数组起始索引 size 拷贝的个数，超过最大值会自动缩减</pre>
适用控制器	通用
例子	<pre>GLOBAL aa(6), bb(6) ' 定义数组 aa, bb FOR i=0 TO 4 ' 赋值 aa 0, 1, 2, 3, 4 aa(i)=i NEXT ?*aa ' 打印数组所有元素 ?*bb DMCPY aa(0), bb(0), 6 ' 把数组 bb 的值赋值给数组 aa ?*aa ' 打印数组复制后所有元素 ?*bb</pre>
相关指令	DMINS , DMDEL

IF -- 条件判断结构

类型	程序结构
描述	条件判断，同标准 BASIC 结构。
语法	<pre>IF <condition1> THEN commands ELSEIF <condition2> THEN commands ELSE commands ENDIF</pre> 参数

	<pre> condition1 条件 condition2 条件 </pre>
适用控制器	通用
例子	举例 1: <pre> dim a ' 定义变量 a=12 ' 赋值 if a>11 then ' 条件判断 trace "the val a is bigger then 11" elseif a<11 then trace "the val a is less then 11" endif </pre> 举例 2: <pre> if in(0) then out(0,on) ' 当单行时可以用不用 endif </pre>
相关指令	THEN , ENDIF

THEN -- 条件判断结构

类型	程序结构
描述	参见: IF
适用控制器	通用

ENDIF -- 条件判断结构

类型	程序结构
描述	参见: IF
适用控制器	通用

ELSEIF -- 条件判断结构

类型	程序结构
描述	参见: IF
适用控制器	通用

FOR -- for 循环结构

类型	程序结构
描述	循环语句，采用标准 BASIC 语法。
语法	<pre>FOR variable=start TO end [STEP increment] ... 'block of commands ... NEXT variable</pre> 参数： - variable 变量名称 - start 起始循环值 - end 结束循环值 - increment 循环步进增量，可选 多任务时，请不要使用(非 local 的)相同的 variable 循环变量，否则会相互干扰。
适用控制器	通用
例子	<pre>local a for a=1 to 100 '1 循环至 100 print a '打印 a next</pre>
相关指令	TO , STEP , NEXT

TO -- for 循环结构

类型	程序结构
描述	参见：FOR
适用控制器	通用

STEP -- for 循环结构

类型	程序结构
描述	参见：FOR

适用控制器	通用
-------	----

NEXT -- for 循环结构

类型	程序结构
描述	参见: FOR
适用控制器	通用

EXIT -- 退出循环

类型	程序结构
描述	循环退出语句。
语法	EXIT FOR , EXIT WHILE
适用控制器	通用
例子	<pre> local a for a=1 to 100 '1 循环至 100 print a '打印 a if a> 20 then EXIT for '必须采用这种方式，否则 IF 和 ENDIF 不匹配。 next </pre>

GOSUB/CALL -- SUB 调用

类型	程序结构
描述	SUB 过程调用，只能调用本文件的 SUB 过程或全局 SUB 过程。 直接调用 SUB 过程时，可以省掉 GOSUB 语句。 SUB 过程没有参数传递时，可以省掉括号 () GOSUB 后当前的内容会压栈，不能在调用的 SUB 程序中访问当前的局部变量，RETURN 返回时出栈。

语法	GOSUB/CALL label Label SUB 过程名
适用控制器	通用
例子	<pre> ' 主程序 Main: GOSUB sub1() sub2(1,2) ' 传入 1 给 para1, 2 给 para2 call sub3 END ' 定义的 SUB SUB sub1() a=100 print "sub1" return SUB sub2(para1, para2) a=200 print "sub2", para1, para2 return GLOBAL SUB sub3() ' 可以在另外一个程序文件中 a=300 print "sub3" return </pre>

GOTO -- 强制跳转

类型	程序结构
描述	强制跳转，与 GOSUB 的区别是 GOTO 不会压栈。
语法	GOTO label
适用控制器	通用
例子	<pre> a=100 goto label1 ' 强制跳转至 label1 a=1000 End ' 主程序结束 </pre>

	<pre> label1: print a ' 结果是 a=100 End ' label1 结束 </pre>
--	--

ON GOSUB -- 条件跳转

类型	程序结构
描述	当 expression 条件为真时，调用过程 label。
语法	<pre> ON expression GOSUB label Expression 判断条件 Label 跳转 sub、label 名 </pre>
适用控制器	通用
例子	<pre> a=100 on a>10 gosub label1 ' a>10 时调用 label 过程 a=1000 print a End ' 主程序结束 label1: print a return ' SUB 过程要返回 </pre>

ON GOTO -- 条件跳转 2

类型	程序结构
描述	条件跳转，当 expression 条件为真时跳转，不会压栈。
语法	<pre> ON expression GOTO label Expression 判断条件 Label 跳转 sub、label 名 </pre>
适用控制器	通用
例子	<pre> a=100 on a>10 goto label1 a=1000 </pre>

	End	' 主程序结束
	labell:	
	print a	
	end	' goto 跳转无法 return 返回。

REPEAT -- 条件循环

类型	程序结构
描述	循环语句。 循环执行 commands，condition 为真时退出循环。
语法	REPEAT commands UNTIL condition
适用控制器	通用
例子	<pre> a=0 repeat ' 循环执行下面语句 print a a=a+1 delay(1000) until IN(4)=ON ' 直到输入 4 有效 </pre>

WHILE -- while 循环结构

类型	程序结构
描述	条件满足时执行循环。
语法	<pre> WHILE condition ... WEND </pre>
适用控制器	通用
例子	<pre> a=0 while IN(4)=OFF ' 直到输入 4 有效，退出循环 a=a+1 print a delay(1000) </pre>

	wend
--	------

WEND -- while 循环结构

类型	程序结构
描述	参见: WHILE
适用控制器	通用

END SUB -- 自定义函数结束

类型	程序结构
描述	用户自定义 SUB 过程结束, 参见 SUB。
适用控制器	通用

RETURN -- 函数返回值

类型	程序结构
描述	用户 SUB 过程返回或返回值。 返回值缺省 0, 在外面可以通过 RETURN 来读取上个 SUB 调用的返回值。 不同任务的返回值不同。
语法	RETURN
适用控制器	通用
例子	<pre> Call sub1 ?return ' 结果为 111 End ' 主程序结束 sub sub1() return 111 ' 返回 111 end sub </pre>

UNTIL -- 条件结构

类型	程序结构
描述	参见: REPEAT WAIT
适用控制器	通用

DELAY -- 延时

类型	语法指令
描述	延时 delay time, 单位毫秒。 别名: wa
语法	DELAY(delay time) Delay time 毫秒数
适用控制器	通用
例子	DELAY(100) ' 延时 100ms

WAIT UNTIL -- 等待条件满足

类型	程序结构
描述	等待直到条件满足。
语法	WAIT UNTIL condition1 [and codition2 or codition3 ...] 可以通过逻辑运算操作多个条件
适用控制器	通用
例子	例一 WAIT UNTIL DPOS(0) > 0 ' 等待轴 0 位置超过 0 ... 例二 与 TICKS 一起使用 TICKS=2000 ' ticks 设为 2000 WAIT UNTIL TICKS<0 ' 等待 2s ?"执行下一步"

	例三 与逻辑条件一起使用 WAIT UNTIL IDLE(0)=-1 AND IDLE(1)=-1 AND IDLE(2)=-1 ’等待轴 0、1、2 都停止
--	--

WAIT IDLE -- 等待轴停止

类型	语法指令
描述	等待 BASE 轴运动完成，BASE 轴运动未完成时，不执行后面的程序。 等效于 WAIT UNTIL IDLE。IDLE 作为轴参数，支持轴参数的语法。
语法	WAIT IDLE
适用控制器	通用
例子	举例 1: <pre>Base(0,1) Move(100,100) Wait idle ’等待当前插补运动结束</pre> 举例 2: <pre>Base(0,1) Move(100,100) Base(2,3) Move(200,200) Wait until idle(0) and idle(1) and idle(2) and idle(3) ’等待轴 0, 1, 2, 3 停止</pre> ?”运动完成”

WAIT LOADED -- 等待轴缓冲空

类型	语法指令
描述	等待 BASE 轴运动缓冲空，此命令会阻塞不执行后面的程序。 缓冲区最后一个运动可以正确执行，同时程序向下扫描。 等效于 WAIT UNTIL LOADED。LOADED 作为轴参数，支持轴参数的语法。
语法	WAIT LOADED

适用控制器

通用

例子

例一 与 WAIT IDLE 的区别

BASE(0)

ATYPE=1

UNITS=100

DPOS=0

SPEED=100

ACCEL=1000

MERGE=1

TRIGGER

MOVE(100) ’ 当前运动

MOVE(50) ’ 缓冲运动，此时缓冲区只有这一条运动

’ 当本条运动执行时，缓冲区就已经清空

WAIT IDLE ’ 使用 WAIT IDLE 时，要等到所有运动完成才能往下执行

OP(0, ON) ’ 打开 op0

编号	数据源	偏移	刻度
0	DPOS	0	100
0	OP	0	1

1 DPOS[0]

2 OP[0]

Min:0.00

Max:150.00

Min:0.00

Max:1.00

200

400

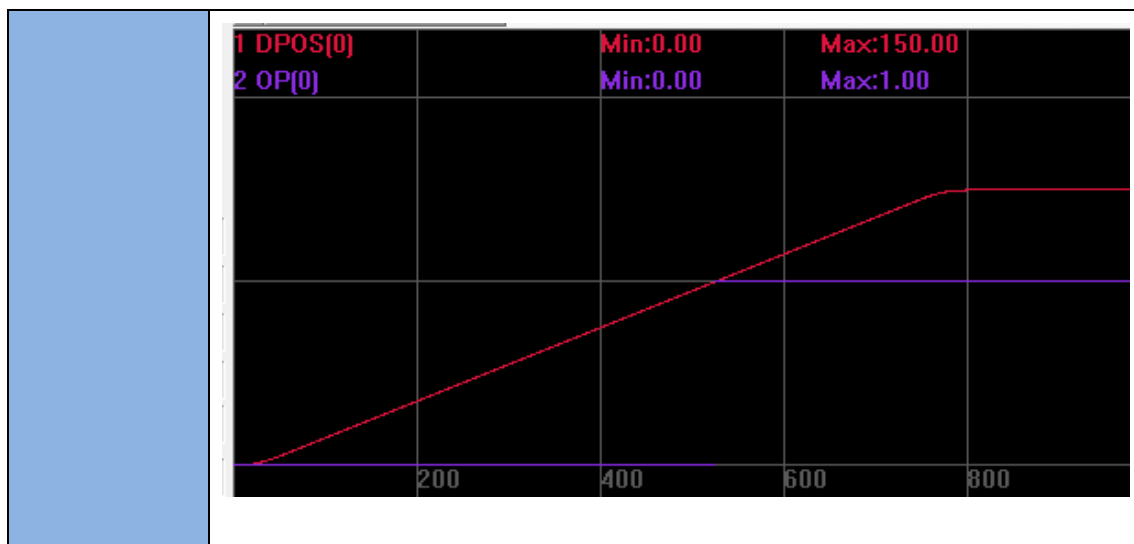
600

800

1000

WAIT LOADED ’ 使用 WAIT LOADED 时，运动缓冲空即可往下执行

OP(0, ON)



第四章 任务相关

ZBASIC 支持实时多任务运行，一个文件上也可以同时运行多个任务。通过 RUN 可以从文件第一行开始运行任务，通过 RUNTASK 可以随意指定 SUB 过程开始运行。

BASE_MOVE -- 指定主轴

类型	任务参数
描述	用于强制指定插补运动函数的 BASE 主轴，此参数不修改实际的运动。 缺省值为-1，此时不生效。 20160326 以上固件版本支持。 每个任务都有独立的 BASE_MOVE 参数。 MOVE MOVEABS MOVECIRC MOVE_OP MOVE_TASK 等插补类型函数有效，凸轮点位等单轴函数无效。
语法	VAR1 = BASE_MOVE, BASE_MOVE = value
适用控制器	通用
例子	BASE_MOVE=2 ' 强制后面本任务的插补运动使用轴 2 为主轴，速度参数也使用轴 2 的 MERGE (2)=ON ' 轴 2 启动连续 SPEED (2)=100 ACCEL (2)=1000

	BASE (0, 1) MOVE (100, 100) ' 轴 0/1 插补, 轴 2 也强制作为主轴参与, 但运动距离为 0 MOVE_OP (1, 1) BASE (1) MOVE (100) ' 1 轴运动 100, 也使用轴 2 为主轴 BASE_MOVE=-1 ' 取消本任务的强制主轴
--	--

RUN -- 启动文件任务

类型	任务指令
描述	新建一个任务来执行控制器上的一个文件。 重复启动同一任务会报错。 多任务操作指令有： END: 当前任务正常结束 STOP: 停止指定文件 STOPTASK 停止指定任务 HALT: 停止所有任务 RUN 启动文件执行 RUNTASK 启动任务在一个 SUB 上执行。
语法	RUN "filename" [, tasknum] filename 程序文件名, BAS 文件可不加扩展名。 tasknum 任务号, 缺省查找第一个有效的。
适用控制器	通用
例子	run "aaa", 1 ' 启动任务 1 运行 aaa.bas 文件
相关指令	RUNTASK

RUNTASK -- 启动 SUB 任务

类型	任务指令
描述	把一个 SUB 过程或是标号作为一个新的任务执行。 重复启动同一任务会报错。
语法	RUNTASK tasknum, label

	tasknum 任务号。 label 自定义 SUB 过程（不能带参数），或标号
适用控制器	通用
例子	<pre> runtask 1, taska '启动任务 1 来跟踪打印位置 MOVE(1000,100) MOVE(1000,100) end taska: '循环打印位置 while 1 print *mpos delay(1000) wend end </pre>
相关指令	RUN

FILE3_RUN -- 执行 FILE3 任务

类型	任务指令
描述	3 次程序文件的启动命令。 3 次程序文件是一种超大文件，可以动态加载，使用 basic 语法。不支持条件判断、跳转等操作。可以通过指令下载，也可以通过工具软件 zfile3view 进行浏览、上传和下载操作。
语法	<pre> FILE3_RUN "filename", tasknum filename 3 次程序的文件名，必须事先下载到控制器里面。 tasknum 任务号，缺省查找第一个有效的。 </pre>
适用控制器	必须带大容量存储的控制器和 2015 以上的固件版本支持。
例子	FILE3_RUN "aaa.z3p", 1 '在任务 1 运行 3 次文件 aaa.z3p
相关指令	FILE3_ONRUN

FILE3_ONRUN --FILE3 回调函数

类型	回调函数
----	------

描述	3 次文件启动时会自动调用。
语法	GLOBAL FILE3_ONRUN: 标号 GLOBAL SUB FILE3_ONRUN() 自定义 SUB 过程（不能带参数） 回调函数属于 3 次文件任务。
适用控制器	通用
例子	<pre>FILE3_RUN "aaa.z3p", 1 '在任务 1 运行 3 次文件 aaa.z3p END GLOBAL SUB FILE3_ONRUN() '3 次任务启动时自动调用 IF 1= PROCNUMBER THEN Base (0, 1, 2) '选择 3 次文件的运行轴列表 SPEED=1000 ACCEL=10000 ELSE Base (4, 5, 6) ENDIF end sub</pre>
相关指令	FILE3_RUN

FILE3_GOTO -- FILE3 强制跳转

类型	任务函数
描述	对三次任务有效，强制跳转到指定的行号开始运行。
语法	FILE3_GOTO(linenum) 参数： linenum 要跳转的行号，1 开始编号
适用控制器	通用
相关指令	FILE3_LINE , FILE3_RUN

FILE3_LINE -- FILE3 行号

类型	任务函数
描述	返回当前 3 次文件运行的行号，无论是否三次文件因为 SUB 调用

	进入 BASIC 文件，总是返回 3 次文件的行号。
语法	Value=FILE3_LINE([taskid]) 参数： Tasked 三次文件的任务号, 不填返回函数调用当前任务的
适用控制器	通用
相关指令	FILE3 RUN , FILE3 GOTO

END -- 结束

类型	任务指令
描述	当前任务结束。 当一个文件中有主程序和 SUB 过程时，一定要在主程序结束后写 END，如果不写 END，程序执行完主程序后，就会依次再执行 SUB 子程序。
适用控制器	通用
相关指令	RUN , RUNTASK

STOP -- 停止文件任务

类型	任务指令
描述	程序强制停止，操作文件。 当一个文件有多个任务时，建议用 STOPTASK 指令。 再启动任务前都要停止任务。
语法	STOP program name, [tasknum] program name 程序文件名, bas 文件可不用带扩展名. tasknum 任务号, 当程序文件有启动多个任务时, 缺省任务号最小的任务
适用控制器	通用
例子	RUN aaa, 1 ' 执行 aaa.bas STOP aaa, 1 ' 停止任务 1
相关指令	STOPTASK , HALT

STOPTASK -- 停止 SUB 任务

类型	任务指令
描述	任务强制停止，操作 SUB 和标记。 再启动任务前都要停止任务。
语法	STOPTASK [tasknum] tasknum 任务号，缺省当前任务
适用控制器	通用
例子	STOPTASK 2 ' 停止任务 2
相关指令	STOP , HALT

HALT -- 停止全部任务

类型	任务指令
描述	停止全部任务。
语法	HALT
适用控制器	通用
例子	HALT ' 停止所有任务
相关指令	STOP , STOPTASK

PAUSE -- 暂停全部任务

类型	任务指令
描述	暂停全部任务。 一般只有 PC 软件调用，断点断住后也会进入暂停状态。 暂停任务再恢复后，任务继续往下执行。
语法	PAUSE
适用控制器	通用
例子	PAUSE ' 暂停所有任务
相关指令	PAUSETASK

PAUSETASK -- 暂停指定任务

类型	任务指令
描述	暂停某一个任务。 暂停任务再恢复后，任务继续往下执行。
语法	<pre>PAUSETASK tasknum tasknum 任务号，缺省当前任务</pre>
适用控制器	通用
例子	<pre>PAUSETASK 1 ' 暂停任务 1</pre>
相关指令	PAUSE

RESUMETASK -- 恢复指定任务

类型	任务指令
描述	恢复某一个任务。 暂停任务再恢复后，任务继续往下执行。
语法	<pre>RESUMETASK tasknum tasknum 任务号，缺省当前任务</pre>
适用控制器	通用
例子	<pre>PAUSETASK 1 ' 暂停任务 1 RESUMETASK 1 ' 继续运行任务 1</pre>
相关指令	PAUSETASK

PROC -- 任务编号

类型	任务修正辅助指令
描述	当访问任务参数，任务状态时可以指定其他的任务。
语法	PROC(tasknum)

	tasknum 任务号 可以省略，参考 AXIS
适用控制器	通用
例子	例一 完整写法 Print PROC_STATUS PROC(1) '打印任务 1 运行状态 例二 简写 Print PROC_STATUS(1) '打印任务 1 运行状态
相关指令	PROCNUMBER

PROCNUMBER -- 当前任务编号

类型	任务特殊状态，系统状态
描述	当前任务的任务号。 当程序不知道当前任务号的时候，通过这个来访问。 这个状态不能通过 PROC 来修正。
语法	VAR1 = PROCNUMBER
适用控制器	通用
例子	Print PROCNUMBER '打印当前任务号 0
相关指令	PROC

ERROR_LINE -- 任务错误行号

类型	任务状态
描述	当前任务的错误行号。 一般只用在出错后，在线命令查看。
语法	VAR1 = ERROR_LINE(tasknum) tasknum 任务号
适用控制器	通用
例子	在线命令输入 >>ERROR_LINE(1) '打印任务 1 错误行
相关指令	PROC , ERROR_SET

PROC_LINE -- 任务行号

类型	任务状态
描述	任务的当前行号，只能获取其他任务的。
语法	VAR1 = PROC_LINE(tasknum) tasknum 任务号
适用控制器	通用
例子	Print proc_line (0) '打印任务 0 运行到那一行 在线命令输入 >>print PROC_LINE 输出：100
相关指令	PROC

PROC_STATUS -- 任务状态

类型	任务状态
描述	当前任务的状态。 0 任务停止 1 任务正在运行 3 任务暂停中
语法	VAR1 = PROC_STATUS(tasknum) Tasknum 任务号
适用控制器	通用
例子	Print PROC_STATUS(0) '打印任务 0 状态 在线命令输入 >>print PROC_STATUS(0) 输出：1
相关指令	PROC

RUN_ERROR -- 任务错误码

类型	任务状态
描述	任务中最先出现的错误编号。
语法	VAR1 = RUN_ERROR(tasknum) Tasknum 任务号
适用控制器	通用
例子	?* run_error(0) '打印任务 0 最先出现的错误号 2043
相关指令	ERROR LINE

TICKS -- 任务计数周期

类型	任务参数
描述	当前任务的计数周期数，每个周期减一。 每个任务都有独立的 TICKS 参数，ZMC00x 系列，ZMC1xx 系列的周期为 1 毫秒。 系统刷新周期修改不影响本 TICKS 的计时。
语法	VAR1 = TICKS, TICKS = value
适用控制器	通用
例子	ticks = 1000 wait until ticks < 0 '等待 ticks < 0 后 move(100)

第五章 运算符及数学函数

ZBASIC 支持标准 BASIC 的所有运算符，也采用标准 BASIC 的优先级。

优先级顺序：算术运算符 > 比较运算符 > 计算逻辑运算符。相同优先级的运算符采用从左到右的顺序计算。

算术运算符		比较运算符		逻辑运算符	
描述	符号	描述	符号	描述	符号
求幂	$\wedge$	等于	=	逻辑非	Not
负号	-	不等于	<>	逻辑与	And
乘	*	小于	<	逻辑或	Or 或
除	/	大于	>	逻辑异或	Xor
整除	\	小于等于	<=	逻辑等价	Eqv
求余	Mod 或 %	大于等于	>=		
加	+				
减	-				
左移位	<<				
右移位	>>				

+ -- 加法运算

类型	运算符
描述	将两个表达式相加。
语法	expression1 + expression2 expression1: 任意有效的表达式 expression2: 任意有效的表达式
适用控制器	通用
例子	在线命令输入 >>print 1+2 输出: 3

- -- 减法运算

类型	运算符
描述	将表达式 1 减去表达式 2。

语法	$\text{expression1} - \text{expression2}$ expression1 : 任意有效的表达式 expression2 : 任意有效的表达式
适用控制器	通用
例子	在线命令输入 >>print 2-(2-1) 输出: 1

* -- 乘法运算

类型	运算符
描述	将表达式 1 与表达式 2 相乘。
语法	$\text{expression1} * \text{expression2}$ expression1 : 任意有效的表达式 expression2 : 任意有效的表达式
适用控制器	通用
例子	在线命令输入 >>print 10*(1+2) 输出: 30

/ -- 除法运算

类型	运算符
描述	将表达式 1 除以表达式 2。
语法	$\text{expression1} / \text{expression2}$ expression1 : 任意有效的表达式 expression2 : 任意有效的表达式
适用控制器	通用
例子	在线命令输入

	<pre>>>print 10/3</pre> 输出: 3.3333
--	---

= -- 比较/赋值运算

类型	运算符
描述	比较运算符: 如果表达式 1 等于表达式 2, 返回 TRUE, 否则返回 FALSE 赋值运算符: 将表达式 2 赋值给前面的变量或参数等。
语法	<pre>expression1 = expression2</pre> expression1: 任意有效的表达式 expression2: 任意有效的表达式
适用控制器	通用
例子	举例 1: <pre>ON IN(0)=ON GOTO label1</pre> ’如果输入通道 0 为 ON, 程序跳转到标识"label1:"开始的行开始执行。 <pre>label1: Print 12 ’打印 12</pre> 举例 2: <pre>DIM aaa aaa = 100 ’VAR1 赋值为 100. PRINT aaa</pre>

<> -- 不等于

类型	运算符
描述	如果表达式 1 不等于表达式 2, 返回 TRUE, 否则返回 FALSE。
语法	<pre>expression1 <> expression2</pre> expression1: 任意有效的表达式 expression2: 任意有效的表达式

适用控制器	通用
例子	<pre>ON MODBUS_BIT(0)<>0 GOTO label1 ' 如果 MODBUS 位寄存器 0 非零值，程序跳转到标识"label1:"开始执行。 label1: Print 11 ' 打印 11</pre>

> -- 大于

类型	运算符
描述	如果表达式 1 大于表达式 2，返回 TRUE，否则返回 FALSE。
语法	<pre>expression1 > expression2</pre> expression1: 任意有效的表达式 expression2: 任意有效的表达式
适用控制器	通用
例子	<pre>WAIT UNTIL MPOS>100 该条语句将使程序循环等待，直到测量反馈位置大于 100 为止。</pre> 例一： <pre>Dim q ' 定义变量 q= 2>1 ' 2 大于 1，所以返回 TRUE print q ' 打印返回值</pre> 例二： <pre>dim a ' 定义变量 a=0 ' 变量赋值 repeat ' 循环执行 a=a+1 ' 加一 ?a ' 打印 DELAY(200) ' 延时 until a>10 ' 条件判断，直到 a 里面的值大于 10 时，停止循环执行</pre>

>= -- 大于等于

类型	运算符
描述	如果表达式 1 大于或等于表达式 2，返回 TRUE，否则返回 FALSE。
语法	expression1 >= expression2 expression1: 任意有效的表达式 expression2: 任意有效的表达式
适用控制器	通用
例子	dim a ' 定义变量 a= 1>=3 ' 1 小于 3，所以返回 FALSE print a ' 打印返回值

< -- 小于

类型	运算符
描述	如果表达式 1 小于表达式 2，返回 TRUE，否则返回 FALSE。
语法	expression1 < expression2 expression1: 任意有效的表达式 expression2: 任意有效的表达式
适用控制器	通用
例子	VAR1=1<0 因为 1 大于 0，所以 VAR1 的值将等于 FALSE(0)。

<= -- 小于等于

类型	运算符
描述	如果表达式 1 小于或等于表达式 2，返回 TRUE，否则返回 FALSE。

语法	<code>expression1 <= expression2</code> <code>expression1:</code> 任意有效的表达式 <code>expression2:</code> 任意有效的表达式
适用控制器	通用
例子	<code>VAR1=1<=1</code> VAR1 的值将等于 TRUE (-1)。

>> -- 右移位

类型	运算符
描述	右移位。
语法	<code>expression1 >> expression2</code> <code>expression1:</code> 任意有效的表达式 <code>expression2:</code> 任意有效的表达式 运算优先级低于其他四则运算符，共同使用时需要加括号()，见例三。
适用控制器	通用
例子	例一 直接操作数值 在线命令输入 <code>>>print 8>>1</code> '二进制右移一位 输出: 4 在线命令输入 <code>>>print 8>>2</code> '二进制右移两位 输出: 2 例二 操作变量、寄存器 <code>dim bb</code> <code>bb=8</code> <code>Modbus_reg(0)=8</code> <code>print bb>>1,bb>>2</code> <code>print Modbus_reg(0)>>1,Modbus_reg(0)>>2</code> 例三 优先级比较 <code>?print 8>>1+1</code> '此时二进制右移 2 位

	输出：2 ?print (8>>1)+1 ' 此时二进制右移 1 位 输出：5
--	--

<< -- 左移位

类型	运算符
描述	左移位。
语法	expression1 << expression2 expression1: 任意有效的表达式 expression2: 任意有效的表达式 运算优先级低于其他四则运算符，共同使用时需要加括号()，见例三。
适用控制器	通用
例子	例一 直接操作数值 在线命令输入 >>print 8<<1 ' 二进制左移一位 输出：16 在线命令输入 >>print 8<<2 ' 二进制左移两位 输出：32 例二 操作变量、寄存器 dim bb bb=8 Modbus_reg(0)=8 print bb<<1,bb<<2 print Modbus_reg(0)<<1,Modbus_reg(0)<<2 例三 优先级比较 ?print 8<<1+1 ' 此时二进制左移 2 位 输出：32 ?print (8<<1)+1 ' 此时二进制左移 1 位 输出：17

\ -- 整除运算

类型	运算符
描述	整数除法。
语法	expression1 \ expression2 expression1: 任意有效的表达式 expression2: 任意有效的表达式
适用控制器	通用
例子	在线命令输入 >>print 10 \ (1+2) 输出: 3

\$ -- 16 进制

类型	特殊字符
描述	表示紧接着的数据是 16 进制格式。
语法	\$hexnum
适用控制器	通用
例子	在线命令输入 >>print \$F 输出: 15

AND -- 按位与

类型	运算符				
描述	按位与操作符，只操作整数部分。 <table border="1"> <thead> <tr> <th>AND</th><th>结果</th></tr> </thead> <tbody> <tr> <td>0 0</td><td>0</td></tr> </tbody> </table>	AND	结果	0 0	0
AND	结果				
0 0	0				

	<table><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td><td>1</td></tr></table>	0	1	0	1	0	0	1	1	1
0	1	0								
1	0	0								
1	1	1								
语法	expression1 AND expression2 expression1: 任意有效的表达式 expression2: 任意有效的表达式 expression1 和 expression2 的二进制形式的每一个位上的二进制数字进行按位与（AND）运算之后的结果									
适用控制器	通用									
例子	在线命令输入 >>print 1 and 2 输出：0 具体操作过程 1 对应二进制 01 2 对应二进制 10 与计算后 对应二进制 00 输出十进制 0									

OR -- 按位或

类型	运算符										
描述	按位或操作符，只操作整数部分。 <table border="1"> <tr> <th>OR</th><th>结果</th></tr> <tr><td>0</td><td>0</td></tr> <tr><td>0</td><td>1</td></tr> <tr><td>1</td><td>0</td></tr> <tr><td>1</td><td>1</td></tr> </table>	OR	结果	0	0	0	1	1	0	1	1
OR	结果										
0	0										
0	1										
1	0										
1	1										

语法	expression1 OR expression2 expression1: 任意有效的表达式 expression2: 任意有效的表达式 expression1 和 expression2 的二进制形式的每一个位上的二进制数字进行按位或（OR）运算之后的结果
适用控制器	通用
例子	在线命令输入 <pre>>>print 1 or 2</pre> 输出: 3 具体操作过程 1 对应二进制 01 2 对应二进制 10 或计算后 对应二进制 11 输出十进制 3

NOT -- 按位非

类型	运算符						
描述	按位非操作符，只操作整数部分，小心对 0N 等整数 NOT。 <table border="1"> <thead> <tr> <th>NOT</th><th>结果</th></tr> </thead> <tbody> <tr> <td>0</td><td>1</td></tr> <tr> <td>1</td><td>0</td></tr> </tbody> </table>	NOT	结果	0	1	1	0
NOT	结果						
0	1						
1	0						
语法	NOT expression1 expression1: 任意有效的表达式						
适用控制器	通用						
例子	在线命令输入 <pre>>>print NOT 1</pre> 输出: -2 具体操作过程						

	1 对应二进制 ... 0000 0001 非计算后 对应二进制 ... 1111 1110 输出十进制 -2
--	--

XOR -- 按位异或

类型	运算符										
描述	逻辑异或操作符，按位异或，只操作整数部分。 <table border="1"> <thead> <tr> <th>XOR</th><th>结果</th></tr> </thead> <tbody> <tr> <td>0 0</td><td>0</td></tr> <tr> <td>0 1</td><td>1</td></tr> <tr> <td>1 0</td><td>1</td></tr> <tr> <td>1 1</td><td>0</td></tr> </tbody> </table>	XOR	结果	0 0	0	0 1	1	1 0	1	1 1	0
XOR	结果										
0 0	0										
0 1	1										
1 0	1										
1 1	0										
语法	expression1 XOR expression2 expression1: 任意有效的表达式 expression2: 任意有效的表达式										
适用控制器	通用										
例子	在线命令输入 <pre>>>print 1 xor 1</pre> 输出: 0 具体操作过程 1 对应二进制 01 异或计算后 对应二进制 00 输出十进制 0										

EQV -- 按位同或

类型	运算符															
描述	按位同或操作符，只操作整数部分。 <table><tr><th colspan="2">EQV</th><th>结果</th></tr><tr><td>0</td><td>0</td><td>1</td></tr><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td><td>1</td></tr></table>	EQV		结果	0	0	1	0	1	0	1	0	0	1	1	1
EQV		结果														
0	0	1														
0	1	0														
1	0	0														
1	1	1														
语法	expression1 EQV expression2 expression1: 任意有效的表达式 expression2: 任意有效的表达式															
适用控制器	通用															
例子	在线命令输入 >>print 2 eqv 1 输出：-4 具体操作过程 2 对应二进制 ... 0000 0010 1 对应二进制 ... 0000 0001 同或计算后 对应二进制 ... 1111 1100 输出十进制 -4															

MOD -- 求余数

类型	运算符
描述	求余数。
语法	expression1 MOD expression2 expression1: 任意有效的表达式，取整数部分。 expression2: 任意有效的表达式，取整数部分。

适用控制器	通用
例子	在线命令输入 >>print 10 MOD (1+2) 输出: 1

ABS -- 绝对值

类型	数学函数
描述	求绝对值。
语法	ABS(expression) expression 任意有效的表达式
适用控制器	通用
例子	Print ABS(-11) ' 结果是 11

TAN -- 三角函数正切

类型	数学函数
描述	求正切三角函数，输入参数为弧度单位。
语法	TAN(expression) expression 任意有效的表达式
适用控制器	通用
例子	Print tan(pi/3) ' 结果是 1.732

ATAN -- 三角函数反正切

类型	数学函数
描述	求反正切三角函数，返回值为弧度单位。
语法	ATAN(expression) expression 任意有效的表达式

适用控制器	通用
例子	<code>print atan(1)</code> ' 结果是 $0.7854 = (45/180)*\pi$

ATAN2 -- 三角函数反正切 2

类型	数学函数
描述	反正切三角函数，返回值为弧度单位。
语法	<code>ATAN2(y, x)</code> y: y 坐标 x: x 坐标
适用控制器	通用
例子	<code>print atan2(1,0)</code> ' 结果是 1.5708

ASIN -- 三角函数反正弦

类型	数学函数
描述	反正弦三角函数，返回值为弧度单位。
语法	<code>ASIN(expression)</code> expression 任意有效的表达式
适用控制器	通用
例子	<code>print asin(0.5)</code> ' 结果是 0.52360

SIN -- 三角函数正弦

类型	数学函数
描述	正弦三角函数，输入参数为弧度单位。

语法	SIN(expression) expression 任意有效的表达式
适用控制器	通用
例子	print sin(pi/6) ' 结果是 0.5000

COS -- 三角函数余弦

类型	数学函数
描述	余弦三角函数，输入参数为弧度单位。
语法	COS(expression) expression 任意有效的表达式
适用控制器	通用
例子	print cos(pi/3) ' 结果是 0.5000

CRC16 -- CRC 检验计算

类型	数学函数
描述	CRC16 CCITT 计算。
语法	CRC16(arrayname, index, size[, initial] [, poly]) arrayname 数据存储所在的数组，一个字节占一个位置。 Index 数据存储所在的数组起始索引。 Size 计算字节数。 Initial CRC 计算初始值，缺省\$FFFF Poly 多项式，暂时只支持 modbus 的\$A001 和 CCITT 的\$1021，缺省\$A001
适用控制器	通用
例子	TABLE(0, \$FE, \$48, \$06, \$00, \$6D, \$00, \$00, \$00) ' 8 个数据存储存储在 TABLE CRCVALUE = CRC16(TABLE, 0, 8) ' 计算 CRC，结果\$1A0D TABLE(8) = CRCVALUE \ 256 ' 计算的 CRC 加在数据的后面，大端模式

	TABLE(9)= CRCVALUE and \$ff
--	-----------------------------

ACOS -- 三角函数反余弦

类型	数学函数
描述	反余弦三角函数，返回值为弧度单位。
语法	ACOS(expression) expression 任意有效的表达式
适用控制器	通用
例子	print acos(0.5) '结果是 1.04720=PI/3

EXP -- 指数

类型	数学函数
描述	指数函数。
语法	exp([base,] expvalue) base 底数，缺省为 e expvalue 指数
适用控制器	通用
例子	举例 1: print exp(2,4) '结果是 16 (2*2*2*2) 举例 2: print exp(1) '结果是 2.7183

SQR -- 平方根

类型	数学函数
描述	平方根函数。

语法	SQR(expression) expression 任意有效的表达式
适用控制器	通用
例子	a= sqr(4) print a '结果是 2

LN -- 自然对数

类型	数学函数
描述	自然对数函数。
语法	LN(expression) expression 任意有效的表达式
适用控制器	通用
例子	a= ln(1) print a '结果是 0

LOG -- 对数底为 10

类型	数学函数
描述	对数，底数为 10。
语法	LOG(expression) expression 任意有效的表达式
适用控制器	通用
例子	a= log(100) print a '结果是 2

CLEAR_BIT -- 按位置 0

类型	数学指令或函数
描述	位操作，只对整数，修改对应位为 0。 分为命令语法和函数语法。
语法	命令语法：CLEAR_BIT(bit#,vr#) 直接操作 VR 寄存器 bit#, 位编号：0-31 vr# 要操作的 VR 变量编号，整数部分 命令语法使用时没有返回值，直接操作，修改操作对象的值。 函数语法：ret = CLEAR_BIT(bit#,int) Ret 操作结果 bit# 位编号：0-31 int 要操作的表达式，取整数部分 函数语法使用时返回操作后的结果，操作对象的值不变。
适用控制器	通用
例子	举例 1：命令语法 <pre>vr(23)=3.333 clear_bit (0,23) 'VR(23)的第 0 位将被清除(设置为 0) ?vr(23) '打印结果 2，小数被去除</pre> 举例 2：函数语法 <pre>Dim a,b a=3.333 b=0 b= clear_bit(0,a) '返回清除 a 的第 0 位和小数后的结果给 b Print a,b '结果为 3.333,2 a 不变,b 为 2</pre>

READ_BIT -- 按位读取

类型	数学函数
描述	位操作，只对整数，读取对应位状态。 只能操作 VR 寄存器，非 VR 参考 READ_BIT2
语法	Ret = READ_BIT(bit#, vr#) Ret 读取结果：1 或 0

	bit# 位编号：0-31 vr# 要操作的 VR 变量编号
适用控制器	通用
例子	vr(23)=3.333 Print read_bit(0,23) ' 读取 vr(23) 的第 0 位，结果为 1

READ_BIT2 -- 按位读取 2

类型	数学函数，20130813 以后的固件版本提供了支持。
描述	位操作，只对整数，读取对应位状态。
语法	Ret = READ_BIT2(bit#, int) Ret 读取结果：1 或 0 bit#, 位编号：0-31 int 要操作的表达式，取整数部分
适用控制器	通用
例子	dim a,b b=1.64 a=read_bit2(0,b) ' 读取 b 的第 0 位, 赋值给 a print a ' 输出 a 值，结果为 1

SET_BIT -- 按位设置

类型	数学指令或函数
描述	位操作，只对整数，修改对应位为 1。 分为命令语法和函数语法。
语法	命令语法：SET_BIT(bit#, vr#) 直接操作 VR 寄存器 bit#, 位编号：0-31 vr# 要操作的 VR 变量编号，整数部分 命令语法使用时没有返回值，直接操作， 修改操作对象的值。 函数语法：ret = SET_BIT(bit#,int) Ret 操作结果

	bit#, 位编号: 0-31 int 要操作的表达式, 取整数部分 函数语法使用时返回操作后的结果, 操作对象的值不变。
适用控制器	通用
例子	举例 1: 命令语法 <pre>vr(23)=0.333 set_bit (0,23) 'VR(23)的第0位将置为1, 并清除小数部分 ?vr(23) '结果为 1</pre> 举例 2: 函数语法 <pre>Dim a,b a=0.333 b=0 b=set_bit (0,a) '设置 a 的第0位的, 结果赋值到 b, 并清除小数 Print a,b '打印结果 0.333, 1 a 没有改变, b 为 1</pre>

FRAC -- 返回小数

类型	数学函数
描述	返回小数部分, 总是大于 0 的部分。
语法	FRAC(expression) expression 要操作的数
适用控制器	通用
例子	<pre>a=frac(1.235) print a '结果是 0.235</pre>

INT -- 返回整数

类型	数学函数
描述	返回整数部分。
语法	INT(expression) expression 任意有效的表达式

适用控制器	通用
例子	<pre>a=int(1.235) print a '结果是1 ?int(-1.1) '打印结果，-2，因为小数部分总处理为正数。</pre>

SGN -- 返回符号

类型	数学函数
描述	返回符号。 1 大于 0 0 等于 0 -1 小于 0
语法	<pre>SGN(expression) expression 任意有效的表达式</pre>
适用控制器	通用
例子	<pre>a=SGN(-1.235) print a '结果是-1</pre>

TURN_POSMAKE -- 旋转中点计算

类型	数学函数
描述	旋转功能的中点计算使用函数,旋转的正向与XY的正向要一致(右手法则)。
语法	<pre>TURN_POSMAKE(tablenum, posx, posy, disR, tableout) tablenum 旋转功能参数存储 table 编号。 posx X 方向的坐标。 posy Y 方向的坐标。 disR 旋转轴的相对偏移。 tableout 存储计算后的坐标。</pre>
适用控制器	通用
相关指令	MCIRC_TURNABS

B_SPLINE -- B 样条平滑

类型	数学函数
描述	将 TABLE 中的数据进行 B 样条平滑。
语法	B_SPLINE(type, data_start, points, data_out, ratio) type 类型，目前只支持 1-B 样条。 data_start 图形数据在 TABLE 中的起始位置。 points 图形数据的个数。 data_out 平滑后的图形数据在 TABLE 中起始位置。 ratio B_SPLINE 函数的平滑比率。平滑后的个数为 points * ratio.
适用控制器	通用
例子	B_SPLINE(1, 0, 10, 100, 10) ’平滑一个 10 点的图形数据，源图形点在 Table 的位置为从 0 到 9，平滑为 100 点的数据，并且平滑后的数据从 Table 地址 100 开始存放。

DTSMOOTH -- table 平滑

类型	数学函数
描述	对 TABLE 存储的点坐标进行平滑调整。
语法	DTSMOOTH (axes, dtfirst, space, points, imode, referradius) axes 轴数 Dtfirst 第一个点的 TABLE 索引 Space 两个点之间的索引间隔(就是一个点存储占用的空间). points 总点个数. imode 0- 绝对方式，对曲率半径小于参考值的进行调整. Referradius 参考曲率半径，可以根据 半径=(速度平方)/拐弯加 速度 来计算参考
适用控制器	3X 系列 20161206 以上固件支持. 4 系列 20170508 以上固件支持
例子	TABLE(0, 0, 0) TABLE(5, 99, 0) TABLE(10, 100, 0) TABLE(15, 100, 1)

	TABLE(20, 101, 1) TABLE(25, 200, 1) DTSMOOTH(2, 0, 5, 6, 0, 5) ?*TABLE(0, 2) ?*TABLE(5, 2) ?*TABLE(10, 2) ?*TABLE(15, 2) ?*TABLE(20, 2) ?*TABLE(25, 2)
--	--

DMADD -- 数组批量增加

类型	数学函数
描述	对数组元素值进行批量增加。 不要一次修改超过 500 元素。
语法	DMADD ArrayName (pos, size , data) ArrayName 数组名 pos 起始的索引 size 要修改的个数，注意加上 pos 不要超过数组大小 data 要增加的值
适用控制器	通用
例子	dim aaa(20) ' 定义一个空间为 20 的数组 ?*aaa ' 打印，全为 0 DMADD aaa(10,5,2) ' 从元素 10 开始，修改 5 个元素值+2 ?*aaa ' 打印，其中 10、11、12、13、14 为 2，其余为 0 DMADD aaa(10,5,2) ' 从元素 10 开始，修改 5 个元素值+2 ?*aaa ' 打印，其中 10、11、12、13、14 为 4，其余为 0

CHR -- ASCII 码打印

类型	字符串函数
----	-------

描述	返回 ASCII 打印，只用于 PRINT。
语法	CHR(expression) expression 任意有效的表达式
适用控制器	通用
例子	在线命令输入 >>PRINT CHR(66) 输出: B

TOSTR -- 格式化输出

类型	字符串函数
描述	格式化输出函数。 变量转换成字符串。
语法	TOSTR(VAR1, [N], [DOT]) VAR1 任意有效的表达式 N 输出总位数，包括小数点和符号位。当 N 设置负数时，表示右对齐。 DOT 输出的小数个数，当 N 太小没有小数位置时，不输出小数。 输出的是字符串类型。
适用控制器	通用
例子	例一 在线命令输入 >> print tostr(2-100,6,2) 输出: -98.00 例二 Dim aa(20) Aa="asd13"+tostr(354) ?aa ' 打印结果 asd13354.000

HEX -- 16 进制打印

类型	字符串函数
描述	返回十六进制格式，只用于 PRINT。
语法	HEX (expression) expression 任意有效的表达式，只取整数部分。
适用控制器	通用
例子	在线命令输入 >>PRINT HEX (15); 输出: f '十六进制

STRCOMP -- 字符串比较

类型	字符串函数
描述	字符串比较函数，根据两个字符串的情况返回 >0 =0 <0。 比较长度在 500 字节内，否则返回值出错误。
语法	STRCOMP (str1, str2) str1 字符串 1 Str2 字符串 2
适用控制器	通用
例子	DIM AAA(10) AAA = "abc" 在线命令输入 >>PRINT strcmp(AAA, "abc") 输出: 0

STRFIND -- 字符串搜索

类型	字符串函数
描述	字符串搜索函数。
语法	STRFIND(str1, str2 [, firstindex])

	str1 待搜索的字符串 str2 搜索模板字符串 Firstindex 从那个位置开始搜索，缺省 0 返回： >= 0, 返还查找到的 index < 0 没有找到
适用控制器	通用
例子	DIM AAA(10), BBB(3) AAA="AD23GF41" BBB="23G" ?STRFIND(AAA, BBB) '打印搜索到的索引位置，2

VAL -- 字符转数值

类型	字符串函数
描述	字符串转换为数值。 只能转换数字字符，遇到字母、符号字符停止。
语法	VAL(str1) str1 字符串
适用控制器	通用
例子	例一 VAR1 = VAL("123") ?VAR1 '打印结果，123 例二 VAR2 = VAL("123QWE23") ?VAR2 '打印结果，123

IEEE_IN -- 组合浮点数

类型	数学函数
描述	把四个字节组合成一个单精度浮点数。

语法	IEEE_IN(byte0,byte1,byte2,byte3) byte0 - byte3 四个字节
适用控制器	通用
例子	VAR = IEEE_IN(VR(10),VR(11),VR(12),VR(13))

IEEE_OUT -- 提取单字节

类型	数学函数
描述	从一个单精度浮点数里面取一个字节。
语法	byte_n = IEEE_OUT(VAR, n) VAR 单精度浮点数 N 0-3, 取第几个字节
适用控制器	通用
例子	VAR = IEEE_OUT(VR(1),2)

PI -- 圆周率

类型	常数
值	3.14159
适用控制器	通用

TRUE -- 真值

类型	常数
值	-1
适用控制器	通用

FALSE -- 假值

类型	常数
值	0
适用控制器	通用

ON -- 开启

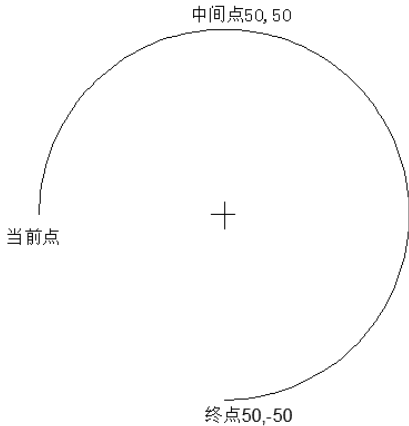
类型	常数
值	1
适用控制器	通用

OFF -- 关闭

类型	常数
值	0
适用控制器	通用

ZCUSTOM -- 运动参数计算

类型	数学函数
描述	计算各种运动指令中的参数，详细功能请看下方语法功能描述。
语法	功能 2：计算空间圆弧或直线上一定距离后的点的位置。 语法：zcustom (2, tableend, tablemid, tableOut, mode, vectdis) Table 表参数按三点画圆模式填写其他两个点参数。 填写相对坐标，返回也是相对的位置。 Tableend：存储圆弧终点的 table 索引。 Tablemid：存储圆弧中间点的 table 索引，与当前点一起构成圆弧的 3 个点。 tableOut： 输出计算数据的 table 索引。 mode = 1 相对起点的空间圆弧弧长。 2 相对终点的空间圆弧弧长。 mode = 3 相对起点的直线距离，此时 tablemid 不起作用。 4 相对终点的直线距离，此时 tablemid 不起作用。 mode = 5 相对计算空间圆弧的圆心，此时 vectdis 无效。

	此时 tableout 依次输出 xyz 圆心，弧度范围， 弧长。 vectdis: 要计算的点相对 mode 的距离，负数表示往前。 圆弧模式时，正数表示顺时针，负数表示逆时针。 见例一使用。 功能 6: 计算空间圆弧的起点和终点的切线角度方向，弧度单位。 语法: zcustom (2, tableend, tablemid, tableOut) 填写相对坐标，返回也是相对的位置。 Tableend: 存储圆弧终点的 table 索引。 Tablemid: 存储圆弧中间点的 table 索引，与当前点一起构成圆弧的 3 个点。 tableOut: 输出计算数据的 table 索引，依次输出 起点 xy 方向，起点 Z 方向，终点 xy 方向，终点 Z 方向，角度弧度单位。 见例二使用。 功能 8: MOVESLINK 输入从轴位置，反算主轴。 语 法: zcustom (8, distance, link dist, start sp, end sp, distancemoved, TableOut) Distance: 从连接开始到结束，跟随轴移动的距离。 link dist: 从连接开始到结束，主轴移动的绝对距离。 start sp: 起始脉冲速度比例。 end sp: 结束脉冲速度比例。 distancemoved: 从轴已经运动距离。 TableOut: 输出 Table 索引，输出对应主轴的位置，如多个解返回第一个。 见例三使用。
适用控制器	通用
例子	例一 功能 2 使用  用平面圆弧做演示 左图圆半径 50 圆心 50,0 弧长 235.62 table (0, 50, -50, 0) ' 设置终点坐标，相对位置 table (3, 50, 50, 0) ' 设置中间点坐标，相对位置

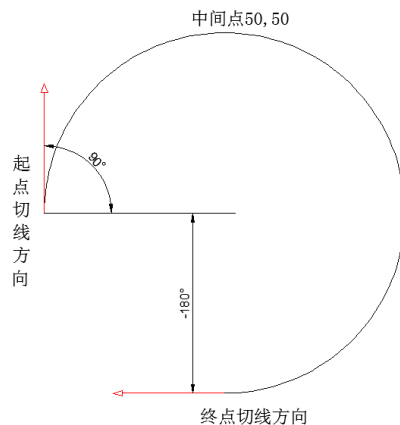

```

' 模式 1, 相对起点
zcustom(2, 0, 3, 10, 1, 78.54) ' 相对起点位置顺时针弧长为 78.54 的圆上点
?table(10), table(11), table(12) ' 输出相对坐标为 50, 50, 0
zcustom(2, 0, 3, 10, 1, -78.54) ' 相对起点位置逆时针弧长为 78.54 的圆上点
?table(10), table(11), table(12) ' 输出相对坐标为 50, -50, 0

' 模式 2, 相对终点
zcustom(2, 0, 3, 10, 2, 78.54) ' 相对终点位置顺时针弧长为 78.54 的圆上点
?table(10), table(11), table(12) ' 输出坐标为 50, -50, 0
zcustom(2, 0, 3, 10, 2, -78.54) ' 相对终点位置逆时针弧长为 78.54 的圆上点
?table(10), table(11), table(12) ' 输出坐标为 100, 0, 0

' 模式 5, 计算此时三点画圆的圆心、弧度、弧长
zcustom(2, 0, 3, 10, 5, 0) ' 此时距离参数不起作用
?table(10), table(11), table(12) ' 输出坐标为 50, -50, 0
?table(13), table(14) ' 输出弧度 4.712, 弧长 235.62
    
```

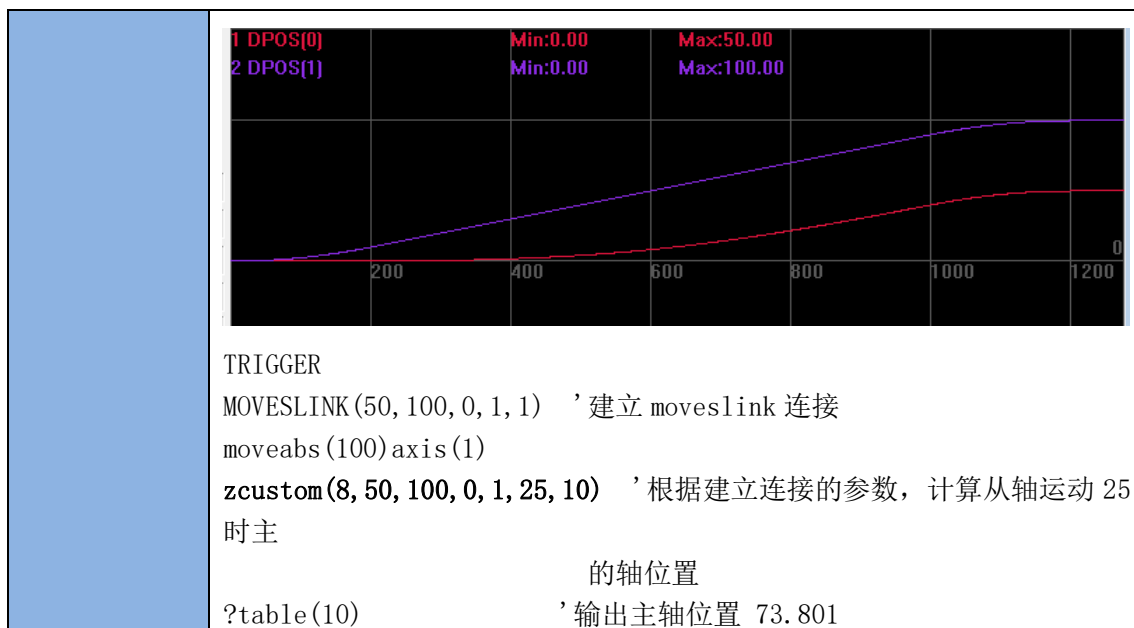
例二 功能 6, 计算圆弧起点和终点的切线弧度



```

table(0, 50, -50, 0) ' 设置终点坐标, 相对位置
table(3, 50, 50, 0) ' 设置中间点坐标, 相对位置
zcustom(6, 0, 3, 10) ' 计算当前点和终点切线的弧度
?table(10), table(11) ' 输出当前点 1.571, 0 1.571=90*pi/180
?table(12), table(13) ' 输出终点 -3.142, 0 -3.142=-180*pi/180
    
```

例三 功能 8, 反算 MOVESLINK 主轴位置



第六章 轴参数与轴状态

轴参数修改语法: `speed = value` 'BASE 轴。也可以通过 `speed axis(axisnum) = value` 来强制取特定轴, 其中 `axis` 可以省掉, 变为 `speed(axisnum)=value`。

可以通过 `SPEED=value1, vlaue2...` 来同时设置多个 BASE 轴的参数。

轴参数可写可读, 读取时 `axis` 也可以省掉: `var1 = speed(axisnum)`。

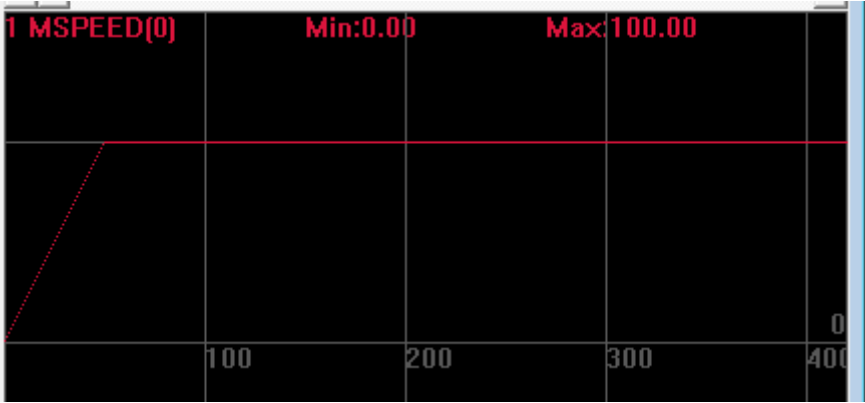
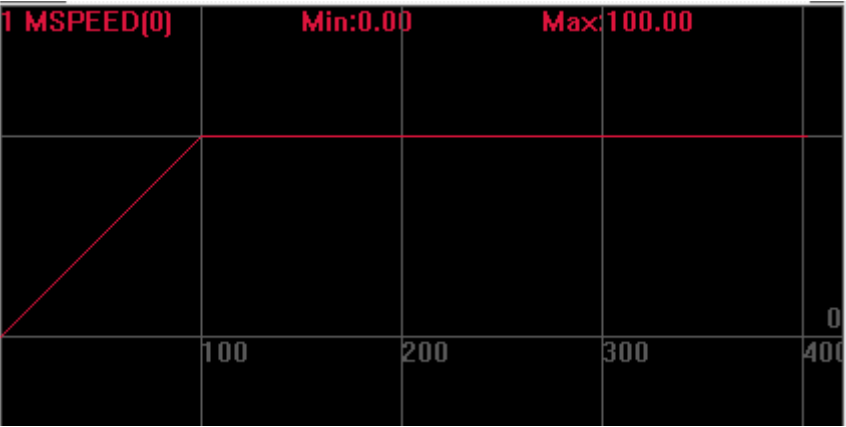
轴状态一般是只读的, 值会随时内部变动, MPOS, DPOS 等可以直接修改的除外。



当进行插补运动时, 主轴(BASE 轴)的参数作为插补的参数。

ACCEL -- 加速度

类型	轴参数
描述	轴加速度, 单位为 units/s/s。 当多轴运动时, 插补运动的加速度为主轴加速度。 建议运动前设置好加速度和减速度, 运动中不要修改, 运动中调整会导致速度曲线变化。
语法	可读: <code>VAR1 = ACCEL(轴号)</code> 可写 <code>ACCEL(轴号) = expression</code>
适用控制器	通用

例子	例一 设置方式 BASE 选择轴设置 BASE (1, 2, 3, 4) ACCEL=100, 100, 100, 100 ’ 设置轴 1, 2, 3, 4 的加速度 直接加轴号设置 ACCEL (2)=200 ’ 设置轴 2 的加速度 例二 BASE (0) ACCEL=2000 SPEED=100 MOVE (100) 加速过程  ACCEL=1000 时 
相关指令	DECEL, SPEED

ADDAX_AXIS -- 叠加轴号

类型	轴状态
描述	ADDAX 指令所叠加轴的轴号，-1 表示没有叠加。
语法	VAR1 = ADDAX_AXIS
适用控制器	通用
例子	ADDAX(0) AXIS(1) ' 轴 0 的运动叠加到轴 1 ?ADDAX_AXIS(1) ' 打印出轴 1 叠加的轴，结果，0 ADDAX(-1) AXIS(1) ' 取消叠加
相关指令	ADDAX

ALM_IN -- 映射报警输入

类型	轴参数
描述	驱动器告警对应的输入口编号，-1 无效。 控制器报警信号生效后，会立即停止轴，停止减速度为 FASTDEC。 设置了告警输入口后，ZMC 控制器缺省为 OFF 有效，常开信号用 INVERT_IN 反转电平；ECI 控制器缺省为 ON 有效，常闭信号用 INVERT_IN 反转电平
语法	VAR1 = ALM_IN, ALM_IN = expression
适用控制器	通用
例子	BASE(0, 1) ALM_IN = 10, 11 ' 将轴 0 告警信号定义到输入口 10，轴 1 定义到 11 INVERT_IN(10, ON) ' 反转电平开启 INVERT_IN(11, ON)
相关指令	DATUM_IN , FWD_IN , REV_IN , INVERT_IN , FASTDEC

ATYPE -- 轴类型

类型	轴参数
描述	轴功能类型设置，只能设置为轴具备的特性（轴特性可查询硬件手册或使用 ZDevelop 软件连接上控制器以后查看控制器状态）。

	最好是在程序初始化的时候就设置好 ATYPE。 ZCAN 扩展轴要先设置 AXIS_ADDRESS，并在设置后延迟 2 个 tick 再调用运动指令，受总线带宽限制，ZCAN 扩展轴不要设置超过 2 个。 对部分产品型号带有独立的编码器，可以使用相应虚拟轴来做编码器轴使用，例如 ZMC206 的电机轴为 0-5 轴，编码器可以通过轴 6-11 来控制，详细可通过 ZDevelop 软件连接上控制器以后查看控制器状态。																																								
语法	VAR1 = ATYPE, ATYPE = expression <table border="1"> <thead> <tr> <th>Atype 类型</th><th>描述</th></tr> </thead> <tbody> <tr> <td></td><td>电机模式 INVERT_STEP 指令设置，默认脉冲方向</td></tr> <tr> <td>0</td><td>虚拟轴</td></tr> <tr> <td>1</td><td>脉冲方向方式的步进或伺服</td></tr> <tr> <td>2</td><td>模拟信号控制方式的伺服</td></tr> <tr> <td>3</td><td>正交编码器</td></tr> <tr> <td>4</td><td>脉冲方向输出+正交编码器输入</td></tr> <tr> <td>5</td><td>脉冲方向输出+脉冲方向编码器输入</td></tr> <tr> <td>6</td><td>脉冲方向方式的编码器</td></tr> <tr> <td>7</td><td>脉冲方向方式步进或伺服+EZ 信号输入</td></tr> <tr> <td>8</td><td>ZCAN 扩展脉冲方向方式步进或伺服</td></tr> <tr> <td>9</td><td>ZCAN 扩展正交编码器。</td></tr> <tr> <td>10</td><td>ZCAN 扩展脉冲方向方式的编码器。</td></tr> <tr> <td>50</td><td>RTEX 周期位置模式，需 Rtex 控制器。</td></tr> <tr> <td>51</td><td>RTEX 周期速度模式，需 Rtex 控制器。</td></tr> <tr> <td>52</td><td>RTEX 周期力矩模式，需 Rtex 控制器。 请先关闭驱动器 2 自由度控制模式，并设置速度限制。</td></tr> <tr> <td>65</td><td>ECAT 周期位置模式，需支持 EtherCAT</td></tr> <tr> <td>66</td><td>ECAT 周期速度模式，需支持 EtherCAT Profile 要设置为 20 或以上</td></tr> <tr> <td>67</td><td>ECAT 周期力矩模式，需支持 EtherCAT Profile 要设置为 30 或以上</td></tr> <tr> <td>70</td><td>ECAT 自定义操作，只读取编码器，需支持 EtherCAT</td></tr> </tbody> </table>	Atype 类型	描述		电机模式 INVERT_STEP 指令设置，默认脉冲方向	0	虚拟轴	1	脉冲方向方式的步进或伺服	2	模拟信号控制方式的伺服	3	正交编码器	4	脉冲方向输出+正交编码器输入	5	脉冲方向输出+脉冲方向编码器输入	6	脉冲方向方式的编码器	7	脉冲方向方式步进或伺服+EZ 信号输入	8	ZCAN 扩展脉冲方向方式步进或伺服	9	ZCAN 扩展正交编码器。	10	ZCAN 扩展脉冲方向方式的编码器。	50	RTEX 周期位置模式，需 Rtex 控制器。	51	RTEX 周期速度模式，需 Rtex 控制器。	52	RTEX 周期力矩模式，需 Rtex 控制器。 请先关闭驱动器 2 自由度控制模式，并设置速度限制。	65	ECAT 周期位置模式，需支持 EtherCAT	66	ECAT 周期速度模式，需支持 EtherCAT Profile 要设置为 20 或以上	67	ECAT 周期力矩模式，需支持 EtherCAT Profile 要设置为 30 或以上	70	ECAT 自定义操作，只读取编码器，需支持 EtherCAT
Atype 类型	描述																																								
	电机模式 INVERT_STEP 指令设置，默认脉冲方向																																								
0	虚拟轴																																								
1	脉冲方向方式的步进或伺服																																								
2	模拟信号控制方式的伺服																																								
3	正交编码器																																								
4	脉冲方向输出+正交编码器输入																																								
5	脉冲方向输出+脉冲方向编码器输入																																								
6	脉冲方向方式的编码器																																								
7	脉冲方向方式步进或伺服+EZ 信号输入																																								
8	ZCAN 扩展脉冲方向方式步进或伺服																																								
9	ZCAN 扩展正交编码器。																																								
10	ZCAN 扩展脉冲方向方式的编码器。																																								
50	RTEX 周期位置模式，需 Rtex 控制器。																																								
51	RTEX 周期速度模式，需 Rtex 控制器。																																								
52	RTEX 周期力矩模式，需 Rtex 控制器。 请先关闭驱动器 2 自由度控制模式，并设置速度限制。																																								
65	ECAT 周期位置模式，需支持 EtherCAT																																								
66	ECAT 周期速度模式，需支持 EtherCAT Profile 要设置为 20 或以上																																								
67	ECAT 周期力矩模式，需支持 EtherCAT Profile 要设置为 30 或以上																																								
70	ECAT 自定义操作，只读取编码器，需支持 EtherCAT																																								
适用控制器	通用																																								
例子	例一 脉冲类型 BASE (0, 1) ATYPE = 1, 1 ' 轴 0, 1 设为脉冲控制类型 UNITS=100, 100 ' 脉冲当量设为 100																																								

	SPEED=100,100 '速度 100 units/s ACCEL=1000 '加速度 1000 units/s/s MOVE(100,100) '直线插补 例二 EtherCAT 总线控制 SLOT_SCAN(0) '总线扫描 BASE(0) AXIS_ADDRESS(0)=1 '第一个驱动器映射到轴 0 ATYPE(0)=65 '轴类型 65, 位置控制 SLOT_START(0) '开启总线 AXIS_ENABLE=1 '轴使能 WDOG=1 '所有轴使能 UNITS=100 '脉冲当量设为 100 SPEED=100 '速度 100 units/s ACCEL=1000 '加速度 1000 units/s/s DECEL=1000 '减速度 1000 units/s/s MOVE(5000) 例三 Rtex 力矩模式 SLOT_SCAN(0) '总线扫描 BASE(0) AXIS_ADDRESS(0)=1 '第一个驱动器映射到轴 0 ATYPE(0)=52 '轴类型 52, Rtex 力矩控制 DRIVE_WRITE(6*256+47,0) '关闭 2 自由度控制 DRIVE_WRITE(3*256+17,0) '选择参数 3.21 作为速度限制 DRIVE_WRITE(3*256+21,2000) '最大速度限制为 2000r/min SLOT_START(0) '开启总线 AXIS_ENABLE=1 '轴使能 WDOG=1 '所有轴使能 DAC=100 '此时 DAC 发送值控制, 具体查看 DAC 指令
相关指令	AXIS_ADDRESS , INVERT_STEP

AXISSTATUS -- 轴状态

类型	轴状态			
描述	查看轴的各种状态。 按十进制显示数值，按二进制对应位判断状态。			
语法	VAR1 = AXISSTATUS			
	位	说明	打印值	
	1	随动误差超限告警	2	2h
	2	与远程轴通讯出错	4	4h

	3	远程驱动器报错	8	8h
	4	正向硬限位	16	10h
	5	反向硬限位	32	20h
	6	找原点中	64	40h
	7	HOLD 速度保持信号输入	128	80h
	8	随动误差超限出错	256	100h
	9	超过正向软限位	512	200h
	10	超过负向软限位	1024	400h
	11	CANCEL 执行中	2048	800h
	12	脉冲频率超过 MAX_SPEED 限制. 需要修改降速或修改 MAX_SPEED.	4096	1000h
	14	机械手指令坐标错误	16384	4000h
	18	电源异常	262144	40000h
	21	运动中触发特殊运动指令失败	2097152	200000h
	22	告警信号输入	4194304	400000h
	23	轴进入了暂停状态	8388608	800000h
适用控制器	通用			
例子	例一 直接读取位（推荐编程时使用） 碰到了正限位时 VAR = READ_BIT2(4, AXISSTATUS(0)) ' 读取轴 0 是否正限位 Print VAR ' 打印结果，1，发生了正向硬限位 例二 返回数值判断（推荐直接查看时使用） ?AXISSTATUS(1) ' 查看轴 1 的状态，结果，48 ' 48=32+16，同时处于正负限位中，一般是没有反转正负限位开关电平。 例三 总线通讯错误 在正确使能电机后，将通讯接线断开，AXISSTATUS 会显示 4，与远程轴通讯出错。将编码器线断开，对应轴 AXISSTATUS 会显示 8，远程驱动器报错。			
相关指令	AXIS_STOPPREASON			

AXIS_ADDRESS -- 轴地址设置

类型	轴参数
描述	扩展轴时的轴地址配置。 ZCAN 扩展轴时，低 5 位表示 CAN ID(通过拨码来组合)，6 位以上表示扩展模块的本地轴号。

	受总线带宽限制，ZCAN 扩展轴不要设置超过 2 个。 必须先设置 AXIS_ADDRESS，再设置 ZCAN 扩展轴 ATYPE 类型，修改后必须重新设置 ATYPE。见例一 <table border="1" data-bbox="504 318 1238 360"> <tr> <td>位 0-3</td><td>扩展板的 CANID，0-31</td></tr> </table> 规则：AXIS_ADDRESS (轴号)=(32*0)+ID ‘扩展板的本地轴口 0 AXIS_ADDRESS (轴号)=(32*1)+ID ‘扩展板的本地轴口 1 总线驱动器映射轴号，将连接的驱动器按编号一一映射。 驱动器编号---根据接线顺序排列，从 0 开始编号到 ecat 驱动器个数-1。 驱动器编号与设备号不同，设备号包括 ECAT 接口所有连接的设备，驱动器编号只算连接的驱动器。 必须先设置 AXIS_ADDRESS，再设置 ECAT 轴 ATYPE 类型，修改后必须重新设置 ATYPE。见例二 <table border="1" data-bbox="504 734 1238 822"> <tr> <td>位 0-15</td><td>驱动器编号+1，0-自动指定.</td></tr> <tr> <td>位 16-31</td><td>SLOT 编号（多槽位时）</td></tr> </table> 规则：AXIS_ADDRESS (轴号)=(槽位号<<16)+驱动器编号+1 本地脉冲轴号重映射，4 系列控制器支持本地脉冲或编码器轴号重新映射，固件 160608 以上版本支持。 重新映射时注意先把原本地脉冲轴设置为虚拟轴。见例四 <table border="1" data-bbox="504 1030 1238 1117"> <tr> <td>位 0-15</td><td>映射的本地脉冲轴号</td></tr> <tr> <td>位 16-31</td><td>高 16 位全 1 或 高 16 位= -1</td></tr> </table> 规则： BASE(重映射的轴号) ATYPE=0，设置轴类型为 0 BASE(要修改的轴号) ATYPE=0，设置轴类型为 0 AXIS_ADDRESS(重映射的轴号)= (-1<<16) + 要修改的轴号 BASE(重映射的轴号) ATYPE=1/7	位 0-3	扩展板的 CANID，0-31	位 0-15	驱动器编号+1，0-自动指定.	位 16-31	SLOT 编号（多槽位时）	位 0-15	映射的本地脉冲轴号	位 16-31	高 16 位全 1 或 高 16 位= -1
位 0-3	扩展板的 CANID，0-31										
位 0-15	驱动器编号+1，0-自动指定.										
位 16-31	SLOT 编号（多槽位时）										
位 0-15	映射的本地脉冲轴号										
位 16-31	高 16 位全 1 或 高 16 位= -1										
语法	VAR1 = AXIS_ADDRESS, AXIS_ADDRESS = expression										
适用控制器	通用										
例子	举例 1：ZCAN 扩展轴 AXIS_ADDRESS (6)=2+(32*1) ‘轴 6 映射到 ZCAN 扩展模块 ID2 的轴 1 ATYPE (6)=8 ‘ZCAN 扩展轴类型 脉冲方向方式步进或伺服 UNITS (6)=100 ‘脉冲当量 100 SPEED (6)=100 ‘速度 100units/s ACCEL (6)=1000 ‘加速度 1000units/s/s MOVE (100) AXIS (6) ‘扩展轴运动 100units 举例 2：EtherCAT 手动映射轴号										

	AXIS_ADDRESS (0)=0+1 ' 第一个 Ecat 驱动器, 编号 0, 绑定为轴 0 AXIS_ADDRESS (2)=1+1 ' 第二个 Ecat 驱动器, 编号 1, 绑定为轴 2 AXIS_ADDRESS (1)=2+1 ' 第三个 Ecat 驱动器, 编号 2, 绑定为轴 1 ATYPE (0)=65 ' 设置为 Ecat 类型 ATYPE (1)=65 ATYPE (2)=65 举例 3: EtherCAT 自动映射轴号 AXIS_ADDRESS (0)=0 ' 自动指定 slot0 的驱动器, 按照接线顺序, 从轴 0 开始依次映射轴号, (不建议使用, 建议举例 2). ATYPE (0)=65 ' 轴 0 配置为 ECAT 模式 举例 4: EtherCAT 控制器修改脉冲轴号 ' 修改前, 操作轴 0, 对应控制器上的轴 0 接口 ATYPE (0)=0 ' 先将本地脉冲轴 0 设为虚拟轴 AXIS_ADDRESS (16)= (-1<<16)+0 ' 绑定本地脉冲轴 0, 高 16 位=-1 ATYPE (16)=1 ' 轴 16 配置为脉冲轴, 使用本地脉冲轴 0 ' 此时, 操作轴 0, 对应 ECAT 驱动器, 操作轴 16, 对应控制器轴 0 接口
相关指令	ATYPE

AXIS_ENABLE -- 单轴使能

类型	轴参数
描述	各轴使能设置。 EtherCAT 总线轴需设置。 还要设置 WDOG=1 总使能。
语法	AXIS_ENABLE = 1/0, 1 使能 0 关闭
适用控制器	通用
例子	AXIS_ENABLE (0) = 1 ' 打开轴 0 使能
相关指令	WDOG

AXIS_STOPREASON -- 轴停止原因

类型	轴状态
----	-----

描述	轴历史停止原因锁存。
语法	写 0 清除，按位锁存，意义与 AXISSTATUS 相同。 20150731 以上版本支持。
适用控制器	通用
例子	If AXIS_STOPREASON and (512+1024) then Print "axis el stoped" Endif
相关指令	AXISSTATUS

AXIS_ZSET -- 精准 op 设置

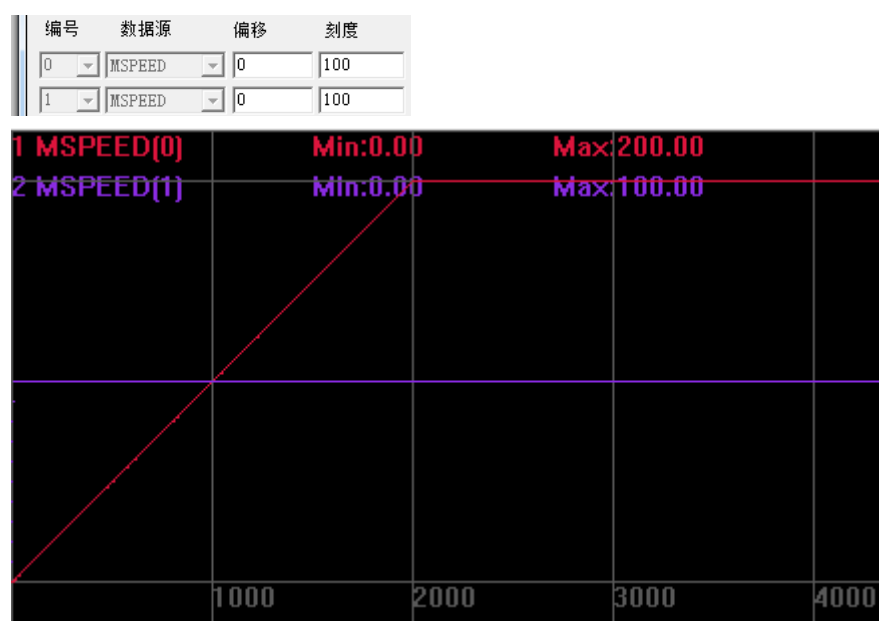
类型	轴参数
描述	用于设置不同的轴是否支持 MOVEOP 精准功能。 SYSTEM_ZSET 修改时同时会修改当前 BASE 轴的 AXIS_ZSET，以兼容旧的程序。 BIT1 - 1 使用 MOVE_OP 精确输出功能，0- MOVE_OP 原来的方式 BIT4 - 1 对带编码器功能的轴，使用编码器位置的 MOVE_OP 精准方式 多个编码器轴插补时，使用 BASE 运动主轴的设置
语法	可读：value=AXIS_ZSET 可写：AXIS_ZSET=value
适用控制器	20170517 以上固件版本
相关指令	SYSTEM_ZSET ， MOVE_OP

CLUTCH_RATE -- 连接速度

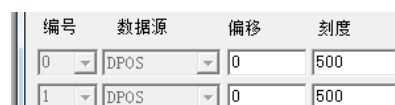
类型	轴参数
描述	connect 连接的速度，默认值 1000000。 用于定义连结率从 0 到设置倍率的改变时间，ratio/秒，见例一 设置值如果不能远大于 CONNECT 连接比例的话，实际连接比例会减小，见例一位移曲线图。 当设置为 0 时，根据跟随轴的速度/加速度参数来跟踪连接，比较适合于手轮运动（当速度不够高时可能导致要持续运动一段时间才能结束）。
语法	clutch_rate= value
适用控制器	通用
例子	例一

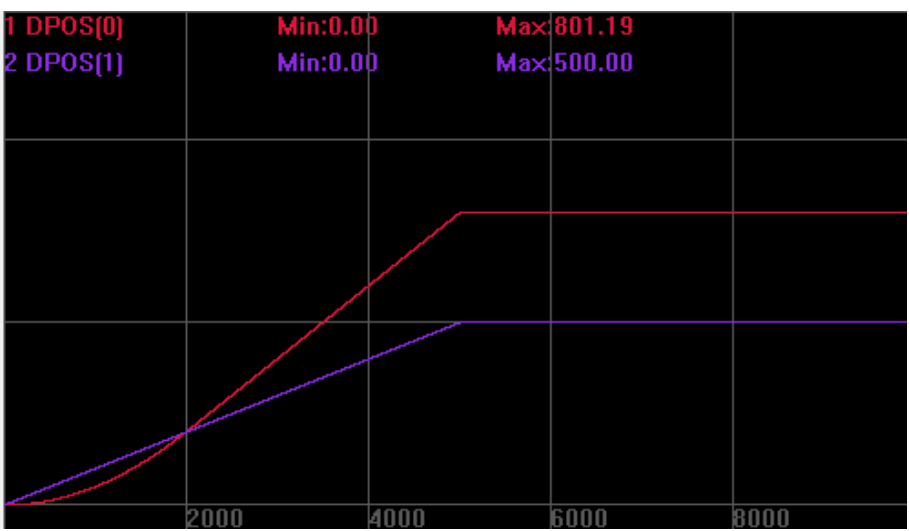
BASE(0, 1)	
ATYPE=1, 1	
DPOS=0, 0	
UNITS=100, 100	
SPEED=100, 100	
ACCEL=10000, 10000	
DECEL=10000, 10000	
CLUTCH_RATE =1	' 设置连接速度为 1ratio/s
TRIGGER	' 自动触发示波器
CONNECT(2, 1) AXIS(0)	' 连接倍率为 2, 需要 2 秒建立连接
MOVE(500) AXIS(1)	' 运动轴 1, 轴 0 跟随

速度曲线，连接时间根据连接比例和 clutch rate 确定



位移曲线, clutch_rate 值较小, 实际运动比例并没有达到 2:1





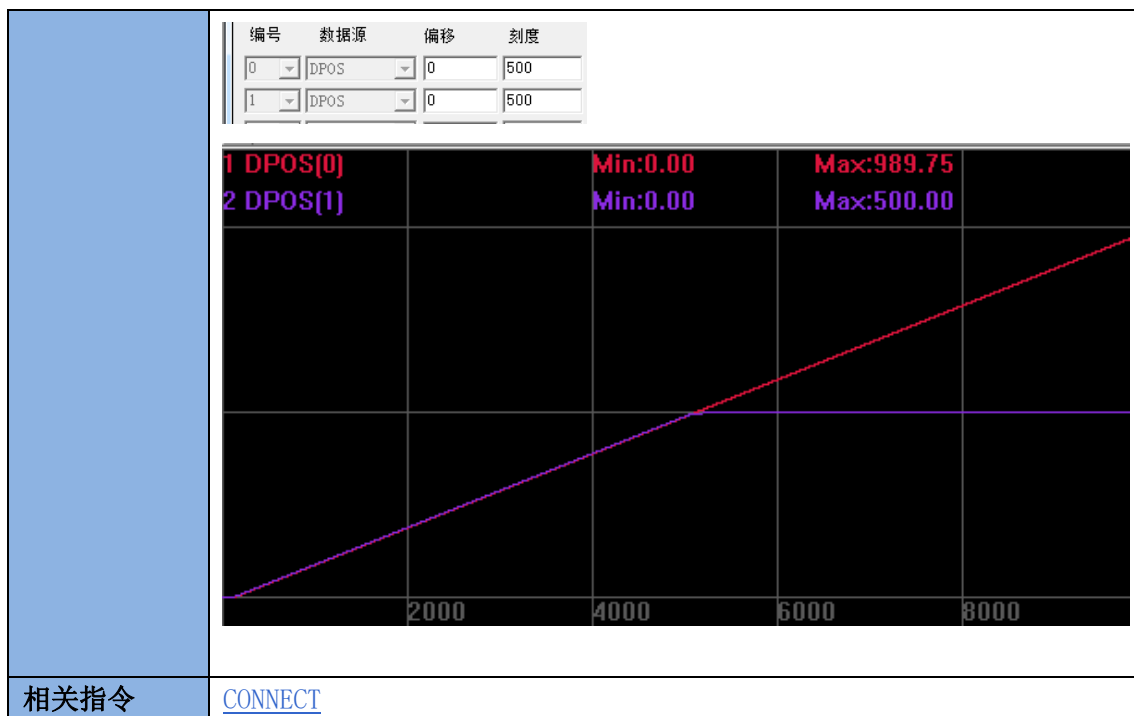
例二
BASE (0, 1)
ATYPE=1, 1
UNITS=100, 100
SPEED=100, 100
ACCEL=500, 500
DECEL=500, 500
CLUTCH_RATE = 0 ’ 根据跟随轴的速度/加速度来跟踪连接，**不一定同步**
TRIGGER ’ 自动触发示波器
CONNECT (2, 1) AXIS (0) ’ 此时，连接时间根据跟随轴速度加速度确定，本例中 0.2s
VMOVE (1) AXIS (1)

速度曲线

编号	数据源	偏移	刻度
0	MSPEED	0	100
1	MSPEED	0	100



位移曲线



CLOSE_WIN -- 锁存范围

类型	轴参数
描述	锁存触发的结束坐标范围点。 预留
语法	CLOSE_WIN = pos
相关指令	REGIST , OPEN_WIN

CORNER_MODE -- 拐角设置

类型	轴参数									
描述	拐角减速等模式设置。									
语法	CORNER_MODE = mode mode: 不同的位代表不同的意义，位可以同时使用。 <table><tr><td>位</td><td>值</td><td>描述</td></tr><tr><td>0</td><td>1</td><td>预留</td></tr><tr><td>1</td><td>2</td><td>自动拐角减速。 按 ACCEL, DECEL 加减速速度。 此参数是在 MOVE 函数调用前生效。</td></tr></table>	位	值	描述	0	1	预留	1	2	自动拐角减速。 按 ACCEL, DECEL 加减速速度。 此参数是在 MOVE 函数调用前生效。
位	值	描述								
0	1	预留								
1	2	自动拐角减速。 按 ACCEL, DECEL 加减速速度。 此参数是在 MOVE 函数调用前生效。								

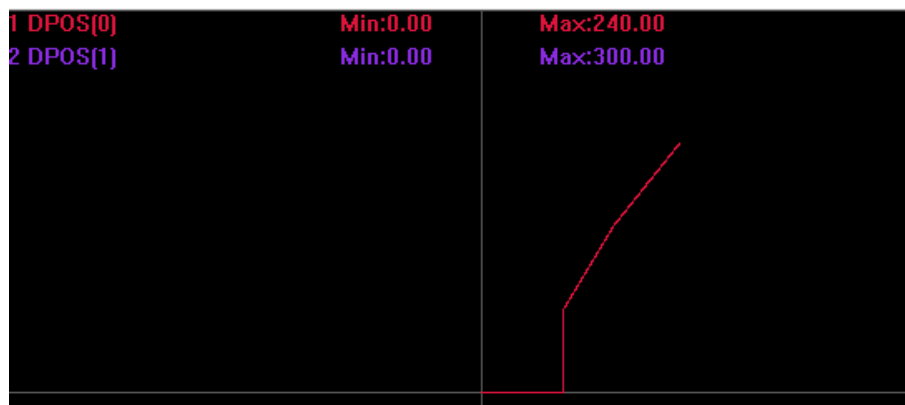
			减速角度定义查看 DECEL_ANGLE 和 STOP_ANGLE 指令。 减速拐角参考速度以 force_speed 速度为参考，一定要设置合理的 force_speed。
	2	4	预留
	3	8	自动小圆限速，半径小于设置值时限速，大于限制值时不限速。 此参数在 MOVE 函数调用前修改生效。 限速速度按 FORCE_SPEED。 限速= FORCE_SPEED * 实际半径/FULL_SP_RADIUS 限速半径 FULL_SP_RADIUS 设置。
	4	16	预留
	5	32	自动倒角设置。 此参数在 MOVE 函数调用前修改生效。 此 MOVE 运动自动和前面的 MOVE 运动做倒角处理，倒角半径参考 ZSMOOTH。 此倒角针对插补的所有轴，20150701 以后固件支持。
适用控制器	通用		
例子	为了方便说明每个位的功能，例程都是单独位设置，实际应用时可以多位同时使用。 比如 CONNER_MODE=2+8，打开自动拐角减速和小圆限速 举例 1：拐角减速 <pre> BASE(0) DPOS=0,0 ACCEL=500 ' 设置加速度 DECEL=500 ' 设置减速度 SPEED=100 ' 设置速度 MERGE=ON ' 开启连续插补 CORNER_MODE=2 ' 启动拐角减速 DECEL_ANGLE = 15 * (PI/180) ' 设置开始减速角度 STOP_ANGLE = 45 * (PI/180) ' 设置结束减速角度 FORCE_SPEED=100 ' 等比减速时起作用 TRIGGER ' 自动触发示波器 MOVE(100,0) MOVE(0,100) ' 运动角度大于 45°，完全减速 MOVE(60,100) ' 运动角度 30.96° 处于 15°~45°，等比减速 MOVE(70,100) ' 运动角度 4.03° 小于 15°，不减速 </pre>		

轨迹曲线

水平刻度: 1000 ☒ XY模式

手动触发 导入 导出

编号	数据源	偏移	刻度
0	DPOS	0	200
1	DPOS	0	200



速度曲线

编号	数据源	偏移	刻度
0	MSPEED	0	100
1	MSPEED	0	100



使用仿真器查看时曲线可能会有误差，最好连控制器查看。

举例 2：小圆限速

```

BASE (0)
CORNER_MODE=8          ' 启动小圆限速
DPOS=0, 0
ACCEL=500              ' 设置加速度
DECEL=500              ' 设置减速度
SPEED=100              ' 运行速度
FORCE_SPEED=120        ' 小圆限速
FULL_SP_RADIUS=60      ' 限速半径 60
TRIGGER                ' 自动触发示波器
    
```

半径大于限制值时，不限制速度，按 SPEED 运行。

MOVECIRC(200, 0, 100, 0, 1) ' 此时速度 100

轨迹曲线

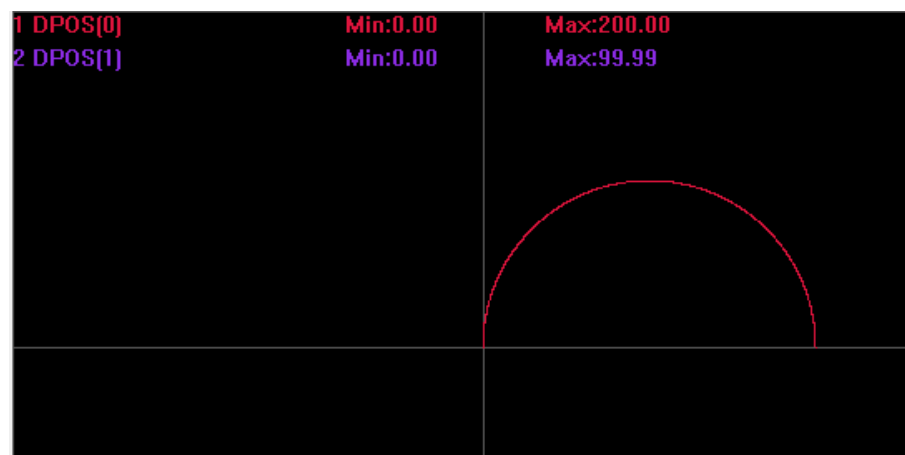
示波器

设置 启动 停止

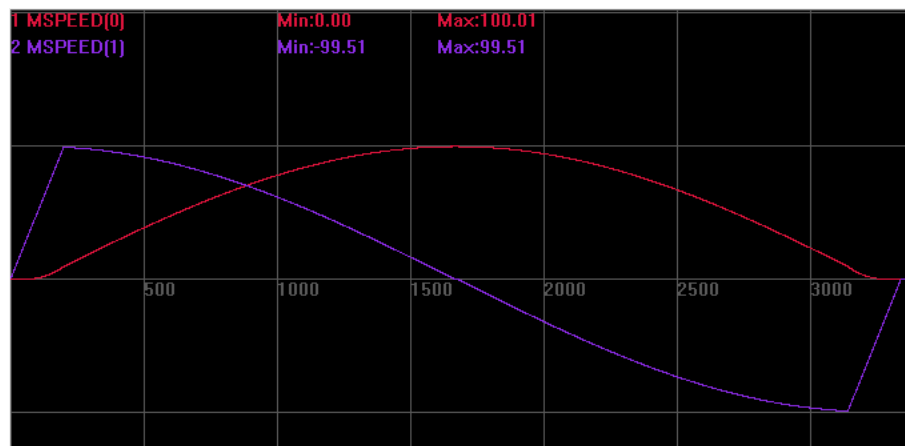
水平刻度: 500 ☒ XY模式

手动触发 导入 导出

编号	数据源	偏移	刻度
0	DPOS	0	100
1	DPOS	0	100



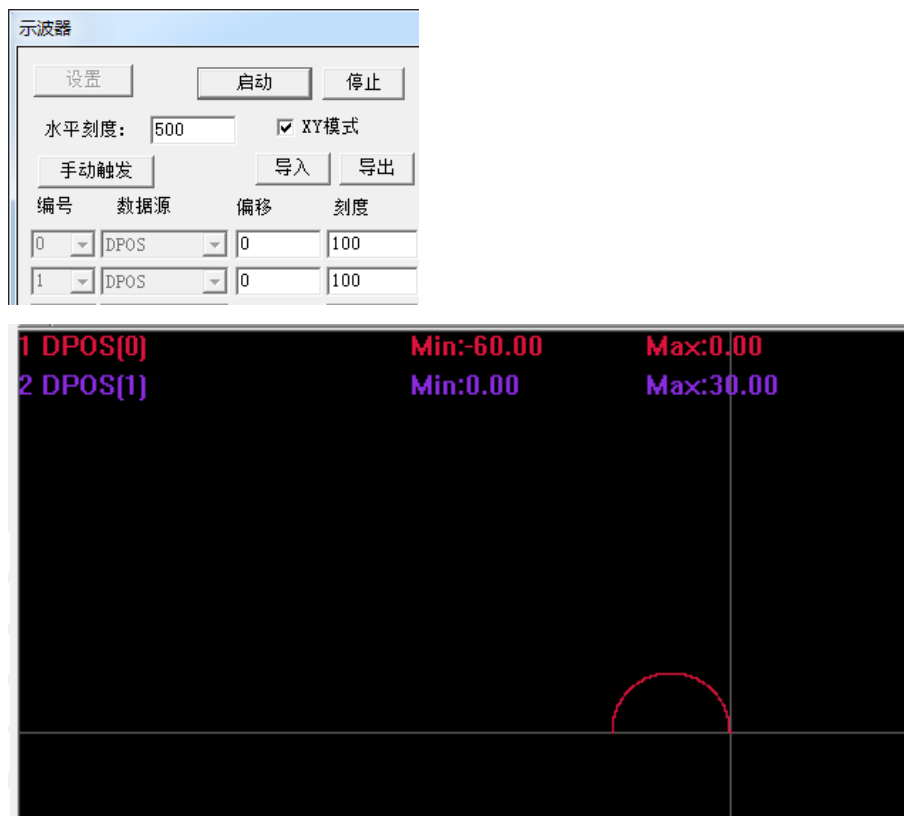
速度曲线



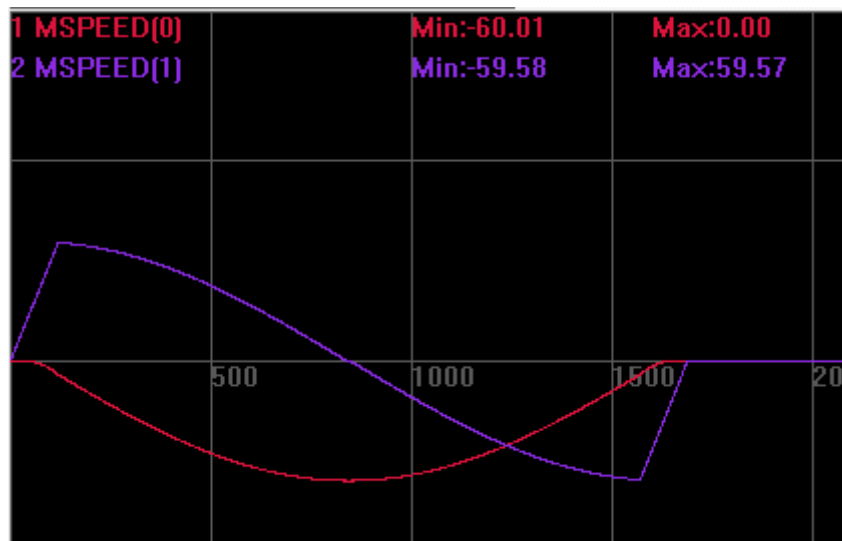
半径小于限制值时，限速=FORCE_SPEED * 实际半径/FULL_SP_RADIUS

MOVECIRC(-60, 0, -30, 0, 0) ' 此时速度 60=30/60*120

轨迹曲线



速度曲线



举例 3: 倒角

倒角是可以用于直线、圆弧、螺旋等运动的，例程为了直观显示，只做了直线拐角

```
BASE(0)
DPOS=0,0
ACCEL=500           ' 设置加速度
DECAL=500           ' 设置减速度
SPEED=100           ' 运行速度
```

CORNER_MODE=32 ' 启动倒角
ZSMOOTH = 10 ' 倒角参考半径
TRIGGER ' 自动触发示波器
MOVE(100, 0)
MOVE(0, 100) ' 上面两条直线间自动倒角。

插补轨迹
使用倒角

示波器

设置

启动

停止

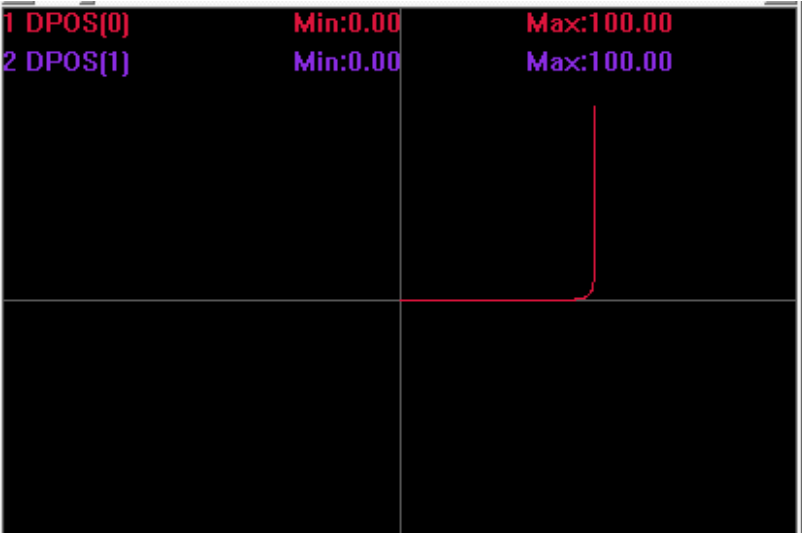
水平刻度: ☒ XY模式

手动触发

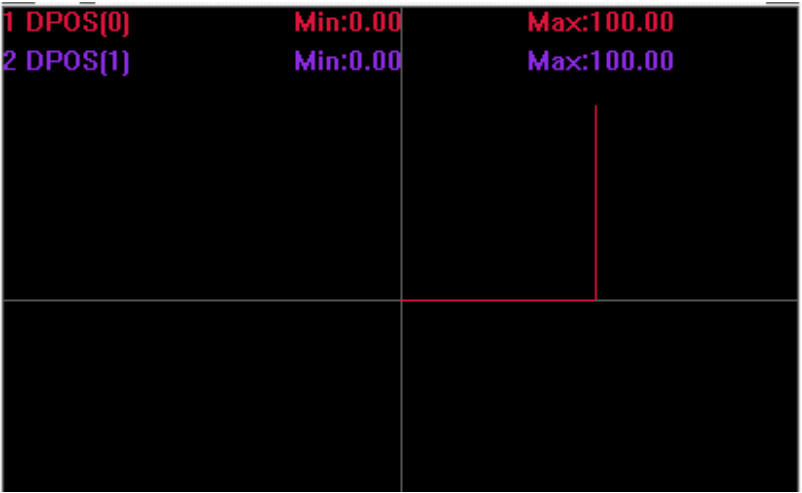
导入

导出

编号	数据源	偏移	刻度
0	DPOS	0	100
1	DPOS	0	100



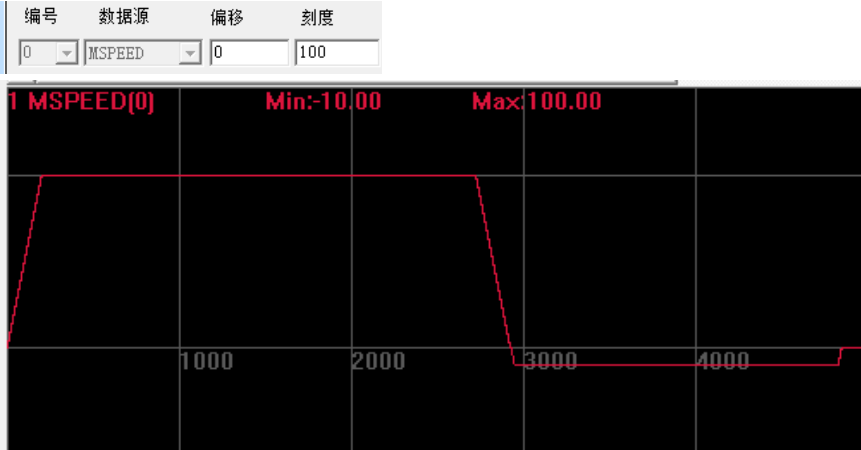
不使用倒角



相关指令

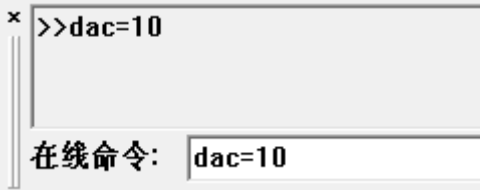
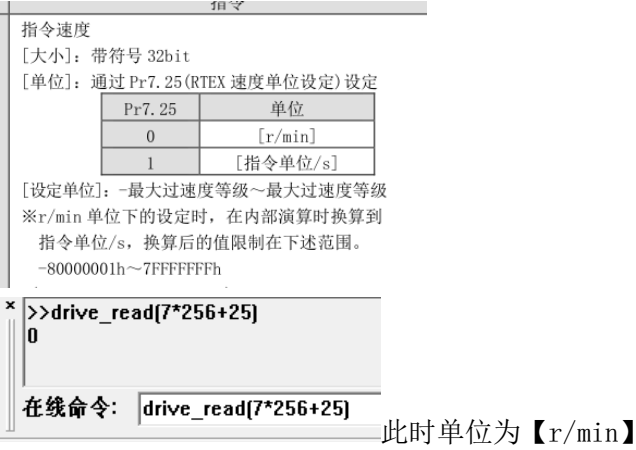
[MERGE](#), [STOP_ANGLE](#), [DECEL_ANGLE](#), [FULL_SP_RADIUS](#)
[ZSMOOTH](#).

CREEP -- 爬行速度

类型	轴参数
描述	轴回零时爬行速度，用于原点搜寻。 单位为 units/s。
语法	VAR1 = CREEP, CREEP = expression
适用控制器	通用
例子	Base(0) Units=100 Accel=1000 Decel=1000 Speed = 100 ' 运行速度设置为 100units/s Creep = 10 ' 爬行速度设置为 10units/s Datum_in=0 ' 设置 IN0 为轴零原点 Invert_in(0,on) ' 反转电平 TRIGGER ' 自动触发示波器 Datum(3) ' 先按速度 100 反找回零，遇原点后按速度 10 反向离开原点 速度曲线 
相关指令	DATUM

DAC -- 总线轴模拟量控制

类型	轴参数
描述	伺服轴 DA 直接控制，速度或力矩模式下支持。

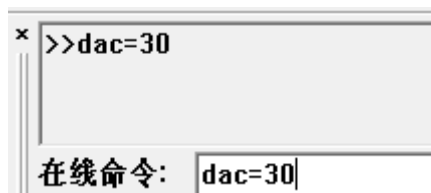
	单位为 DA 模块的刻度，12 为或 16 位。 速度控制时要看驱动器具体单位。 力矩控制时单位为千分之一，等于 1000 时表示 100%力矩。
语法	VAR1 = DAC, DAC = expression
使用控制器	带 EtherCAT 接口或 Rtex 接口，2017 以后固件支持
例程	例一 Rtex 速度控制 请先使用第十三章的 Rtex 初始化例程，并将例程中的 ATYPE 设置为 51。然后可在 zdevelop 在线命令直接发送 dac，如下图  此时电机将以 10r/min 的速度旋转，发送 dac= - 10 会反向旋转。也可以在程序中发送 dac 指令。 速度单位根据驱动器手册确认，如下图  例二 Rtex 力矩控制 请先将驱动器参数 pr6.47 的第一位置为 0，关闭 2 自由度控制模式；参数 pr3.17 设置速度限制，如下图。（参照松下 Rtex 手册）

Pr3.17(速度制限选择)的设定值是1时，可以通过SL_SW切换转矩控制时的速度限制值。

分类	No.	属性	参数名称	设定范围	单位	功能		
3	17	B	速度制限选择	0~1	—	设定转矩控制时的速度限制值的选择方式。		
						设定值	SL_SW = 0	SL_SW = 1
						0	Pr3. 21	
						1	Pr3. 21	Pr3. 22
3	21	B	速度限制值1	0~20000	r/min	设定转矩控制时的速度限制。 转矩控制中在不超过速度制限值设定的速度内进行控制。 另外，内部值被 Pr5. 13「过速度等级设定」、Pr6. 15「第2过速度等级设定」、Pr9. 10「最大过速度等级」的最小设定速度进行限制。		
3	22	B	速度限制值2	0~20000	r/min	Pr3. 17「速度制限选择」=1 设定时，设定 SL_SW 为1时的速度制限值。 另外，内部值被 Pr5. 13「过速度等级设定」、Pr6. 15「第2过速度等级设定」、Pr9. 10「最大过速度等级」的最小设定速度进行限制。		

再使用第十三章的 [Rtex 初始化例程](#)，并将例程中的 ATYPE 设置为 52。

然后可在 zdevelop 在线命令直接发送 dac，此时电机开始转动。



当前驱动器为 0.3 的力矩，如果 dac 发送过小时，电机无法克服摩擦力，不能转动。

也可以在程序中发送 dac 指令。

力矩控制时 dac 的单位为千分之一的力矩，dac=1000 是表示 100%

指令

指令转矩

[大小]: 带符号 32bit

[单位]: 0.1%

[设定范围]: -电机最大转矩~电机最大转矩

* 最大转矩限制[%] = $100 \times \text{Pr9.07} / (\text{Pr9.06} \times \sqrt{2})$

例三 EtherCAT 速度控制

```

for i=0 to 1      ' 初次使用将所有轴设为普通脉冲类型
    atype(i)=1
next
slot_scan(0)      ' 总线扫描开始
IF NODE_COUNT(0,0)>0 THEN
    AXIS_ADDRESS(0)=1      ' 将第一个驱动器映射到轴 0
    atype(0)=66            ' 66 为速度控制模式
    DRIVE_PROFILE(0)=20    ' 速度控制要设置为 20
    
```

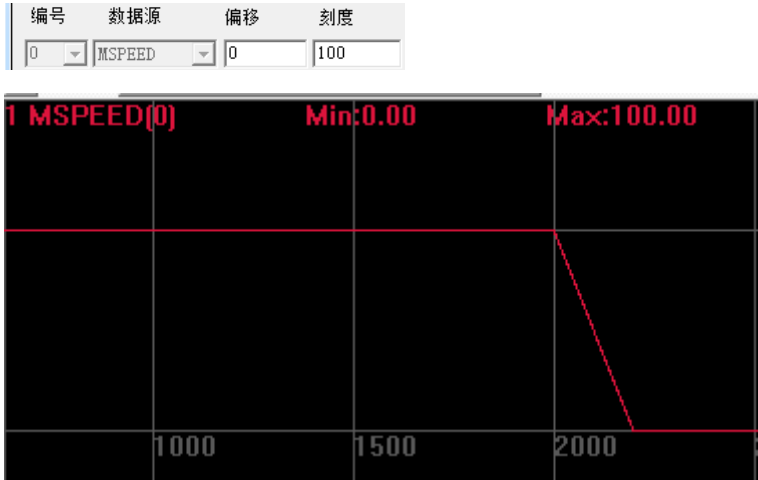
	<pre> delay (200) slot_start(0) ' 总线扫描成功, 启动总线 DRIVE_CONTROLWORD(0)=128 ' 清除驱动器错误 delay (2) DRIVE_CONTROLWORD(0)=6 delay (2) DRIVE_CONTROLWORD(0)=15 delay (2) delay(20) datum(0) ' 清除控制器错误 base(0) axis_enable=1 ' 映射轴使能打开 wdog=1 ' 轴使能 DAC=10000 ' 电机以 10000 脉冲每秒的速度转动 ENDIF 例四 EtherCAT 力矩控制 将例三中的 atype 改为 67, drive_profile 改为 30 即可。 此时 dac 发送范围 0~1000, 1000 表示 100%力矩, 反向选择发送负值即可。 </pre>
相关指令	SERVO

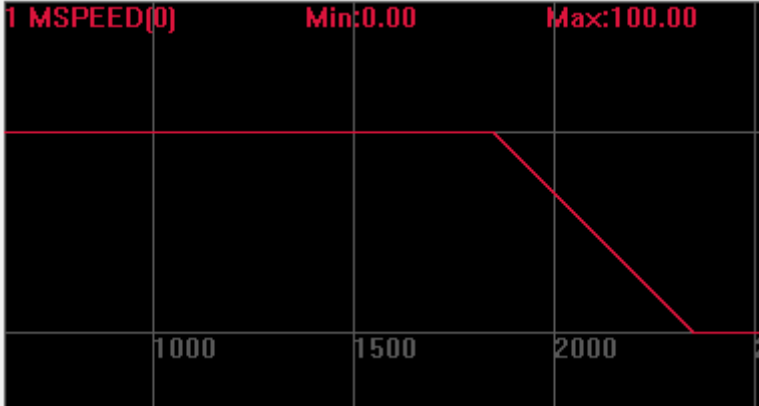
DATUM_IN -- 映射原点输入

类型	轴参数
描述	通用输入口设置为原点开关信号, -1 无效。 设置了原点开关后, ZMC 控制器输入 OFF 时认为有信号输入, 要相反效果可以用 INVERT_IN 反转电平。 (ECI 系列控制器除外)。
语法	VAR1 = DATUM_IN, DATUM_IN = expression
适用控制器	通用
例子	<pre> Base(0, 1, 2, 3) Datum_in = 6, 7, 8, 9 ' 将轴 0, 1, 2, 3 原点输入对应到输入口 6, 7, 8, 9 Invert_in(6, on) ' 把原点信号反转 Invert_in(7, on) Invert_in(8, on) </pre>

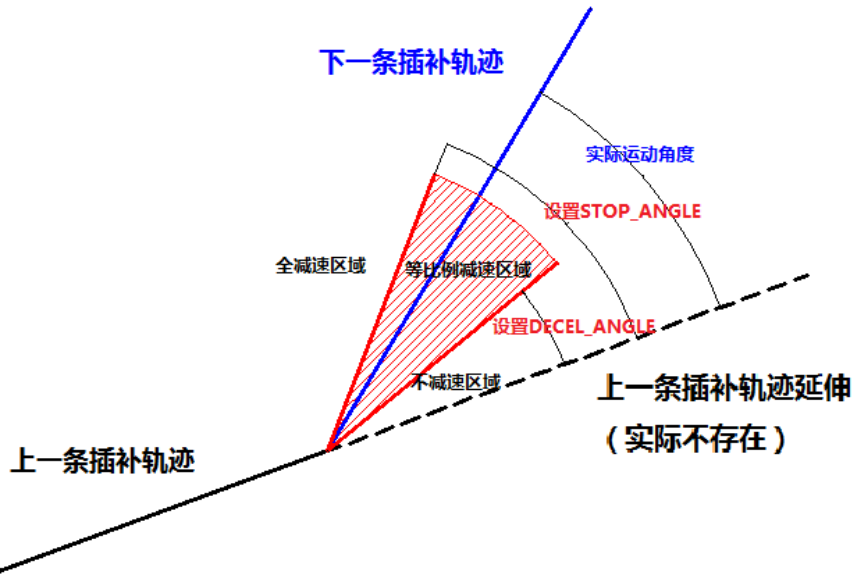
	Invert_in(9,on)
相关指令	DATUM , FWD_IN , REV_IN , INVERT_IN

DECEL -- 减速度

类型	轴参数
描述	轴减速度，单位为 units/s/s。 当多轴运动时，作为插补运动的减速度。 当设置为 0 时，自动等于加速度 ACCEL 值，进行对称的加减速。 建议运动前设置好加速度和减速度，运动中不要修改，运动中调整会导致速度曲线变化。
语法	VAR1 = DECEL, DECEL = expression
适用控制器	通用
例子	例一 <pre>BASE(1,2,3,4) DECEL=100,100,100,100 '设置轴1,2,3,4的减速度 DECEL(0)=200 '设置轴0的减速度 PRINT DECEL(0)</pre> 例二 <pre>BASE(0) '选择轴0 SPEED=100 '设置速度为100 units/s DECEL=500 '减速度为500units/s/s TRIGGER '自动触发示波器 MOVE(200) '移动200units</pre> 速度曲线 

	Decel=200 时 
相关指令	ACCEL , FASTDEC

DECEL_ANGLE -- 拐角减速开始

类型	轴参数
描述	开始减速的角度，单位是弧度。 角度转换弧度公式： 角度值*(PI/180) 减速角度 是指下一插补运动与上一插补运动方向延长线的角度，如下图。 下一插补运动轨迹处于下方时取绝对值。 直线与圆弧相连时按照圆弧的切线方向计算角度。 与 STOP_ANGLE 一起使用，当实际运动的角度处于 DECEL_ANGLE（上限）与 STOP_ANGLE（下限）时才会减速。 

语法	VAR1 = DECEL_ANGLE, DECEL_ANGLE = expression
适用控制器	通用
例子	CORNER_MODE=2 DECEL_ANGLE = 25 * (PI/180) ' 设置开始减速拐角，弧度 STOP_ANGLE = 45 * (PI/180) ' 设置结束减速拐角，弧度 FORCE_SPEED=SPEED ' 一定要设置 FORCE_SPEED 参考 CORNER_MODE 例程一
相关指令	STOP_ANGLE , CORNER_MODE

DPOS -- 轴指令位置

类型	轴状态
描述	轴的虚拟坐标位置，或称需求位置。 写 DPOS 会自动转换为 OFFPOS 偏移，不会移动电机。 以 UNITS 作为单位。
语法	VAR1 = DPOS, DPOS=expression
适用控制器	通用
例子	Dpos(0) = 0 ' 轴 0 坐标偏移至 0 ?*dpos ' 命令行查看当前 DPOS ' 打印结果, 0 0 0 0 0 0 0 0 0 0 0 0
相关指令	MPOS , ENDMOVE , OFFPOS , DEFPOS

D_GAIN -- 微分增益

类型	轴参数
描述	微分增益，非模拟量伺服不支持。 预留
语法	VAR1 = D_GAIN
相关指令	P_GAIN , I_GAIN , OV_GAIN , VFF_GAIN

ENCODER -- 编码器原始值

类型	轴状态
描述	编码器硬件寄存器原始值。 内部参数，只有配置为需要使用编码器的 ATYPE 时才可以读取。 驱动器有多圈绝对值编码器时，读取的就是多圈值。
语法	VAR1 = ENCODER
适用控制器	通用
例子	?*encoder ' 打印各轴编码器值 ' 打印结果, 0 0 0 0 0 0 0 0 0 0 0 0
相关指令	MPOS , ENCODER_RATIO

ENCODER_STATUS -- 编码器状态

类型	轴状态		
描述	编码器的 EA EB EZ 的状态。		
语法	VAR1 = ENCODER_STATUS		
	位	值	描述
	0	1	EA 状态
	1	2	EB 状态
	2	4	EZ 状态
适用控制器	通用		
例子	?*ENCODER_STATUS ’ 打印全部轴的编码器状态		
相关指令	ATYPE, MPOS		

ENDMOVE -- 当前运动目标位置

类型	轴状态
描述	轴当前运动的最终目标绝对位置。 对 VMOVE, DATUM 等非固定距离的运动, ENDMOVE 并不准确, 或时刻在变化。

语法	VAR1 = ENDMOVE
适用控制器	通用
例子	<pre> Base(0) Speed = 10 '速度 10units/s Dpos = 0 '坐标清 0 Move(100) '运动 100units Wait idle Print ENDMOVE(0) '运行到该条语句时当前运动 move(100) 完成 '打印结果, 100 Move(200) Wait idle Print ENDMOVE(0) '运行到该条语句时当前运动 move(200) 完成 '打印结果, 300 </pre>
相关指令	DPOS , MPOS , ENDMOVE_BUFFER

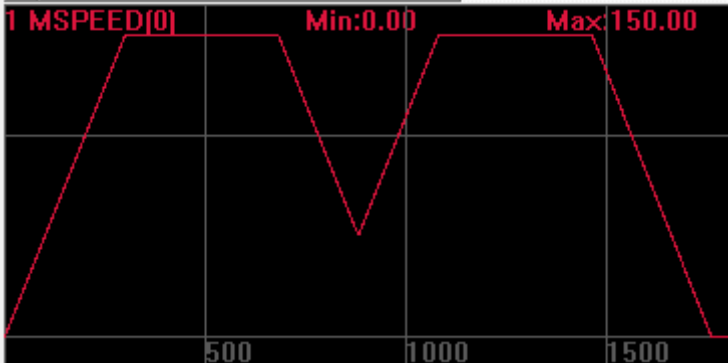
ENDMOVE_BUFFER -- 缓冲最终位置

类型	轴状态
描述	轴当前和缓冲的所有运动的最终目标位置。 可用来进行绝对和相对位置的转换，见例二。 对 VMOVE, DATUM 等非固定距离的运动，ENDMOVE_BUFFER 并不准确，或时刻在变化。 使用了循环坐标指令 REP_OPTION 后，ENDMOVE_BUFFER 根据 REP_OPTION 模式按 REP_DIST 设置值依次递减，即最小精度是 REP_DIST（模式 1）或 2 倍 REP_DIST（模式 0），见例三。
语法	VAR1 = ENDMOVE_BUFFER
适用控制器	通用
例子	<pre> 例一 Base(0) Speed = 10 Dpos = 0 Move(100) Move(200) Print ENDMOVE_BUFFER (0) '直接打印出最终运动完后的绝对坐标 '打印结果, 300 例二 相对绝对转换 </pre>

	<pre> BASE (0) UNITS=100 SPEED=100 ACCEL=1000 DPOS=0 WHILE 1 MOVE(100- ENDMOVE_BUFFER(0)) ' 运动到 100 位置，不会继续运动 WEND 使用 endmove_buffer 与相对运动指令可实现绝对运动的功能，常用于 MSPHERICAL 等只有相对模式的指令。 例三 循环坐标时的返回值 BASE (0) UNITS=100 SPEED=100 ACCEL=1000 DPOS=0 TRIGGER move(1000) REP_DIST=100 ' 设置坐标循环范围 REP_OPTION=1 ' 0~100 循环 WHILE 1 ?ENDMOVE_BUFFER(0) ' 此时打印出 1000, 900, 800..., 100, 0 ' 打印的最小精度是 100 WEND </pre>
相关指令	DPOS , MPOS , ENDMOVE

ENDMOVE_SPEED -- SP 运动结束速度

类型	轴参数
描述	自定义速度的 SP 运动的结束速度，这个参数被带入运动缓冲。 只有使用了带 SP 的运动指令才生效。 不使用时请设置较大值。
语法	VAR1 = ENDMOVE_SPEED, ENDMOVE_SPEED = expression
适用控制器	通用
例子	<pre> BASE (0) MERGE=1 </pre> ' 开启连续插补

	SPEED=100 ACCEL=500 DEVEL=500 FORCE_SPEED=150 ENDMOVE_SPEED=50 TRIGGER MOVESP(100) MOVESP(100) 速度曲线 编号 数据源 偏移 刻度 0 MSPEED 0 100 1 MSPEED[0] Min:0.00 Max:150.00  不限速度时 1 MSPEED[0] Min:0.00 Max:150.00 
相关指令	FORCE_SPEED , *SP , STARTMOVE_SPEED

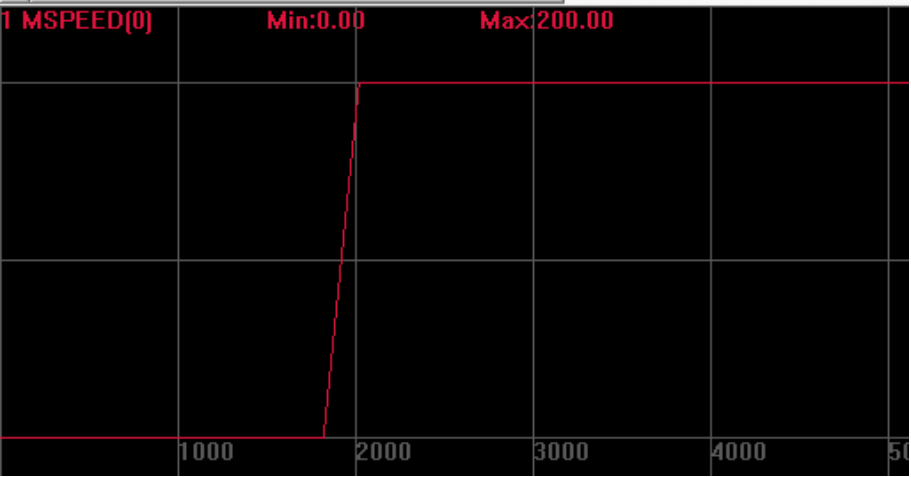
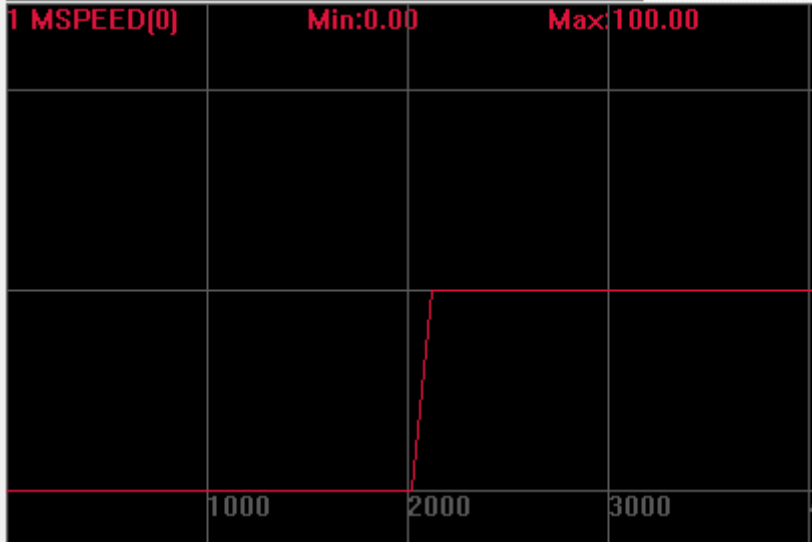
ERRORMASK -- 错误时操作

类型	轴参数
描述	和 AXISSTATUS 做与运算来决定哪些错误需要关闭 WDOG。

语法	VAR1 = ERRORMASK, ERRORMASK = expression
适用控制器	通用
相关指令	AXISSTATUS , WDOG

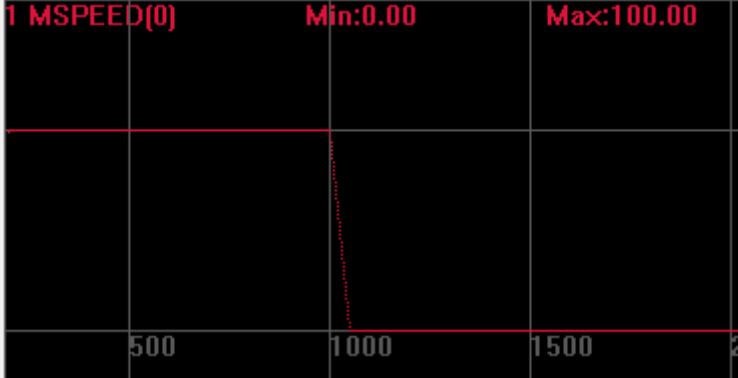
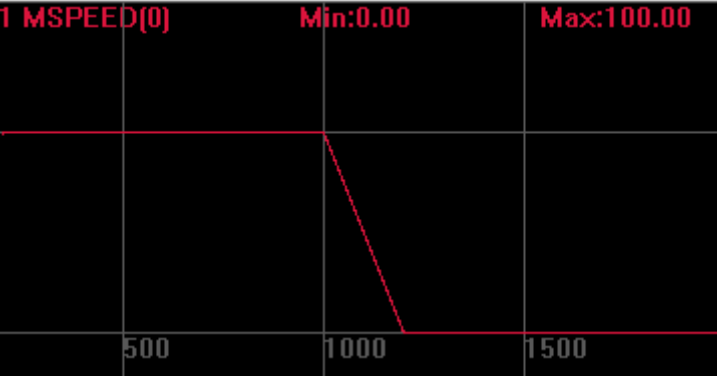
FAST_JOG -- 映射点动输入

类型	轴参数								
描述	快速点动的输入的编号，-1 为无效。 如果设置快速点动输入，速度由 SPEED 参数给出。如果没有输入设置，速度由 JOGSPEED 参数给出。见例一 输入 OFF 时，认为有信号输入，要相反效果可以用 INVERT_IN 反转电平 (ECI 系列控制器除外)。								
语法	VAR1 = FAST_JOG, FAST_JOG = expression								
适用控制器	通用								
例子	例一 BASE(0) ’ 选择轴 0 SPEED=100 ’ 设置速度为 100 units/s ACCEL=500 ’ 加速度为 500units/s/s JOGSPEED=200 ’ 点动速度设为 200units/s FAST_JOG(0)=0 ’ 轴 0 的快速输入设为 IN0 口 FWD_JOG(0)=1 ’ 正向点动开关设为 IN1 口 INVERT_IN(0, ON) ’ 反转电平 INVERT_IN(1, ON) TRIGGER ’ 自动触发示波器 IN0 无输入时，按下 IN1，轴速度为 JOGSPEED=200 <table><tr><th>编号</th><th>数据源</th><th>偏移</th><th>刻度</th></tr><tr><td>0</td><td>MSPEED</td><td>0</td><td>100</td></tr></table>	编号	数据源	偏移	刻度	0	MSPEED	0	100
编号	数据源	偏移	刻度						
0	MSPEED	0	100						

	
	IN0 有输入时，按下 IN1，轴速度为 SPEED=100
	
相关指令	REV_JOG ， FWD_JOG ， SPEED ， JOGSPEED

FASTDEC -- 快减减速度

类型	轴参数
描述	快减减速度，单位为 units/s/s。 在 CANCEL，RAPIDSTOP，达到限位或异常停止时自动采用。 当设置为 0 值或小于 DECEL 值时自动为 DECEL。
语法	VAR1 = FASTDEC，FASTDEC= expression
适用控制器	通用

例子	BASE (0)		' 选择轴 0							
	SPEED=100		' 速度设为 100							
	DECEL=500		' 减速度 500							
	FASTDEC=2000		' 快减减速度 2000							
	TRIGGER		' 自动触发示波器							
	VMOVE (1)		' 持续正转							
	delay (1000)		' 等待 1s							
	CANCEL (2)		' 停止							
	减速曲线									
	<table><tr><th>编号</th><th>数据源</th><th>偏移</th><th>刻度</th></tr><tr><td>0</td><td>MSPEED</td><td>0</td><td>100</td></tr></table>			编号	数据源	偏移	刻度	0	MSPEED	0
编号	数据源	偏移	刻度							
0	MSPEED	0	100							
										
FASTDEC=10 时										
										
相关指令	DECEL									

FE -- 当前随动误差

类型	轴状态
描述	随动误差，该值等于 DPOS-MPOS。
语法	VAR1 = DPOS

适用控制器	通用
相关指令	MPOS , DPOS

FE_LIMIT -- 最大随动误差设置

类型	轴参数
描述	最大允许的随动误差值。 预留
语法	VAR1 = FE_LIMIT, FE_LIMIT = expression
相关指令	FE , FE_RANGE

FE_RANGE -- 报警时随动误差

类型	轴参数
描述	报警时的随动误差值。 预留
语法	VAR1 = FE_RANGE, FE_RANGE = expression
相关指令	FE , FE_LIMIT

FHOLD_IN -- 映射保持输入

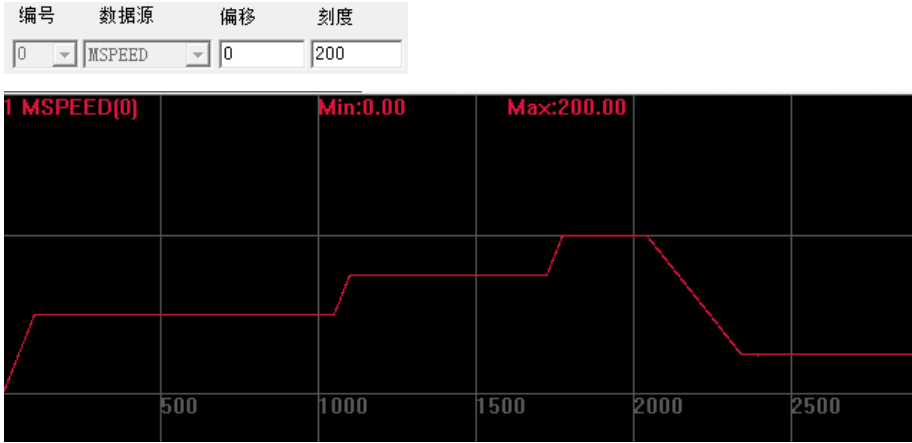
类型	轴参数
描述	保持输入对应的输入点编号，-1 无效。 如果有输入信号，运动轴的速度由 FHSPEED 参数设置。当前运动没有被取消。当取消输入，过程中的运动速度返回程序速度。见例一 输入 OFF 时，认为有信号输入，要相反效果可以用 INVERT_IN 反转电平 (ECI 系列控制器除外)。 此参数只适用速度控制方式(带 SP 后缀指令)。如果运动不是速度控制 (CAMBOX, CONNECT, MOVELINK)，不会起作用。
语法	VAR1 = FHOLD_IN, FHOLD_IN = expression
适用控制器	通用
例子	例一 BASE(0) ’ 选择轴 0 FORCE_SPEED=100 ’ 设置速度为 100 units/s

FHSPEED -- 保持速度

类型	轴参数
描述	轴保持速度，在 FHOLD_IN 被按下时的速度，单位为 units/s。
语法	VAR1 = FHSPEED, FHSPEED = expression
适用控制器	通用
例子	见 FHOLD_IN 例程
相关指令	FHOLD_IN , SPEED

FORCE_SPEED -- SP 速度

类型	轴参数
描述	自定义速度的 SP 运动的强制速度。

	这个参数被带入运动缓冲，单位为 units/s。 当 FORCE_SPEED 大于 SPEED 时，SPEED 同时也会生效限制住最高速度(140716 以后版本 SPEED 不起作用)。 如果要求进入本段的时候 FORCE_SPEED 已经降为对应的速度，请设置 STARTMOVE_SPEED。
语法	VARI = FORCE_SPEED, FORCE_SPEED = expression
适用控制器	通用
例子	BASE(0) UNITS=100 ' 脉冲当量 100 ACCEL=500 DECEL=500 SPEED = 100 ' 速度 100units/s FORCE_SPEED=150 ' 自定义速度 150units/s TRIGGER ' 自动触发示波器 MOVE(100) ' 不带 SP 使用 SPEED 速度 MOVESP(100) ' 速度为 150 FORCE_SPEED=200 MOVESP(100) ' 速度为 200 FORCE_SPEED=50 MOVESP(100) ' 速度为 50 速度曲线 
相关指令	*SP , SPEED

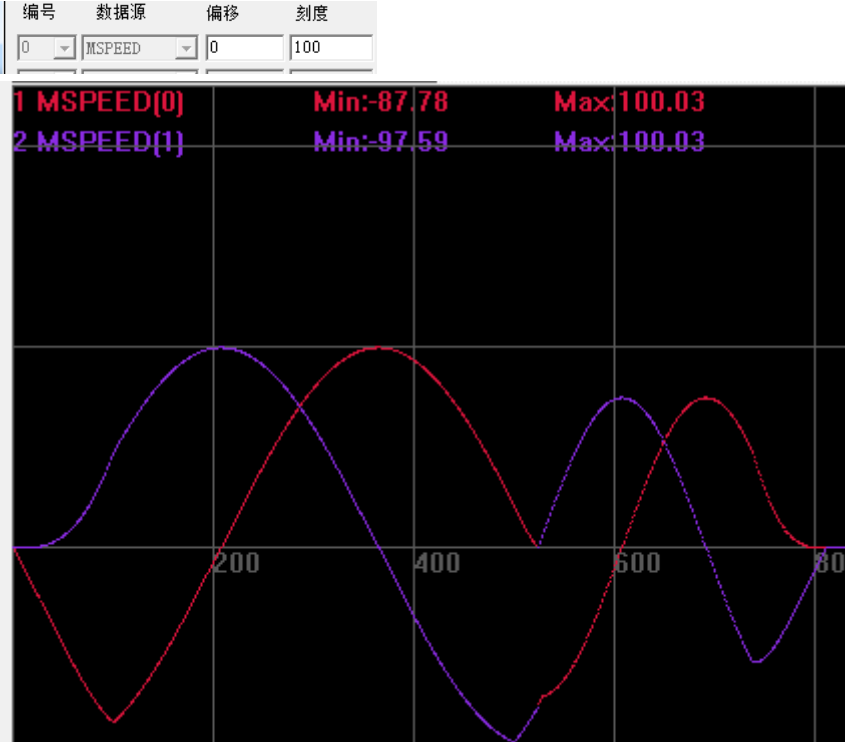
FS_LIMIT -- 正向软限位设置

类型	轴参数
----	-----

描述	正向软限位位置，单位是 units。 当 FS_LIMIT 大于 REP_DIST 时，参数不起作用，正向软限位被禁止。 取消软限位时，建议不要去修改 REP_DIST 的值，将 FS_LIMIT 设置一个较大值即可。 软限位无法作为 DATUM 回零时的限位信号参考。
语法	VAR1 =FS_LIMIT, FS_LIMIT = expression
适用控制器	通用
例子	例一 BASE(0) ’ 选择轴 0 ATYPE=1 ’ 轴类型设置 UNITS=100 ’ 脉冲当量 100 DPOS=0 ’ 坐标清 0 SPEED=100 ’ 速度 100units/s ACCEL=1000 ’ 加速度 1000units/s/s FS_LIMIT=200 ’ 设置正向软限位 200units MOVE(300) ’ 运动 300units 此时轴运动到 200 位置时会停止，并报错 Axis:0 AXISSTATUS:200h, FSOFT 继续操作轴时只能向负向运行。 取消设置时，只需要把值设大些。 FS_LIMIT=2000000 ’ 取消正向软限位设置
相关指令	RS_LIMIT , , FWD_IN , REV_IN

FULL_SP_RADIUS -- 限速半径

类型	轴参数
描述	小圆限速的最大圆弧半径，单位是 units。 当半径大于 FULL_SP_RADIUS 时，速度是用户程序指定的速度值，当半径小于 FULL_SP_RADIUS 时，控制器会按比例减小速度。 $VP_SPEED = FORCE_SPEED * radius / FULL_SP_RADIUS$ 设置自动倒角时为倒角的参考半径。
语法	VAR1 = FULL_SP_RADIUS, FULL_SP_RADIUS = expression
适用控制器	通用
例子	例一 BASE(0, 1) ’ 选择轴 0 为主轴 ATYPE=1, 1 ’ 轴类型设置 DPOS=0, 0 ’ 坐标清 0 SPEED=100 ’ 主轴速度 100units/s

	FORCE_SPEED=150 ' 自定义速度 159units/s ACCEL=1000 ' 加速度 1000units/s/s CORNER_MODE=8 ' 开启小圆限速 FULL_SP_RADIUS=8 ' 限速半径 8 TRIGGER ' 自动触发示波器 MOVECIRC(10, 10, 0, 10, 1) ' 圆弧半径 10，不限速，按 speed=100 运行 MOVECIRC(4, 4, 0, 4, 1) ' 圆弧半径 4，限速，速度按 4/8*150=75 运行 速度曲线 
相关指令	CORNER_MODE ， FORCE_SPEED ， SPLIMIT_RADIUS

SPLIMIT_RADIUS -- 限速值

类型	轴参数
描述	小圆限速的最小速度，单位是 units。 当小于于 LSPEED 时，按 LSPEED 值。
语法	VAR1 = SPLIMIT_RADIUS, SPLIMIT_RADIUS = expression
适用控制器	4 系列控制器 170518 以上固件版本
相关指令	CORNER_MODE ， FULL_SP_RADIUS

FWD_IN -- 映射正限位输入

类型	轴参数
描述	正向硬件限位开关对应的输入点编号，-1 无效。 控制器限位信号生效后，会立即停止轴，停止减速度为 FASTDEC。 输入 OFF 时，认为有信号输入，要相反效果可以用 INVERT_IN 反转电平 (ECI 系列控制器除外)。
语法	VAR1 = FWD_IN, FWD_IN = expression
适用控制器	通用
例子	例一 BASE (0, 1, 2, 3) ’ 选择轴 0, 1, 2, 3 FWD_IN=6, 7, 8, 9 ’ 分别设置正向限位开关 INVERT_IN (6, on) ’ 反转信号 INVERT_IN (7, on) INVERT_IN (8, on) INVERT_IN (9, on) 当开关输入信号时，对应轴的轴状态会报警 Axis:0 AXISSTATUS:10h, FWD。 此时只可以负向运动。
相关指令	REV_IN , FS_LIMIT , FASTDEC

FWD_JOG -- 映射正向 JOG 输入

类型	轴参数
描述	正向 JOG 输入对应的输入点编号，-1 无效。 输入 OFF 时，认为有信号输入，要相反效果可以用 INVERT_IN 反转电平 (ECI 系列控制器除外)。 当有信号输入时，对应轴按照 JOGSPEED 速度正向运动。
语法	VAR1 = FWD_JOG, FWD_JOG= expression
适用控制器	通用
例子	例一 BASE (0) ’ 选择轴 0 SPEED=100 ’ 速度 100 JOGSPEED=50 ’ JOG 速度 50 FWD_JOG=0 ’ 输入 IN0 作为正向 JOG 开关

INVERT_IN(0, on) ' 反转信号
TRIGGER ' 自动触发示波器

输入 0 口有信号输入时，轴 0 正向运行，速度为 50。
输入 0 口取消信号输入时，轴 0 停止。

速度曲线

1 MSPEED[0] Min:0.00 Max:50.00

1600 1800 2000 2200 2400

0

I GAIN -- 积分增益

类型	轴参数
描述	积分增益，非模拟量伺服不支持。 预留
语法	I_GAIN = expression
相关指令	P_GAIN ， D_GAIN ， OV_GAIN ， VFF_GAIN

IDLE -- 运动状态

类型	轴状态
描述	轴运动状态判断，只能判断是运动中还是停止。 运动中，返回 0，运动结束，返回-1。

	当轴关联为机械手，CONNFRAME 逆解时，关节轴一直返回 0；CONNREFRAME 正解时，虚拟轴一直返回 0。
语法	VAR1 = IDLE
适用控制器	通用
例子	举例 1: <pre>If IDLE(0) then ' 如果轴 0 停止 Base(1) Move(100) EndIf</pre> 举例 2: <pre>Base(0, 1) Move(100, 100) Base(2, 3) Move(200, 200) Wait until idle(0) and idle(1) and idle(2) and idle(3) ' 等待轴 0, 1, 2, 3 停止</pre>
相关指令	LOADED , WAIT IDLE

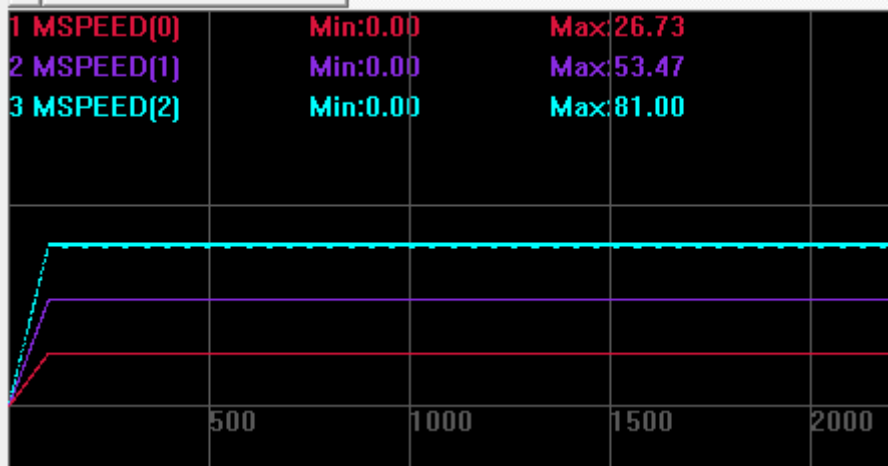
INVERT_STEP -- 脉冲模式设置

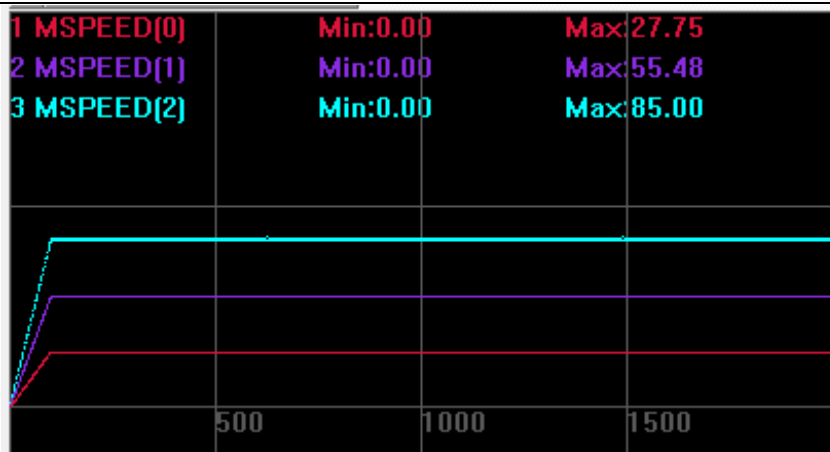
类型	轴参数																											
描述	步进脉冲输出模式设置。 有脉冲方向和双脉冲两种模式，控制器默认为脉冲方向控制。																											
语法	INVERT_STEP = mode 参数：mode(缺省 0) 低 8 位（位 0-位 7）表示的模式值如下。 0-3 脉冲方向模式。4-7 双脉冲方式（部分控制器版本不支持，详细咨询厂商），8/9 AB 输出(部分控制器定制) 各个模式对应的电平如下： <table border="1"> <thead> <tr> <th rowspan="2">模式值</th><th colspan="2">正向运动</th><th colspan="2">负向运动</th></tr> <tr> <th>脉冲线</th><th>方向线</th><th>脉冲线</th><th>方向线</th></tr> </thead> <tbody> <tr> <td>0</td><td></td><td>High</td><td></td><td>Low</td></tr> <tr> <td>1</td><td></td><td>High</td><td></td><td>Low</td></tr> <tr> <td>2</td><td></td><td>Low</td><td></td><td>High</td></tr> </tbody> </table>				模式值	正向运动		负向运动		脉冲线	方向线	脉冲线	方向线	0		High		Low	1		High		Low	2		Low		High
模式值	正向运动		负向运动																									
	脉冲线	方向线	脉冲线	方向线																								
0		High		Low																								
1		High		Low																								
2		Low		High																								

	3		Low		High
	4		High	High	
	5	High			High
	6		Low	Low	
	7	Low			Low
	8	AB 输出, 定制			
	9	BA 输出, 定制			
高 8 位（位 8-位 15）表示方向变化保护时间，单位微秒：0-255 如果模式设定不正确，步进马达可能会在换向时丢失一步的位置，当不确定步进马达的设置时，可以设置 100 微秒左右的保护时间。					
适用控制器	通用				
例子	设置为脉冲方向模式 <code>INVERT_STEP = 256*100+0</code> ' 设置 100 微秒的保护时间，模式为 0。 设置为双脉冲模式 <code>INVERT_STEP = 256*100+6</code> ' 设置 100 微秒的保护时间，模式为 6。				
相关指令	STEP_RATIO				

INTERP_FACTOR -- 插补速度计算

类型	轴参数
描述	插补时轴是否参与速度计算，缺省参与（1）。 此参数只对直线的任意轴和螺旋的第三个轴起作用。 运动后请及时取消，否则会导致后续运动不正确。 部分轴不参与速度计算时，先按参与计算的轴的插补计算出参与轴的分速度和运动总时间，再把不参与计算的轴的运动距离依次除以总时间，得到不参与计算的各轴的相应速度。见例二 不能把插补的实际运动的所有轴都设置 0，会导致实际速度无穷大。
语法	<code>INTERP_FACTOR = 0/1</code>

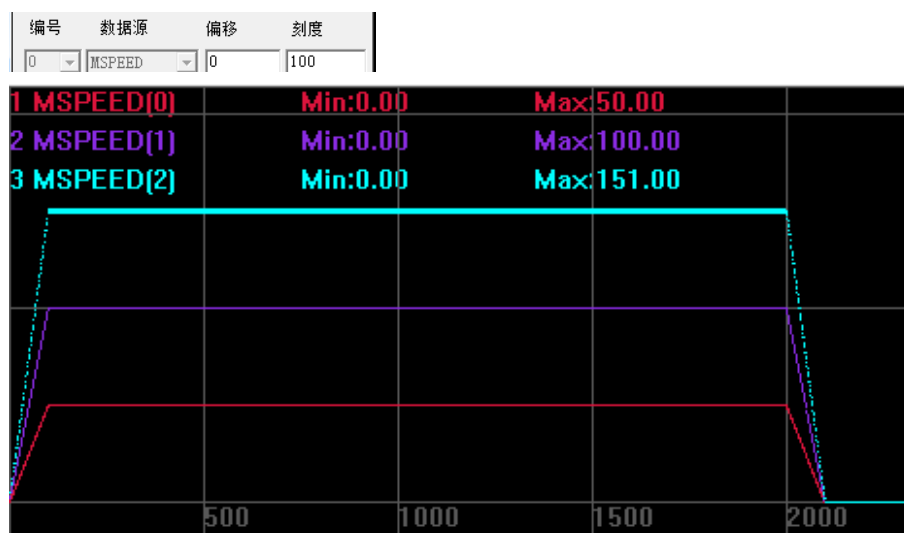
适用控制器	通用
例子	例一 全部参与速度计算 BASE (0, 1, 2) ’ 选择轴 0 为主轴 UNITS=100, 100, 100 ’ 脉冲当量 100 SPEED=100 ’ 主轴速度 100units/s INTERP_FACTOR=1, 1, 1 ’ 3 轴都参与速度计算 TRIGGER ’ 自动触发示波器 MOVE (100, 200, 300) ’ 各轴分运动距离 此时根据合成运动速度 100 可以计算出各轴分速度，如下图。 编号 数据源 偏移 刻度 0 MSPEED 0 100  例二 部分轴不参与速度计算 BASE (0, 1, 2) ’ 选择轴 0 为主轴 UNITS=100, 100, 100 ’ 脉冲当量 100 SPEED=100 ’ 主轴速度 100units/s INTERP_FACTOR=0, 1, 1 ’ 轴 0 不参与速度计算。 TRIGGER ’ 自动触发示波器 MOVE (100, 200, 300) ’ 各轴分运动距离 此时先计算出仅有 2、3 轴插补时的分速度、运动总时间，然后再以轴 0 的运动距离除以总时间，得到轴 0 的速度，如下图。 编号 数据源 偏移 刻度 0 MSPEED 0 100



例三 只有一个轴参与计算

INVERT_FACTOR=0, 1, 0 只有轴 1 进行运动计算。

此时轴 1 的速度就是主轴速度 100，然后计算出运动总时间。其他轴依次用运动距离除以总时间，得到各轴速度，如下图。



相关指令

[BASE_MOVE](#)

JOGSPEED -- JOG 速度

类型	轴参数
描述	JOG 时的速度，单位为 units/s。 当 REV_JOG/FWD_JOG 被设置，并对应输入点按下时，电机将以 JOGSPEED 慢速运动。
语法	JOGSPEED= value

适用控制器	通用
例子	例一 BASE(0) ' 选择轴 0 SPEED=100 ' 速度 100 JOGSPEED=50 ' JOG 速度 50 FWD_JOG=0 ' 输入 IN0 作为正向 JOG 开关 REV_JOG=1 ' 输入 IN1 作为负向 JOG 开关 INVERT_IN(0, on) ' 反转信号 INVERT_IN(1, on) TRIGGER ' 自动触发示波器 输入 0 口有信号输入时，轴 0 正向运行，速度为 50。 输入 1 口有信号输入时，轴 0 负向运行，速度为 50。 同时有信号输入时，轴 0 正向运行。 速度曲线 编号 数据源 偏移 刻度 0 MSPEED 0 100 I MSPEED(0) Min:-50.00 Max:50.00 

LINK_AXIS -- 连接轴号

类型	轴状态
描述	返回当前连接运动的参考轴号。 无连接时返回-1。
语法	VAR1 = LINKAX
适用控制器	通用

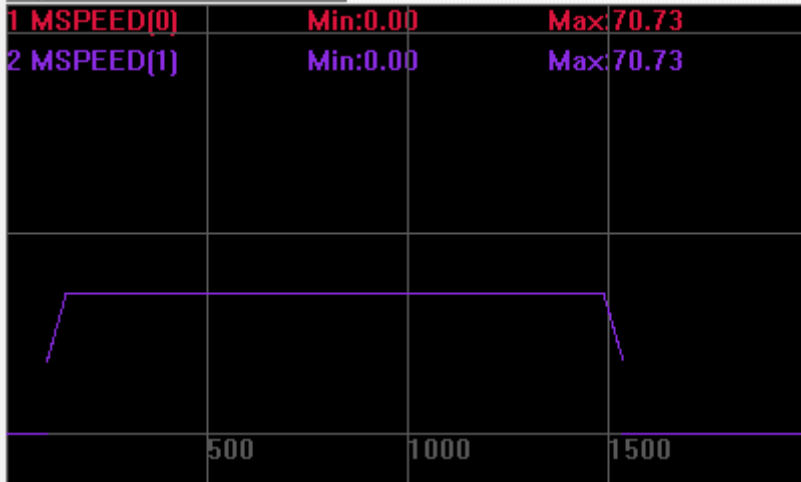
例子	CONNECT (2,1) AXIS (0) ' 轴 0 连接到轴 1 上 ?LINK_AXIS (0) ' 打印轴 0 的参考轴号，结果，1
相关指令	CAMBOX ， MOVELINK ， CONNECT

LOADED -- 缓冲空

类型	轴状态
描述	除了当前运动外，没有缓冲的指令了，此时返回 ture。
语法	VAR1 = LOAED
适用控制器	通用
相关指令	IDLE ， WAIT LOADED

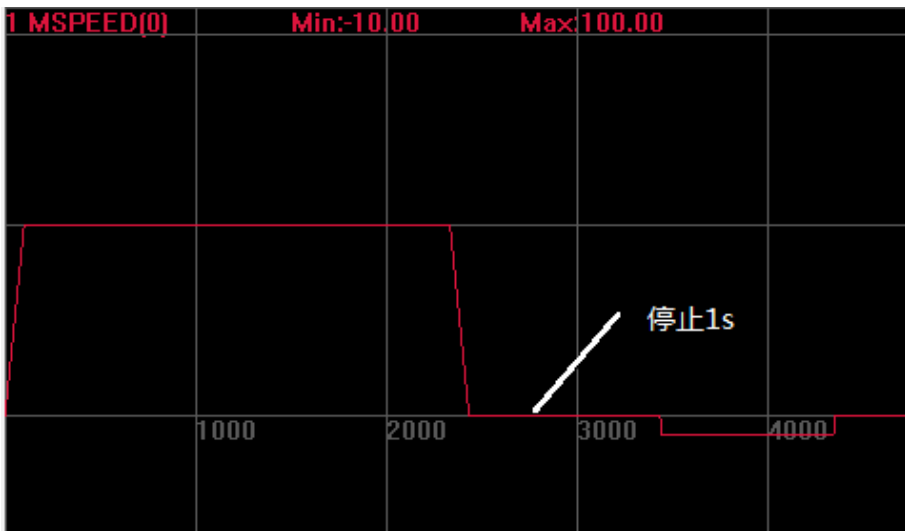
LSPEED -- 起始速度

类型	轴参数
描述	轴起始速度，同时用于停止速度，缺省 0，单位为 units/s。 当多轴运动时，作为插补运动的起始速度。 当需要追求效率时，可以考虑设置起始速度。
语法	VAR1 = LSPEED, LSPEED = expression
适用控制器	通用
例子	例一 BASE (0, 1)

	
相关指令	SPEED

HOMEWAIT -- 回零反找延时

类型	轴参数
描述	对脉冲方式的伺服驱动器，回原点运动时，当反找时要等待一定时间。此参数设置等待的时间，单位是毫秒。 控制器默认值为 2ms。
语法	VAR1 = HOMEWAIT, HOMEWAIT = expression
适用控制器	通用
例子	BASE(0) ' 选择轴 0 SPEED=100 ' 找原点速度 100units/s CREEP=10 ' 爬行速度 10units/s DATUM_IN=0 ' 原点信号设为输入 IN0 INVERT_IN(0, ON) ' 反转信号 HOMEWAIT=1000 ' 设置反找等待 1s TRIGGER ' 自动触发示波器 DATUM(3) ' 正向找原点 速度曲线 碰到 IN0 口时，会先停止 1s，然后在反向爬行 

		相关指令
DATUM		

MARK -- 锁存触发

类型	轴状态
描述	返回锁存事件是否产生。 当 REGIST 指令执行时，MARK 变真，返回-1。当执行完成时，MARK 变成假，返回 0。
语法	VAR1 = MARK
适用控制器	通用

例子	BASE(0) ' 选择轴 0 MOVE(100) ' 运动 100units REGIST(3) ' 上升沿 R0 WAIT UNTIL MARK ' 等待触发
相关指令	REG_POS , REGIST

MARKB -- 锁存 2 触发

类型	轴状态
描述	返回对应第二个锁存位置的锁存事件是否产生。 当 REGIST 指令执行时, MARK 变真, 返回-1。当执行完成时, MARK 变成假, 返回 0。
语法	VAR1 = MARKB
适用控制器	通用
例子	BASE(0) ' 选择轴 0 MOVE(100) ' 运动 100units REGIST(14) ' 上升沿 R1 WAIT UNTIL MARKB ' 等待触发
相关指令	REG_POSB , REGIST

MARKC -- 锁存 3 触发

类型	轴状态
描述	返回对应第三个锁存位置的锁存事件是否产生。 当 REGIST 指令执行时, MARK 变真, 返回-1。当执行完成时, MARK 变成假, 返回 0。
语法	VAR1 = MARKC
适用控制器	通用
例子	BASE(0) ' 选择轴 0 MOVE(100) ' 运动 100units REGIST(18) ' 上升沿 R2 WAIT UNTIL MARKC ' 等待触发
相关指令	REG_POSC , REGIST

MARKD -- 锁存 4 触发

类型	轴状态
描述	返回对应第四个锁存位置的锁存事件是否产生。 当 REGIST 指令执行时，MARK 变真，返回-1。当执行完成时，MARK 变成假，返回 0。
语法	VAR1 = MARKD
适用控制器	通用
例子	BASE(0) ' 选择轴 0 MOVE(100) ' 运动 100units REGIST(20) ' 上升沿 R3 WAIT UNTIL MARKD ' 等待触发
相关指令	REG_POSD , REGIST

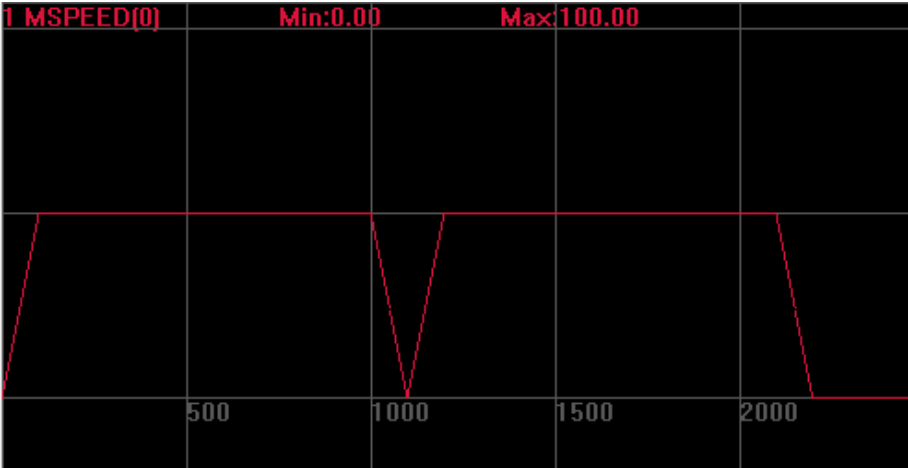
MAX_SPEED -- 脉冲频率限制

类型	轴参数
描述	脉冲输出的最高频率限制。 一旦发现超过此设置会强制，并且设置 AXISSTATUS。 对编码器轴，设置值低于 500K 时会启用编码器滤波，设置值高于 1M 时会取消编码器滤波设置。
语法	MAX_SPEED = value
适用控制器	通用
例子	Base(0) MAX_SPEED =500000 ' 设置脉冲速度限制 500K
相关指令	AXISSTATUS

MERGE -- 连续插补

类型	轴参数
描述	前后缓冲的运动连接到一起而不减速，用于连续插补。

	当 MERGE 设置为 ON 时，多段插补间仍减速，可能原因如下： 1. 可能 MERGE 并没有设置成功，可以打印查看 2. 控制器是点位运动型号，不支持连续插补，可联系厂家 3. 设置了 CORNER_MODE 拐角减速，打印确认 4. 使用了带 SP 的运动指令，并设置了 ENDMOVE_SPEED, STARTMOVE_SPEED, 此时速度由这两条指令确定 5. 多条插补间切换了主轴，主轴速度参数改变了 6. 多条插补间加入了 MOVE_DELAY 延时指令，即使延时写 0，也会导致减速
语法	MERGE = ON/OFF
适用控制器	通用
例子	BASE(0) ' 选择轴 0 UNITS=100 SPEED=100 ' 速度 100units ACCEL=1000 MERGE=ON ' 打开连续插补 TRIGGER ' 自动触发示波器 MOVE(100) ' 运动 100units MOVE(100) 速度曲线 编号 数据源 偏移 刻度 0 MSPEED 0 100 1 MSPEED[0] Min:0.00 Max:100.00 500 1000 1500 2000 MERGE=OFF 时

				
相关指令	*SP, CORNER MODE			

MOVES_BUFFERED -- 当前缓冲数

类型	轴状态
描述	返回当前被缓冲起来的运动指令个数。 不要用 MOVES_BUFFERED 减这个参数来判断是否有剩余的缓冲空间，特殊的运动指令可能会占用多个缓冲空间，采用 REMAIN_BUFFER 状态函数来判断更准确。
语法	VAR1 = MOVES_BUFFERED
适用控制器	通用
例子	Print MOVES_BUFFERED ' 打印结果，0
相关指令	LOADED , LIMIT_BUFFERED , REMAIN_BUFFER

MOVE_CURMARK -- 当前运动标号

类型	轴状态
描述	返回当前轴正在运动指令的 MOVE_MARK 标号。
语法	VAR1 = MOVE_CURMARK
适用控制器	通用
例子	MOVE_MARK =1 MOVE (100)

	<pre> MOVE_MARK =2 MOVE(200) MOVE_MARK =3 MOVE(300) WAIT UNTIL MOVE_CURMARK = 2 '等待 MOVE(200)开始执行的时候，开输出 口 1 OP(1,ON) </pre>
相关指令	MOVE_MARK

MOVE_MARK -- 运动标号

类型	轴参数
描述	下一条要调用的运动指令的 MARK 标号，这个标号会和运动指令一起写入运动缓冲。 每调用一条运动指令，MOVE_MARK 会自动加一。 如果要强制指定 MOVE_MARK，需要每次运动前都设定一次。 通过 MOVE_PAUSE 指令可以在 MARK 不同的边界处暂停。
语法	VAR1 = MOVE_MARK, MOVE_MARK = expression
适用控制器	通用
例子	<pre> MOVE_MARK =1 ' 设置为标号 1 MOVE(100) MOVE_MARK =1 ' 设置为标号 1 MOVE(100) MOVE_MARK =2 ' 设置为标号 2 MOVE(200) MOVE_PAUSE (2) ' MOVE(200)前暂停 </pre>
相关指令	MOVE_CURMARK

MPOS -- 编码器反馈位置

类型	轴状态
描述	轴的测量反馈位置，单位是 units。 写 MPOS 会自动转换为 OFFPOS 偏移。
语法	VAR1 = MPOS, MPOS=expression

适用控制器	通用
例子	<pre>Mpos(0) = 50 'MPOS 偏移至 50 ?*mpos '打印结果, 50 0 0 0 0 0 0 0 0 0 0 0</pre>
相关指令	DPOS , ENDMOVE

MSPEED -- 实际反馈速度

类型	轴状态
描述	轴的测量反馈位置速度，单位是 units/s。
语法	VAR1 = MSPEED
适用控制器	通用
相关指令	UNITS , VP SPEED

MTYPE -- 当前运动类型

类型	轴状态																						
描述	当前正在进行的运动指令类型。 当插补联动时，对从轴总是返回主轴的运动指令类型。																						
语法	VAR1 = MTYPE <table border="1"> <thead> <tr> <th>MTYPE</th><th>运动指令类型</th></tr> </thead> <tbody> <tr> <td>0</td><td>IDLE (没有运动)</td></tr> <tr> <td>1</td><td>MOVE</td></tr> <tr> <td>2</td><td>MOVEABS</td></tr> <tr> <td>3</td><td>MHELICAL</td></tr> <tr> <td>4</td><td>MOVECIRC</td></tr> <tr> <td>5</td><td>MOVEMODIFY</td></tr> <tr> <td>6</td><td>MOVESP</td></tr> <tr> <td>7</td><td>MOVEABSSP</td></tr> <tr> <td>8</td><td>MOVECIRCSP</td></tr> <tr> <td>9</td><td>MHELICALSP</td></tr> </tbody> </table>	MTYPE	运动指令类型	0	IDLE (没有运动)	1	MOVE	2	MOVEABS	3	MHELICAL	4	MOVECIRC	5	MOVEMODIFY	6	MOVESP	7	MOVEABSSP	8	MOVECIRCSP	9	MHELICALSP
MTYPE	运动指令类型																						
0	IDLE (没有运动)																						
1	MOVE																						
2	MOVEABS																						
3	MHELICAL																						
4	MOVECIRC																						
5	MOVEMODIFY																						
6	MOVESP																						
7	MOVEABSSP																						
8	MOVECIRCSP																						
9	MHELICALSP																						

	10	FORWARD
	11	REVERSE
	12	DATUMING
	13	CAM
	14	FWD_JOG
	15	REV_JOG
	20	CAMBOX
	21	CONNECT
	22	MOVELINK
	23	CONNPATH
	25	MOVESLINK
	26	MSPIRAL
	27	MECLIPSE/ MECLIPSEABS/ MECLIPSESP/ MECLIPSEABSSP
	28	MOVE_OP/MOVE_OP2 MOVE_TABLE MOVE_TASK MOVE_PARA MOVE_PWM MOVE_ASYNMOVE MOVE_AOUT
	29	MOVE_DELAY MOVE_WAIT MOVE_SYNMOVE
	31	MSPHERICAL/ MSPHERICALSP
	32	MOVE_PT
	33	CONNFRAME
	34	CONNREFRAME
适用控制器	通用	

例子	<pre> WHILE 1 ' 循环判断 IF MTYPE=0 THEN ?"没有运动" ELSEIF MTYPE=1 THEN ?"直线插补" ELSEIF MTYPE=4 THEN ?"圆弧插补" ... ENDIF WEND </pre>
相关指令	NTYPE , REMAIN BUFFER

NTYPE -- 下条运动类型

类型	轴状态
描述	当前正在进行的运动指令后面的第一条指令类型。 当插补联动时，对从轴总是返回主轴的运动指令类型。
语法	VAR1 = NTYPE
适用控制器	通用
例子	<pre> WHILE 1 ' 循环判断 IF NTYPE=0 THEN ?"下一条结束运动" ELSEIF NTYPE=1 THEN ?"下一条直线插补" ELSEIF NTYPE=4 THEN ?"下一条圆弧插补" ... ENDIF WEND </pre>
相关指令	MTYPE

OFFPOS -- 偏移位置

类型	轴参数
描述	相对偏移修改所有的坐标，不会对已运行/进入缓冲区的运动产生影响。

	当修改完成后，OFFPOS 还原为 0。
语法	VAR1 = OFFPOS, OFFPOS = expression
适用控制器	通用
例子	例一 相对偏移位置 <pre>Base (0) Moveabs (1000) Wait idle OFFPOS = -1000 ' 坐标偏移 1000 PRINT Dpos (0) ' 打印结果 0</pre> 例二 不改变在运行运动 <pre>Base (0) Moveabs (1000) ' 运动到绝对位置 1000 OFFPOS =500 ' 位置偏移 500 Wait idle PRINT Dpos (0) ' 打印当前位置 1500，此时电机仍运动 1000</pre> 例三 与 DEFPOS 改变为绝对位置不同，OFFPOS 是相对改变 <pre>BASE (0, 1) ' 选择轴 0，轴 1 DPOS=100, 100 ' 设置当前位置为 100, 100 ?DPOS (0), DPOS (1) ' 打印确认</pre> <pre>DEFPOS (10, 20) ' 设置当前位置为 10, 20 ?DPOS (0), DPOS (1) ' 当前位置为 10, 20 DEFPOS (10, 20) ' 多次调用 DEFPOS DEFPOS (10, 20) ?DPOS (0), DPOS (1) ' 此时当前位置仍为 10, 20</pre> <pre>OFFPOS=10, 20 ' 多次调用 OFFPOS OFFPOS=10, 20 ?DPOS (0), DPOS (1) ' 此时当前位置变为 30, 60 (10+10+10, 20+20+20)</pre>
相关指令	DPOS , DEFPOS

OPEN_WIN -- 锁存范围

类型	轴参数
----	-----

描述	锁存触发的结束坐标范围点。 预留
语法	OPEN_WIN = pos
相关指令	REGIST , CLOSE_WIN

OV_GAIN -- 速度增益

类型	轴参数
描述	速度增益，非模拟量伺服不支持。 预留
语法	VAR1 = OV_GAIN, OV_GAIN = expression
相关指令	P_GAIN , D_GAIN , I_GAIN , VFF_GAIN

P_GAIN -- 比例增益

类型	轴参数
描述	比例增益，非模拟量伺服不支持。 预留
语法	VAR1 = P_GAIN, P_GAIN = expression
相关指令	I_GAIN , D_GAIN , OV_GAIN , VFF_GAIN

PP_STEP -- 编码器内部比例

类型	轴参数
描述	编码器输入内部会乘以这个比例。 这个参数效果与 ENCODER_RATIO 叠加，却省 1。
语法	PP_STEP = value
适用控制器	通用
相关指令	ENCODER_RATIO

REG_POS -- 锁存位置

类型	轴状态
描述	保存锁存的测量反馈位置(MPOS)，units 单位。
语法	VAR1 = REG_POS
适用控制器	通用
例子	<pre> MOVE(100) ' 运动 100units REGIST(3) ' 上升沿 R0 WAIT UNTIL MARK ' 等待锁存发生 PRINT REG_POS ' 打印出锁存位置 </pre>
相关指令	REGIST ， MARK

REG_POSB -- 锁存 2 位置

类型	轴状态
描述	返回锁存寄存器 2 的测量反馈位置(MPOS)，units 单位。
语法	VAR1 = REG_POSB
适用控制器	通用
例子	<pre> MOVE(100) ' 运动 100units REGIST(16) ' 上升沿 EZ 信号触发 WAIT UNTIL MARKB ' 等待第二个锁存触发 PRINT REG_POSB ' 打印触发时位置 </pre>
相关指令	REGIST ， MARKB

REG_POSC -- 锁存 3 位置

类型	轴状态
描述	返回锁存寄存器 3 的测量反馈位置(MPOS)，units 单位。
语法	VAR1 = REG_POSC
适用控制器	通用
例子	<pre> MOVE(100) ' 运动 100units REGIST(18) ' 上升沿信号触发 </pre>

	WAIT UNTIL MARKC '等待第二个锁存触发 PRINT REG_POSC '打印触发时位置
相关指令	REGIST , MARKC

REG_POSD -- 锁存 4 位置

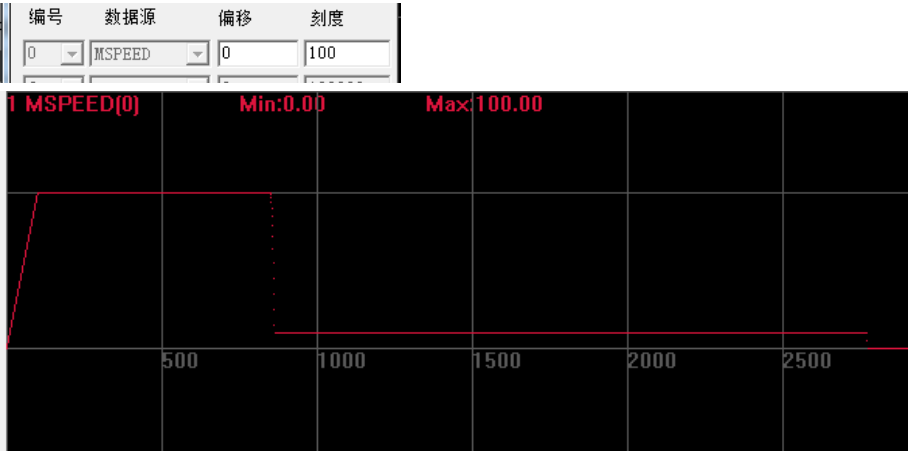
类型	轴状态
描述	返回锁存寄存器 4 的测量反馈位置(MPOS)，units 单位。
语法	VAR1 = REG_POSD
适用控制器	通用
例子	MOVE(100) '运动 100units REGIST(20) '上升沿信号触发 WAIT UNTIL MARKD '等待第二个锁存触发 PRINT REG_POSD '打印触发时位置
相关指令	REGIST , MARKD

REG_INPUTS -- 锁存输入映射

类型	轴状态		
描述	设置 R0-R3 输入锁存的输入口映射，每 4bit 对应一个锁存口。		
语法	VAR1 = REG_INPUTS		
	bit	锁存口	输入口范围 (ZMC304)
	Bit0-3	R0	0-7
	Bit4-7	R1	0-7
	Bit8-11	R2	0-7
	Bit12-15	R3	0-7
	 输入口范围：不同的控制器可以使用的范围不一样。		
适用控制器	4 系列及以上，最新固件		
例子	BASE (6) ATYPE=3 REG_INPUTS=\$3210 ‘R0-R3 分别对应输入口 0，1，2，3 REG_INPUTS=\$6666 ‘R0-R3 都对输入口 6		

相关指令	REGIST

REMAIN -- 当前运动剩余距离

类型	轴状态
描述	返回轴当前运动还未完成的距离，units 单位。
语法	VECTOR_BUFFERED
适用控制器	通用
例子	BASE (0) ' 选择轴 0 SPEED=100 ' 速度 100units/s ACCEL=1000 ' 加速度 1000units/s TRIGGER ' 自动触发示波器 MOVE(100) ' 运动 100units WAIT UNTIL REMAIN<20 ' 等待剩余距离小于 20 SPEED=10 ' 修改速度 速度曲线 
相关指令	VECTOR_BUFFERED

REMAIN_BUFFER -- 剩余缓冲数

类型	特殊轴状态
描述	返回可以剩余使用的缓冲个数。 此状态比较特殊，本身可以带参数，因此修正轴号时 AXIS 不能省掉。

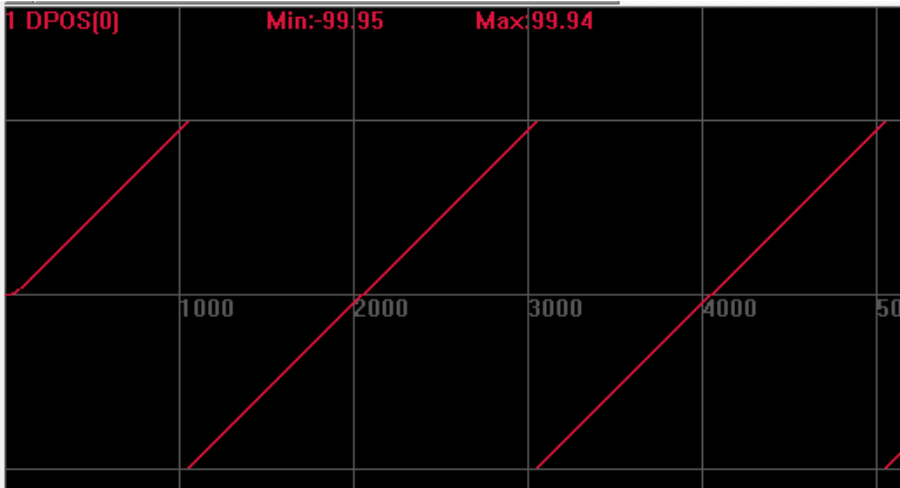
	当返回 0 时表示当前轴的缓冲空间满，此时如果继续调用当前轴的运动指令会阻塞任务直到缓冲有空位。
语法	VAR1 = REMAIN_BUFFER ([mtype]) AXIS (AXISNUM) Mtype 缺省为最复杂的运动，如空间圆弧。
适用控制器	通用
例子	<pre> DIM movetime ' 定义变量 movetime = 0 WHILE movetime < 100 ' 条件循环 IF REMAIN_BUFFER(1) > 0 THEN ' 如果有剩余缓冲，调用直线运动指令 MOVE(10) movetime = movetime +1 ENDIF WEND ' 调用了 100 次 move(10) </pre>
相关指令	LOADED , LIMIT_BUFFERED

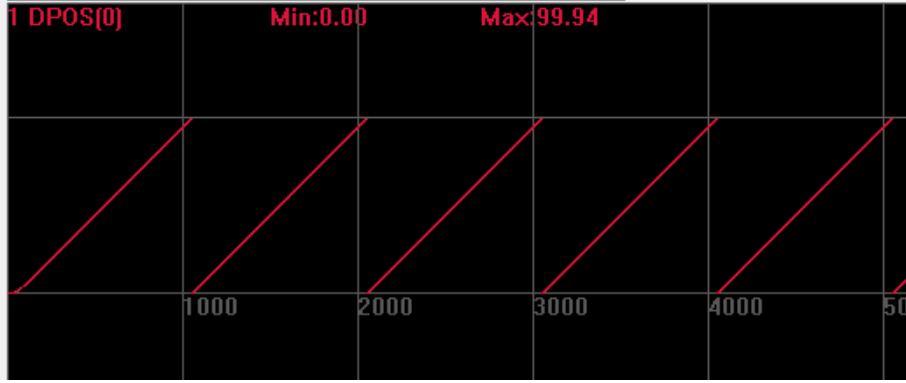
REP_DIST -- 坐标循环位置

类型	轴参数
描述	根据 REP_OPTION 设置来自动循环轴 DPOS 和 MPOS 坐标。
语法	VAR1 = REP_DIST, REP_DIST = expression
适用控制器	通用
例子	参考 REP_OPTION
相关指令	REP_OPTION

REP_OPTION -- 坐标循环模式

类型	轴参数
描述	坐标重复设置。 可用于限制凸轮主轴的坐标循环范围，来实现多条凸轮曲线连续。 使用绝对运动模式时，如果目标位置处于坐标循环范围内，可以正确运动；如果处于坐标循环范围外，运动不正确。 相对运动不受影响。

语法	VAR1 = REP_OPTION, REP_OPTION = opt opt, 按位来表示不同的意义。 <table><tr><th>位</th><th>值</th><th>描述</th></tr><tr><td>0</td><td>1</td><td>0-循环范围 - REP_DIST 到 + REP_DIST. 1-循环范围为 0 到 + REP_DIST。</td></tr><tr><td>1</td><td>2</td><td>写 1 禁止 CAMBOX 和 MOVELINK 的重复运动, 禁止生效后还原为 0。</td></tr><tr><td>2</td><td>4</td><td>预留</td></tr><tr><td>4</td><td>16</td><td>1-不使用 REP_DIST, 0-使用 REP_DIST。</td></tr></table>	位	值	描述	0	1	0-循环范围 - REP_DIST 到 + REP_DIST. 1-循环范围为 0 到 + REP_DIST。	1	2	写 1 禁止 CAMBOX 和 MOVELINK 的重复运动, 禁止生效后还原为 0。	2	4	预留	4	16	1-不使用 REP_DIST, 0-使用 REP_DIST。
位	值	描述														
0	1	0-循环范围 - REP_DIST 到 + REP_DIST. 1-循环范围为 0 到 + REP_DIST。														
1	2	写 1 禁止 CAMBOX 和 MOVELINK 的重复运动, 禁止生效后还原为 0。														
2	4	预留														
4	16	1-不使用 REP_DIST, 0-使用 REP_DIST。														
适用控制器	通用															
例子	BASE (0) ' 选择轴 0 UNITS=100 ' 脉冲当量 100 DPOS=0 ' 坐标清 0 SPEED=100 ' 速度 100units/s ACCEL=1000 ' 加速度 1000units/s/s REP_DIST=100 ' 设置坐标循环范围 REP_OPTION=0 ' 设置循环模式 TRIGGER ' 自动触发示波器 VMOVE (1) ' 持续运动 坐标曲线 <table><tr><th>编号</th><th>数据源</th><th>偏移</th><th>刻度</th></tr><tr><td>0</td><td>MSPEED</td><td>0</td><td>100</td></tr></table> REP_OPTION=1 时	编号	数据源	偏移	刻度	0	MSPEED	0	100							
编号	数据源	偏移	刻度													
0	MSPEED	0	100													

	
相关指令	CAMBOX , MOVELINK , REP_DIST

REV_IN -- 映射负限位输入

类型	轴参数
描述	负向硬件限位开关对应的输入点编号，-1 无效。 控制器限位信号生效后，会立即停止轴，停止减速度为 FASTDEC。 输入 OFF 时，认为有信号输入，要相反效果可以用 INVERT_IN 反转电平 (ECI 系列控制器除外)。
语法	VAR1 = REV_IN, REV_IN = expression
适用控制器	通用
例子	参考 FWD_IN 例子
相关指令	FWD_IN , RS_LIMIT , FASTDEC

REV_JOG -- 映射负向 JOG 输入

类型	轴参数
描述	负向 JOG 输入对应的输入点编号，-1 无效。 输入 OFF 时，认为有信号输入，要相反效果可以用 INVERT_IN 反转电平 (ECI 系列控制器除外)。 当有信号输入时，对应轴按照 JOGSPEED 速度负向运动。 当 REV_JOG 和 FWD_JOG 同时有信号输入时，轴按照 FWD_JOG 运行。
语法	VAR1 = REV_JOG, REV_JOG= expression
适用控制器	通用

例子	参考 FWD_JOG 例子
相关指令	FWD_JOG , JOGSPEED , FAST_JOG

RS_LIMIT -- 负向软限位设置

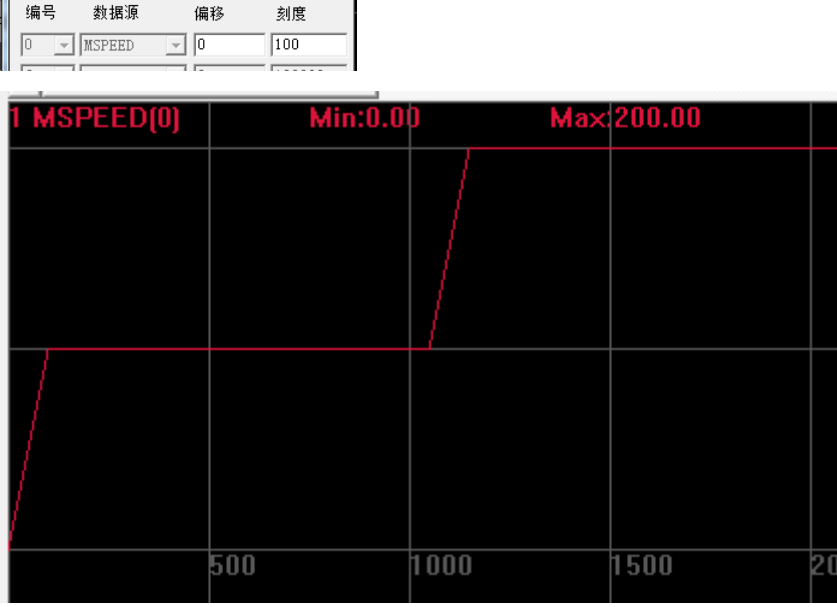
类型	轴参数
描述	负向软限位位置，单位是 units。 当 RS_LIMIT 大于 REP_DIST 时，参数不起作用，正向软限位被禁止。 取消软限位时，建议不要去修改 REP_DIST 的值，将 RS_LIMIT 设置一个较大值即可。 软限位无法作为 DATUM 回零时的限位信号参考。
语法	VAR1 =RS_LIMIT, RS_LIMIT = expression
适用控制器	通用
例子	参考 FS_LIMIT 例子
相关指令	FS_LIMIT , FWD_IN , REV_IN

SERVO -- 闭环开关

类型	轴参数
描述	闭环开关设置。 预留
语法	VAR1 = SERVO, SERVO = ON/OFF
相关指令	WDOG

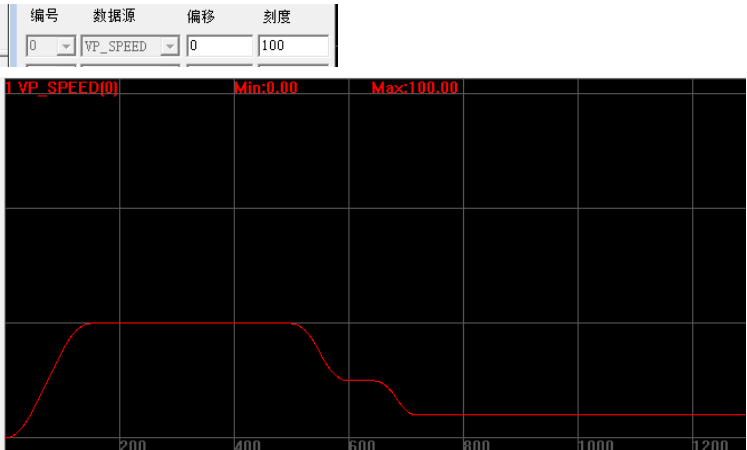
SPEED -- 运动速度

类型	轴参数
描述	轴速度，单位为 units/s。 当多轴运动时，作为插补运动的速度。 SPEED 修改后，立刻生效，可以实现动态变速，但是改变瞬间会抖。希望平滑变速请使用 SPEED_RATIO 指令。 当 SP 指令的 FORCE_SPEED 大于 SPEED 时，SPEED 速度也会生效。(140716 以后版本 SPEED 不起作用)

语法	VAR1 = SPEED, SPEED = expression
适用控制器	通用
例子	BASE (0) UNITS=100 ' 脉冲当量 SPEED =100 ' 设置 0 轴速度 100units/s ACCEL=1000 ' 加速度 1000units/s/s DPOS=0 ' 坐标清 0 TRIGGER ' 自动触发示波器 VMOVE (1) ' 持续运动 WAIT UNTIL DPOS (0)>100 ' 等待轴 0 运行到 100 SPEED=200 ' 速度改为 200 速度曲线 
相关指令	FORCE SPEED , SPEED_RATIO

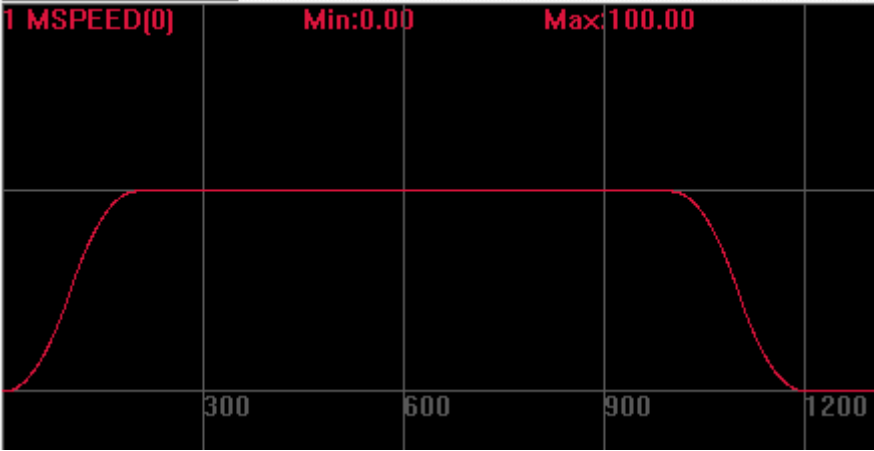
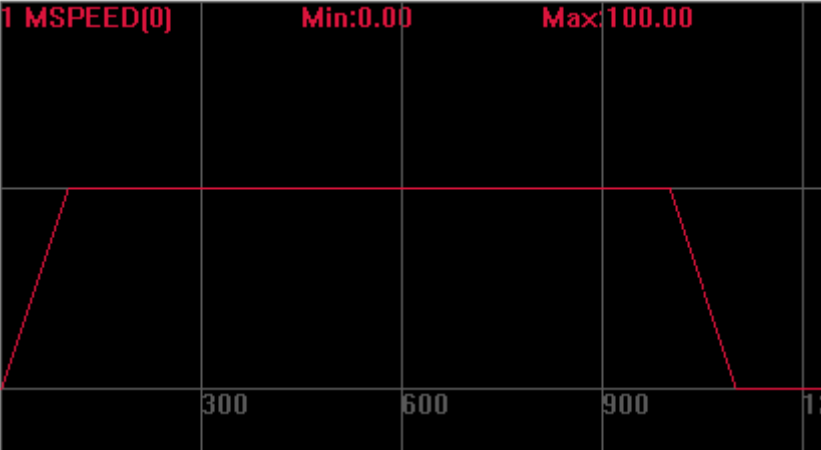
SPEED_RATIO -- 速度比例

类型	轴参数
描述	轴速度比例，0-1。 轴实际运动速度为 SPEED*SPEED_RATIO。 用于运动中根据加减速的平滑速度改变。
语法	SPEED_RATIO = value Value: 比例，0-1
适用控制器	最新固件支持

例子	<pre> rapidstop(2) wait idle speed_ratio = 1 trigger dpos = 0 speed = 100 accel = 1000 decel = 1000 merge = on sramp = 50 move(1111111) delay(500) ’等待 0.5s speed_ratio = 0.5 ’速度降为 50 wait until vp_speed < 80 Delay(100) ’等待 0.1s speed_ratio = 0.2 ’速度降为 20 end </pre> 速度曲线 
相关指令	FORCE_SPEED, SPEED

SRAMP -- 加减速曲线

类型	轴参数
描述	加减速过程 S 曲线设置。
语法	VAR1 = SRAMP, SRAMP = smoothms smoothms 0-250 毫秒，设置后加减速过程会变长相应的时间。
适用控制器	通用

例子	BASE (0)		' 选择轴 0
	UNITS=100		' 脉冲当量 100
	SPEED=100		' 速度 100units/s
	ACCEL=1000		' 加速度 1000units/s/s
	SRAMP=100		' S 曲线时间 100ms
	TRIGGER		' 自动触发示波器
	MOVE(100)		' 运动 100units
	速度曲线		
	编号 数据源 偏移 刻度 0 MSPEED 0 100		
			
	SRAMP=0 时		
			
相关指令	ACCEL , DECEL		

STARTMOVE_SPEED -- SP 运动开始速度

类型	轴参数
描述	自定义速度的 SP 运动的开始速度，这个参数被带入运动缓冲。 只有使用了带 SP 的运动指令才生效。 不使用时请设置较大值。
语法	VAR1 =STARTMOVE_SPEED, STARTMOVE_SPEED = expression
适用控制器	通用
例子	参考 ENDMOVE_SPEED 例程
相关指令	FORCE_SPEED , *SP , ENDMOVE_SPEED

STOP_ANGLE -- 拐角减速结束

类型	轴参数
描述	停止减速到的角度，单位是弧度。 角度转换弧度公式： 角度值*(PI/180) 减速角度 是指下一插补运动与上一插补运动方向延长线的角度，如下图。 下一插补运动轨迹处于下方时取绝对值。 直线与圆弧相连时按照圆弧的切线方向计算角度。 与 DECEL_ANGLE 一起使用，当实际运动的角度处于 DECEL_ANGLE（上限）与 STOP_ANGLE（下限）时才会减速。

语法	VAR1 = STOP_ANGLE, STOP_ANGLE= expression
适用控制器	通用
例子	CORNER_MODE=2 DECEL_ANGLE = 25 * (PI/180) STOP_ANGLE = 90 * (PI/180) FORCE_SPEED=SPEED ’一定要设置 FORCE_SPEED 参考 CORNER_MODE 例程一
相关指令	DECEL_ANGLE , CORNER_MODE

TRANS_DPOS

类型	轴状态
描述	预留

UNITS -- 脉冲当量

类型	轴参数																
描述	脉冲当量，指定每单位发送的脉冲数，支持 5 位小数精度。 控制器以 UNITS 作为基本单位，修改后坐标显示会随改 UNITS 变比例变化。 比如 <table><tr><td>UNITS</td><td>10</td><td>DPOS</td><td>3000</td></tr><tr><td></td><td></td><td>MPOS</td><td>3000</td></tr></table> 修改 UNITS 后 <table><tr><td>UNITS</td><td>100</td><td>DPOS</td><td>300</td></tr><tr><td></td><td></td><td>MPOS</td><td>300</td></tr></table>	UNITS	10	DPOS	3000			MPOS	3000	UNITS	100	DPOS	300			MPOS	300
UNITS	10	DPOS	3000														
		MPOS	3000														
UNITS	100	DPOS	300														
		MPOS	300														
语法	可读：VAR1 = UNITS 或 VR1=UNITS(轴号) 可写：UNITS= expression 或 UNITS(轴号)= expression																
适用控制器	通用																
例子	如何设置 假设电机 U=3600 脉冲转一圈，丝杠一圈螺距 P=2mm。 电机转 1° 对应的 UNITS 则 UNITS=U/360=3600/360=10，此时 MOVE(1)，电机转 1° 工作台走 1mm 对应的 UNITS																

则 $\text{UNITS} = U/P = 3600/2 = 1800$ ，此时 MOVE(1)，工作台走 1mm
机台存在减速比时，要把减速比算上，假设减速比 i=2:1。
则 $\text{UNITS} = U*i/P = 3600*2/2 = 3600$

编程使用

BASE(0, 1, 2) ’ 选择轴 0，轴 1，轴 2

UNITS=10, 100, 1000, 30 ’ 轴 0 设为 10，轴 1 设为 100，轴 2 设为 1000，
 ’ 轴 3 设为 30

与 BASE 联合使用时，数据多出 BASE 选择的轴数，依次往后设置轴。
数据少于 BASE 选择的轴数，就只设置对应的轴。

UNTIS(2)=100 ’ 直接设置轴 2，与 BASE 选择轴无关。

VECTOR BUFFERED --缓冲剩余距离

类型	轴状态
描述	返回轴当前当前运动和缓冲运动还未完成的距离，units 单位。 对多轴插补是矢量距离。
语法	VAR1 = VECTOR_BUFFERED
适用控制器	通用
例子	<pre> BASE(0) ' 选择轴 0 UNITS=100 ' 脉冲当量 100 SPEED=100 ' 速度 100units/s ACCEL=1000 ' 加速度 1000units/s/s MOVE(100) ' 当前运动 100units MOVE(300) ' 缓冲运动 300units MOVE(-1000) ' 缓冲运动-1000units ?VECTOR_BUFFERED ' 返回剩余运动距离， 结果， 1400 </pre>
相关指令	REMAIN

VECTOR MOVED -- 当前运动距离

类型	轴状态
描述	返回轴当前运动的距离，units 单位。 对多轴插补是矢量距离。

	用之前最好手动清零。
语法	VAR1 = VECTOR_MOVED VECTOR_MOVED=0
适用控制器	通用
例子	VECTOR_MOVED=0 '手动清 0 MOVE(100) WAIT IDLE ? VECTOR_MOVED '打印出轴 0 运动距离，结果，100
相关指令	ENDMOVE

VFF_GAIN -- 前馈增益

类型	轴参数
描述	速度反馈的前馈增益，非模拟量伺服不支持。 预留
语法	VAR1 = VFF_GAIN, VFF_GAIN = expression
相关指令	P_GAIN , I_GAIN , D_GAIN , OV_GAIN

VP_SPEED -- 当前运动速度

类型	轴状态
描述	返回轴当前运动的速度，单位为 units/s。 当多轴运动时，主轴返回的是插补运动的速度，不是主轴的分速度。 非主轴返回的是相应的分速度，与 MSPEED 效果一致。
语法	VAR1 = VP_SPEED
适用控制器	通用
例子	BASE(0,1) UNITS=100,100 '脉冲当量 SPEED =100 '主轴速度 100units/s ACCEL=1000 '加速度 1000units/s/s DPOS=0 '坐标清 0 TRIGGER '自动触发示波器 MOVE(100,100) '两轴各运动 100units 速度曲线

主轴 VP_SPEED 是插补合成运动的速度。
 非主轴 VP_SPEED 是轴当前分速度，与 MSPEED 一致。

编号	数据源	偏移	刻度
0	VP_SPEED	0	100
0	MSPEED	0	100
1	VP_SPEED	-300	100

1 VP_SPEED[0]

Min:0.00

Max:100.00

2 MSPEED[0]

Min:0.00

Max:70.72

3 VP_SPEED[1]

Min:0.00

Max:70.72

相关指令
 [MSPEED](#)，
 [SPEED](#)

ZSMOOTH -- 倒角半径

类型	轴参数
描述	参考倒角半径，CONER_MODE 使用。 根据拐角角度自动计算实际拐角半径大小。超过路径时限制为 50%。 90° 时拐角半径为设置值。
语法	VAR1 = ZSMOOTH, ZSMOOTH = smoothdistance
适用控制器	通用
例子	参考 CONER_MODE 例三
相关指令	CORNER_MODE

第七章 输入输出相关

AIN -- 模拟量输入

类型	输入输出函数
描述	读取模拟输入，返回 AD 转换模块的刻度值。 刻度值范围 0~4096 对应 0~10V 电压。
语法	Var=AIN(channel) channel 模拟输入通道 0-127 扩展板 AD 通道号与拨码有关，起始值为（8 + 拨码组合值*8），详细查看硬件手册。
适用控制器	通用
例子	a=AIN(1) ' 读取通道 1 的 AD a=AIN(1) *10 /4096 ' 通道 1 的电压值
相关指令	AOUT

AOUT -- 模拟量输出

类型	输入输出指令或函数
描述	模拟通道输出。 刻度值范围 0~4096 对应 0~10V 电压。
语法	AOUT(channel) = value channel 模拟输出通道 0-63 扩展板 DA 通道号与拨码有关，起始值为（4 + 拨码组合值*4），详细查看硬件手册。
适用控制器	通用
例子	AOUT(1) = 0 ' 关闭输出 DA 通道 0 AOUT(1) = 4096 ' DA1 口输出 10V 电压
相关指令	AIN

IN -- 输入口

类型	输入输出函数
描述	读取输入，没有参数时返回 0-31 的状态。 读取的是 INVERT_IN 翻转以后的状态。 扩展板 IO 通道号与拨码有关，起始值为（16 +拨码组合值*16），详细查看硬件手册。
语法	IN([channel1],[channel2]) channel1 要读取的起始输入通道 channel2 要读取的结束输入通道，没有结束通道时，返回的单个输入的状态。
适用控制器	通用
例子	a=IN(1) ' 读取通道 1 的输入。
相关指令	OP , INVERT_IN

INFILTER -- 输入口滤波

类型	系统参数
描述	本地输入口的滤波参数。 值越大，滤波时间越长， 2-9，缺省 2。
语法	VAR1 = INFILTER, INFILTER= expression
适用控制器	通用
例子	INFILTER= 5 ' 在干扰严重场合加大滤波时间

IN_SCAN -- 扫描输入变化

类型	输入输出函数
描述	扫描输入变动，返回值 1(true)-变动，0(false)-没有变动。 此函数必须固定不断的扫描，返回的是两次扫描之间的变动，可以通过 IN_EVENT 读取变动的具体情况，使用的是 INVERT_IN 翻转以后的状态。 固件版本 20140214 以后的才提供这个支持，扫描范围有宽度限制。 00x 系列控制器只能在单个任务中使用。

语法	<pre>VAR1=IN_SCAN([channel1][, channel2])</pre> channel1 要读取的起始输入通道 channel2 要读取的结束输入通道，没有结束通道时，扫描单个输入。
适用控制器	通用
例子	<pre>while 1 if in_scan(0,23) then '扫描 IN0-23 口电平变化 if in_event(0) > 0 then ' IN0 上升沿触发 print "in0 up", in_buff(0) elseif in_event(0) < 0 then ' IN0 下降沿触发 print "in0 down", in_buff(0) endif endif wend</pre>
相关指令	IN_EVENT , SCAN_EVENT , IN_BUFF

IN_EVENT -- 读取输入变化

类型	输入输出函数
描述	读取输入的变化情况。 1-上升沿，-1-下降沿，0-没有变化。 此函数必须与 IN_SCAN 配合使用。
语法	VAR1 = IN_EVENT(IONUM)
适用控制器	通用
例子	参见 IN_SCAN
相关指令	IN_SCAN , SCAN_EVENT

SCAN_EVENT -- 检测变化

类型	输入输出函数
描述	检测表达式的内容变化。

	off-on 返回 1, on-off 返回-1, 不变返回 0。 150810 以上固件版本支持, 以下版本用 IN_EVENT 和 IN_SCAN
语法	ret = SCAN_EVENT (expression) expression 任意有效的表达式, 结果会转成 BOOL 类型。
适用控制器	通用
例子	例一 输入信号扫描 <pre>While 1 If scan_event(in(0)) > 0 then ' IN0 上升沿触发 Print "IN0 On" Elseif scan_event(in(0))<0 then ' IN0 下降沿触发 Print "IN0 Off" Endif Wend</pre> 例二 寄存器、变量扫描 <pre>While 1 If scan_event(TABLE(0)) > 0 then ' table0 上升沿触发 Print "TABLE0 On" Elseif scan_event(TABLE(0))<0 then ' table0 下降沿触发 Print "TABLE0 Off" Endif Wend</pre> 在线命令操作 table(0), 打印相关结果
相关指令	IN_SCAN , IN_EVENT

IN_BUFF -- 读取输入缓冲

类型	输入输出函数
描述	读取 IN_SCAN 缓冲的当前输入, 没有参数时返回 0-31 的状态。 读取的是 INVERT_IN 翻转以后的状态。
语法	IN_BUFF([channel1],[channel2]) channel1 要读取的起始输入通道, 必须是 IN_SCAN 的输入范围。 channel2 要读取的结束输入通道, 没有结束通道时, 返回的单个输入的状态。

适用控制器	通用
例子	参见 IN_SCAN
相关指令	IN_SCAN

INVERT_IN -- 反转输入

类型	特殊指令
描述	反转输入状态，可以读取判断是否有反转。
语法	<pre>INVERT_IN(channel, state); VAR1= INVERT_IN(channel)</pre> channel 输入通道 state ON/OFF
适用控制器	通用
例子	INVERT_IN(1, ON) '特殊信号默认 OFF 有效，反转避免来了限位信号. (ECI 系列控制器除外) FWD_IN(0)=1 '输入 IN1 作为轴 0 的正向限位信号
相关指令	IN

OP -- 输出口

类型	输入输出指令和函数
描述	输出或读取输出状态。 当在表达式中使用时，自动为函数语法。 扩展板 IO 编号与拨码有关，扩展板起始编号为（16 + 拨码组合值*16），详细查看硬件手册。
语法	<pre>OP([ionum],value) 或 OP(ionum1, ionum2,value[,mask]) OP([firstnum[, [finalnum]])</pre> ionum 输出编号，0-，没有这个参数时输出 0-31。 value 输出状态，多个输出 口操作时按位来指明多个口状态。 ionum1 要操作的第一个输出通道 ionum2 要操作的最后一个输出通道 mask 用位来指定哪些 IO 需要操作，不填时第一个通道到最后一个通道都操作。

	firstnum 输出编号, 0-, finalnum 输出编号, 0-, 没有这个参数时, 读取单个输出口的状态。
适用控制器	通用
例子	<pre>' 翻转输出口 0 IF OP (0) = ON THEN OP (0, OFF) ELSE OP (0, ON) ENDIF</pre>
相关指令	READ_OP , MOVE_OP

READ_OP -- 读取输出口

类型	输入输出函数
描述	读取输出状态。 同 OP, 多个输出口操作时按位来指明多个口状态。
语法	READ_OP ([firstnum[, [finalnum]]) firstnum 输出编号, 0-, finalnum 输出编号, 0-, 没有这个参数时, 读取单个输出口的状态。
适用控制器	通用
例子	<pre>' 翻转输出口 0 IF READ_OP (0) = ON THEN OP (0, OFF) ELSE OP (0, ON) ENDIF</pre>
相关指令	OP

PWM_FREQ -- pwm 频率

类型	PWM 控制函数
描述	pwm 频率设置或读取。

语法	PWM_FREQ (index, freq) 或 PWM_FREQ (index)=freq index 编号, 0-, freq 频率, 硬件 PWM 1M, 软件 PWM 2k
适用控制器	支持 PWM 功能的控制器才支持此函数。
例子	PWM_FREQ (0)=1000 '1K 频率 ?PWM_FREQ (0)
相关指令	PWM_DUTY , MOVE_PWM

PWM_DUTY -- pwm 占空比

类型	PWM 控制函数
描述	pwm 占空比设置或读取。 占空比指有效电平占整个周期的比例。 一个周期中先输出有效电平, 再输出无效电平。
语法	PWM_DUTY(index, duty) PWM_DUTY(index)=duty index 编号, 0-, Duty 占空比, 0-1, 当设置 0 的时候, PWM 关闭。
适用控制器	支持 PWM 功能的控制器才支持此函数。
例子	PWM_DUTY (0)=0.5 ?PWM_DUTY (0)
相关指令	PWM_FREQ , MOVE_PWM

PSWITCH -- 软件位置比较输出

类型	输入输出指令
描述	根据位置比较来操作输出口。 如果多个 pswitch 操作同一个输出口, 需要编号顺序排在一起。
语法	PSWITCH(num, enable, [, axis, op num, op state, set pos, reset pos]) num: 比较器的编号, ZMC1xx 系列有 16 个比较器, 编号: 0-15 enable: 操作比较器的使能 - ON 启动 / OFF 取消 axis: 指定要获取位置的轴号 op num: 操作的 IO 编号

	op state: 输出的状态, 1 表示在下面位置范围内输出为 ON, 0 表示在下面位置范围内输出为 OFF set pos: 设定产生输出的起始位置, 采用 units 单位 reset pos: 设定输出复位的位置, 采用 units 单位
适用控制器	通用
例子	PSWITCH(0, ON, 0, 1, ON, 0, 200)' 轴 0 的 0-200 位置范围内打开输出 1
相关指令	HW_PSWITCH

HW_PSWITCH -- 硬件位置比较输出

类型	轴指令
描述	硬件比较输出, 不同的轴对应不同的输出口。 缺省轴 0-5 对应输出 0 1 2 3 0 1。 总共 4 个比较输出口。 可以连续调用两个 HW_PSWITCH 指令, 通过函数方式可以获取能调用的指令个数。 每个比较点触发都会使得当前输出口电平翻转。 HW 缓冲数 1024 个, 可以连续调用 1024 个 HW 指令。 HW 指令调用后, 不受后面的坐标修改功能的影响, HW 指令 TABLE 存储的坐标必须在调用时是正确的, 因此尽量手动修改坐标, 要规避 HW 指令与坐标循环自动修改的随机冲突。 因为自动循环修改坐标不受程序控制, 无法确定是在 HW 的前面还是后面, 这样 TABLE 里面的坐标就无法确定。 总线轴请使用 HW_PSWITCH2 指令。
语法	HW_PSWITCH(mode, direction, reserve, tablestart, tableend) Buff = HW_PSWITCH([axisnum]) mode 1-启动比较器, 2- 停止并删除没完成的比较点. Direction 0-坐标负向, 1- 坐标正向, 2-不判断方向 Reserve 预留 Tablestart 第一个比较点坐标所在 TABLE 编号 tableend 最后一个比较点坐标所在 TABLE 编号
适用控制器	4 系列及以上产品, 最新固件
例子	Base(0) Atype=1 Units=100 Speed=100

函数语法: Buff = HW_PSWITCH2([axisnum])

Mode=1 :单轴比较, 见例一

HW_PSWITCH2(1, Opnum, Opstate, Tablestart, Tableend[, Direction])

参数: mode 1-启动比较器,
Opnum 对应的输出口
Opstate 第一个比较点的输出状态.
Tablestart 第一个比较点绝对坐标所在 TABLE 编号
Tableend 最后一个比较点绝对坐标所在 TABLE 编号
Direction 第一个点判断方向, 0-坐标负向, 1- 坐标正向,
-1-不使用方向

Mode=2 :清除比较点

HW_PSWITCH2(2)

参数: mode 2- 停止并删除没完成的比较点.

Mode=3 :矢量比较方式, 见例二

HW_PSWITCH2(3, Opnum, Opstate, Tablestart, Tableend)

参数: mode 3-启动比较器,
Opnum 对应的输出口
Opstate 第一个比较点的输出状态.
Tablestart 第一个比较点绝对坐标所在 TABLE 编号
Tableend 最后一个比较点绝对坐标所在 TABLE 编号

使用矢量距离比较时, 与 VECTOR_MOVED 进行比较, 建议连续运动前设置 VECTOR_MOVED 初始值

Mode=4 :矢量比较方式, 单个比较点

HW_PSWITCH2(4, Opnum, Opstate, Vectstart)

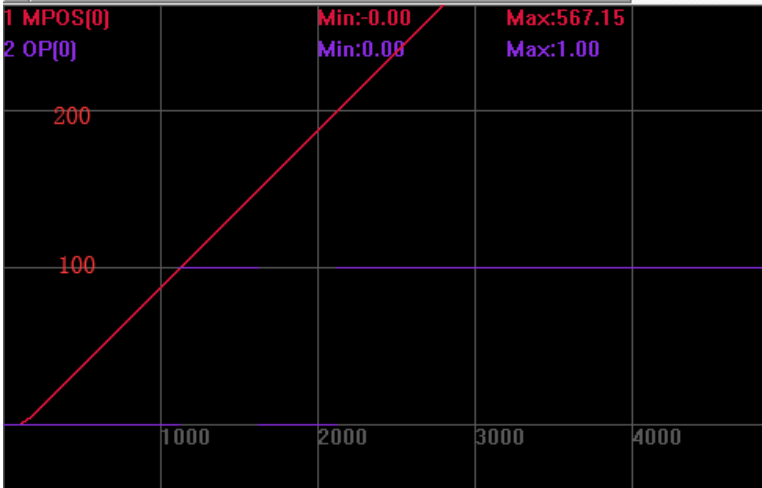
参数: mode 4-启动比较器,
Opnum 对应的输出口
Opstate 第一个比较点的输出状态.
Vectstart 比较点绝对坐标

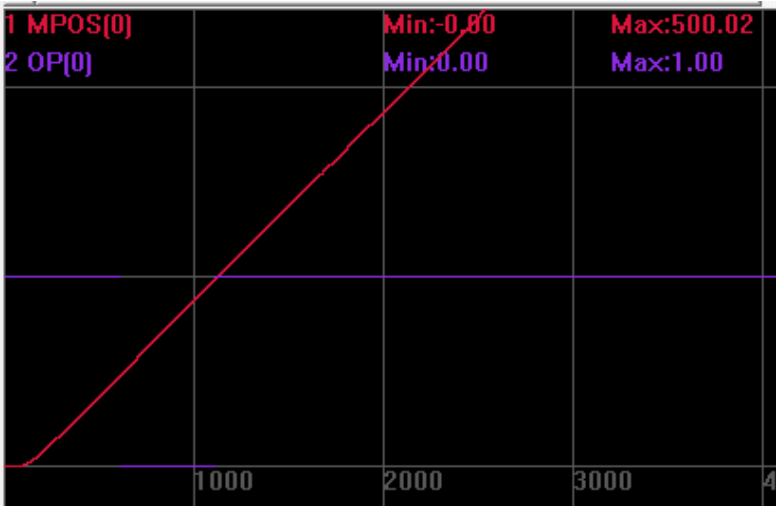
Mode=5 :矢量比较方式, 周期脉冲模式, 见例三

HW_PSWITCH2(5, Opnum, Opstate, Vectstart, Repes, Cycledis, Ondis)

参数: mode 5-启动比较器,
Opnum 对应的输出口
Opstate 第一个比较点的输出状态, 认为是有效状态, 反之认为无效状态.
Vectstart 比较点绝对坐标
Repes 重复周期, 一个周期内比较两次, 先输出有效状态, 再输出无效状态.
Cycledis 周期距离, 每隔这个距离输出 Opstate, Ondis 后还原为 无效状态.
Ondis 输出有效状态的距离, (Cycledis- Ondis)为无效状态距离

离

	Mode=6 :矢量比较方式，周期模式，与 HW_TIMER 一起使用，见例四 HW_PSWITCH2(6, Opnum, Opstate, Vectstart, Repes, Cycledis) 参数: mode 6-启动比较器, Opnum 对应的输出口 Opstate 第一个比较点的输出状态. Vectstart 比较点绝对坐标 Repes 重复周期, 一个周期只比较一次. Cycledis 周期距离, 每隔这个距离输出一次.
适用控制器	4 系列及以上产品，最新固件
例子	例一 单轴比较 总线轴使能过程省略，参考总线初始化例程。 Base(0) Dpos=0 Mpos=0 Table(0, 50, 100, 150, 200) ' 比较点坐标设置 Hw_pswitch2(1, 0, 0, 0, 3, 1) ' 比较 4 个点，操作输出口 0 TRIGGER ' 触发示波器 move(500)  运动到 50 时，关闭 OUT0；位置 100 时，打开 OUT0；位置 150 时，关闭 OUT0；位置 200 时，打开 OUT0。 例二 矢量单轴比较模式 总线轴使能过程省略，参考总线初始化例程。 Base(0) Dpos=0 Mpos=0 Table(0, 50, 100, 150, 200) ' 比较点坐标设置 VECTOR_MOVED=100 ' 设置矢量起始位置 Hw_pswitch2(3, 0, 0, 0, 3) ' 比较 4 个点，操作输出口 0 TRIGGER ' 触发示波器 move(500)



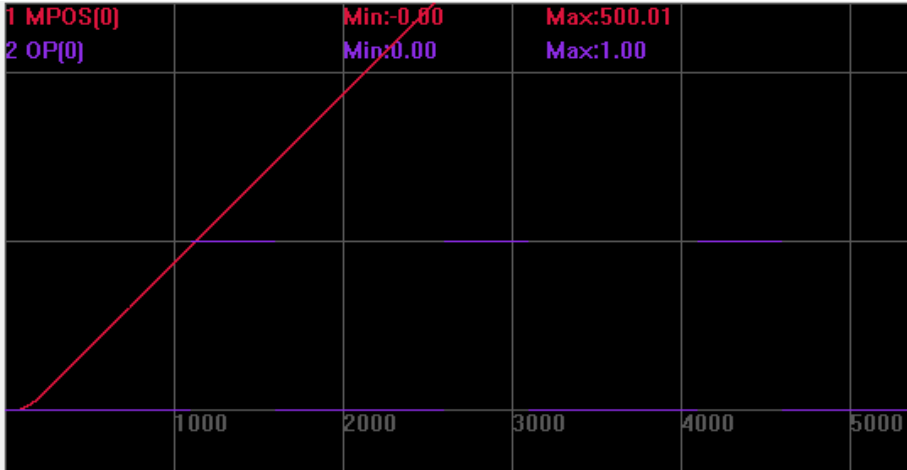
可以看出信号操作直接从例一的位置 100 处开始，矢量比较模式就是将 MPOS(当前位置)+VECTOR_MOVED(之前的位移)与设置位置进行比较。

例三 周期比较模式，距离复位

总线轴使能过程省略，参考[总线初始化例程](#)。

```
Base(0)
Dpos=0
Mpos=0
Table(0,50,100,150,200) '比较点坐标设置
VECTOR_MOVED=0 '设置矢量起始位置为0，方便观察
Hw_pswitch2(5,0,1,100,3,150,50) '位置100开始比较，比较3次
                                '周期距离150，输出有效距离50

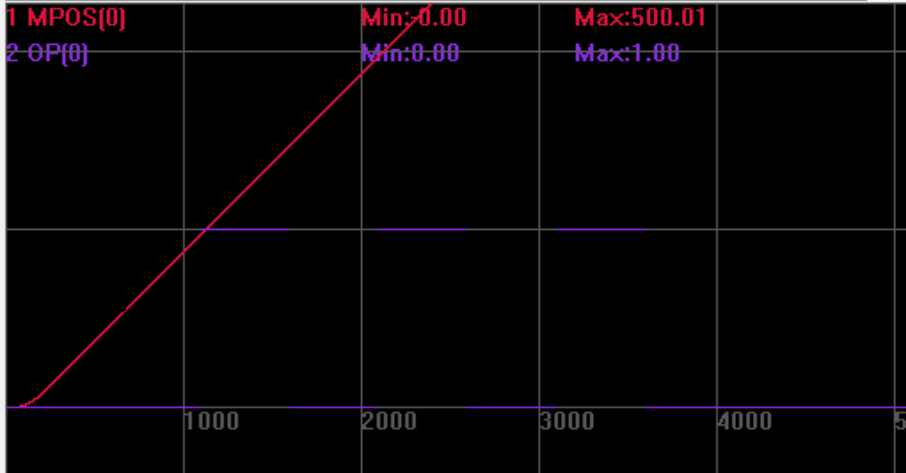
TRIGGER '触发示波器
move(500)
```



位置 100 开始比较，比较 3 次，一个和周期中前 50 输出有效状态，后 100 输出无效状态。

例四 周期比较模式，时间复位

总线轴使能过程省略，参考[总线初始化例程](#)。

	<pre> Base(0) Dpos=0 Mpos=0 Table(0, 50, 100, 150, 200) ' 比较点坐标设置 VECTOR_MOVED=0 ' 设置矢量起始位置为 0，方便观察 Hw_pswitch2(6, 0, 1, 100, 3, 100) ' 位置 100 开始比较，比较 3 次 ' 周期距离 100，输出有效时间由 hw_timer 指令确定 Hw_timer(2, 1000000, 500000, 1, off, 0) ' 输出变为 on 后 500ms 变为 off move(500) </pre>  位置 100 时开始比较，比较 3 次，输出有效信号后 500ms 反转。
相关指令	PSWITCH , HW_PSWITCH , MOVEOP_DELAY , REG_IMPUTS

HW_TIMER -- 硬件定时

类型	特殊命令
描述	硬件定时器，用于硬件比较输出后一段时间后还原电平。 HW_TIMER 只有 1 个，每次调用会强制停止之前的调用。
语法	<pre> HW_TIMER(mode, cyclonetime, optime, reptimes, opstate, opnum) done = HW_TIMER_DONE mode 0 停止， 2-启动 cyclonetime 周期时间，us 单位 optime 有效时间,us 单位 reptimes 重复次数 opstate 输出缺省状态，输出口变为非此状态后开始计时。 opnum 输出口编号，必须能硬件比较输出的口。 </pre>

适用控制器	4 系列产品支持
例子	HW_TIMER(2, 100, 50, 1, off, 0) ‘输出变为 on 后, 50us 后切换为 off
相关指令	HW_PSWITCH

ZSIMU_IN -- 仿真 IN 输入

类型	仿真器专用指令
描述	模拟输入 IN 口的输入。
语法	ZSIMU_IN([ionum ,] value) ionum 输入编号, 0-, 没有这个参数时输出 0-31。 value 输入状态
适用控制器	通用
例子	ZSIMU_IN(0, 1) ’ 仿真输入 0 有效
相关指令	IN

ZSIMU_AIN -- 仿真模拟量输入

类型	仿真器专用指令
描述	模拟模拟量输入口的输入。
语法	ZSIMU_AIN(ionum, value)
适用控制器	通用
例子	ZSIMU_AIN(0, 1024) ’ 仿真通道 0
相关指令	AIN

ZSIMU_ENCODER -- 仿真编码器输入

类型	仿真器专用指令
描述	模拟编码器输入口的输入。
语法	ZSIMU_IN(axis num, value)

	axis num 轴编号, 0- value ENCODER 的仿真值
适用控制器	通用
例子	ZSIMU_ENCODER(0, 1024) ' ENCODER = 1024
相关指令	ENCODER

第八章 通讯相关

ADDRESS -- 控制器站号

类型	系统参数
描述	控制器的所有串口的 MODBUS 协议站号 1- 127, 缺省=1。
语法	ADDRESS = value
适用控制器	通用
例子	Print ADDRESS ' 打印出协议站号 打印结果 1
相关指令	SETCOM , , PORT , PROTOCOL

CANIO_ADDRESS -- CAN 通讯设置

类型	系统参数						
描述	控制器上 CAN 的 CANID 和速度的设置。 扩展板的 CAN 速度通过拨码开关设置。 自动存储到 FLASH, 重启后生效。 低 8 位 (位 0-7) 表示 CANID, 范围 0-32, 缺省 32, 32 表示为主控制器。 高 8 位 (位 8-15) 表示 CAN 速度 <table border="1"> <thead> <tr> <th>高 8 位值</th><th>CAN 波特率:</th></tr> </thead> <tbody> <tr> <td>0</td><td>500 KHz (缺省值)</td></tr> <tr> <td>1</td><td>250 KHz</td></tr> </tbody> </table>	高 8 位值	CAN 波特率:	0	500 KHz (缺省值)	1	250 KHz
高 8 位值	CAN 波特率:						
0	500 KHz (缺省值)						
1	250 KHz						

	<table border="1"> <tr> <td>2</td><td>125 KHz</td></tr> <tr> <td>3</td><td>1 MHz</td></tr> </table> 一个网络中，不要配置多个主控制器。 CAN 地址和速度设置必须重启后生效。	2	125 KHz	3	1 MHz
2	125 KHz				
3	1 MHz				
语法	CANIO_ADDRESS = value				
适用控制器	通用				
例子	CANIO_ADDRESS = 32 ’ 设置主端，CAN 波特率 500KHz CANIO_ADDRESS = 32+ 512 ’ 设置主端，CAN 波特率为 125KHz CANIO_ADDRESS = 1 ’ 设置 CANID 为 1，此时为从端，500KHz，不能连接 IO 板 具体使用参考第十三章的 CAN 通讯例子				
相关指令	CANIO_ENABLE ， CAN ， CANIO_STATUS				

CANIO_ENABLE -- CAN 使能

类型	系统参数
描述	使能或禁止内部 CAN 主端功能。 当 CANIO_ADDRESS 设置 32 时，缺省是使能的。
语法	CANIO_ENABLE = ON/OFF
适用控制器	通用
例子	CANIO_ENABLE = ON ’ 打开 CAN 主端功能 CANIO_ENABLE = OFF ’ 关闭 CAN 主端功能
相关指令	CANIO_ADDRESS

CANIO_STATUS -- ZIO 扩展版状态

类型	系统状态
描述	获取当前的 IO 板的状态，返回 ON/OFF。 20140325 以上的固件版本支持

语法	CANIO_STATUS(cardnum) cardnum: 扩展 IO 板号
适用控制器	通用
例子	举例 1: ? *CANIO_STATUS ’ 输出所有 IO 板的状态 举例 2: If CANIO_STATUS(1) =0 then ’ 判断 IO 板的连接状态 Print “扩展模块 1 没有连接好” Endif
相关指令	CAN , CANIO_ENABLE , CANIO_ADDRESS

CAN -- CAN 通讯

类型	系统指令										
描述	直接通过 CAN 总线收发数据。 可以实现多个控制器通过 CAN 来通讯，但是同一个 CAN 网络上只能有一个主端 (CANIO_ADDRESS=32)。 较新的固件版本才支持这个功能，如不支持请联系厂家。 接线 CANL - CANL CANH - CANH LH 间连接 1 个 120 欧电阻										
语法	CAN(channel, function, tablenum) 参数: channel CAN 通道, 0 表示第一个通道, -1 表示缺省通道。 function 功能号 <table border="1"> <thead> <tr> <th>值</th><th>描述</th></tr> </thead> <tbody> <tr> <td>6</td><td>接收, 没有数据时, identifier<0</td></tr> <tr> <td>7</td><td>发送</td></tr> <tr> <td>16 需要升级固件</td><td>带扩展支持接收, 没有数据时, identifier<0</td></tr> <tr> <td>17 需要升级固件</td><td>发送扩展数据, 普通数据使用 7 发送</td></tr> </tbody> </table> Tablenum 数据存储的 TABLE 位置 6, 7 模式时依次存储: identifier: Can 通讯对象 (Cob-Id), 11 位, 前 4 位是功能码, 后 7 位是节点 ID, ZCAN 使用的高位数据预留, 建议	值	描述	6	接收, 没有数据时, identifier<0	7	发送	16 需要升级固件	带扩展支持接收, 没有数据时, identifier<0	17 需要升级固件	发送扩展数据, 普通数据使用 7 发送
值	描述										
6	接收, 没有数据时, identifier<0										
7	发送										
16 需要升级固件	带扩展支持接收, 没有数据时, identifier<0										
17 需要升级固件	发送扩展数据, 普通数据使用 7 发送										

	0-511。接收时如果小于 0 则表明没有收到数据。 bytes: 数据区域字节数，最多为 8。 data: 数据区域，字节（0-FF）。 16, 17 模式时依次存储： identifier: Can 通讯对象 (Cob-Id)，11 位，前 4 位是功能码，后 7 位是节点 ID，ZCAN 使用的高位数据预留，建议 0-511。接收时如果小于 0 则表明没有收到数据。 Identifier extend : 扩展 id，增加高 18 位，与低 11 位共同组成 CAN 的 29 位 ID，不存在填-1。 bytes: 数据区域字节数，最多为 8。 data: 数据区域，字节（0-FF）。 29 位扩展帧 ID 从左边非 0 开始拆分为低 11 位和高 18 位，例子如下。 扩展帧 ID 为 18EF1300 对应二进制为 0001 1000 1110 1111 0001 0011 0000 0000 低 11 位为 11000111011 对应 10 进制 1595 高 18 位为 110001001100000000 对应 10 进制 201472 所以使用 CAN17 模式时，identifier=1595，Identifier extend=201472
适用控制器	通用
例子	例一 发送端 TABLE (0, 1, 8, 1, 2, 3, 4, 5, 6, 7, 8) ' 发送 cobid=1，8 个字节，依次为 1-8. CAN (0, 7, 0) ' 发送 接收端： CANIO_ADDRESS=1 ' 设置为非主控，此参数设置一次即可 CAN (0, 6, 0) ' 接收 ?TABLE (0) 例二 发送端 TABLE (0, 1, 10, 8, 1, 2, 3, 4, 5, 6, 7, 8) ' 发送 cobid=1，扩展 id10，8 个字节，依次为 1-8 CAN (0, 17, 0) ' 发送 接收端： CANIO_ADDRESS=1' 设置为非主控，此参数设置一次即可. CAN (0, 16, 0) ' 接收 ?TABLE (0)
相关指令	CANIO_ADDRESS ， CANIO_STATUS ， CANIO_ENABLE

GET # -- 读取字符

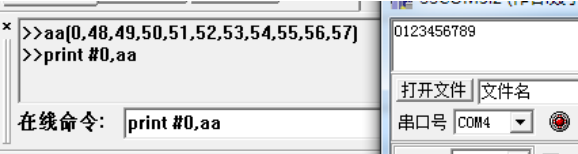
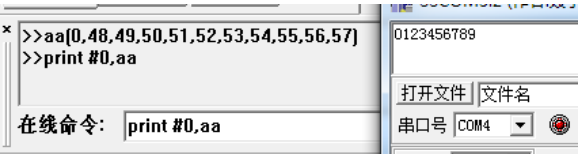
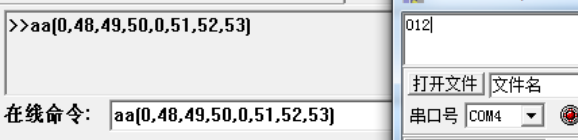
类型	系统指令
描述	从 RAW 方式通讯或自定义网口通讯的通道里面读取一个字节，存入一个变量。 语法 1/2 没有读取到会阻塞，这个函数一般在多任务里面进行调用。 语法 3, 4 会返回读取到的字节数。 20150522 版本以前只支持语法 1。
语法	语法 1: GET #PORT, VARIABLE 语法 2: GET #PORT, ARRAY[(startindex)] [,maxchares] 语法 3: charesget = GET #PORT, VARIABLE 语法 4: charesget = GET #PORT, ARRAY[(startindex)] [,maxchares] PORT: 通道号 VARIABLE: 存放的变量名 Startindex: 存放数组的起始地址 Maxchares: 存放的最多数量
适用控制器	通用
例子	举例 1: <pre> DIM VAR1 SETCOM(38400, 8, 1, 0, 0, 0) ' 开启 RAW 方式 GET #0, VAR1 ' 从通道 0 读取数据 PRINT VAR1 ' 打印读取数据 </pre> 举例 2: <pre> DIM ARRAY1(101) SETCOM(38400, 8, 1, 0, 0, 0) ' 开启 RAW 方式 CHARS = GET #0, ARRAY1, 100 ' 从通道 0 最多读取 100 个字符 If CHARS > 0 THEN ARRAY1(CHARS) = 0 ' 设置结束 0 PRINT ARRAY1 ' 字符串打印出来 ENDIF </pre>
相关指令	PORT , SETCOM , PRINT #

OPEN # -- 打开自定义网口通讯

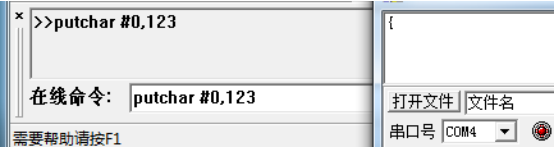
类型	系统指令
描述	开启自定义的以太网通讯。 新固件版本提供此功能。
语法	<pre>OPEN #PORT, "mode", portnum[, ipaddress]</pre> PORT 通讯通道， 见 PORT 描述。 "mode" 通讯主从， "TCP_CLIENT"-从， "TCP_SERVER" - 主 portnum TCP 端口号 ipaddress 字符串，从端的时候要提供
适用控制器	通用
例子	<pre>open #11, "TCP_SERVER", 10 ' 主端设置</pre> <pre>open #10, "TCP_CLIENT", 10, "192.168.1.112" ' 从端设置</pre> 详细例程参考第十三章-自定义网口通讯
相关指令	PORT , SETCOM , PRINT #

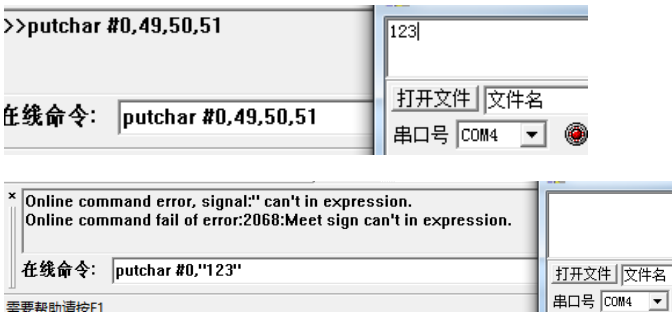
PRINT # -- 输出字符串

类型	系统指令
描述	从 RAW 方式通讯或自定义网口通讯的通道口里面输出字符串，碰到 0 结束。 PRINT #直接发送为字符串(“”符号自动去除)，不需要 ASCII 转码处理，一次只能发一个数据，如下所示： 

	  PRINT #发送数组为 ASCII 码，遇 0 停止，如下所示：  
语法	PRINT #PORT, “字符串” PORT: 通道号
适用控制器	通用
例子	DIM VAR(10) ’ 定义数组 VAR = “AAAA” ’ 数组赋值 SETCOM(38400, 8, 1, 0, 0, 0) ’ 开启 RAW 方式 PRINT #0, VAR ’ 从通道 0 输出数组数据
相关指令	PUTCHAR # , GET # , PORT , SETCOM

PUTCHAR # -- 输出字符

类型	系统指令
描述	从 RAW 方式通讯或自定义网口通讯的通道口输出字符，数组可以代表多个字符。 PUTCHAR #发送数据为 ASCII 码，多个数据逗号隔开，不能直接发送字符串，如下所示： 

	 PUTCHAR #发送数组为 ASCII 码，遇 0 为空格，会将整个数组完全发送，所以要确保数组数据正确，如下所示： 
语法	PUTCHAR #PORT, 字符 PORT: 通道号 20170503 以后固件支持打印出部分数据 PUTCHAR #PORT, ARRAY (INDEX, NUMES) PORT: 通道号 INDEX: 开始输出的位置 NUMES: 输出的字节个数，二进制方式.
适用控制器	通用
例子	DIM VAR1, ARRAY1 (10) ' 定义数组 VAR1 = \$FE ' 数组 1 赋值 ARRAY1 = "ABCDEFGHJIJ" ' 数组 2 赋值 SETCOM (38400, 8, 1, 0, 0, 0) ' 开启 RAW 方式 PUTCHAR #0, VAR1 ' 从通道 0 输出数组 1 数据 PUTCHAR #0, ARRAY1 ' 从通道 0 输出数组 2 数据
相关指令	PORT , SETCOM , GET #

PRINT -- 打印信息

类型	打印输出函数
描述	打印信息到 PC 端 ZDEVELOP 软件的输出窗口。 别名：？ 通过*参数名可以打印参数值；*特殊字符串可以打印一些特殊的信息。
语法	PRINT expression, "string" 或 ? expression, "string" expression 有效表达式 *SET 打印所有参数值 *TASK 打印任务信息 任务正常时只打印任务状态 任务出错时还会打印出错误任务号，具体错误行 *MAX 打印所有规格参数 *FILE 打印程序文件信息 *SETCOM 打印当前串口的配置信息。 *BASE 打印当前任务的 BASE 列表（140123 以后版本支持） *数组名 打印数组的所有元素，数组长度不能太长 *参数名 打印一个所有轴的单个参数 ?*ETHERCAT 打印 EtherCAT 总线连接设置状态 ?*RTEX 打印 Rtex 总线连接设置状态 ?*FRAME 打印机械手参数，需要 161022 及以上固件支持 ?*SLOT 打印出控制器槽位口信息（RTEX 口，EtherCAT 口） ?*PORT 打印所有 PORT 通讯口
适用控制器	通用
例子	举例 1: 在线命令输入 >>print 1+2 输出: 3 举例 2: 在线命令输入 >>print *task '打印所有任务状态 Task:0 Running. file:"hmi.bas" line:280: Task:1 Stopped. Task:2 Stopped. Task:3 Stopped. Task:4 Stopped. Task:5 Stopped. Task:6 Stopped. 举例 3: 在线命令输入

	<pre>>> ?*mpos 输出: 21872.400 0 0 0 0 0 0 0 0 0 0</pre>
相关指令	TRACE

TRACE -- 打印信息 2

类型	打印输出函数
描述	打印信息到 PC 端 ZDEVELOP 软件的输出窗口。 受 ERRSWITCH 开关的控制。
语法	同 PRINT
适用控制器	通用
例子	<pre>ERRSWITCH = 3 ' trace 打印全输出 Dpos(0) =0 Trace "dpos(0) =" dpos(0)</pre> 打印结果 <pre>0</pre>
相关指令	ERRSWITCH , PRINT , WARN , ERROR

WARN -- 警告信息

类型	打印输出函数
描述	程序自动打印报警信息到 PC 端 ZDEVELOP 软件的输出窗口。 也可手动打印信息。 受 ERRSWITCH 开关的控制。
语法	程序报警时自动打印。 手动打印同 PRINT
适用控制器	通用
例子	参考 TRACE 例子
相关指令	ERRSWITCH , PRINT , TRACE , ERROR

ERROR -- 错误信息

类型	打印输出函数
描述	程序自动打印错误信息到 PC 端 ZDEVELOP 软件的输出窗口。 也可手动打印信息。 受 ERRSWITCH 开关的控制。
语法	程序错误时自动打印。 手动打印同 PRINT。
适用控制器	通用
例子	参考 TRACE 例子
相关指令	ERRSWITCH , PRINT , TRACE , WARN

ERRSWITCH -- 信息输出设置

类型	系统参数
描述	调试输出开关，控制 TRACE WARN ERROR 调试输出语句是否真的输出信息。
语法	ERRSWITCH = switch switch 调试输出的开关， 0- TRACE WARN ERROR 指令全部不输出 1- 只输出 ERROR 指令 2- 输出 WARN ERROR 指令 3- TRACE WARN ERROR 指令全部输出 4- TRACE WARN ERROR 指令全部输出，以及运动指令监控
适用控制器	通用
例子	例一 ERRSWITCH = 3 ' 打印全输出，此时 WARN（报警）、ERROR（错误）、TRACE 信息都会输出 例二 ERRSWITCH = 0 ' 不输出，控制器错误信息和报警信息都不输出 TRACE DPOS(0) ' 此时无法用 trace 打印出轴 0 dpos ?DPOS(0) ' 可以用 print 打印，结果，0 例三

	<pre>ERRSWITCH = 4 ' 输出所有信息，以及运动指令监控 >>MOVE(111) MOVE(111)AXIS(0)TASK(23) ' 打印当前操作的轴及运动的任务</pre>
相关指令	TRACE , WARN , ERROR

PORT_STATUS -- 通道状态

类型	通道参数，只读						
描述	返回当前通道的状态，串口总是返回 1。 MODBUS_TCP 链接上以后, PORT_STATUS 会变为 1。 当对应链接已经作为从端使用时，控制器自动搜索没有使用的 PORT 作为 MODBUS_TCP 主端链接，此时程序无法确定实际使用的那个 PORT。 新固件版本提供此功能。						
语法	<pre>VAR1 = PORT_STATUS(port) PORT: 通道号</pre> <table border="1"> <tr> <th>值</th><th>协议</th></tr> <tr> <td>0</td><td>没有连接。</td></tr> <tr> <td>1</td><td>有连接使用。</td></tr> </table>	值	协议	0	没有连接。	1	有连接使用。
值	协议						
0	没有连接。						
1	有连接使用。						
适用控制器	通用						
例子	?PORT_STATUS(0) ' 返回 232 串口通道状态，结果，1						
相关指令	PORT , SETCOM , ADDRESS						

PORT -- 通道编号

类型	通道修正辅助指令
描述	当访问通道参数时可以指定其他的通道。 其他指令需要选择通道时查看。
语法	PORT (portnum)

portnum 通道号

不同型号的控制器支持的通道数不同。在线命令?*port 查看，见例一

ZMC00x 系列

通道号	协议
0	串口 A
1	串口 B

ZMC1-2xx 系列，支持同时两个以太网连接，端口号为 502，支持触摸屏的 MODBUS-TCP 协议连接。

通道号	协议
0	RS232 串口
1	RS485 串口
2	以太网接口，连接 1
3	以太网接口，连接 2
10	自定义网络通讯通道 1
11	自定义网络通讯通道 2

ZMC3xx 系列：带 3 个串口。

通道号	协议
0	RS232 串口
1	RS485 串口
2	RS422 串口
3	以太网接口，连接 1
4	以太网接口，连接 2
10	自定义网络通讯通道 1
11	自定义网络通讯通道 2

ZMC4xx 系列：带 2 个串口。

通道号	协议
0	RS232 串口
1	RS485 串口
2	以太网接口，连接 1
3	以太网接口，连接 2
10	自定义网络通讯通道 1
11	自定义网络通讯通道 2
20	网络控制器互联通道，不支持 PORT_STATUS 查询。

ECI 系列：只有一个串口。

通道号	协议
0	RS232 串口
1	以太网接口，连接 1

	2 以太网接口，连接 2
适用控制器	通用
例程	例一 查看控制器通道 <pre>>>?*port Port:0-COM. Port:1-COM. Port:2-ETH. Port:3-ETH. Port:4-ETH. Port:5-ETH. Port:6-ETH. Port:7-ETH. Port:10-ECUSTOM. Port:11-ECUSTOM. Port:12-ECUSTOM. Port:13-ECUSTOM. Port:14-ECUSTOM. Port:15-ECUSTOM. Port:20-CONNECT.</pre> 其中 COM 为串口通道，ETH 为网口通讯通道，ECUSTOM 为自定义网口通讯通道，CONNECT 为控制器互联通道。
相关指令	FILE_PORT ， PORT_STATUS ， PROTOCOL

FILE_PORT -- 当前通道文件号

类型	通道参数
描述	返回或设置当前通道的缺省文件号。 设定后可以通过在线命令来访问对应文件的文件模块变量或数组。 ZDevelop 集成开发环境会自动使用此参数，不要在使用 ZDevelop 时修改此参数。
语法	VAR1 = FILE_PORT, FILE_PORT = filenum
适用控制器	通用

例子	>>FILE_PORT = 0 ' 当前连接的通道的 FILE_PORT 参数
相关指令	PORT

PROTOCOL -- 通道通讯协议

类型	通道参数，只读										
描述	返回当前通道的通讯协议。										
语法	VAR1 = PROTOCOL(port) PORT: 通道号 返回值 <table border="1"> <tr> <th>值</th><th>协议</th></tr> <tr> <td>0</td><td>RAW 数据格式，无协议。</td></tr> <tr> <td>3</td><td>MODBUS 协议，控制器为从端。（缺省）</td></tr> <tr> <td>14</td><td>MODBUS 协议，控制器为主端。</td></tr> <tr> <td>15</td><td>直接命令执行方式。</td></tr> </table>	值	协议	0	RAW 数据格式，无协议。	3	MODBUS 协议，控制器为从端。（缺省）	14	MODBUS 协议，控制器为主端。	15	直接命令执行方式。
值	协议										
0	RAW 数据格式，无协议。										
3	MODBUS 协议，控制器为从端。（缺省）										
14	MODBUS 协议，控制器为主端。										
15	直接命令执行方式。										
适用控制器	通用										
相关指令	PORT , SETCOM , ADDRESS										

SETCOM -- 串口配置

类型	系统指令				
描述	串口的配置。 控制器重新上电后，setcom 参数会还原成默认值，所以请在程序开头写 setcom 设置。 ZMC00x 系列不支持 MODBUS 主端。				
语法	SETCOM (baudrate, databits, stopbits, parity, port[, mode] [, variable] [, timeout]) baudrate 串口波特率: 9600 19200 4800 115200 38400(缺省) 57600 128000 256000 databits 数据位数 8 stopbits 停止位: 只能设置 0/1/2 parity 是否校验: <table border="1"> <tr> <th>值</th><th>描述</th></tr> <tr> <td>0 (缺省)</td><td>无校验</td></tr> </table>	值	描述	0 (缺省)	无校验
值	描述				
0 (缺省)	无校验				

		1	奇校验
		2	偶校验
	port	串口 PORT 编号 0-1 ， 参见 PORT 描述，不同的控	
	制器不一样。		
	mode	协议：	
		值	描述
		0	RAW 数据模式，无协议 此时可以使用 GET, PRINT #
		4（缺省）	MODBUS 从端（16 位整数）
		14	MODBUS 主端（16 位整数）
		15	直接命令执行模式，此时可以直接从串口输入字符串命令(换行符结束)
	variable	寄存器选择，0-VR，1-TABLE，2-系统 MODBUS 寄存器。 variable 参数是全局的设置，所有的端口共一个。	
			描述
		0	VR，此时一个 VR 映射到一个 MODBUS_REG。 VR 是 32 位浮点型，REG 是 16 位整数型，从 VR 传递数据给 REG 会丢失小数部分，当 VR 数据超过正负 15 位时，REG 数据会改变。 REG 传递数据给 VR 不会有问題。
		1	TABLE，此时一个 TABLE 数据映射到一个 MODBUS_REG。（不推荐） TABLE 是 32 位浮点型，REG 是 16 位整数型，从 TABLE 传递数据给 REG 会丢失小数部分，当 TABLE 数据超过正负 15 位时，REG 数据会改变。 REG 传递数据给 TABLE 不会有问題。
		2（缺省）	系统 MODBUS 寄存器，此时 VR 与 MODBUS 寄存器是两片独立区间。
		3	VR_INT 模式，此时一个 VR_INT 映射到两个 MODBUS_REG。
			variable 参数是全局的设置，所有的端口共一个。
			当设置为 VR 或 TABLE 时，通用输出口会映射到 MODBUS_BIT(0)的位置，输入口会映射到 MODBUS_BIT(1000)的位置，因此此时不要使用 MODBUS_BIT 作为触摸屏的按钮等。
			没有用到的输出口与输入口会连在一起。
	timeout	mode=14 时为消息超时时间，毫秒单位，缺省值 1000	
	适用控制器	通用	
	例子	例一 mode0 RAW 模式 DIM char1 ' 定义变量 SETCOM(38400, 8, 1, 0, 0, 0) ' 配置串口为 RAW 模式 WHILE 1	

```
GET #0, char1      ' 发送给通道 0 的字符保存到 char1
PRINT char1        ' 按 ASCII 码打印通道 0 接受的字符
PUTCHAR #0, char1  ' 将接收的字符再发送回去
WEND
```

松下 A6 伺服编码器读取参照第十三章[简易例程](#)

例二 modbus 通讯设置

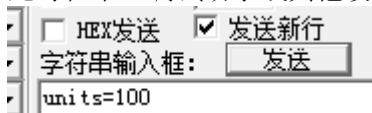
```
SETCOM(38400, 8, 1, 0, 0, 4, 2)      ' 设置串口 0 为 modbus 从端,波特率 38400
SETCOM(38400, 8, 1, 0, 1, 14, 2, 1000) ' 设置串口 1 为 modbus 主端,波特率 38400
```

具体例程参照 MODBUSM_DES 指令例程

例三 直接字符命令方式

```
setcom(38400, 8, 1, 0, 0, 15)      ' 设置串口 0 为直接字符串命令方式
```

此时在串口调试助手或其他设备，直接发送相关指令可直接操作控制器。



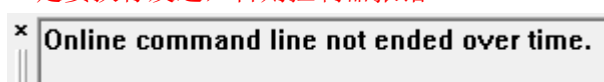
发送前

	轴0	
ATYPE	1	1
UNITS	10000	1

发送后

	轴0	
ATYPE	1	
UNITS	100	
----	----	

一定要换行发送，否则控制器报错



例四 寄存器模式 0

```
VR(0)=0      ' 初始化 VR(0)和 REG(0) 为 0
MODBUS_REG(0)=0
SETCOM(38400, 8, 1, 0, 0, 4, 0)      ' 设置 VR 映射到 MODBUS_REG
VR(0)=100.345      ' 设置 VR(0)=100.345
?MODBUS_REG(0)      ' 打印结果为 100, VR 已经映射到 REG, 但是
                    ' REG 是整型, 所以小数部分丢失
MODBUS_REG(0)=200      ' REG(0) 设为 200
?VR(0)      ' 打印结果为 200, REG 变化也会改变 VR
```

例五 寄存器模式 2

```
VR(0)=0      ' 初始化 VR(0)和 REG(0) 为 0
```

	MODBUS_REG(0)=0 SETCOM (38400, 8, 1, 0, 0, 4, 2) ' 设置 VR 与 MODBUS_REG 独立 VR(0)=100.345 ' 设置 VR(0)=100.345 ?MODBUS_REG(0) ' 打印结果为 0, VR 不影响 REG MODBUS_REG(0)=200 ' REG(0) 设为 200 ?VR(0) ' 打印结果为 100.345, REG 也不影响 VR
相关指令	ADDRESS , PROTOCOL , MODBUSM_DES , PORT

MODBUS_BIT -- 位寄存器

类型	MODBUS 位寄存器						
描述	修改或者读取 BIT 寄存器，布尔型，触摸屏一般叫 0x 寄存器。 另：特殊的 modbus 0x 寄存器，通过这些寄存器可以直接从触摸屏读取和设置 IO，注意读取的 IO 是原始的状态，INVERT_IN 不起作用： <table border="1"> <thead> <tr> <th>寄存器（zero based）</th><th>意义</th></tr> </thead> <tbody> <tr> <td>10000-</td><td>输入 IN, 每个 IN 占 1 个寄存器。</td></tr> <tr> <td>20000-</td><td>输出 OP, 每个 OP 占用 1 个寄存器。</td></tr> </tbody> </table>	寄存器（zero based）	意义	10000-	输入 IN, 每个 IN 占 1 个寄存器。	20000-	输出 OP, 每个 OP 占用 1 个寄存器。
寄存器（zero based）	意义						
10000-	输入 IN, 每个 IN 占 1 个寄存器。						
20000-	输出 OP, 每个 OP 占用 1 个寄存器。						
语法	MODBUS_BIT(first, [last]) = value first 位寄存器编号，编号从 0 开始。 last 位寄存器编号，编号从 0 开始。						
适用控制器	通用						
例子	DIM VAR MODBUS_BIT(100) = 1 ' bit100 赋值 1 VAR = MODBUS_BIT(100) ' 把 bit100 的值赋值到变量 VAR						

MODBUS_IEEE -- 字寄存器-32 位浮点型

类型	MODBUS 字寄存器
描述	修改或者读取字寄存器，32 位浮点型。触摸屏一般叫 4x 寄存器。 ZMOTION 运动控制器专门具备 MODBUS 字寄存器，会占用两个寄存器的地址。字寄存器间的关系查看第一章寄存器。 控制器缺省使用系统 MODBUS 字寄存器，要修改时请设置 SETCOM 的第 7 个参数。

	另：特殊的 modbus 4x 寄存器，通过这些寄存器可以直接从触摸屏读取一些状态： <table><tr><th>寄存器（zero based）</th><th>类型</th><th>意义</th></tr><tr><td>10000-</td><td>IEEE</td><td>DPOS 读取 每个轴占两个寄存器。</td></tr><tr><td>11000-</td><td>IEEE</td><td>MPOS 读取 每个轴占两个寄存器。</td></tr><tr><td>12000-</td><td>IEEE</td><td>VP_SPEED 读取 每个轴占两个寄存器。</td></tr></table>	寄存器（zero based）	类型	意义	10000-	IEEE	DPOS 读取 每个轴占两个寄存器。	11000-	IEEE	MPOS 读取 每个轴占两个寄存器。	12000-	IEEE	VP_SPEED 读取 每个轴占两个寄存器。
寄存器（zero based）	类型	意义											
10000-	IEEE	DPOS 读取 每个轴占两个寄存器。											
11000-	IEEE	MPOS 读取 每个轴占两个寄存器。											
12000-	IEEE	VP_SPEED 读取 每个轴占两个寄存器。											
语法	MODBUS_IEEE (regnum) = value regnum 寄存器编号，编号从 0 开始。												
适用控制器	通用												
例子	DIM VAR MODBUS_IEEE(100) = 100.10 'ieee100 赋值 100.10 VAR = MODEBUS_IEEE(100) 'ieee100 赋值给变量 VAR												
相关指令	MODBUS_REG ， MODBUS_LONG ， MODBUS_STRING												

MODBUS_LONG -- 字寄存器-32 位整型

类型	MODBUS 字寄存器
描述	修改或者读取字寄存器，32 位整型。触摸屏一般叫 4x 寄存器。 ZMOTION 运动控制器专门具备 MODBUS 字寄存器，会占用两个寄存器的地址。字寄存器间的关系查看第一章寄存器。 控制器缺省使用系统 MODBUS 字寄存器，要修改时请设置 SETCOM 的第 7 个参数。
语法	MODBUS_LONG(regnum) = value regnum 寄存器编号，编号从 0 开始。
适用控制器	通用
例子	DIM VAR MODBUS_LONG(100) = 100 'long100 赋值 100 VAR = MODEBUS_LONG(100) 'long100 赋值给 VAR
相关指令	MODBUS_REG , MODBUS_IEEE , MODBUS_STRING

MODBUS_REG -- 字寄存器-16 位整型

类型	MODBUS 字寄存器											
描述	修改或者读取字寄存器，16 位整型，触摸屏一般叫 4x 寄存器。											
	ZMOTION 运动控制器专门具备 MODBUS 字寄存器。											
	不同型号控制器的字寄存器个数有区别。											
	字寄存器间的关系查看第一章 寄存器 。											
描述	控制器缺省使用系统 MODBUS 字寄存器，要修改时请设置 SETCOM 的第 7 个参数。											
	另：特殊的 modbus 4x 寄存器，通过这些寄存器可以直接从触摸屏读取一些状态：											
	<table><tr><td>寄存器（zero based）</td><td>类型</td><td>意义</td></tr><tr><td>13000-</td><td>16</td><td>DA 输出</td></tr><tr><td>14000-</td><td>16</td><td>AD 读取</td></tr></table>			寄存器（zero based）	类型	意义	13000-	16	DA 输出	14000-	16	AD 读取
	寄存器（zero based）	类型	意义									
13000-	16	DA 输出										
14000-	16	AD 读取										
语法	MODBUS_REG(regnum) = value regnum 寄存器编号，编号从 0 开始。											
适用控制器	通用											
例子	DIM VAR MODBUS_REG(100) = 100 'reg100 赋值 100 VAR = MODBUS_REG(100) 'reg100 赋给变量 VAR											
相关指令	MODBUS_IEEE ， MODBUS_LONG ， MODBUS_STRING											

MODBUS_STRING -- 字寄存器-字节

类型	MODBUS 字寄存器，字符串函数
描述	读取 MODBUS 寄存器区域的字符串，按字节来读取。触摸屏一般叫 4x 寄存器。 字寄存器间的关系查看第一章寄存器。
语法	MODBUS_STRING(index, chares) index 起始的 MODBUS 寄存器编号，编号从 0 开始。不同型号控制器的字寄存器个数有区别。 chares 读取的字符总数。
适用控制器	通用

例子	<pre> DIM ARR(8) MODBUS_STRING (0, 8) = "abc" 'string0 开始保存字符串，共 8 字节 print MODBUS_STRING(0, 8) '打印保存的字符串 '打印结果，abc ARR = MODBUS_STRING(0, 8) '字符换赋值给数组 </pre>
相关指令	MODBUS_REG , MODBUS_LONG , MODBUS_IEEE

MODBUSM_DES -- modbus 通讯连接

类型	通讯指令
描述	设置或读取 MODBUS 主端的对方。 通讯等待时只会阻塞当前这个任务，不影响其他任务。
语法	<pre> MODBUSM_DES (address[, port]), ADDRESS1 = MODBUSM_DES([port]) address 对端的 modbus 协议站号 port 当前 modbus 主通讯的 port 号 </pre>
适用控制器	通用
例子	<pre> 例一 多主端-多从端通讯 SETCOM(38400, 8, 1, 0, 0, 14, 2, 1000) '设置串口 0 为 modbus 主端，通讯等待 1s SETCOM(38400, 8, 1, 0, 1, 14, 2, 1000) '设置串口 1 为 modbus 主端，通讯等待 1s WHILE 1 IF IN(0)=1 THEN ' IN0 高电平时使用串口 0 IF IN(1)=1 THEN MODBUSM_DES(1, 0) ' IN1 高电平时与从端站号 1 通讯 MODBUSM_REGSET(0, 10, 0) ' 本地寄存器复制到对端 MODBUSM_REGGET(20, 10, 20) ' 对端寄存器复制到本地 WAIT UNTIL MODBUSM_STATE <> 1 '等待消息结束 ELSE MODBUSM_DES(2, 0) ' IN1 低电平时与从端站号 2 通讯 MODBUSM_REGSET(30, 10, 30) ' 本地寄存器复制到对端 MODBUSM_REGGET(40, 10, 40) ' 对端寄存器复制到本地 WAIT UNTIL MODBUSM_STATE <> 1 '等待消息结束 ENDIF ?"通道 0 状态=", MODBUSM_STATE '打印通讯状态 ELSE ' IN0 低电平时使用串口 1 IF IN(1)=1 THEN MODBUSM_DES(1, 1) ' IN1 高电平时与从端站号 1 通讯 </pre>

	<pre> MODBUSM_REGSET(50, 10, 50) ' 本地寄存器复制到对端 MODBUSM_REGGET(60, 10, 60) ' 对端寄存器复制到本地 WAIT UNTIL MODBUSM_STATE <> 1 ' 等待消息结束 ELSE MODBUSM_DES(2, 1) ' IN1 低电平时与从端站号 2 通讯 MODBUSM_REGSET(70, 10, 70) ' 本地寄存器复制到对端 MODBUSM_REGGET(80, 10, 80) ' 对端寄存器复制到本地 WAIT UNTIL MODBUSM_STATE <> 1 ' 等待消息结束 ENDIF ?"通道 1 状态=", MODBUSM_STATE ' 打印通讯状态 ENDIF WEND </pre>
相关指令	SETCOM , PROTOCOL , PORT , MODBUSM_DES2

MODBUSM_DES2 -- 控制器间网口通讯

类型	通讯指令
描述	控制器间网口通讯，也可作为 MODBUS_TCP 主端通讯。 ?*PORT 可以显示出当前可用的网络链接通道。 <pre> >>?*port Port:0-COM. Port:1-COM. Port:2-ETH. Port:3-ETH. Port:4-ETH. Port:5-ETH. Port:6-ETH. Port:7-ETH. Port:10-ECUSTOM. Port:11-ECUSTOM. Port:12-ECUSTOM. Port:13-ECUSTOM. Port:14-ECUSTOM. Port:15-ECUSTOM. Port:20-CONNECT. </pre> 作为 MODBUS_TCP 主端时 选择一个 ETH 模式的 PORT 口作为 MODBUS_TCP 链接通道(不建议用第一个和最后一个)。 当选择的通道已经作为从端被占用时，控制器自动选择一个没有占用的 ETH 模式的 PORT 口作为 MODBUS_TCP 主端链接。 作为控制器互联使用时 选择 CONNECT 模式的 PORT 口作为控制器互联通道。
语法	<pre>MODBUSM_DES2 (id, port, "desipaddress", [timer])</pre> id 对方控制器的 MODBUS 从端 ID, 缺省 1

	port 支持两种模式，?*PORT 确认通道号及模式 ETH 时，作为 MODBUS_TCP 主端通道 CONNECT 时，做为控制器互联通道 "desipaddress" 字符串，对方控制器的 IP 地址 timer 消息超时时间设置，缺省 1000ms。
适用控制器	4 系列产品，20170117 及以上固件版本支持
例子	例一 MODBUS 主端通讯连接建立 MODBUS_TCP 主端通讯不用考虑丢包。 MODBUSM_des2(1, 4, "192. 168. 0. 12") '按站号和 ip 与从端通讯 '使用控制器通道 4，?*port 确认 <pre> while 1 lasttick=ticks FOR I =0 TO 9999 modbus_reg(0) = I MODBUSM_regset(0, 10, 0) '设置对端寄存器 modbus_reg(0) = 99 MODBUSM_regget(0, 10, 0) '读取对端寄存器 if modbus_reg(0) <> i then ?"reg(0)=" modbus_reg(0), "state=" MODBUSM_STATE '打印在第几次通讯时错误 endif NEXT ?lasttick-ticks '打印通讯时间 wend End </pre> 例二 控制器间通讯连接建立 网络环境不好时，互连通讯有很小的丢包可能。 MODBUSM_des2(\$fe, 20, "192. 168. 0. 25", 10) '控制器从端站号为 fe '控制器主端通道号为 20，?*port 确认 '设置 10ms 超时时间 <pre> MODBUSM_REGSET(0, 10, 0) '本地寄存器复制到对端 WAIT UNTIL MODBUSM_STATE <> 1 '等待消息结束 if MODBUSM_STATE<>0 then MODBUSM_REGSET(0, 10, 0) '出错重发一次 MODBUSM_REGGET(20, 10, 20) '对端寄存器复制到本地 </pre>

	<pre> WAIT UNTIL MODBUSM_STATE <> 1 '等待消息结束 if MODBUSM_STATE<>0 then MODBUSM_REGGET(20,10,20) '出错重发一次 end </pre>
相关指令	ADDRESS , PORT

MODBUSM_STATE -- modbus 通讯状态

类型	通讯状态												
描述	MODBUS 主通讯状。 <table border="1"> <thead> <tr> <th>值</th><th>描述</th></tr> </thead> <tbody> <tr> <td>0</td><td>状态正常</td></tr> <tr> <td>1</td><td>等待应答中</td></tr> <tr> <td>2</td><td>等待超时</td></tr> <tr> <td>3</td><td>应答错误</td></tr> <tr> <td></td><td></td></tr> </tbody> </table>	值	描述	0	状态正常	1	等待应答中	2	等待超时	3	应答错误		
值	描述												
0	状态正常												
1	等待应答中												
2	等待超时												
3	应答错误												
语法	<pre> VAR1 = MODBUSM_STATE </pre> 当前 modbus 主端通讯状态												
适用控制器	通用												
例子	<pre> 例一 SETCOM(38400,8,2,0,1,14,2,1000) '485 口为 modbus 主端，消息超时时间 1s MODBUSM_DES(1,1) '与对端站号 1 通讯 MODBUSM_REGGET(0,10,0) '获取对端寄存器值 WAIT UNTIL MODBUSM_STATE <> 1 '等待消息结束 '最多等待 1s，为消息超时时间 '获取消息或超时后变为相应值 ?MODBUSM_STATE '打印通讯结果 IF MODBUSM_STATE=0 THEN ?'通讯正常" ELSEIF MODBUSM_STATE=2 THEN ?'通讯超时" ELSEIF MODBUSM_STATE=3 THEN ?'通讯错误" ENDIF </pre>												

相关指令	PROTOCOL , PORT , SETCOM
------	--

MODBUSM_REGSET -- 写对端保持寄存器

类型	通讯指令
描述	把本地 MODBUS 保存寄存器设置到对端。 对应标准协议功能码 06 或 16，写保持寄存器。
语法	MODBUSM_REGSET (startreg, num, local_reg) startreg 对端的寄存器起始编号，从 0 开始。 num 寄存器个数 local_reg 从本地系统 MODBUS 寄存器中取值，起始编号。
适用控制器	通用
例子	参考 MODBUSM_REGGET 例一
相关指令	ADDRESS , PROTOCOL , PORT , SETCOM

MODBUSM_REGGET -- 读对端保持寄存器

类型	通讯指令
描述	把对端的 MODBUS 保存寄存器复制到本地。 对应标准协议功能码 03，读保持寄存器。
语法	MODBUSM_REGGET (startreg, num, local_reg) startreg 对端的寄存器起始编号，从 0 开始。 num 寄存器个数 local_reg 从本地系统 MODBUS 寄存器中取值，起始编号。
适用控制器	通用
例子	例一 台达绝对值编码器读取 global dim flag_abs ' 编码器读取正确标志 flag_abs = 0 global dim total_pul ' 读到个总脉冲个数 setcom(38400, 8, 2, 0, 1, 14) ' 设置 485 口为 modbus 主端，波特率 38400 8. N. 2 MODBUSM_DES(1, 1) ' 设置 485 端口，对方站号 1 p3-00 modbus_long(300) = 2 ' 用 300, 301 传输数据

	<pre> modbusm_regset(98,2,300) ' 设置 P0-49 = 2 ' 更新参数 modbusm_regget(98,2,300) ticks = 1000 while (modbus_long(300) and ticks > 0) ' 等到 P-49 变成 0 或者 1 秒 超时, ' 即更新完成或者不成功 modbusm_regget(98,2,300) Wend if ticks < 0 then print "伺服更新不成功" flag_abs = 1 return Endif modbusm_regget(100,6,310) if modbus_long(310) = 0 then ' 编码器正常 flag_abs = 0 total_pul = modbus_long(314) else print "编码器出错" flag_abs = 2 Endif if flag_abs = 0 then ' 正确 dpos(0) = -total_pul / units(0) ' 测试出来的脉冲个数是反的 ' 取反还原坐标 Endif End </pre>
相关指令	ADDRESS , PROTOCOL , PORT , , SETCOM

MODBUSM_3XGET -- 读对端输入寄存器

类型	通讯指令
描述	把对端的 MODBUS 输入寄存器复制到本地。 对应标准协议功能码 04，读输入寄存器。

语法	MODBUSM_3XGET (startreg, num, local_reg) startreg 对端的寄存器起始编号，从 0 开始。 num 寄存器个数 local_reg 从本地系统 MODBUS 寄存器中取值，起始编号。
适用控制器	通用
例子	MODBUSM_3XGET(0, 9, 0) ' 把对端输入寄存器 0-9 复制到通讯本地寄存器 0-9
相关指令	ADDRESS , PROTOCOL , PORT , SETCOM

MODBUSM_BITSET -- 写对端线圈

类型	通讯指令
描述	把本地 MODBUS 位寄存器设置到对端。 对应标准协议功能码 05 或 15，写线圈。
语法	MODBUSM_BITSET (startreg, num, local_reg) startreg 对端的位寄存器起始编号，从 0 开始。 num 寄存器个数 local_reg 从本地系统 MODBUS 寄存器中取值，起始编号。
适用控制器	通用
例子	MODBUSM_BITSET (0, 10, 0) ' 把本地位寄存器 0-9 复制到通讯对端的寄存器 0-9
相关指令	ADDRESS , PROTOCOL , PORT , SETCOM

MODBUSM_BITGET -- 读对端线圈

类型	通讯指令
描述	把对端 MODBUS 位寄存器复制到本地。 对应标准协议功能码 01，读线圈。
语法	MODBUSM_BITGET (startreg, num, local_reg) startreg 对端的寄存器起始编号，从 0 开始。 num 寄存器个数 local_reg 从本地系统 MODBUS 位寄存器中取值，起始编号。

适用控制器	通用
例子	MODBUSM_BITGET (0, 10, 0) ' 把通讯对端的位寄存器 0-9 复制到本地位寄存器 0-9
相关指令	ADDRESS , PROTOCOL , PORT , SETCOM

MODBUSM_1XGET -- 读对端离散输入

类型	通讯指令
描述	把对端 MODBUS 位寄存器复制到本地。 对应标准协议功能码 02，读离散输入。
语法	MODBUSM_1XGET (startreg, num, local_reg) startreg 对端的寄存器起始编号，从 0 开始。 num 寄存器个数 local_reg 从本地系统 MODBUS 位寄存器中取值，起始编号。
适用控制器	通用
例子	MODBUSM_1XGET (0, 10, 0) ' 把通讯对端的位寄存器 0-9 复制到本地位寄存器 0-9
相关指令	ADDRESS , PROTOCOL , PORT , SETCOM

第九章 系统相关

所有的日期时间参数或指令，LOCK 后不让修改。

APP_PASS -- 密码

类型	系统指令
描述	控制器应用密码。 下载 ZAR 包时可以选择校验这个密码，校验错误将不能下载。 LOCK 后不能修改 APP_PASS。

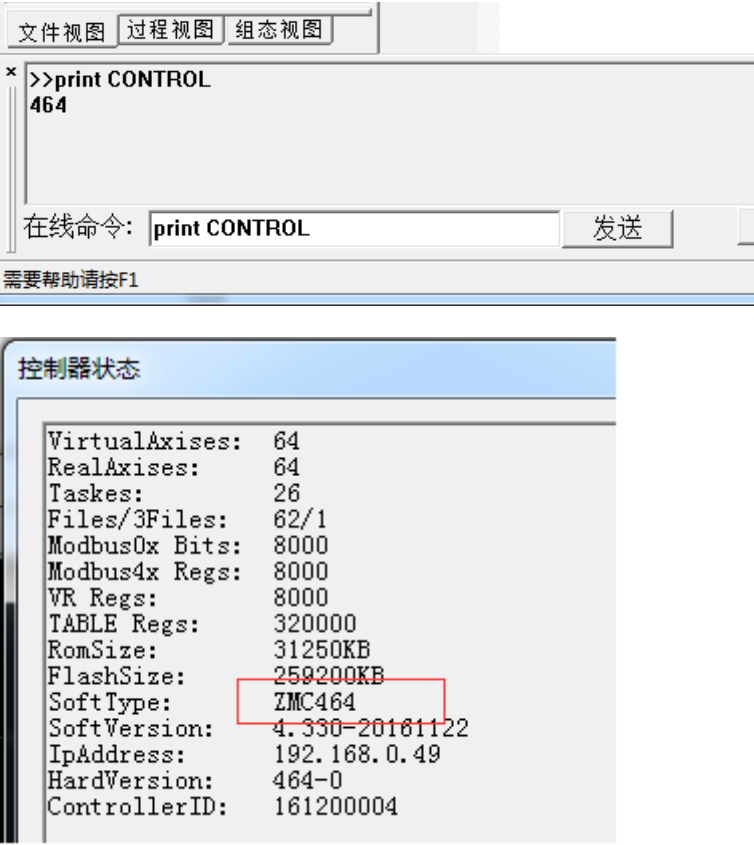
	APP_PASS 采用不可逆算法加密，一旦忘记，将无法获知。
语法	APP_PASS(pass) pass: 字母或数字，”_”等少数特殊符号，总共不要超过 16 个字符。 pass 不能设置变量或表达式，否则变量名等会被认为是密码。
适用控制器	通用
例子	APP_PASS(Zmotion)
相关指令	LOCK

CLEAR -- 清除 VR

类型	系统指令
描述	清除所有 VR 内容。
语法	CLEAR()
适用控制器	通用
例子	CLEAR() '清除 VR 全部数据
相关指令	VR

CONTROL -- 控制器软件型号

类型	系统只读参数
描述	返回控制器软件型号。
语法	Val=CONTROL
适用控制器	通用
例子	在线命令输入 >>print CONTROL '打印控制器型号 输出: 106

	 文件视图 过程视图 组态视图 x >>print CONTROL 464 在线命令: print CONTROL 发送 需要帮助请按F1 控制器状态 <pre> VirtualAxes: 64 RealAxes: 64 Tasks: 26 Files/3Files: 62/1 Modbus0x Bits: 8000 Modbus4x Regs: 8000 VR Regs: 8000 TABLE Regs: 320000 RomSize: 31250KB FlashSize: 259200KB SoftType: ZMC464 SoftVersion: 4.330-20181122 IpAddress: 192.168.0.49 HardVersion: 464-0 ControllerID: 161200004 </pre>
相关指令	ID_HARDWARE

DATE -- 系统日期

类型	系统参数
描述	设置系统日期，掉电保存，或是返回 2000 年 1 月 1 日以后的天数。 仿真器无法修改日期。
语法	DATE= DD:MM:YYYY 或 DD:MM:YY
适用控制器	通用
例子	DATE=27:2:13 在线命令输入 >>print DATE 输出: 4806 即 2013:2:27 - 2000:1:1 = 4806
相关指令	DATE\$, RTC_DATE

DATE\$ -- 系统日期

类型	字符串函数
描述	字符串函数，按 DD:MM:YYYY 格式返回 DATE 设置日期。
语法	DATE\$
适用控制器	通用
例子	DATE=27:2:13 在线命令输入 >>print DATE\$ 输出：27:02:2013
相关指令	DATE ， RTC_DATE

DAY -- 系统星期

类型	系统参数
描述	设置系统时钟的星期，0-6，0 表示星期天，掉电保存。 仿真器无法修改日期。 DAY 不随 DATE 和 RTC_DATE 改变而改变，两者独立。
语法	VAR=DAY，DAY=expression
适用控制器	通用
例子	举例 1： DAY = 3 举例 2 在线命令输入 >>print DAY 输出：3
相关指令	DAY\$

DAY\$ -- 系统星期

类型	字符串函数
----	-------

描述	字符串函数，返回 DAY 设置星期。 DAY\$不随 DATE 和 RTC_DATE 改变而改变，两者独立。
语法	DAY\$
适用控制器	通用
例子	在线命令输入 >>print DAY\$ 输出: Wednesday
相关指令	DAY

RTC_DATE -- 系统日期

类型	系统参数
描述	设置或者获取系统日期，掉电保存，从 2000 年 1 月 1 日起。 注意与 DATE 指令的表示格式是不一样的。 仿真器无法修改日期。
语法	RTC_DATE = YYYYMMDD 或 YYMMDD
适用控制器	通用
例子	RTC_DATE = 20130227 在线命令输入 >>print RTC_DATE 输出: 20130227
相关指令	DATE , DATE\$

DISABLE_GROUP -- 轴分组

类型	系统指令
描述	一停多，把几个轴设置为一组，驱动器告警后会关闭组内的所有使能 (ECAT 产品支持)，脉冲轴设置无意义。 一般用于多工位分组。
语法	DISABLE_GROUP (AXIS1, AXIS2, ...)
适用控制器	通用
例子	DISABLE_GROUP (-1) '取消所有组设置，此时报警关闭 WDOG.

	DISABLE_GROUP(0, 5, 1) ' 轴 0, 5, 1 设置一组 DISABLE_GROUP(4, 2) ' 轴 4, 2 设置一组 当轴 0/5/1 出现问题时, 轴 0、轴 5 和轴 1 都会掉使能, 而轴 4 和轴 2 不会, 同样当轴 4/2 出现问题时, 轴 4 和轴 2 会掉使能, 轴 0、轴 5 和轴 1 不会
相关指令	WDOG , AXIS_ENABLE

ERROR_AXIS -- 报错轴号

类型	系统状态
描述	出现错误的第一个轴号, -1 没有。
语法	Var=ERROR_AXIS
适用控制器	通用
例子	?ERROR_AXIS ' 打印第一个出错轴号 ' 打印结果, -1, 当前没有轴错误
相关指令	MOTION_ERROR

ERROR_SET -- 报错输出

类型	系统指令
描述	当 BASIC 程序运行出错的时候, 输出口自动打开。并且把错误信息写到对应的 MODBUS 寄存器, 输出口状态还原时 BASIC 程序会自动重新运行。 20130906 以上版本提供支持, 寄存器长度至少 32 个字节。
语法	ERROR_SET (输出口, modbus 寄存器地址 [, ERRSUBName]) ERRSUBName: 150721 以上版本提供; 设置一个 SUB 过程进行临时处理, 语法错误停止时此函数自动被调用, 此过程不要使用 WAIT 等可能阻塞的指令, 而且必须很精简。此过程里面再出现语法错误不会再被处理。
适用控制器	通用
例子	ERROR_SET(1, 200, error_deal) Mov(30) ' 拼写错误, 此时运行错误, 输出口 1 打开。并在寄存器

	<pre> ' 记录错误信息。 ' Modbus_string(200,32) ="sample_move.bas,6,e2043" END SUB error_deal() ' 报错时调用函数 ?"进入错误处理 sub" ' 打印信息 ' 以下编写需要的功能程序 END SUB 可以在线命令栏 输入? Modbus_string(200,32) 查看错误信息: Modbus_string(200,32) ="sample_move.bas,6,e2043" sample_move.bas : 表示文件名 6: 表示当前出现错误的行号 e2043;表示错误码 </pre>
--	--

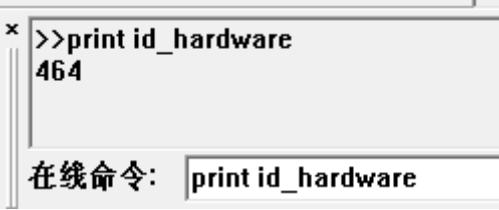
MOTION_ERROR -- 报错轴列表

类型	系统状态
描述	出现错误的轴列表。 每个位表示一个轴号。位 0-n 表示轴 0-n。
语法	Var=MOTION_ERROR
适用控制器	通用
例子	Print MOTION_ERROR ' 打印结果, 0, 未出错
相关指令	ERROR_AXIS

FRAME -- 机械手类型

类型	系统参数
描述	选择机械手类型, 请查看正运动机械手手册。
相关指令	CONFRAME

ID_HARDWARE -- 控制器硬件型号

类型	系统只读参数
描述	返回控制器硬件型号。
语法	Val=CONTROL
适用控制器	通用
例子	在线命令输入 <pre>>>print ID_HARDWARE '打印控制器型号'</pre> 输出: 464  
相关指令	CONTROL

IP_ADDRESS -- IP 地址

类型	系统参数，自动存储到 FLASH
描述	控制器 IP 地址。 只有带以太网接口的控制器支持，读取时以 32 位整数返回，见例一。 修改立刻生效，使用网口连接时，会断开，需重连。 单网卡连接控制器，网卡与控制器处于同一网段即可。

	多网卡连接控制器，网卡之间要为不同网段，控制器连接哪个网卡就设置为对应网段。
语法	IP_ADDRESS = dot.dot.dot.dot
适用控制器	通用
例子	例一 在线命令输入 <pre>>>IP_ADDRESS=192.168.0.26 >>PRINT IP_ADDRESS</pre> 输出：436250816 具体转换过程 四段转换为二进制 <pre>192 -- 1100 0000 168 -- 1010 1000 0 -- 0000 0000 26 -- 0001 1010</pre> 二进制重新组合 <pre>26 0 168 192 0001 1010 0000 0000 1010 1000 1100 0000</pre> 转化为十进制 436250816
相关指令	IP_GATEWAY ， IP_NETMASK

IP_GATEWAY -- IP 网关

类型	系统参数，自动存储到 FLASH
描述	控制器 IP 网关。 只有带以太网接口的控制器支持，读取时以 32 位整数返回。
语法	IP_GATEWAY = dot.dot.dot.dot
适用控制器	通用
例子	在线命令输入 <pre>>>IP_GATEWAY =192.168.0.1 >>PRINT IP_GATEWAY</pre> 输出：16820416 具体转换过程参考 IP_ADDRESS 例程

相关指令	IP_NETMASK , IP_ADDRESS

IP_NETMASK -- IP 掩码

类型	系统参数，自动存储到 FLASH
描述	控制器 IP 网络掩码。 只有带以太网接口的控制器支持，读取时以 32 位整数返回。
语法	IP_NETMASK = dot.dot.dot.dot
适用控制器	通用
例子	在线命令输入 >>IP_NETMASK =255.255.252.0 >>PRINT IP_NETMASK 输出：16580607 具体转换过程参考 IP_ADDRESS 例程
相关指令	IP_GATEWAY , IP_ADDRESS

LOCK -- 锁定控制器

类型	系统指令
描述	锁定控制器，不让对控制器操作。 上位机的 zpj 程序仍可以修改，无法下载到控制器，但生成的 zar 文件仍可下载。 会阻止对控制器的枚举操作，比如打印所有数组等，但可以打印具体数组的值。 LOCK 密码采用不可逆算法加密，一旦忘记，将不可获知，pass 不能设置变量或表达式，否则变量名等会被认为是密码。
语法	LOCK (pass) pass 字母或数字，”_”等少数特殊符号，总共不要超过 16 个字符。
适用控制器	通用
例子	LOCK (passwd) ' 锁定控制器，密码为 passwd
相关指令	UNLOCK

LEDOUT -- 控制器指示灯

类型	系统指令
描述	操作控制器指示灯。 电源指示灯无法操作。
语法	LEDOUT (num, state) Num 指示灯编号, 1-RUN 指示灯, 2-ALM 指示灯 State 状态, 0-关闭, 1-打开
适用控制器	通用
例子	ledout (1, 0) ' 关闭运行灯 ledout (2, 0) ' 关闭报警灯

SERIAL_NUMBER -- 控制器唯一 ID

类型	系统只读参数
描述	返回控制器唯一 ID。 是一个唯一的序列号, 生成 ZAR 包的时候也可以和这个 ID 绑定, 这样这个 ZAR 只能为这个控制器所用。
语法	Var=SERIAL_NUMBER
适用控制器	通用
例子	Print SERIAL_NUMBER ' 打印控制器 ID 打印结果 13080011

SERVO_PERIOD -- 总线通讯周期

类型	系统参数
描述	总线伺服通讯周期。 缺省 1000 微秒, 修改功能预留, 目前只读。 Rtex 驱动使用时请按以下设置 控制器周期 500us 时, 将驱动器的 P7.20 设置为 3, P7.21 设置为 1。 控制器周期 1000us 时, 将驱动器的 P7.20 设置为 6, P7.21 设置为 1。 查看 Rtex 驱动器设置请看例子。

语法	value=SERVO_PERIOD
适用控制器	通用
例子	在线命令打印控制器通讯周期和 Rtex 驱动器设置 >>print SERVO_PERIOD '打印伺服更新周期，结果，1000 >>DRIVE_READ(7*256+20)AXIS(0) '打印 Rtex 轴 0 驱动器周期设置 >>DRIVE_READ(7*256+21)AXIS(0) '打印 Rtex 轴 0 驱动器周期比值设置
相关指令	SERVO

SYSTEM_ZSET -- 控制器设置

类型	系统参数
描述	控制器设置。 BIT0- 1 VP_SPEED 缺省使用插补速度，0- VP_SPEED 使用单轴的速度 BIT1- 1 使用 MOVE_OP 精确输出功能，0- MOVE_OP 普通方式 BIT4 - 1 对带编码器功能的轴，使用编码器位置的 MOVE_OP 精准方式 20170505 以上固件版本支持这个位设置 使用前要求 MPOS 必须跟随 DPOS，编码器精准功能受驱动器的响应影响，输出时刻速度越平稳，效果越好。 通过打印此指令查看 BIT1 的值来判断控制器是否支持精确位置输出，见例一。
语法	可读：value=SYSTEM_ZSET 可写：SYSTEM_ZSET=value
适用控制器	通用
例子	例一 判断控制器是否支持精确位置输出 <pre>if READ_BIT2(1,SYSTEM_ZSET)=1 then '读取 bit1 的值 ?"支持精确位置输出" else ?"不支持精确位置输出" Endif</pre>
相关指令	MOVE_OP

TABLE -- 系统缺省数组

类型	系统数组
描述	系统缺省建立的全局数组，所有程序都可以访问。 数据录波缓冲区，凸轮数据表，螺距补偿表，机械手参数，等等都是用 table 来存储。
语法	TABLE(index) = value , VAR1 = TABLE(index), TABLE(index [, value1..])
适用控制器	通用
例子	TABLE(0) = 10 ' table(0)赋值 10 Table(10,100,200,300) ' table(10)赋值 100, table(11)赋值 200, table(12)赋值 300
相关指令	TSIZE

TSIZE -- table 大小

类型	系统参数
描述	TABLE 的所有元素个数，可以修改 TABLE 空间大小。 请在程序开头修改，不能超过 TABLE 最大空间。
语法	Var=TSIZE TSIZE=Value
适用控制器	通用
例子	读取 Print TSIZE ' 打印出控制器 table 大小 设置 TSIZE=10000 ' 设置 table 的大小，不能超过控制器 table 最大 size
相关指令	TABLE

TIME -- 系统时间

类型	系统参数
描述	设置系统时钟的时间，返回 0 点以后的秒数。 仿真器无法修改日期。

语法	TIME = hh:mm:ss
适用控制器	通用
例子	举例 1: TIME= 11:14:40 举例 2 在线命令输入 >>print TIME 输出: 40541
相关指令	TIME\$, RTC TIME

TIME\$ -- 系统时间

类型	字符串函数
描述	字符串函数，按 24 小时的格式 hh:mm:ss 返回当前时间。
语法	TIME\$
适用控制器	通用
例子	在线命令输入 >>print TIME\$ 输出: 11:29:46
相关指令	TIME , RTC TIME

TRIGGER -- 触发示波器

类型	系统指令
描述	开始执行示波器的数据采样。 150723 以后版本支持，与 zdevelop 的示波器功能合用。
语法	TRIGGER
适用控制器	通用
例子	TRIGGER '示波器开始抓取下面运动的每时刻速度，存储在示波器功能设置的 TABLE 区间。 MOVE(10000)

相关指令	SCOPE
------	-----------------------

RTC_TIME -- 系统时间

类型	系统参数
描述	设置或者获取系统时间。 注意与 TIME 指令的表示方式是不一样的。
语法	RTC_TIME = hhmmss
适用控制器	通用
例子	在线命令输入 >>RTC_TIME = 113706 >>print RTC_TIME 输出: 113706
相关指令	TIME , TIMES

SCOPE -- 数据采样

类型	系统指令
描述	数据采样，保存到 TABLE，最多同时采样 8 类数据。 使用 TRIGGER 开始自动采样，采样时间=采样周期*采样个数。
语法	SCOPE(enable[, period]) SCOPE(enable, period, table_start, table_stop, p0 [,p1 [,p2 [,p3 [,p4 [,p5 [,p6 [,p7]]]]]])) Enable: 使能与否 Period: 系统周期，一般为 1ms，可用 SERVO_PERIOD 查看 Table_start: 采样数据存储在 TABLE 的起始位置 Table_stop: TABLE 结束位置，减去起始位置为采样个数 p0~p7: 采样数据类型，等分存储在 TABLE 范围
适用控制器	通用
例子	BASE(0) ATYPE=1 UNITS=100 DPOS=0

	SPEED=1000 ACCEL=500 SCOPE (ON, 10, 0, 1000, DPOS (0), MSPEED (0)) ' 每隔 10ms 抓取 dpos 和 mspeed ' 存储在 TABLE0~1000 ' 0~499 存 dpos, 500~1000 存 mspeed ' 共采样 1000/2*10=5s TRIGGER ' 开始抓取 MOVE (10000)
相关指令	TRIGGER , SCOPE_POS

SCOPE_POS -- 采样点数

类型	系统指令
描述	只读，返回当前 SCOPE 采样数据的点数
语法	Var = SCOPE_POS
适用控制器	通用
例子	BASE (0) ATYPE=1 UNITS=100 DPOS=0 SPEED=1000 ACCEL=500 SCOPE (ON, 10, 0, 1000, DPOS (0), MSPEED (0)) ' 采样设置 TRIGGER ' 开始抓取 MOVE (10000) WHILE 1 ?SCOPE_POS ' 返回当前已采样保存的点数 WEND
相关指令	SCOPE

UNLOCK -- 解锁控制器

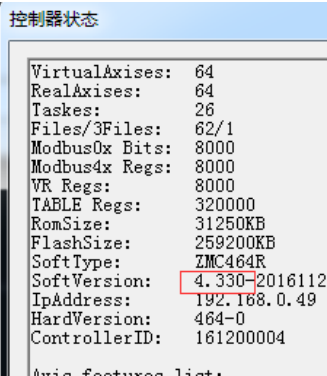
类型	系统指令
描述	解锁控制器。

	LOCK 密码采用不可逆算法加密，一旦忘记，将不可获知。
语法	UNLOCK (pass) pass LOCK 时的密码。
适用控制器	通用
相关指令	LOCK

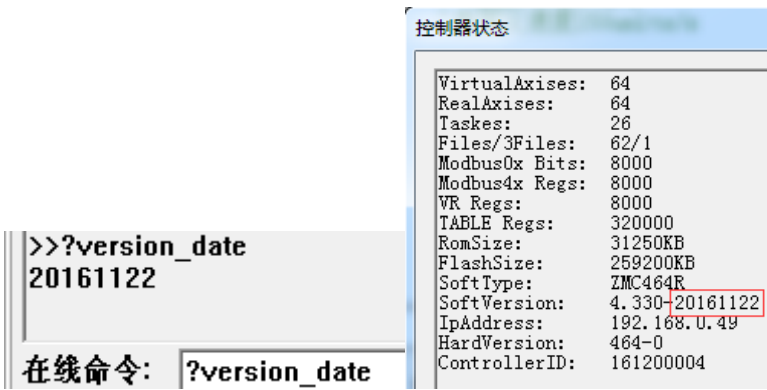
LIMIT_BUFFERED -- 运动缓冲限制

类型	系统参数
描述	限定运动缓冲个数，不能超过控制器的最大值。
语法	LIMIT_BUFFERED = value
适用控制器	通用
相关指令	REMAIN BUFFER , MOVES BUFFERED

VERSION -- 系统软件版本

类型	系统状态
描述	系统软件版本号。
语法	VARI = VERSION
适用控制器	通用
例子	?*version ' 打印出软件版本，结果，4.3  >>?*version 4.3300 在线命令: ?*version

VERSION_DATE -- 系统固件版本

类型	系统状态
描述	系统固件版本号。
语法	VAR1 = VERSION_DATE
适用控制器	通用
例子	?*version_date ' 打印出固件版本，结果，20161122 

VR -- 掉电保存

类型	系统指令
描述	掉电保存寄存器组，32 位浮点型。 注意不同型号控制器的数量有区别。 保存浮点数，与 VR_INT 和 VR_STRIN 共用一个空间。
语法	VR(index) = value , VAR1 = VR(index)
适用控制器	通用
例子	VR(0) = 10 Aaa = VR(0) ?Aaa
相关指令	VR_INT ， VRSTRING

VR_INT -- 掉电保存整型

类型	系统指令
描述	掉电保存寄存器组，32 位整型。

	注意不同型号控制器的数量有区别。 只能保存整型，与 VR 和 VR_STRIN 共用一个空间。
语法	VR_INT(index) = value , VAR1 = VR_INT(index)
适用控制器	通用
例子	VR(0) = 10 Aaa = VR(0) ?Aaa
相关指令	VR , VRSTRING

VRSTRING -- 掉电保存字符串

类型	字符串函数
描述	用 VR 来存储字符串。 保存 ASCII 码，与 VR 和 VR_INT 共用一个空间，一个字符占用一个 VR。
语法	VRSTRING (index[, chares]) index 起始的 VR 编号，编号从 0 开始。 chares 读取的字符总数。
适用控制器	通用
例子	在线命令输入 >> VRSTRING (0, 8) = "abc" ' 保存字符串 >>print VRSTRING (0, 8) 输出: abc
相关指令	VR , VR_INT

WDOG -- 轴总使能

类型	系统参数
描述	控制所有轴的使能。
语法	WDOG=0/1
适用控制器	通用
例子	WDOG=1 ' 所有轴使能打开
相关指令	AXIS_ENABLE

第十章 存储相关指令

ZMotion 运动控制器都拥有内部 FLASH 存储器，部分型号控制器具备外部存储器接口，可以接 U 盘或 SD 卡，参见具体控制器的硬件手册。

U 盘或 SD 卡要格式化为 FAT 格式。

FILE -- U 盘文件操作

类型	文件指令			
描述	加载、搜索控制器或 U 盘文件。 根据对应字符串选择功能。			
语法	value = FILE "function" ,... function 功能选择 <table><tr><td>"LOAD_ZAR"</td><td>FILE "LOAD_ZAR", "filename" 加载 U 盘里面的 ZAR 升级文件。 filename: 程序文件名 升级失败返回 0，同时会 WARN 输出失败原因。 升级成功后会自动启动 ZAR 文件，因此返回值 TRUE 没有意义。  升级后原来的调试状态的断点会被清除掉。</td></tr></table>		"LOAD_ZAR"	FILE "LOAD_ZAR", "filename" 加载 U 盘里面的 ZAR 升级文件。 filename: 程序文件名 升级失败返回 0，同时会 WARN 输出失败原因。 升级成功后会自动启动 ZAR 文件，因此返回值 TRUE 没有意义。  升级后原来的调试状态的断点会被清除掉。
"LOAD_ZAR"	FILE "LOAD_ZAR", "filename" 加载 U 盘里面的 ZAR 升级文件。 filename: 程序文件名 升级失败返回 0，同时会 WARN 输出失败原因。 升级成功后会自动启动 ZAR 文件，因此返回值 TRUE 没有意义。  升级后原来的调试状态的断点会被清除掉。			

	"LOAD_TCF"	FILE "LOAD_TCF", "filename", TableIndex, Maxsize 专有文件 TCF 文件读取。 Filename: 文件名 TableIndex: 存储起始 TABLE 索引 Maxsize: 存储 TABLE 总索引个数 <table><tr><td>TABLE0</td><td>标志位</td><td></td></tr><tr><td>TABLE1</td><td>总点数</td><td></td></tr><tr><td>TABLE2</td><td>胶头编号</td><td></td></tr><tr><td></td><td></td><td></td></tr><tr><td></td><td></td><td></td></tr><tr><td></td><td>TABLE100</td><td>点类型</td><td>第一个点</td></tr><tr><td></td><td>TABLE101</td><td>X 坐标</td><td></td></tr><tr><td></td><td>TABLE102</td><td>Y 坐标</td><td></td></tr><tr><td></td><td>TABLE103</td><td>Z 坐标</td><td></td></tr><tr><td></td><td>TABLE104</td><td>预留</td><td></td></tr><tr><td></td><td>TABLE105</td><td>点类型</td><td>第二个点</td></tr><tr><td></td><td>TABLE106</td><td>X 坐标</td><td></td></tr><tr><td></td><td>TABLE107</td><td>Y 坐标</td><td></td></tr><tr><td></td><td>TABLE108</td><td>Z 坐标</td><td></td></tr><tr><td></td><td>TABLE109</td><td>预留</td><td></td></tr><tr><td></td><td></td><td></td><td></td></tr></table>	TABLE0	标志位		TABLE1	总点数		TABLE2	胶头编号									TABLE100	点类型	第一个点		TABLE101	X 坐标			TABLE102	Y 坐标			TABLE103	Z 坐标			TABLE104	预留			TABLE105	点类型	第二个点		TABLE106	X 坐标			TABLE107	Y 坐标			TABLE108	Z 坐标			TABLE109	预留					
	TABLE0	标志位																																																											
	TABLE1	总点数																																																											
	TABLE2	胶头编号																																																											
	TABLE100	点类型	第一个点																																																										
	TABLE101	X 坐标																																																											
	TABLE102	Y 坐标																																																											
	TABLE103	Z 坐标																																																											
	TABLE104	预留																																																											
	TABLE105	点类型	第二个点																																																										
	TABLE106	X 坐标																																																											
	TABLE107	Y 坐标																																																											
	TABLE108	Z 坐标																																																											
	TABLE109	预留																																																											
	"LOAD_BYTE"	FILE "LOAD_BYTE", "filename", TableIndex, Maxsize, Offset 字节方式加载文件。 Filename: 文件名 TableIndex: 存储起始 TABLE 索引 Maxsize: 存储 TABLE 总索引个数 Offset: 文件开始读取的字节偏移 <table><tr><td>TABLE0</td><td>总字节数</td></tr><tr><td>TABLE1</td><td>读取到的第一个字节</td></tr><tr><td>TABLE2</td><td>第二个字节</td></tr><tr><td></td><td></td></tr><tr><td></td><td></td></tr><tr><td>TABLEn</td><td>第 n 个字节</td></tr></table>	TABLE0	总字节数	TABLE1	读取到的第一个字节	TABLE2	第二个字节					TABLEn	第 n 个字节																																															
TABLE0	总字节数																																																												
TABLE1	读取到的第一个字节																																																												
TABLE2	第二个字节																																																												
TABLEn	第 n 个字节																																																												
	"FIND_FIRST"	FILE "FIND_FIRST", type, vr 搜索 U 盘文件。 Type: 1- 文件 / 2-文件夹 / ".extend" 文件后缀名 vr: VRSTRING(vr) 存储查找的结果, 超过最大 VR 时, 使																																																											

		用 MODBUS_STRING。
	"FIND_NEXT"	FILE "FIND_NEXT", vr 搜索下一个 U 盘文件。 vr: VRSTRING(vr) 存储查找的结果, 超过最大 VR 时, 使用 MODBUS_STRING。
	"FIND_PREV"	FILE "FIND_PREV", vr 搜索上一个 U 盘文件。 vr: VRSTRING(vr) 存储查找的结果, 超过最大 VR 时, 使用 MODBUS_STRING。
	"FLASH_FIRST"	FILE "FLASH_FIRST", type, vr 搜索 FLASH 文件, 只支持 BIN 和 Z3P 文件。 Type: 1- 文件 / 2-文件夹 / ".extend" 文件后缀名 vr: VRSTRING(vr) 存储查找的结果, 超过最大 VR 时, 使用 MODBUS_STRING。
	"FLASH_NEXT"	FILE "FLASH_NEXT", vr 搜索下一个 FLASH 文件。 vr: VRSTRING(vr) 存储查找的结果, 超过最大 VR 时, 使用 MODBUS_STRING。
	"FLASH_PREV"	FILE "FLASH_PREV", vr 搜索上一个 FLASH 文件。 vr: VRSTRING(vr) 存储查找的结果, 超过最大 VR 时, 使用 MODBUS_STRING。
	"FLASH_DEL"	FILE "FLASH_DEL", string 删除指定文件。 String: 文件名全称, 带扩展名。
	"DELETE"	FILE "DELETE", "filename" 删除 U 盘指定文件。 Filename: 文件名全称, 带扩展名。
	"COPY FROM"	FILE "COPY FROM", "FLASH 文件名"[, "U 盘文件名"]

		FLASH 文件拷贝到 U 盘，只支持 BIN 和 Z3P 文件。 FLASH 文件名规则：SD 块号.BIN 如 SD0.BIN 表示 FLASH 块号 0 SD1.BIN 表示 FLASH 块号 1
	"COPY_TO"	FILE "COPY_TO", "U 盘文件名"[, "FLASH 文件名"] U 盘文件拷贝到 FLASH，只支持 BIN 和 Z3P 文件。
适用控制器	通用，使用 U 盘功能时要求控制器带 U 盘接口	
例子	<pre> DIM result ' 定义变量 IF U_STATE=TRUE THEN ' U 盘插入判断 Result = FILE "find_first", ".zar", 10 ' 扫描第一个 zar 格式文件，名字 保存到 VR IF result=TRUE THEN ' 扫描成功判断 FILE"load_zar", VRSTRING(10, 20) ' 下载扫描到文件名与存储到 VR 里字 符相同的 zar 文件 ENDIF ENDIF ENDIF </pre>	

FLASHVR -- 拷贝 RAM 的数据

类型	存储指令
描述	拷贝 RAM 的数据到 FLASH 里面。 ZMC00x（除了 005）、ECI 全系列控制器把 TABLE 数据存储到了 FLASH 的最后一个块，用 FLASH_WRITE, FLASH_READ 指令也可以操作到这个块，要避免冲突。 其他系列控制器把 TABLE 数据存储到另一个独立的存储区域，无法操作。
语法	FLASHVR (function) function 功能选择

	<table border="1"> <tr> <td>-1</td><td>整个 TABLE 都存储到 FLASH。 并且上电时自动读取到 TABLE。</td></tr> <tr> <td>-2</td><td>取消 FLASH 上电时读取到 TABLE</td></tr> </table>	-1	整个 TABLE 都存储到 FLASH。 并且上电时自动读取到 TABLE。	-2	取消 FLASH 上电时读取到 TABLE
-1	整个 TABLE 都存储到 FLASH。 并且上电时自动读取到 TABLE。				
-2	取消 FLASH 上电时读取到 TABLE				
适用控制器	通用				
例子	FLASHVR (-1) 'table 存到 flash 块，重新上电后再从 flash 读回 table				
相关指令	FLASH_WRITE				

STICK_WRITE -- table 输出到 U 盘

类型	存储指令						
描述	拷贝 TABLE 的数据到外部存储器里面。 value=TRUE 表示操作成功，否则操作失败。 建议使用 U_WRITE。 不具备外部存储接口的控制器不支持此命令。						
语法	value = STICK_WRITE(filename, table_start [,length [,format]]) filename 文件号，对应到 SD【filename】 table_start 开始操作的起始 TABLE 号。 length 要操作的 TABLE 元素个数，缺省 128 format 写入文件的格式 <table border="1"> <tr> <th>值</th><th>描述</th></tr> <tr> <td>0 (缺省)</td><td>float 格式存储，.BIN</td></tr> <tr> <td>1</td><td>文本格式存储，.CSV</td></tr> </table>	值	描述	0 (缺省)	float 格式存储，.BIN	1	文本格式存储，.CSV
值	描述						
0 (缺省)	float 格式存储，.BIN						
1	文本格式存储，.CSV						
适用控制器	带 U 盘接口						
例子	STICK_WRITE(0,0,128,0) '拷贝 table 前 128 个元素以 float 格式到外部存储器，产生 SD0.BIN 文件						
相关指令	U_WRITE ， STICK_WRITEVR						

STICK_READ -- U 盘读取到 table

类型	存储指令						
描述	拷贝外部存储器的数据到 TABLE 里面。 value=TRUE 表示操作成功，否则操作失败。 建议使用 U_READ。 不具备外部存储接口的控制器不支持此命令。						
语法	value = STICK_READ (filenum, table_start [,format]) filenum 文件号，对应到 SD【filenum】 table_start 开始操作的起始 TABLE 号。 format 写入文件的格式 <table border="1"> <tr> <th>值</th><th>描述</th></tr> <tr> <td>0 (缺省)</td><td>float 格式存储, .BIN</td></tr> <tr> <td>1</td><td>文本格式存储, .CSV</td></tr> </table>	值	描述	0 (缺省)	float 格式存储, .BIN	1	文本格式存储, .CSV
值	描述						
0 (缺省)	float 格式存储, .BIN						
1	文本格式存储, .CSV						
适用控制器	带 U 盘接口						
例子	STICK_READ (0,10,0) ' 拷贝外部存储器名称为 SDO 的 bin 文件数据到 TABLE, 从 TABLE(10)开始保存						
相关指令	U_READ , STICK_READVR						

STICK_WRITEVR -- vr 输出到 U 盘

类型	存储指令				
描述	拷贝 VR 的数据到外部存储器里面。 value=TRUE 表示操作成功，否则操作失败。 建议使用 U_WRITE。 不具备外部存储接口的控制器不支持此命令。				
语法	value = STICK_READVR (filenum, vr_start [,format]) filenum 文件号，对应到 SD【filenum】 vr_start 开始操作的起始 VR 号。 length 要操作的元素个数，缺省 128 format 文件的格式 <table border="1"> <tr> <th>值</th><th>描述</th></tr> <tr> <td>0 (缺省)</td><td>float 格式存储, .BIN</td></tr> </table>	值	描述	0 (缺省)	float 格式存储, .BIN
值	描述				
0 (缺省)	float 格式存储, .BIN				

	1	文本格式存储, .CSV
适用控制器	带 U 盘接口	
例子	STICK_WRITEVR (0,0,128,0) ' 拷贝 VR 前 128 个元素以 float 格式存贮到外部存储器, 生成 SD0.BIN 文件	
相关指令	U_WRITE , STICK_WRITE	

STICK_READVR -- U 盘读取到 vr

类型	存储指令						
描述	拷贝外部存储器的数据到 VR 里面。 value=TRUE 表示操作成功, 否则操作失败。 建议使用 U_READ。 不具备外部存储接口的控制器不支持此命令。						
语法	value = STICK_READVR (filenum, vr_start [,format]) filenum 文件号, 对应到 SD【filenum】.BIN vr_start 开始操作的起始 VR 号。 format 文件的格式 <table border="1"> <tr> <th>值</th><th>描述</th></tr> <tr> <td>0 (缺省)</td><td>float 格式存储</td></tr> <tr> <td>1</td><td>文本格式存储</td></tr> </table>	值	描述	0 (缺省)	float 格式存储	1	文本格式存储
值	描述						
0 (缺省)	float 格式存储						
1	文本格式存储						
适用控制器	带 U 盘接口						
例子	STICK_READVR (0,20,0) ' 拷贝外部存储器名为 SD0 的 bin 文件数据到 VR, 从 VR(20)开始存储。						
相关指令	U_READ , STICK_READ						

FLASH_WRITE -- flash 存储

类型	存储指令
描述	存储变量或者数组, 数组的单个或部分元素到内部 FLASH 里面, 掉电保存。

	内部 FLASH 采用顺序存储的方式，读取的顺序必须与存储时的顺序一致。 内部 FLASH 有存储次数限制，不要随意循环操作。
语法	<pre>FLASH_WRITE sect_num [, varname] [, arrayname] [, arrayname(a)] [, arrayname(a, length)]</pre> sect_num FLASH 块编号, 不同类型不一样 varname 变量名 Arrayname 数组名, 可以为 TABLE, VR a 操作的数组索引 length 操作的数组元素个数
适用控制器	通用
例子	<pre>FLASH_WRITE 1, VAR, ARRAY1, ARRAY2(1)</pre> ' 把 VAR, ARRAY1, ARRAY2(1) 的数据依次写入 flash 块 1
相关指令	FLASHVR , FLASH_READ

FLASH_READ -- flash 读取

类型	存储指令
描述	从内部 FLASH 读取数据到变量，或数组里面。 内部 FLASH 采用顺序存储的方式，读取的顺序必须与存储时的顺序一致。 读取未被写入过的 Flash 块时，会提示 Online command warn, File:C:\SDn.BIN read error，不影响使用。
语法	<pre>FLASH_READ sect_num [, varname] [, arrayname] [, arrayname(a)] [, arrayname(a, length)]</pre> sect_num FLASH 块编号, 不同类型不一样 varname 变量名 arrayname 数组名, 可以为 TABLE, VR a 操作的数组索引 length 操作的数组元素个数
适用控制器	通用
例子	<pre>FLASH_READ 1, VAR, ARRAY1, ARRAY2(1)</pre> ' 读取 flash 块 1 的数据，依次保存到变量 VAR，数组 ARRAY1，ARRAY2(1)
相关指令	FLASHVR , FLASH_WRITE

FLASH_SECTSIZE -- flash 变量数

类型	系统状态函数
描述	读取内部 FLASH 一个块可以存储的变量个数。 不同控制器个数不同。
语法	value = FLASH_SECTSIZE
适用控制器	通用
例子	? FLASH_SECTSIZE 打印结果 20480 'ZMC1xx 系列 1024 'ZMC00x 系列 ...
相关指令	FLASH_SECTES

FLASH_SECTES -- flash 块数

类型	系统状态函数
描述	读取控制器内部 FLASH 的总块数。 不同控制器块数不同。 对 ZMC00x 系列控制器，FLASHVR 会使用用最后一个块，要避免冲突。
语法	value = FLASH_SECTES
适用控制器	通用
例子	?FLASH_SECTES ' 打印出 flash 块数 打印结果 128 'ZMC005 ...
相关指令	FLASH_SECTSIZE

U_WRITE -- 输出到 U 盘

类型	存储指令
描述	存储变量或者数组, 数组的单个或部分元素到外部存储器里面。 不具备外部存储接口的控制器不支持此命令。 文件格式为 32 位 ieee 浮点数顺序存储, 一个变量或一个数组元素占用一个浮点数, 可以通过 PC 事先做好文件, 然后用 U_READ 指令读取。
语法	<pre>U_WRITE sect_num, [, varname] [, arrayname] [, arrayname(a)][, arrayname(a, length)]</pre> sect_num 文件编号, 对应到 SD 【filenum】.BIN。 varname 变量名 Arrayname 数组名, 可以为 TABLE, VR。 a 操作的数组索引 length 操作的数组元素个数
适用控制器	带 U 盘接口
例子	<pre>If U_STATE = TRUE then 'U 盘已插入 U_WRITE 0, TABLE(0, 10) 'TBAL0-10 的数据写入 U 盘文件 SDO Endif</pre>
相关指令	U_READ , U_STATE

U_READ -- 从 U 盘读取

类型	存储指令
描述	从外部存储器读取数据到变量或数组里面。 不具备外部存储接口的控制器不支持此命令。 文件格式为 32 位 ieee 浮点数顺序存储, 一个变量或一个数组元素占用一个浮点数, 可以通过 PC 事先做好文件, 然后用 U_READ 指令读取。
语法	<pre>U_READ sect_num, [, varname] [, arrayname] [, arrayname(a)][, arrayname(a, length)]</pre> sect_num 文件编号, 对应到 SD 【filenum】.BIN varname 变量名

	Arrayname 数组名, 可以为 TABLE, VR a 操作的数组索引 length 操作的数组元素个数
适用控制器	带 U 盘接口
例子	<pre>If U_STATE = TRUE then 'U 盘已插入 U_READ 1, VAR, table(0), ARRAY2(1) '读取 U 盘文件 SD1 数据 Endif</pre>
相关指令	U_WRITE , U_READ2 , U_STATE

U_READ2 -- 从 U 判读取 2

类型	存储指令
描述	从外部存储器读取数据到变量或数组里面。 不具备外部存储接口的控制器不支持此命令。 文件格式为 32 位 ieee 浮点数顺序存储, 一个变量或一个数组元素占用一个浮点数, 可以通过 PC 事先做好文件, 然后用 U_READ 指令读取。
语法	<pre>U_READ sect_num, star_num[, varname] [, arrayname] [, arrayname(a)] [, arrayname(a, length)]</pre> sect_num 文件编号, 对应到 SD【filenum】.BIN start_num 文件内读取的起始位置 varname 变量名 Arrayname 数组名, 可以为 TABLE, VR a 操作的数组索引 length 操作的数组元素个数
适用控制器	带 U 盘接口
例子	<pre>If U_STATE = TRUE then 'U 盘已插入 U_READ2 1, 10, VAR, table(0), ARRAY2(1) '读取 U 盘文件 SD1 数据 '从 SD1 位置 10 开始 Endif</pre>
相关指令	U_READ , U_WRITE , U_STATE

U_READDSB -- DSB 文件读取

类型	存储指令	
描述	DSB 文件读取。 不具备外部存储接口的控制器不支持此命令。	
语法	U_READDSB "filename", TableIndex, Maxsize [, flag] filename 文件名 TableIndex 存储起始 TABLE 索引 Maxsize 存储 TABLE 总索引个数 flag 预留.	
	编号	内容
	0	标志位: 标明一行的变量数;(暂无用)
	1	针迹数:
	2--29	文件名:28 个字符
	30	换色次数:
	31	+X:
	32	-X:
	33	+Y:
	34	-Y:
	35	AX: +
	36	AY: -
	37	MX: +
	38	MY: +
	39	PD:
		
	40	L_limt 左边界坐标
	41	R_limt 右边界坐标
	42	U_limt 上边界坐标
	43	D_limt 下边界坐标
	99	已读到总行数
	100	行类型 第一个点
	101	参数 1: X 相对位移
	102	参数 2: Y 相对位移
	103	参数 3: 相对起针点 X 坐标
	104	参数 4: 相对起针点 Y 坐标
	105	行类型 第二个点
	106	参数 1: X 相对位移
	107	参数 2: Y 相对位移

适用控制器	带 U 盘接口
相关指令	U_READ , U_WRITE , U_STATE

U_STATE -- U 盘状态

类型	系统状态函数
描述	检查 U 盘是否插入。 有插入返回 TRUE，否则返回 FALSE。 不具备外部存储接口的控制器不支持此命令。 U 盘里面的文件不要放太多。
语法	Val=U_STATE
适用控制器	带 U 盘接口
例子	<pre>?U_STATE '打印 U 盘状态 If U_STATE = TRUE then 'U 盘已插入 U_READ 1, VAR, ARRAY1, ARRAY2(1) 'U 盘数据读取 Endif</pre>
相关指令	U_READ , U_WRITE

第十一章 中断相关

INT_ENABLE -- 中断总开关

类型	系统参数						
描述	中断总开关。 为了避免程序没有初始化好进入中断，中断开关缺省是关闭的。						
语法	INT_ENABLE = switch <table border="1"> <thead> <tr> <th>值</th><th>描述</th></tr> </thead> <tbody> <tr> <td>0 (缺省)</td><td>关闭</td></tr> <tr> <td>1</td><td>打开</td></tr> </tbody> </table>	值	描述	0 (缺省)	关闭	1	打开
值	描述						
0 (缺省)	关闭						
1	打开						

适用控制器	通用
例子	INT_ENABLE=1 ' 开启总中断 具体例程参考 ONPOWEROFF 和 ONTIMERn
相关指令	ONPOWEROFF , ONTIMERn

ONPOWEROFF -- 掉电中断 SUB

类型	中断
描述	掉电中断程序入口，必须是全局的 SUB 过程。 控制器只有 1 个掉电中断。 掉电中断执行的时间特别有限，只能写少数几条语句。
语法	GLOBAL ONPOWEROFF () END SUB
适用控制器	通用
例子	INT_ENABLE = 1 END GLOBAL SUB ONPOWEROFF () vr(0) = dpos(0) ' 保存坐标 vr(1) = dpos(1) vr(2) = dpos(2) END SUB
相关指令	INT_ENABLE

INT_ONn -- 外部输入中断 SUB

类型	中断
描述	外部输入中断程序入口，上升沿触发，必须是全局的 SUB 过程。 必须支持 PLC 功能的固件才可使用。 中断 IN 口 0~31。

语法	<pre>GLOBAL SUB INT_ONn() n 是 IN 编号 END SUB</pre>
适用控制器	支持 PLC 功能的固件
例子	<pre>INT_ENABLE=1 ' 开启中断 END GLOBAL SUB INT_ON0() ' 中断程序 print "输入 IN0 上升沿触发" END SUB</pre>
相关指令	INT_OFFn , INT_ENABLE

INT_OFFn -- 外部输入中断 SUB

类型	中断
描述	外部输入中断程序入口，下降沿触发，必须是全局的 SUB 过程。 必须支持 PLC 功能的固件才可使用。 中断 IN 口 0~31。
语法	<pre>GLOBAL SUB INT_OFFn() n 是 IN 编号 END SUB</pre>
适用控制器	支持 PLC 功能的固件
例子	<pre>INT_ENABLE=1 ' 开启中断 END GLOBAL SUB INT_OFF0() ' 中断程序 print "输入 IN0 下降沿触发" END SUB</pre>
相关指令	INT_ONn , INT_ENABLE

ONTIMERn -- 定时器中断 SUB

类型	中断
描述	定时器中断程序入口，必须是全局的 SUB 过程。 定时器个数根据控制器型号查看。
语法	GLOBAL SUB ONTIMERn() n 是定时器编号 END SUB
适用控制器	通用
例子	INT_ENABLE=1 ' 开启中断 TIMER_START(0,100) ' 定时器 0 开启，周期 100ms END GLOBAL SUB ONTIMER0() ' 中断程序 print "ontimer0 enter" END SUB
相关指令	INT_ENABLE ， TIMER_START

TIMER_IFEND -- 定时器状态

类型	系统函数
描述	返回定时器是否结束。
语法	value = TIMER_IFEND (timernum) 返回 0 定时器正在定时，即未执行对应中断程序。 返回 1 定时器定时完成，开始执行对应中断程序。 支持 PLC 的固件在启动定时器前打印 0，不支持 PLC 的固件打印 1。
适用控制器	通用
例子	INT_ENABLE=1 ' 开启中断 ?TIMER_IFEND(0) ' 定时器未启动，打印结果 0 ' 不支持 PLC 的固件则打印 1 TIMER_START(0,2000) ' 定时器 0 启动，周期 2s ?TIMER_IFEND(0) ' 打印结果，0，定时器正在定时 DELAY(2000) ?TIMER_IFEND(0) ' 打印结果，1，定时器定时完成，执行中断

相关指令	ONTIMER , TIMER_START

TIMER_START -- 启动定时器

类型	系统指令
描述	启动系统定时器，定时器只执行 1 次。
语法	TIMER_START(timernum, time_ms) timernum 编号 0-定时器个数-1 time_ms 定时器长度，单位毫秒
适用控制器	通用
例子	参考 TIMER_IFEND 例程
相关指令	ONTIMER , TIMER_IFEND

TIMER_STOP -- 停止定时器

类型	系统指令
描述	强制停止系统定时器。
语法	TIMER_STOP (timernum) timernum 编号 0-定时器个数-1
适用控制器	通用
例子	<pre> INT_ENABLE=1 ' 开启中断 TIMER_START(0,2000) ' 定时器 0 启动，周期 2s ?TIMER_IFEND(0) ' 打印结果，0，未运行 DELAY(2000) ?TIMER_IFEND(0) ' 打印定时器 0 状态 ' 打印结果，1，已运行 TIMER_STOP(0) ' 停止计时器 0 ?TIMER_IFEND(0) ' 打印结果，0，未运行 </pre>
相关指令	ONTIMER , TIMER_IFEND

第十二章 总线相关

槽位号

槽位号是指控制器上接口的编号，总线接口缺省为 0。当控制器上有多个总线接口，?*slot 查看。

在指令说明中，用 slot 代表槽位号。

设备号

设备号是指一个槽位上连接的所有设备的编号，从 0 开始，按连接顺序依次增加，可以通过 NODE_COUNT(slot) 查看。

在指令说明中，用 node 代表设备号。

驱动器编号

控制器会自动识别出槽位上的驱动器，编号从 0 开始，按连接顺序依次增加。

驱动器编号与设备号不同，假设控制器连接了 3 个设备，驱动器前面连接了两个其他 IO 设备，那么驱动器此时的设备号 node=2，而驱动器编号为 0。

?*SLOT -- 打印总线接口

类型	EtherCAT 辅助指令
描述	查看控制器总线接口编号及类型。
语法	?*SLOT
适用控制器	带总线接口
例子	?*SLOT 打印结果 Slot:0-ETHERCAT '当前只有一个 EtherCAT 总线，编号为 0

SLOT_SCAN -- 总线扫描

类型	总线指令
描述	总线扫描

	通过 RETURN 返回成功与否， 返回-1 扫描成功， 0 扫描失败。 Rtex 扫描失败会直接报错 6209。 遇到不支持的设备类型，RETURN 会返回 0。 Rtex 控制器未连接设备时会报错，EtherCAT 控制器不会。 扫描后可以使用 NODE 指令读取相关设备信息，并使用 DRIVE 相关指令配置。
语法	SLOT_SCAN (slot) slot 控制器 EtherCAT 槽位号 0-缺省
适用控制器	带 EtherCAT 接口或 RTEX 接口
例子	<pre> aa: ' 标记 aa SLOT_SCAN(0) ' 总线扫描 ? RETURN ' 打印返回值， -1 成功， 0 失败 IF RETURN THEN ?NODE_COUNT(0) ' 扫描成功， 返回当前连接的设备个数 ELSE ?"扫描失败" delay (1000) ' 等待 1s GOTO aa ' 扫描失败， 跳转到 aa， 重新扫描 ENDIF </pre>
相关指令	SLOT_START , SLOT_STOP

SLOT_START -- 总线开启

类型	总线指令
描述	总线启动。 通过 RETURN 返回成功与否。返回-1 启动成功， 0 启动失败。 需在总线扫描 SLOT_SCAN 成功后再执行。 执行前， 先设置好 AXIS_ADDRESS, ATYPE, DRIVE_PROFILE
语法	SLOT_START (slot [,opstate]) slot 控制器 EtherCAT 槽位号 0-缺省 opstate 启动到的 EtherCAT 状态, 4-SAFEOP, 8- OP(缺省) NODE_PDOBUFF 指令才使用
适用控制器	带 EtherCAT 接口或 RTEX 接口
例子	<pre> aa: ' 标记 aa SLOT_SCAN(0) ' 总线扫描 IF RETURN THEN ' 扫描成功， 进入轴设置 AXIS_ADDRESS(0)=1 ' 第一个驱动器映射到轴 0 </pre>

	ATYPE(0)=65 '轴类型 65，位置控制 DRIVE_PROFILE(0)=0 'PDO 周期扫描设置 SLOT_START(0) '开启总线 ELSE ?"扫描失败" delay (1000) '等待 1s GOTO aa '扫描失败，跳转到 aa，重新扫描 ENDIF
相关指令	SLOT_STOP , SLOT_SCAN , NODE_PDOBUF

SLOT_STOP -- 总线停止

[illegible]

	ENDIF
相关指令	SLOT_START , SLOT_SCAN

SDO_WRITE -- 数据字典写入

类型	总线指令，仅 EtherCAT 可用
描述	通过设备号和槽位号进行 SDO 写入。 通过 RETURN 返回成功与否，-1 写入成功，0 写入失败 需连接好设备，扫描总线后才能执行 只有可写的数据字典才能写入
语法	SDO_WRITE (slot, node, index, subindex ,type ,value) slot 槽位号 0-缺省 Node 设备编号 0- Index 数据字典编号，前面可加"\$"表示 16 进制，如\$6060 Subindex 子编号 Type 数据类型 1 Boolean 2 Integer 8 3 Integer 16 4 Integer 32 5 Unsigned 8 6 Unsigned 16 7 Unsigned 32 Value 数据值.
适用控制器	带 EtherCAT 接口
例子	<pre> SLOT_SCAN(0) IF NODE_COUNT(0)>0 THEN SDO_WRITE(0,0,\$6060,0,2,8) '0 号设备设置控制模式为 8，位置控制 ENDIF </pre>
相关指令	SDO_WRITE_AXIS , SDO_READ

SDO_WRITE_AXIS -- 数据字典写入

类型	总线指令，仅 EtherCAT 可用
描述	通过轴号 SDO 写入。 通过 RETURN 返回成功与否，-1 写入成功，0 写入失败 需连接好设备，扫描总线后才能执行 只有可写的数据字典才能写入
语法	SDO_WRITE_AXIS(axis, index, subindex ,type ,value) axis 轴号 Index 数据字典编号，前面可加"\$"表示 16 进制，如\$6060 Subindex 子编号 Type 数据类型 1 Boolean 2 Integer 8 3 Integer 16 4 Integer 32 5 Unsigned 8 6 Unsigned 16 7 Unsigned 32 Value 数据值.
适用控制器	带 EtherCAT 接口
例子	正确的连接了一个 EtherCAT 轴设备再使用例程 <pre> SLOT_SCAN(0) ' 总线扫描 IF NODE_COUNT(0)>0 THEN AXIS_ADDRESS(0)=1 ' 第一个驱动器映射到轴 0 ATYPE(0)=65 ' 轴类型 65，位置控制 DRIVE_PROFILE(0)=0 ' PDO 周期扫描设置 SDO_WRITE_AXIS(0,\$6060,0,2,8) ' 轴 0 设备设置控制模式为 8, 位置控制 ENDIF </pre>
相关指令	SDO_WRITE ， SDO_READ_AXIS

SDO_READ -- 数据字典读取

类型	总线指令，仅 EtherCAT 可用
描述	通过设备号和槽位号进行 SDO 读取。

	通过 RETURN 返回成功与否，-1 写入成功，0 写入失败 需连接好设备，扫描总线后才能执行 只有可读的数据字典才能读取
语法	SDO_READ (slot, node, index, subindex ,type, tablenum) slot 槽位号 0-缺省 Node 设备编号 0- Index 数据字典编号，前面可加"\$"表示 16 进制，如\$6060 Subindex 子编号 Type 数据类型 1 Boolean 2 Integer 8 3 Integer 16 4 Integer 32 5 Unsigned 8 6 Unsigned 16 7 Unsigned 32 tablenum 读取的数据存储的 TABLE 位置
适用控制器	带 EtherCAT 接口
例子	<pre> SLOT_SCAN(0) IF NODE_COUNT(0)>0 THEN SDO_ READ (0,0,\$6061,0,2,0) '读取 0 号设备的控制模式 '数据存到 table(0) ?table(0) '打印出数据 ENDIF </pre>
相关指令	SDO_READ_AXIS ， SDO_WRITE

SDO_READ_AXIS -- 数据字典读取

类型	总线指令，仅 EtherCAT 可用
描述	通过轴号进行 SDO 读取。 通过 RETURN 返回成功与否，-1 写入成功，0 写入失败 需连接好设备，扫描总线后才能执行 只有可读的数据字典才能读取

语法	SDO_READ_AXIS(axis, index, subindex ,type, tablenum) axis 轴号 Index 数据字典编号, 前面可加"\$"表示 16 进制, 如\$6060 Subindex 子编号 Type 数据类型 1 Boolean 2 Integer 8 3 Integer 16 4 Integer 32 5 Unsigned 8 6 Unsigned 16 7 Unsigned 32 tablenum 读取的数据存储的 TABLE 位置
适用控制器	带 EtherCAT 接口
例子	正确的连接了一个 EtherCAT 轴设备再使用例程 <pre> SLOT_SCAN(0) ' 总线扫描 IF NODE_COUNT(0)>0 THEN AXIS_ADDRESS(0)=1 ' 第一个驱动器映射到轴 0 ATYPE(0)=65 ' 轴类型 65, 位置控制 DRIVE_PROFILE(0)=0 ' PDO 周期扫描设置 SDO_WRITE_AXIS(0,\$6060,0,2,8) ' 轴 0 设备设置控制模式为 8, 位置控制 制 SDO_READ_AXIS (0,\$6061,0,2,0) ' 读取 0 号设备的控制模式, 数据存 到 table(0) ?table(0) ' 打印出数据 ENDIF </pre>
相关指令	SDO_READ , SDO_WRITE_AXIS

?*ETHERCAT -- 打印 EtherCAT 总线状态

类型	EtherCAT 辅助指令
描述	调试时使用, 可以显示每个 NODE 的重要状态。 总线扫描后才能显示设备状态。
语法	?*ETHERCAT
适用控制器	带 EtherCAT 接口

例子	?*ETHERCAT
	打印结果 <pre>Slot:0 contain 1 nodes. Lostcount:0-0. Node:0 status:1 manid:7595h productid:0h axes:1 Alstate:8 Node_profile:0. BindAxis:0 Drive_profile:0 Controlword:fh drive_status:1237h Drive_mode:8h target:ffffe067h encode:ffffe068h.</pre> Slot 0 contain 1 nodes 0 槽位口共连接了 1 个设备 Lostcount 0-0 丢包数 Node 设备连接 NODE 编号 Status 设备连接状态，参考 NODE_STATUS Mainid 厂商 ID Productid 设备 ID Axes 设备总轴数 Alstate 设备 OP 状态 Node_profile 设备 Profile 设置 Bindaxis 映射到控制器轴号 Drive_profile 设备收发 PDO 设置 Controlword 控制字 Drive_status 设备当前状态，参考 DRIVE_STATUS Drive_mode 设备控制模式 Target 电机位置 Encode 编码器位置
相关指令	PRINT

?*RTEX -- 打印 Rtex 总线状态

类型	Rtex 辅助指令
描述	调试时使用，可以显示每个 NODE 的重要状态。 总线启动后才能显示设备状态。
语法	?*RTEX
适用控制器	带 RTEX 接口

例子	?*RTEX 打印结果 <pre>Slot:0 contain 1 nodes. Lostcount:0-0. Node:0 status:1 manid:616e6150h devicetype:31h axes:1 Alstate:1. BindAxis:0 Drive_profile:0 Controlword:80h drive_status:3c1h target:ffffe5ch encode:ffffe5ch.</pre> Slot 0 contain 1 nodes 0 槽位口共连接了 1 个设备 Lostcount 0-0 丢包数 Node 设备连接 NODE 编号 Status 设备连接状态，参考 NODE_STATUS Mainid 厂商 ID Productid 设备 ID Axes 设备总轴数 Alstate 设备 RTEX 状态 Node_profile 设备 Profile 设置 Bindaxis 映射到控制器轴号 Drive_profile 设备收发 PDO 设置 Controlword 控制字 Drive_status 设备当前状态，参考 DRIVE_STATUS Drive_mode 设备控制模式 Target 电机位置 Encode 编码器位置
相关指令	PRINT

NODE_COUNT -- 设备个数

类型	总线指令
描述	通过总线连接的设备总个数。 总线扫描后可用。
语法	只读：var = NODE_COUNT (slot) slot, 槽位号, 0-缺省 可以直接打印出来，见例一 可以直接作为数据，见例二
适用控制器	带 EtherCAT 接口或 RTEX 接口
例子	例一 SLOT_SCAN(0) ? NODE_COUNT(0) ’ 打印出 0 槽口连接的设备数 例二 SLOT_SCAN(0) IF NODE_COUNT(0) = 3 THEN ’ 指定连接了多少设备，才进行下一步

	’轴映射、轴类型设置等程序代码块 ENDIF
相关指令	NODE_INFO

NODE_STATUS -- 设备状态

类型	总线指令								
描述	设备状态，总线扫描后可用 <table border="1"> <thead> <tr> <th>BIT 位</th><th>意义</th></tr> </thead> <tbody> <tr> <td>0</td><td>指明 node 是否存在； 1- 存在</td></tr> <tr> <td>1</td><td>通讯状态； 1-出错；</td></tr> <tr> <td>2</td><td>节点状态； 1-出错；</td></tr> </tbody> </table> 值为 1 时，bit0 为 1，bit1 和 bit2 为 0，设备通讯正常 值为 3 时，bit0 和 bit1 为 1，bit2 为 0，设备通讯出错	BIT 位	意义	0	指明 node 是否存在； 1- 存在	1	通讯状态； 1-出错；	2	节点状态； 1-出错；
BIT 位	意义								
0	指明 node 是否存在； 1- 存在								
1	通讯状态； 1-出错；								
2	节点状态； 1-出错；								
语法	只读：var = NODE_STATUS (slot, node) slot, 槽位号, 0-缺省 node, 设备编号 0-								
适用控制器	带 EtherCAT 接口或 RTEX 接口								
例子	<pre> SLOT_SCAN(0) IF NODE_COUNT(0)>0 THEN ?NODE_STATUS(0,0) ’读取设备 0 的状态 ’打印值 1，通讯正常 ENDIF </pre>								
相关指令	NODE_INFO								

NODE_AXIS_COUNT -- 设备电机数

类型	总线指令
描述	每个设备带电机个数读取。 必须总线扫描后才能读取。
语法	只读：var = NODE_AXIS_COUNT (slot, node) slot, 槽位号, 0-缺省 node, 设备编号 0-
适用控制器	带 EtherCAT 接口或 RTEX 接口

例子	<pre> SLOT_SCAN(0) IF NODE_COUNT(0)>0 THEN ? NODE_AXIS_COUNT (0,0) ' 打印出设备 0 带电机个数 ENDIF </pre>
相关指令	NODE_INFO

NODE_IO -- 设备 IO

类型	EtherCAT 总线指令
描述	设备的 IO 起始编号设置，单个设备的输入输出的起始编号一样。 设置结果只会是 8 的倍数。 必须总线扫描后才能读取。 一般用于 EIO 扩展板的 IO 设置，其他设备含有 IO 口时也可使用。
语法	可读: var = NODE_IO (slot, node) 可写: NODE_IO(slot, node)=iobase slot, 槽位号, 0-缺省 node, 设备编号 0-
适用控制器	带 EtherCAT 接口或 RTEXT 接口
例子	<pre> SLOT_SCAN(0) IF NODE_COUNT(0)>0 THEN NODE_IO(0,0)=32 ' 设置设备 0 的 IO 起始编号为 32 ?NODE_IO(0,0) ' 打印出设备 0 的 IO 起始编号 ENDIF </pre>
相关指令	NODE_AIO

NODE_AIO -- 设备模拟量

类型	总线指令
描述	设备的 AIO 起始编号设置，单个设备的输入输出的起始编号一样。 必须总线扫描后才能读取。 一般用于 EIO 扩展板的 AIO 设置，其他设备含有 AIO 口时也可使用。
语法	可读: var = NODE_AIO (slot, node[,idir]) 可写: NODE_AIO(slot, node[,idir])=Aiobase slot, 槽位号, 0-缺省 node, 设备编号 0-

	idir AD/DA 选择 0-缺省, 设置时同时 ain, aout, 读取时只 ain。 3- ain 4-aout
适用控制器	带 EtherCAT 接口或 RTEX 接口
例子	<pre> SLOT_SCAN(0) IF NODE_COUNT(0)>0 THEN NODE_AIO(0,0,3)=3 ' 设置设备 0 的 AIN 起始编号为 3 ?NODE_AIO(0,0,3) ' 打印出设备 0 的 AIN 起始编号 ENDIF </pre>
相关指令	NODE_IO

NODE_INFO -- 设备信息

类型	EtherCAT 总线指令
描述	总线设备的信息读取。 必须总线扫描后才能读取。
语法	只读: var = NODE_INFO (slot, node, sel) slot, 槽位号, 0-缺省 node, 设备编号 0- sel 信息编号 0- VENDER, 厂商编号 1 - DEVICE, 设备编号 2- VERSION, 版本 3 - ALIAS, 别名, 一般用来识别驱动器。 4 - 预留 下面是 IO 的个数: 10- in 个数 11- op 个数 12-ain 个数 13-aout 个数 14- 预留
适用控制器	带 EtherCAT 接口或 RTEX 接口

例子	<pre> SLOT_SCAN(0) IF NODE_COUNT(0)>0 THEN ?NODE_INFO(0,0,10) '读取设备 0 的输入 IN 个数 ?NODE_INFO(0,0,0) '读取设备 0 的厂商编号 ?NODE_INFO(0,0,11) '读取设备 0 的输出 OP 个数 ENDIF </pre>
相关指令	SLOT_SCAN

NODE_PROFILE -- PDO 预设置

类型	总线指令
描述	总线设备的 profile 设置。 预留，总线启动后不能再修改。
语法	可读：var= NODE_PROFILE(slot,node) 可写：NODE_PROFILE(slot, node) = iprofile[, reserve]
适用控制器	带 EtherCAT 接口或 RTEK 接口

NODE_PDOBUFF -- 特殊设备 PDO 设置

类型	总线指令								
描述	特殊 ecat 设备的 PDO 支持。 非轴和 IO 的设备，通过这个指令来读写 PDO，例如电源设备。 轴和 IO 类型的设备，已经可以通过轴参数和 IO 指令来访问 PDO，不能使用这个指令。 SLOT_START 会自动提前读取当前的 PDO 列表，以及可写 PDO 的当前值。可以在 SLOT_START 调用前通过 SDO 修改 PDO 列表，或相关数据字典的当前值。 需要启动以后才能修改，可以先用 SOD_START 启动到 SAFEOP，然后设置初始化 PDO 状态.，再启动到 OP。								
语法	命令语法：NODE_PDOBUFF (slot, node, index, subindex ,type) 函数语法：Buff = NODE_PDOBUFF (slot, node, index, subindex ,type) 参数： <table> <tr> <td>slot</td><td>槽位号 0-缺省</td></tr> <tr> <td>Node</td><td>设备编号 0-</td></tr> <tr> <td>Index</td><td>数据字典编号，前面可加"\$"表示 16 进制，如\$6060 必须是 PDO 列表中的字典才能访问。</td></tr> <tr> <td>Subindex</td><td>子编号</td></tr> </table>	slot	槽位号 0-缺省	Node	设备编号 0-	Index	数据字典编号，前面可加"\$"表示 16 进制，如\$6060 必须是 PDO 列表中的字典才能访问。	Subindex	子编号
slot	槽位号 0-缺省								
Node	设备编号 0-								
Index	数据字典编号，前面可加"\$"表示 16 进制，如\$6060 必须是 PDO 列表中的字典才能访问。								
Subindex	子编号								

	Type 数据类型 1 Boolean 2 Integer 8 3 Integer 16 4 Integer 32 5 Unsigned 8 6 Unsigned 16 7 Unsigned 32
适用控制器	带 EtherCAT 接口，4 系列产品，20170508 以上版本支持
例子	<pre>>>node_pdobuff(0,0, \$6040, 0, 3) = 15 >>?node_pdobuff(0,0, \$6041, 0, 3)</pre>
相关指令	SDO WRITE ， SDO READ

NODE_PRESET -- 设备预配置

类型	EtherCAT 总线指令
描述	总线设备预先配置，配置以后没有挂上外设也提前启动总线，总线启动后不能再修改。 预设置后，可以通过 NODE_STATUS 判断外设是否挂上。 预设值的类型与实际不一样，将不能启动总线。 中间的外设没有预设值，也没有扫描到，也不能启动总线。 固件 20160601 以上版本支持
语法	命令语法 1: NODE_PRESET (slot, node, manuid, productid) 命令语法 2: NODE_PRESET (slot, -1) 清除所有预设置 函数语法 1: VALUE = NODE_PRESET (slot, node) 返回是否有预设置 函数语法 1: VALUE = NODE_PRESET (slot) 返回预设值的最大个数。 slot, 槽位号, 0-缺省 node, 设备编号 0- manuid, 厂商 ID 编号, 参考 NODE_INFO productid, 设备 ID 编号, 参考 NODE_INFO
适用控制器	带 EtherCAT 接口或 RTEX 接口
例子	<pre>node_preset(0,-1)' 清除原来的预设置 node_preset(0,0, \$83, 5) ' 设置第一个 NODE 为 OMRON 驱动器 slot_scan(0) ?"scan result:",RETURN, "max", node_count(0) ' 此时会自动显示总数为 1 个</pre>

	<pre> for i= 0 to node_count(0) -1 ? "node", i ? "status", node_status(0,i) ? "manu:", NODE_INFO(0,i,0) ? "dev:", NODE_INFO(0,i,1) ? "motor:", NODE_AXIS_COUNT(0,i) Next </pre>
相关指令	NODE STATUS

DRIVE_FE -- 驱动器误差

类型	轴状态
描述	驱动器的当前误差超限，对应数据字典 0x60F4。 必须设置正确的 ATYPE 以后才能设置。
语法	只读：var = DRIVE_FE (axis) Axis: 轴号
适用控制器	带 EtherCAT 接口或 RTEX 接口
例子	<pre> SLOT_SCAN(0) ' 总线扫描 IF NODE_COUNT(0)>0 THEN AXIS_ADDRESS(0)=1 ' 第一个驱动器映射到轴 0 ATYPE(0)=65 ' 轴类型 65，位置控制 DRIVE_PROFILE(0)=0 ' PDO 周期报文设置，根据 DRIVE_PROFILE ?DRIVE_FE (0) ' 读取轴 0 设备的跟踪误差值 ENDIF </pre>
相关指令	DRIVE_FE_LIMIT

DRIVE_FE_LIMIT -- 驱动器误差限制

类型	EtherCAT 轴参数
描述	驱动器的当前误差超限设置。 必须设置正确的 ATYPE 以后才能设置。 预留。
语法	可读：var=DRIVE_FE_LIMIT(axis) 可写：DRIVE_FE_LIMIT(axis)= value Axis: 轴号

适用控制器	带 EtherCAT 接口或 RTEX 接口
相关指令	DRIVE_FE

DRIVE_STATUS -- 驱动器状态

类型	轴状态																																											
描述	驱动器的当前状态，按位判断状态。 必须设置正确的 ATYPE 以后才能读取。 对于 EtherCAT 驱动器，对应数据字典 6041 bit6,5,3-0(switch on disabled / quick stop / fault / operation enabled / switched on / ready to switch on) 根据此 Bit 可以确认 PDS 的状态。以下表示状态和对应的 Bit。 <table><tr><th>状态字</th><th colspan="2">PDS state</th></tr><tr><td>xxxx xxxx x0xx 0000 b</td><td>Not ready to switch on</td><td>初始化 未完成状态</td></tr><tr><td>xxxx xxxx x1xx 0000 b</td><td>Switch on disabled</td><td>初始化 完成状态</td></tr><tr><td>xxxx xxxx x01x 0001 b</td><td>Ready to switch on</td><td>主电路电源 OFF 状态</td></tr><tr><td>xxxx xxxx x01x 0011 b</td><td>Switched on</td><td>伺服使能 OFF / 伺服准备</td></tr><tr><td>xxxx xxxx x01x 0111 b</td><td>Operation enabled</td><td>伺服使能 ON</td></tr><tr><td>xxxx xxxx x00x 0111 b</td><td>Quick stop active</td><td>即停止</td></tr><tr><td>xxxx xxxx x0xx 1111 b</td><td>Fault reaction active</td><td>异常(报警)判断</td></tr><tr><td>xxxx xxxx x0xx 1000 b</td><td>Fault</td><td>异常(报警)状态</td></tr></table> 其他位意义请查看对应驱动器手册说明。 对于 Rtex 驱动器 Rtex 驱动器的状态字位意义如下图，详细含义请查看松下 Rtex 手册第 4-2-3 章节。 Bit7~bit0 <table><tr><td>Servo _Active</td><td>Servo _Ready</td><td>Alarm</td><td>Warning</td><td>Torque _Limited</td><td>Homing _Complete</td><td>In_Progress</td><td>In_Position</td></tr><tr><td>SI-MON5 /E-STOP</td><td>SI-MON4 /EX-SON</td><td>SI-MON3 /EXT3</td><td>SI-MON2 /EXT2</td><td>SI-MON1 /EXT1</td><td>HOME</td><td>POT /NOT</td><td>NOT /POT</td></tr></table> Bit15~bit8	状态字	PDS state		xxxx xxxx x0xx 0000 b	Not ready to switch on	初始化 未完成状态	xxxx xxxx x1xx 0000 b	Switch on disabled	初始化 完成状态	xxxx xxxx x01x 0001 b	Ready to switch on	主电路电源 OFF 状态	xxxx xxxx x01x 0011 b	Switched on	伺服使能 OFF / 伺服准备	xxxx xxxx x01x 0111 b	Operation enabled	伺服使能 ON	xxxx xxxx x00x 0111 b	Quick stop active	即停止	xxxx xxxx x0xx 1111 b	Fault reaction active	异常(报警)判断	xxxx xxxx x0xx 1000 b	Fault	异常(报警)状态	Servo _Active	Servo _Ready	Alarm	Warning	Torque _Limited	Homing _Complete	In_Progress	In_Position	SI-MON5 /E-STOP	SI-MON4 /EX-SON	SI-MON3 /EXT3	SI-MON2 /EXT2	SI-MON1 /EXT1	HOME	POT /NOT	NOT /POT
状态字	PDS state																																											
xxxx xxxx x0xx 0000 b	Not ready to switch on	初始化 未完成状态																																										
xxxx xxxx x1xx 0000 b	Switch on disabled	初始化 完成状态																																										
xxxx xxxx x01x 0001 b	Ready to switch on	主电路电源 OFF 状态																																										
xxxx xxxx x01x 0011 b	Switched on	伺服使能 OFF / 伺服准备																																										
xxxx xxxx x01x 0111 b	Operation enabled	伺服使能 ON																																										
xxxx xxxx x00x 0111 b	Quick stop active	即停止																																										
xxxx xxxx x0xx 1111 b	Fault reaction active	异常(报警)判断																																										
xxxx xxxx x0xx 1000 b	Fault	异常(报警)状态																																										
Servo _Active	Servo _Ready	Alarm	Warning	Torque _Limited	Homing _Complete	In_Progress	In_Position																																					
SI-MON5 /E-STOP	SI-MON4 /EX-SON	SI-MON3 /EXT3	SI-MON2 /EXT2	SI-MON1 /EXT1	HOME	POT /NOT	NOT /POT																																					
语法	只读：toq = DRIVE_STATUS (axis) Axis: 轴号																																											
适用控制器	带 EtherCAT 接口或 RTEX 接口																																											
例子	<pre>SLOT_SCAN(0) IF NODE_COUNT(0)>0 THEN ATYPE(0)=65 ' 总线位置控制 ? DRIVE_STATUS (0) ' 打印轴 0 设备的状态 ENDIF</pre>																																											
相关指令	ATYPE , DRIVE PROFILE																																											

DRIVE_TORQUE -- 驱动器力矩

类型	EtherCAT 轴状态
描述	驱动器的当前力矩。 必须设置正确的 ATYPE 与 DRIVE_PROFILE 以后才能读取。
语法	只读: var = DRIVE_TORQUE(axis) Axis: 轴号
适用控制器	带 EtherCAT 接口或 RTEX 接口
例子	<pre> SLOT_SCAN(0) IF NODE_COUNT(0)>0 THEN ATYPE(0)=65 ' 总线位置控制 DRIVE_PROFILE(0)=1 ' pdo 设为 1, 含有返回力矩功能 ? DRIVE_TORQUE (0) ' 打印轴 0 的力矩 ENDIF </pre>
相关指令	ATYPE , DRIVE_PROFILE

DRIVE_MODE -- 驱动器模式

类型	轴参数
描述	驱动器的控制模式，对应数据字典 0x6060。 必须设置正确的 ATYPE 以后才能操作这个参数。
语法	可读: var=DRIVE_MODE (axis) 可写: DRIVE_MODE (axis)= value Axis: 轴号
适用控制器	带 EtherCAT 接口或 RTEX 接口
例子	<pre> SLOT_SCAN(0) ... ' 轴使能过程，参考第十三章简易例程 IF NODE_COUNT(0)>0 THEN DRIVE_MODE(0)=8 ' 轴 0 设备设置为位置控制模式 ? DRIVE_MODE(0) ' 打印轴 0 的控制模式 ENDIF </pre>

DRIVE_PROFILE -- 驱动器 PDO 设置

类型	EtherCAT 轴参数
描述	每个轴的发送 pdo 接收 pdo 的配置选择。 必须设置正确的 ATYPE 以后才能操作这个参。 详细配置请咨询厂家。 EtherCAT 总线 -1, 表示使用驱动器的内置缺省 PDO 列表, 20160601 以上版本支持. 缺省 PDO 不带 0X6060 时, 无法使用 datum(21)回零指令。 -1 - 驱动器默认设置, 需要控制器版本 20160601 及以上 0-缺省配置, csp 位置模式 {0x60400010, 0x607a0020, 0x60600008}, //控制字 目标位置 模式 {0x60410010, 0x60640020}, //状态字 反馈位置 1-csp 位置模式+力矩反馈 {0x60400010, 0x607a0020, 0x60600008}, //控制字 目标位置 模式 {0x60410010, 0x60640020, 0x60770010}, //状态字 反馈位置 当前力矩 2-csp 位置模式+力矩反馈+锁存 1up {0x60400010, 0x607a0020, 0x60b80010, 0x60600008}, //控制字 目标位置 模式 probe 设置 {0x60410010, 0x60640020, 0x60770010, 0x60b90010, 0x60ba0020}, //状态字 反馈位置 当前力矩 probe 状态 probe 位置 3-csp 位置模式+力矩限制+力矩反馈+锁存 1 上升沿 {0x60400010, 0x607a0020, 0x60b80010, 0x60720010, 0x60600008}, //控制字 目标位置 模式 probe 设置 力矩限制 {0x60410010, 0x60640020, 0x60770010, 0x60b90010, 0x60ba0020}, //状态字 反馈位置 当前力矩 probe 状态 probe 位置 4-csp 位置模式+力矩反馈+驱动器 IO 输入 {0x60400010, 0x607a0020, 0x60600008}, //控制字 目标位置 模式 {0x60410010, 0x60640020, 0x60770010, 0x60fd0020}, //状态字 反馈位置 当前力矩 驱动器 IO 输入

	5-csp 位置模式+力矩反馈+驱动器 IO 输出+驱动器 IO 输入 {0x60400010, 0x607a0020, 0x60fe0120, 0x60600008}, //控制字 目标位置 IO 输出 mode {0x60410010, 0x60640020, 0x60770010, 0x60fd0020}, //状态字 反馈位置 当前力矩 驱动器 IO 输入 6-特殊驱动器专用 7-特殊驱动器专用 8-特殊驱动器专用 9-固件版本 160504 支持 {0x60400010, 0x607a0020, 0x60fe0120, 0x60b80010, 0x60720010, 0x60600008}, //控制字 目标位置 IO 输出(32 个) probe 设置 力矩限制 模式 {0x60410010, 0x60640020, 0x60770010, 0x60fd0020, 0x60b90010, 0x60ba0020}, //状态字 反馈位置 当前力矩 驱动器 IO 输入(32 个) probe 状态 probe 位置 10-固件版本 160504 以上支持, 加 drive_fe 部分 {0x60400010, 0x607a0020, 0x60fe0120, 0x60b80010, 0x60720010, 0x60600008}, //控制字 目标位置 IO 输出 probe 设置 力矩限制 模式 {0x60410010, 0x60640020, 0x60770010, 0x60fd0020, 0x60b90010, 0x60ba0020, 0x60f40020}, //状态字 反馈位置 当前力矩 驱动器 IO 输入 probe 状态 probe 位置 drive_fe 11-固件版本 160504 以上支持, probe 专用测试 {0x60400010, 0x607a0020, 0x60b80010, 0x60600008}, //控制字 目标位置 probe 设置 模式 {0x60410010, 0x60640020, 0x60770010, 0x60b90010, 0x60ba0020, 0x60bb0020, 0x60bc0020, 0x60bd0020}, //状态字 反馈位置 当前力矩 probe 状态 probe 位置 1/位置 2/位置 3/位置 4 12-固件版本 160504 以上支持, 特殊驱动器专用 13-固件版本 160504 以上支持, 带速度前馈与加速度前馈 {0x60400010, 0x60B20010, 0x607a0020, 0x60B10020, 0x60600008}, //控制字 加速度前馈 目标位置 速度前馈 模式 {0x60410010, 0x60640020, 0x60770010, 0x60fd0020, 0x606c0020}, //状态字 反馈位置 当前力矩 IO 实际速度
--	--

17-固件版本 160504 以上支持, csp/csv/cst 三种模式可以切换
 {0x60400010, 0x60710010, 0x60ff0020, 0x607a0020, 0x60b80010, 0x60720010, 0x60600008},
 //控制字 周期力矩 周期速度 目标力矩 目标位置 probe 模式 力矩限制
 {0x60410010, 0x60770010, 0x60640020, 0x60fd0020, 0x60b90010, 0x60ba0020, 0x60bb0020},
 //状态字 当前力矩 反馈位置 IO probe 全部

18-固件版本 160504 以上支持, csp/csv/cst 三种模式可以切换+力矩反馈读取
 {0x60400010, 0x60710010, 0x60ff0020, 0x607a0020, 0x60b80010, 0x60720010, 0x60600008},
 //控制字 周期力矩 周期速度 目标力矩 目标位置 probe 模式 力矩限制
 {0x60410010, 0x60770010, 0x60640020, 0x60fd0020, 0x60b90010, 0x60ba0020, 0x60bb0020, 0x60bc0020, 0x60bd0020},
 //状态字 当前力矩 反馈位置 IO probe 全部

20-固件版本 160504 以上支持, csp 位置+csv 速度
 {0x60400010, 0x60ff0020, 0x607a0020, 0x60600008},
 //控制字 目标速度 目标位置 模式
 {0x60410010, 0x60640020},
 //状态字 反馈位置

21-固件版本 160504 以上支持, csp 位置+csv 速度+力矩反馈
 {0x60400010, 0x60ff0020, 0x607a0020, 0x60600008},
 //控制字 目标速度 目标位置 模式
 {0x60410010, 0x60640020, 0x60770010},
 //状态字 反馈位置 当前力矩

22-固件版本 160504 以上支持, csp 位置+csv 速度+力矩反馈+色标锁存 1 上升沿
 {0x60400010, 0x60ff0020, 0x607a0020, 0x60b80010, 0x60600008},
 //控制字 目标速度 目标位置 模式 probe 设置
 {0x60410010, 0x60640020, 0x60770010, 0x60b90010, 0x60ba0020},
 //状态字 反馈位置 当前力矩 probe 状态 probe 位置

23-固件版本 160504 以上支持, csp 位置+csv 速度+力矩反馈+锁存 1 上升沿+力矩限制
 {0x60400010, 0x60ff0020, 0x607a0020, 0x60b80010, 0x60720010,

```

0x60600008},
//控制字    目标速度    目标位置    模式    probe 设置 力矩限制
{0x60410010, 0x60640020, 0x60770010, 0x60b90010, 0x60ba0020},
//状态字    反馈位置    当前力矩    probe 状态    probe 位置

24-固件版本 160504 以上支持, csp 位置+csv 速度+IO 输入+位置+力矩反馈
{0x60400010, 0x60ff0020, 0x607a0020, 0x60600008},
//控制字    目标速度    目标位置    模式
{0x60410010, 0x60640020, 0x60770010, 0x60fd0020},
//状态字    反馈位置    当前力矩    驱动器 IO 输入

25-固件版本 160504 以上支持, csp 位置+csv 速度+IO 输入+位置+力矩反馈
{0x60400010, 0x60ff0020, 0x607a0020, 0x60fe0120, 0x60600008},
//控制字    目标速度    目标位置    IO 输出    模式
{0x60410010, 0x60640020, 0x60770010, 0x60fd0020},
//状态字    反馈位置    当前力矩    驱动器 IO 输入

30-固件版本 160504 以上支持, csp 位置+cst 转矩
{0x60400010, 0x60710010, 0x607a0020, 0x60600008},
//控制字    目标转矩    目标位置    模式
{0x60410010, 0x60640020},
//状态字    反馈位置

31-固件版本 160504 以上支持, csp 位置+cst 转矩+力矩反馈
{0x60400010, 0x60710010, 0x607a0020, 0x60600008},
//控制字    目标力矩    目标位置    模式
{0x60410010, 0x60640020, 0x60770010},
//状态字    反馈位置    当前力矩

32-固件版本 160504 以上支持, csp 位置+cst 转矩+力矩反馈+锁存 1 上升沿
{0x60400010, 0x60710010, 0x607a0020, 0x60b80010 , 0x60600008},
//控制字    目标转矩    目标位置    模式    probe 设置
{0x60410010, 0x60640020, 0x60770010, 0x60b90010, 0x60ba0020},
//状态字    反馈位置    当前力矩    probe 状态    probe 位置

33-固件版本 160504 以上支持, csp 位置+cst 转矩+力矩反馈+锁存 1 上升沿+ 力矩限制
{0x60400010, 0x60710010, 0x607a0020, 0x60b80010, 0x60720010, 0x60600008 },
//控制字    目标转矩    目标位置    模式    probe 设置 力矩限制
{0x60410010, 0x60640020, 0x60770010, 0x60b90010, 0x60ba0020},
//状态字    反馈位置    当前力矩    probe 状态    probe 位置

```

	33-固件版本 160504 以上支持, csp 位置+cst 转矩+力矩反馈+锁存 1 上升沿+ 力矩限制 <pre>{0x60400010, 0x60710010, 0x607a0020, 0x60b80010, 0x60720010, 0x60600008 },</pre> <pre>//控制字 目标转矩 目标位置 模式 probe 设置 力矩限制</pre> <pre>{0x60410010, 0x60640020, 0x60770010, 0x60b90010, 0x60ba0020},</pre> <pre>//状态字 反馈位置 当前力矩 probe 状态 probe 位置</pre> 34-固件版本 160504 以上支持, csp 位置+cst 转矩+力矩反馈+驱动器 I0 输入 <pre>{0x60400010, 0x60710010, 0x607a0020, 0x60600008},</pre> <pre>//控制字 目标转矩 目标位置 模式</pre> <pre>{0x60410010, 0x60640020, 0x60770010, 0x60fd0020},</pre> <pre>//状态字 反馈位置 当前力矩 驱动器 I0 输入</pre> 35-固件版本 160504 以上支持, csp 位置+cst 转矩+力矩反馈+I0 输入+I0 输出 <pre>{0x60400010, 0x60710010, 0x607a0020, 0x60fe0120, 0x60600008},</pre> <pre>//控制字 目标转矩 目标位置 I0 输出 模式</pre> <pre>{0x60410010, 0x60640020, 0x60770010, 0x60fd0020},</pre> <pre>//状态字 反馈位置 当前力矩 驱动器 I0 输入</pre> Rtex 总线 0- 不带 I0 映射 1- 带驱动器 I0 映射 带 I0 映射时, 按 DRIVE I0 指令设置起始地址
语法	可读: var= DRIVE_ PROFILE(axis) 可写: DRIVE_ PROFILE(axis)= value Axis: 轴号
适用控制器	带 EtherCAT 接口或 RTEX 接口
例子	<pre>SLOT_SCAN(0)</pre> <pre>IF NODE_COUNT(0)>0 THEN</pre> <pre> AXIS_ADDRESS(1)=1</pre> <pre> ATYPE(1)=65</pre> <pre> DRIVE_ PROFILE(1)=-1 ' 轴 1 PDO 设置为-1, 设备默认值</pre> <pre> ? DRIVE_ PROFILE(1) ' 打印出轴 1 的 PDO 设置</pre> <pre>ENDIF</pre>
相关指令	ATYPE , NODE_PROFILE

DRIVE_CONTROLWORD -- 驱动器控制字

类型	轴参数																																																																																									
描述	驱动控制字，按位操作 必须设置正确的 ATYPE 以后才能操作这个参数。 对于 EtherCAT 驱动器， 对应数据字典 0x6040 EtherCAT 控制器 ATYPE=65 时控制字会根据 WDOG/AXIS_ENABLE 自动切换以使能驱动器，主要位操作如下图。具体位意义请查看对应驱动器手册。 <table><tr><th rowspan="3">Command</th><th colspan="5">bits of the controlword</th><th rowspan="3">PDS Transitions</th></tr><tr><th>bit 7</th><th>bit 3</th><th>bit 2</th><th>bit 1</th><th>bit 0</th></tr><tr><th>fault reset</th><th>enable operation</th><th>quick stop</th><th>enable voltage</th><th>switch on</th></tr><tr><td>Shutdown</td><td>0</td><td>-</td><td>1</td><td>1</td><td>0</td><td>2, 6, 8</td></tr><tr><td>Switch on</td><td>0</td><td>0</td><td>1</td><td>1</td><td>1</td><td>3</td></tr><tr><td>Switch on + Enable operation</td><td>0</td><td>1</td><td>1</td><td>1</td><td>1</td><td>3+4 (*1)</td></tr><tr><td>Enable operation</td><td>0</td><td>1</td><td>1</td><td>1</td><td>1</td><td>4, 16</td></tr><tr><td>Disable voltage</td><td>0</td><td>-</td><td>-</td><td>0</td><td>-</td><td>7, 9, 10, 12</td></tr><tr><td>Quick stop</td><td>0</td><td>-</td><td>0 (*2)</td><td>1</td><td>-</td><td>7, 10, 11</td></tr><tr><td>Disable operation</td><td>0</td><td>0</td><td>1</td><td>1</td><td>1</td><td>5</td></tr><tr><td>Fault reset</td><td></td><td>-</td><td>-</td><td>-</td><td>-</td><td>15</td></tr></table> 对于 Rtex 驱动器 Rtex 控制器控制字缺省自动设置，需要手动时先将 DRIVE_CW_MODE 设为 1。不能随便修改，位的意义如下图，详细说明请看松下 Rtex 手册的 4-2-3 章节。 Bit7-bit0 <table><tr><td>Servo On</td><td>0</td><td>0</td><td>Gain SW</td><td>TL SW</td><td>HM Ctrl</td><td>0</td><td>0</td></tr><tr><td>Hard Stop</td><td>SMT Stop</td><td>Pause</td><td>0</td><td>SL SW</td><td>0</td><td>EX-OUT2</td><td>EX-OUT1</td></tr></table> Bit15-bit8	Command	bits of the controlword					PDS Transitions	bit 7	bit 3	bit 2	bit 1	bit 0	fault reset	enable operation	quick stop	enable voltage	switch on	Shutdown	0	-	1	1	0	2, 6, 8	Switch on	0	0	1	1	1	3	Switch on + Enable operation	0	1	1	1	1	3+4 (*1)	Enable operation	0	1	1	1	1	4, 16	Disable voltage	0	-	-	0	-	7, 9, 10, 12	Quick stop	0	-	0 (*2)	1	-	7, 10, 11	Disable operation	0	0	1	1	1	5	Fault reset		-	-	-	-	15	Servo On	0	0	Gain SW	TL SW	HM Ctrl	0	0	Hard Stop	SMT Stop	Pause	0	SL SW	0	EX-OUT2	EX-OUT1
Command	bits of the controlword					PDS Transitions																																																																																				
	bit 7		bit 3	bit 2	bit 1		bit 0																																																																																			
	fault reset	enable operation	quick stop	enable voltage	switch on																																																																																					
Shutdown	0	-	1	1	0	2, 6, 8																																																																																				
Switch on	0	0	1	1	1	3																																																																																				
Switch on + Enable operation	0	1	1	1	1	3+4 (*1)																																																																																				
Enable operation	0	1	1	1	1	4, 16																																																																																				
Disable voltage	0	-	-	0	-	7, 9, 10, 12																																																																																				
Quick stop	0	-	0 (*2)	1	-	7, 10, 11																																																																																				
Disable operation	0	0	1	1	1	5																																																																																				
Fault reset		-	-	-	-	15																																																																																				
Servo On	0	0	Gain SW	TL SW	HM Ctrl	0	0																																																																																			
Hard Stop	SMT Stop	Pause	0	SL SW	0	EX-OUT2	EX-OUT1																																																																																			
语法	可读：var=DRIVE_CONTROLWORD(axis) 可写：DRIVE_CONTROLWORD(axis)=value Axis： 轴号																																																																																									
适用控制器	带 EtherCAT 接口或 RTEX 接口																																																																																									
例子	例一 EtherCAT SLOT_SCAN(0) IF NODE_COUNT(0)>0 THEN																																																																																									

	<pre> AXIS_ADRESS(0)=1 ATYPE(0)=65 ' EtherCAT 总线位置控制 DRIVE_PROFILE(0)=0 ' pdo 设为 0 DRIVE_CONTROLWORD(0)=128 ' 伺服错误清除 DELAY (100) DRIVE_CONTROLWORD(0)=6 ' 伺服 shutdown DELAY (100) DRIVE_CONTROLWORD(0)=15 ' 伺服 switch on ENDIF 例二 Rtex SLOT_SCAN(0) IF NODE_COUNT(0)>0 THEN AXIS_ADRESS(0)=1 ATYPE(0)=50 ' Rtex 总线位置控制 DRIVE_PROFILE(0)=1 ' 伺服带 IO DRIVE_CW_MODE=1 ' 手动设置控制字 DRIVE_CONTROLWORD(0)=128 ' 伺服使能 ENDIF </pre>
相关指令	ATYPE , DRIVE_CW_MODE

DRIVE_CW_MODE -- 驱动器设置

类型	轴参数
描述	必须设置正确的 ATYPE 以后才能操作这个参数。 部分版本为使用方便，总是可以直接操作 MODE。 EtherCAT 预留 Rtex 0- 控制器自动调整 DRIVE_CONTROLWORD，此时 DRIVE_CONTROLWORD 指令无效。 1- 可以手动调整，此时可以使用 DRIVE_CONTROLWORD 指令。 RTEX 的控制字不要随便修改。
语法	可读：var= DRIVE_CW_MODE(axis) 可写：DRIVE_CW_MODE(axis)=value
适用控制器	带 EtherCAT 接口或 RTEX 接口
相关指令	ATYPE , DRIVE_CONTROLWORD

DRIVE_IO -- 驱动器 IO

类型	轴参数																												
描述	直接配置 DRIVE 的 IO 的起始编号，输入输出都是同样的编号。 DRIVE_PROFILE 支持读取驱动器 IO 时起作用。																												
语法	可读：var= DRIVE_IO (axis) 可写：DRIVE_IO (axis)=value Axis: 轴号 EtherCAT 总线伺服的输入输出点功能及地址参考数据字典 60FD, 60FE（某些厂家不是按照标准协议地址，请查看对应驱动器手册） Rtex 总线伺服的输入输出点功能对应控制器的地址=DRIVE_IO+编号 <table border="1"> <tr> <th>DRIVE_IO+ 以下编号</th><th>伺服功能</th></tr> <tr> <td>输入点</td><td></td></tr> <tr> <td>0</td><td>NOT/POT</td></tr> <tr> <td>1</td><td>POT/NOT</td></tr> <tr> <td>2</td><td>HOME</td></tr> <tr> <td>3</td><td>SI-MON1/EXT1</td></tr> <tr> <td>4</td><td>SI-MON2/EXT2</td></tr> <tr> <td>5</td><td>SI-MON3/EXT3</td></tr> <tr> <td>6</td><td>SI-MON4/EX-S ON</td></tr> <tr> <td>7</td><td>SI-MON5/E-ST OP</td></tr> <tr> <td></td><td></td></tr> <tr> <td>输出点</td><td></td></tr> <tr> <td>0</td><td>EX-OUT1</td></tr> <tr> <td>1</td><td>EX-OUT2</td></tr> </table>	DRIVE_IO+ 以下编号	伺服功能	输入点		0	NOT/POT	1	POT/NOT	2	HOME	3	SI-MON1/EXT1	4	SI-MON2/EXT2	5	SI-MON3/EXT3	6	SI-MON4/EX-S ON	7	SI-MON5/E-ST OP			输出点		0	EX-OUT1	1	EX-OUT2
DRIVE_IO+ 以下编号	伺服功能																												
输入点																													
0	NOT/POT																												
1	POT/NOT																												
2	HOME																												
3	SI-MON1/EXT1																												
4	SI-MON2/EXT2																												
5	SI-MON3/EXT3																												
6	SI-MON4/EX-S ON																												
7	SI-MON5/E-ST OP																												
输出点																													
0	EX-OUT1																												
1	EX-OUT2																												
适用控制器	带 EtherCAT 接口或 RTEX 接口																												

例子	<pre> SLOT_SCAN(0) ... IF NODE_COUNT(0)>0 THEN DRIVE_IO(1)=32 DIM var var=DRIVE_IO(1) ?var ENDIF </pre> ’轴使能过程，参考第十三章简易例程 ’轴 1 设备的 IO 起始编号设为 32 ’定义变量 var ’var 赋值为轴 1 设备的 IO 起始编号 ’直接打印出轴 1 设备的 IO 起始编号
相关指令	NODE IO ， DRIVE PROFILE

DRIVE_CLEAR -- 清除报警

类型	总线指令，仅 Rtex 控制器可用
描述	操作当前 BASE 轴，清除驱动器报警。 通过 RETURN 返回成功与否。 驱动器没有错误时，使用指令控制器会警告 6015，不影响程序运行。
语法	<pre> BASE(轴号) DRIVE_CLEAR(para) Para: 0 清除当前告警 1 清除历史告警 2 清除外部输入告警 </pre>
适用控制器	带 RTEX 接口
例子	<pre> SLOT_SCAN(0) IF NODE_COUNT(0)>0 THEN BASE(0) DRIVE_CLEAR(0) ENDIF </pre> ’选择轴 0 ’清除当前告警
相关指令	DRIVE_READ ， DRIVE_WRITE

DRIVE_READ -- 参数读取

类型	总线指令，仅 Rtex 控制器可用
----	-------------------

描述	操作当前 BASE 轴，读取驱动器参数。 通过 RETURN 返回成功与否。																																											
语法	DRIVE_READ(para [,vr_index]) Para: - 参数分类*256+ 参数编号。(Pr7.20=7*256+20) - 参数= 130, 读取钳位的状态, BIT0, BIT1 表示两个通道的状态; - 参数=\$10000+(ssid), 读取 RTEX 驱动器系统信息, 字符串存储在 VRSTRING。 - 参数=\$20000+(报警功能码)+(\$1000*索引) 读取报警信息。 - 参数=\$30000+(监视功能码)+(\$1000*索引) 读取监视器信息。 vr_index: 读取数据存储在 vr 里面, 如果没有, 直接在输出栏打印出来。 以下为常用参数 1. 伺服参数 <table border="1"> <thead> <tr> <th>伺服参数</th><th>功能</th><th>值</th></tr> </thead> <tbody> <tr> <td>Pr0.00</td><td>设置电机转向</td><td>0: CW 1: CCW</td></tr> <tr> <td>Pr0.01</td><td>控制模式设置</td><td>一般设置为 0: 半闭环控制</td></tr> <tr> <td>Pr0.08</td><td>电机一圈脉冲数</td><td>0-8388608 (根据实际电机)</td></tr> <tr> <td>Pr0.09</td><td>齿轮比分子设定</td><td>0-1073741824</td></tr> <tr> <td>Pr0.10</td><td>齿轮比分母设定</td><td>1-1073741824</td></tr> <tr> <td>Pr4.01</td><td>正限位信号设置</td><td>常闭: 00818181h (8487297) 常开: 00010101h (65793)</td></tr> <tr> <td>Pr4.02</td><td>负限位信号设置</td><td>常闭: 00828282h (8553090) 常开: 00020202h (131586)</td></tr> <tr> <td>Pr4.03</td><td>Home 信号设置</td><td>常闭: 00A2A2A2h (10658466) 常开: 00222222h (2236962)</td></tr> <tr> <td></td><td></td><td></td></tr> <tr> <td>Rtex 通讯参数</td><td></td><td></td></tr> <tr> <td>Pr7.20</td><td>Rtex 通讯周期</td><td>-1: 将 Pr7.91 的设定生效 3: 0.5ms 6: 1.0ms</td></tr> <tr> <td>Pr7.21</td><td>Rtex 指令更新周期比</td><td>1: 1 倍 2: 2 倍</td></tr> <tr> <td>Pr7.91</td><td>Rtex 通讯周期扩展</td><td>62500 ns 125000 ns 250000 ns</td></tr> </tbody> </table>		伺服参数	功能	值	Pr0.00	设置电机转向	0: CW 1: CCW	Pr0.01	控制模式设置	一般设置为 0: 半闭环控制	Pr0.08	电机一圈脉冲数	0-8388608 (根据实际电机)	Pr0.09	齿轮比分子设定	0-1073741824	Pr0.10	齿轮比分母设定	1-1073741824	Pr4.01	正限位信号设置	常闭: 00818181h (8487297) 常开: 00010101h (65793)	Pr4.02	负限位信号设置	常闭: 00828282h (8553090) 常开: 00020202h (131586)	Pr4.03	Home 信号设置	常闭: 00A2A2A2h (10658466) 常开: 00222222h (2236962)				Rtex 通讯参数			Pr7.20	Rtex 通讯周期	-1: 将 Pr7.91 的设定生效 3: 0.5ms 6: 1.0ms	Pr7.21	Rtex 指令更新周期比	1: 1 倍 2: 2 倍	Pr7.91	Rtex 通讯周期扩展	62500 ns 125000 ns 250000 ns
伺服参数	功能	值																																										
Pr0.00	设置电机转向	0: CW 1: CCW																																										
Pr0.01	控制模式设置	一般设置为 0: 半闭环控制																																										
Pr0.08	电机一圈脉冲数	0-8388608 (根据实际电机)																																										
Pr0.09	齿轮比分子设定	0-1073741824																																										
Pr0.10	齿轮比分母设定	1-1073741824																																										
Pr4.01	正限位信号设置	常闭: 00818181h (8487297) 常开: 00010101h (65793)																																										
Pr4.02	负限位信号设置	常闭: 00828282h (8553090) 常开: 00020202h (131586)																																										
Pr4.03	Home 信号设置	常闭: 00A2A2A2h (10658466) 常开: 00222222h (2236962)																																										
Rtex 通讯参数																																												
Pr7.20	Rtex 通讯周期	-1: 将 Pr7.91 的设定生效 3: 0.5ms 6: 1.0ms																																										
Pr7.21	Rtex 指令更新周期比	1: 1 倍 2: 2 倍																																										
Pr7.91	Rtex 通讯周期扩展	62500 ns 125000 ns 250000 ns																																										

			500000 ns 1000000 ns 2000000 ns
	3. 驱动器系统信息		
	SSID		
	\$01	厂商名	
	\$05	设备类型	
	\$12	驱动器型号	
	\$13	驱动器序列号	
	\$14	驱动器软件版本	
	\$15	驱动器类型	
	\$22	电机型号	
	\$23	电机序列号	
	4. 报警信息		
	功能码	功能	索引
	\$000	读取当前报警 /报警履历	0: 本次的报警码 1: 上次的报警码 2: 前两次的报警码 ... 14: 前 14 次的报警码
	\$001	清除当前报警	0: 清除本次的报警
	\$011	清除所有报警	0: 清除报警履历
	\$012	外部位移传感器的 错误清除	0: 通过串行通信类型的外部位 移传感器清除箝位的错误。 进行外部位移传感器的错误清 除后, 请断开控制电源后重启。
	5. 监视器		
	功能码	功能	索引
	\$01	位置偏差, 指令单位	0: 滤波后的指令的偏差
	\$02	编码器分辨率, 脉冲/转	0: 电机编码器分辨率
适用控制器	\$04	指令位置, 指令单位	0: 滤波后的内部指令位置
	\$05	实际速度, Pr7.25 单位	0: 电机实际速度
	\$06	内部指令转矩, 0.1%	0: 到电机的指令转矩
	\$07	实际位置, 指令单位	0: 电机实际的位置
	\$09	箝位位置 1, 指令单位	0: CH1 箝位的电机的实际位置
	\$0A	箝位位置 2, 指令单位	0: CH2 箝位的电机的实际位置
	带 RTEX 接口		

例子	总线开启后才能使用 例一 <pre>IF NODE_COUNT(0)>0 THEN BASE(0) ' 选择轴 0 DRIVE_READ(7*256+11, 0) ' 读取 Pr7.11 参数数值, 保存到 vr(0) ?vr(0) ' 打印 ENDIF</pre> 例二 <pre>IF NODE_COUNT(0)>0 THEN BASE(0) ' 选择轴 0 DRIVE_READ(\$10000+\$01) ' 读取厂商名, 直接打印出来 ENDIF</pre>
相关指令	DRIVE_WRITE , DRIVE_CLEAR

DRIVE_WRITE -- 参数写入

类型	总线指令, 仅 Rtex 控制器可用																												
描述	操作当前 BASE 轴, 驱动器参数修改。 通过 RETURN 返回成功与否。																												
语法	DRIVE_WRITE(para value) Para: - 参数分类*256 + 参数编号。(Pr7.20=7*256+20) - 特殊参数=128, 当前参数写入 EEPROM。(此时 value=1) - 特殊参数=\$40000 + typecode + (设置值*256), 使用驱动器的回零箝位功能。 value: 参数值。 1. 伺服参数 <table border="1"> <thead> <tr> <th>伺服参数</th><th>功能</th><th>值</th></tr> </thead> <tbody> <tr> <td>Pr0.00</td><td>设置电机转向</td><td>0: CW 1: CCW</td></tr> <tr> <td>Pr0.01</td><td>控制模式设置</td><td>一般设置为 0: 半闭环控制</td></tr> <tr> <td>Pr0.08</td><td>电机一圈脉冲数</td><td>0-8388608 (根据实际电机)</td></tr> <tr> <td>Pr0.09</td><td>齿轮比分子设定</td><td>0-1073741824</td></tr> <tr> <td>Pr0.10</td><td>齿轮比分母设定</td><td>1-1073741824</td></tr> <tr> <td>Pr4.01</td><td>正限位信号设置</td><td>常闭: 00818181h (8487297) 常开: 00010101h (65793)</td></tr> <tr> <td>Pr4.02</td><td>负限位信号设置</td><td>常闭: 00828282h (8553090) 常开: 00020202h (131586)</td></tr> <tr> <td>Pr4.03</td><td>Home 信号设置</td><td>常闭: 00A2A2A2h (10658466)</td></tr> </tbody> </table>		伺服参数	功能	值	Pr0.00	设置电机转向	0: CW 1: CCW	Pr0.01	控制模式设置	一般设置为 0: 半闭环控制	Pr0.08	电机一圈脉冲数	0-8388608 (根据实际电机)	Pr0.09	齿轮比分子设定	0-1073741824	Pr0.10	齿轮比分母设定	1-1073741824	Pr4.01	正限位信号设置	常闭: 00818181h (8487297) 常开: 00010101h (65793)	Pr4.02	负限位信号设置	常闭: 00828282h (8553090) 常开: 00020202h (131586)	Pr4.03	Home 信号设置	常闭: 00A2A2A2h (10658466)
伺服参数	功能	值																											
Pr0.00	设置电机转向	0: CW 1: CCW																											
Pr0.01	控制模式设置	一般设置为 0: 半闭环控制																											
Pr0.08	电机一圈脉冲数	0-8388608 (根据实际电机)																											
Pr0.09	齿轮比分子设定	0-1073741824																											
Pr0.10	齿轮比分母设定	1-1073741824																											
Pr4.01	正限位信号设置	常闭: 00818181h (8487297) 常开: 00010101h (65793)																											
Pr4.02	负限位信号设置	常闭: 00828282h (8553090) 常开: 00020202h (131586)																											
Pr4.03	Home 信号设置	常闭: 00A2A2A2h (10658466)																											

		常开：00222222h（2236962）																													
转矩相关																															
Pr0.13	第一转矩限制	0~500%																													
Pr5.21	转矩限制选择 转矩控制时，固定为 Pr0.13（第1转矩限制）。	如下图所示																													
		<table><tr><th rowspan="2">设定值</th><th colspan="2">TL_SW = 0</th><th colspan="2">TL_SW = 1</th></tr><tr><th>负方向</th><th>正方向</th><th>负方向</th><th>正方向</th></tr><tr><td>0, [1]</td><td colspan="4">Pr0.13</td></tr><tr><td>2</td><td>Pr5.22</td><td>Pr0.13</td><td>Pr5.22</td><td>Pr0.13</td></tr><tr><td>3</td><td colspan="2">Pr0.13</td><td colspan="2">Pr5.22</td></tr><tr><td>4</td><td>Pr5.22</td><td>Pr0.13</td><td>Pr5.26</td><td>Pr5.25</td></tr></table>	设定值	TL_SW = 0		TL_SW = 1		负方向	正方向	负方向	正方向	0, [1]	Pr0.13				2	Pr5.22	Pr0.13	Pr5.22	Pr0.13	3	Pr0.13		Pr5.22		4	Pr5.22	Pr0.13	Pr5.26	Pr5.25
		设定值		TL_SW = 0		TL_SW = 1																									
			负方向	正方向	负方向	正方向																									
		0, [1]	Pr0.13																												
2	Pr5.22	Pr0.13	Pr5.22	Pr0.13																											
3	Pr0.13		Pr5.22																												
4	Pr5.22	Pr0.13	Pr5.26	Pr5.25																											
Pr5.22	第二转矩限制	0~500%																													
Pr5.25	正方向转矩限制	0~500%																													
Pr5.26	负方向转矩限制	0~500%																													
速度相关																															
Pr3.12	加速时间设置	0~10000ms（达到1000.r/min）																													
Pr3.13	减速时间设置	0~10000ms（达到1000.r/min）																													
Pr3.14	S加减速设置	0~1000ms																													
Pr3.17	速度限制选择 转矩控制时的速度限制 值的方式选择	<table><tr><th>设定值</th><th>SL_SW = 0</th><th>SL_SW = 1</th></tr><tr><td>[0]</td><td colspan="2">Pr3.21</td></tr><tr><td>1</td><td>Pr3.21</td><td>Pr3.22</td></tr></table> 设定为1时，根据RTEX通信指令SL_SW的数值进行选择。	设定值	SL_SW = 0	SL_SW = 1	[0]	Pr3.21		1	Pr3.21	Pr3.22																				
设定值	SL_SW = 0	SL_SW = 1																													
[0]	Pr3.21																														
1	Pr3.21	Pr3.22																													
Pr3.21	速度限制值1	0~20000r/min																													
Pr3.22	速度限制值2	0~20000r/min																													
Rtex 通讯 参数																															
Pr7.20	Rtex 通讯周期	-1：将Pr7.91的设定生效 3：0.5ms 6：1.0ms																													
Pr7.21	Rtex 指令更新周期比	1：1倍 2：2倍																													
Pr7.91	Rtex 通讯周期扩展	62500 ns 125000 ns 250000 ns 500000 ns 1000000 ns 2000000 ns																													

3. 回零箝位模式

typecode	
----------	--

	\$50	位置箝位状态监视器																							
	\$51	位置箝位 1 启动																							
	\$52	位置箝位 2 启动																							
	\$53	位置箝位 1, 2 启动																							
	\$54	位置箝位 1 解除																							
	\$58	位置箝位 2 解除																							
	\$5c	位置箝位 1, 2 解除																							
	设置值																								
	设置值=钳位 1 设置 + (\$10* 钳位 2 设置)																								
	<table><tr><th colspan="8">指令</th></tr><tr><td>bit7</td><td>6</td><td>5</td><td>4</td><td>3</td><td>2</td><td>1</td><td>0</td></tr><tr><td colspan="4">Latch_Sel2</td><td colspan="4">Latch_Sel1</td></tr></table>		指令								bit7	6	5	4	3	2	1	0	Latch_Sel2				Latch_Sel1		
指令																									
bit7	6	5	4	3	2	1	0																		
Latch_Sel2				Latch_Sel1																					
<table><tr><td>箝位 1/2 设置</td><td></td></tr><tr><td>\$0</td><td>Z 相信号触发</td></tr><tr><td>\$1</td><td>EXT1 的逻辑上升沿</td></tr><tr><td>\$2</td><td>EXT2 的逻辑上升沿</td></tr><tr><td>\$3</td><td>EXT3 的逻辑上升沿</td></tr><tr><td>\$9</td><td>EXT1 的逻辑下降沿</td></tr><tr><td>\$10</td><td>EXT2 的逻辑下降沿</td></tr><tr><td>\$11</td><td>EXT3 的逻辑下降沿</td></tr></table>		箝位 1/2 设置		\$0	Z 相信号触发	\$1	EXT1 的逻辑上升沿	\$2	EXT2 的逻辑上升沿	\$3	EXT3 的逻辑上升沿	\$9	EXT1 的逻辑下降沿	\$10	EXT2 的逻辑下降沿	\$11	EXT3 的逻辑下降沿								
箝位 1/2 设置																									
\$0	Z 相信号触发																								
\$1	EXT1 的逻辑上升沿																								
\$2	EXT2 的逻辑上升沿																								
\$3	EXT3 的逻辑上升沿																								
\$9	EXT1 的逻辑下降沿																								
\$10	EXT2 的逻辑下降沿																								
\$11	EXT3 的逻辑下降沿																								
适用控制器	支持 RTEX 的控制器																								
例子	总线开启后才能使用 例一 IF NODE_COUNT(0)>0 THEN BASE(0)																								

第十三章 简易例程

IO 操作

```
While 1                                     ' 循环
    if in(0) = on then                      ' 输入口 0 有效
        op(2, off)                         ' 关闭输出口 2
    else
        op(2, on)                          ' 打开输出口 2
    endif
wend
```

字符串使用和自定义通讯

例程 1:

```
SETCOM(38400, 8, 1, 0, 0, 0)
DIM TEMPVAR
DIM VALUE
DIM CHLIST(10)
For i=0 to 9
    GET #0, TEMPVAR
    CHLIST(i)=TEMPVAR
Next
TRACE CHLIST ' 调试
VALUE = VAL(CHLIST) ' 转成变量
PRINT #0, TOSTR(CHLIST) ' 转成字符串输出
```

字符串与数据相互转化

```
dim val1, val2, array1(15)

val1=1234 ' 变量 1 赋值 1234
for i=0 to 14
    array1(i)=0
next
```

```
?val1, val2    ' 打印两个变量确认值
?*array1       ' 打印数组确认值

array1=tostr(val1) ' 数据转成字符串，赋值给数组
?*array1        ' 再次打印数组确认

array1=TOSTR(val1)+"1asf" ' tostr 也可与其他字符串合并
?*array1

val2=val(array1) ' 字符串转化为数据赋值给变量 2
?val2           ' 打印变量 2 确认

' 只有数字字符可以来回转化，字母、符号字符不可以
```

编码器读取

松下 A6 编码器读取例程

```
' *****绝对值编码器部分*****
setcom(38400, 8, 1, 0, 1, 0) ' 设置 485 口 PORT1 为自定义协议

global dim tempchar ' 接收的一个字节

global dim neqbuff(2) ' 发送识别码 485 为 81H, 05H
neqbuff(0) = $81
neqbuff(1) = $05

global dim eotbuff(2) ' 接收识别码 485 为 80H, 04H
eotbuff(0) = $80
eotbuff(1) = $04

global dim ackbuff ' 接收应答 06H
ackbuff = $06

global dim cmdbuff(20) ' 发送命令数组
global dim getbuff(20) ' 接收的支付串

global dim getnum ' 接收的字节数
getnum = 0

global dim highdata ' 编码器多圈数据
global dim lowdata ' 单圈数据
```

```

runtask 4, get_char          ' 启动接收字符串任务

' ///////////////////////////////////
modbus_reg(0)=0
while 1
    if modbus_reg(0)=1 then
        modbus_reg(0)=0
        getmpos(1, 45)      ' 读取站号 1 的多圈与单圈值。
    endif
wend
end

' 读坐标
global sub getmpos(sifunum, rcr)          ' 读伺服编号为 1 的电机的绝对值位置
    cmdbuff(0) = $00
    cmdbuff(1) = sifunum
    cmdbuff(2) = $d2
    cmdbuff(3) = rcr

    neqbuff(0) = $80 + sifunum
    neqbuff(1) = $05

    eotbuff(0) = $80
    eotbuff(1) = $04

    getnum = 0
    putchar #1, neqbuff
    ticks = 2000
    wait until (getnum = 2) or ticks < 0
    if getnum = 2 then          ' 如果接到了 2 个字符
        if (getbuff(0) = $80 + sifunum) and (getbuff(1) = $04) then      ' 收到应
            答发送命令
                getnum = 0
                putchar #1, cmdbuff          ' 发送读取编码器命令
                ticks = 2000
                wait until (getnum = 3) or ticks < 0
                if (getbuff(0) = $06) and (getbuff(1) = $80) and (getbuff(2) = $05) then
                    ' 收到发送请求, 给应答
                        getnum = 0
                        putchar #1, eotbuff      ' 发送应答, 等待接收数据
                        ticks = 2000
                        wait until (getnum = 15) or ticks < 0
                        if getnum = 15 then          ' 读到数据 11-10 为多圈数据 9-7 为单圈数

```

据

```

        putchar #1,ackbuff
        highdata = getbuff(11) * $100 + getbuff(10)
        lowdata = getbuff(9) * $10000 + getbuff(8) * $100 +getbuff(7)
        'print
getbuff(11),getbuff(10),getbuff(9),getbuff(8),getbuff(7),getnum
    else
        print getnum,getbuff(0) ,getbuff(1),"超时重新读取 1"
    endif
    else
        print getbuff(0) ,getbuff(1),"超时重新读取 2"
    endif
    else
        print getbuff(0) ,getbuff(1),"驱动器无应答"
    endif
    else
        print "驱动器无应答"
    endif
end sub

' 串口接收
global sub get_char()
    while 1
        get #1,tempchar
        getbuff(getnum) = tempchar
        getnum = getnum + 1
    wend
end sub

```

SP 指令连续插补

```

ERRSWITCH = 3    ' 全部信息输出
base(0,1,2,3)    ' 选择 X Y Z U
rapidstop
wait idle

dpos = 0,0,0,0
atype=1,1,1,1    ' 脉冲方式步进或伺服
units = 100,100,100,100 ' 脉冲当量，每 mm100 脉冲
speed = 200,200,200,200 ' 此速度会对 FORCE_SPEED 进行限制
accel = 2000,2000,2000,2000
decel = 2000,2000,2000,2000
merge = on ' 启动连续插补

```



```

CORNER_MODE = 0          ' 不启动自动拐角减速, 手动设置 STARTMOVE_SPEED,
                           ' ENDMOVE_SPEED
'DECEL_ANGLE = 15 * (PI/180)' 开始减速的角度 15 度
'STOP_ANGLE = 45 * (PI/180)' 降到最低速度的角度 45 度

while 1 ' 循环运动

    if in(0) = on then ' 输入 0 有效启动运动
        TRACE "start movesp"
        ' 走一个方框, 每一段的速度和停止速度都不一样
        FORCE_SPEED = 100' 第一段速度 100
        ENDMOVE_SPEED = 10' 第一段结束的速度 10,
        MOVESP(100, 0)

        FORCE_SPEED = 150' 第二段速度 150
        ENDMOVE_SPEED = 15
        STARTMOVE_SPEED = 15    ' 第二段的起始速度 15, 因为高于第一段的结束速度
                                ' 10, 因此实际的起始速度为 10
        MOVESP(0, 100)

        FORCE_SPEED = 200' 第三段速度 200
        ENDMOVE_SPEED = 20
        STARTMOVE_SPEED = 20    ' 第三段的起始速度 20, 因为高于第二段的结束速度
                                ' 15, 因此实际的起始速度为 15
        MOVESP(-100, 0)

        FORCE_SPEED = 300' 第四段速度 300, 受 SPEED 限制其实为 200
        ENDMOVE_SPEED = 30
        STARTMOVE_SPEED = 30    ' 第三段的起始速度 30, 因为高于第二段的结束速
                                ' 度' 20, 因此实际的起始速度为 20
        MOVESP(0, -100)

        wait idle    ' 等待运动停止
        delay(100)   ' 延时
    endif

wend

end

```

手轮

```

ERRSWITCH = 3      ' 全部信息输出
const axishand = 0
base(axishand)      ' 选择第 0 轴接手轮
atype=3            ' 脉冲+方向 的手轮, 正交输入手轮使用 3

base(1)' 轴 1 被手轮控制
atype=1' 步进
dpos = 0
units = 100 ' 脉冲当量, 每 mm100 脉冲
speed = 200
accel = 3000
decel = 3000
sramp = 20
clutch_rate = 0' 采用速度加速度来进行限制

dim poslast          ' 记录上一个位置变量
poslast = dpos
while 1

    if in(0) = on and in(1) = off then
        connect(1, axishand)      ' 链接到轴 0, 倍率 1
    elseif in(1) = on and in(0) = off then
        connect(10, axishand)     ' 链接到轴 0, 倍率 10
    elseif in(0)= on and in(1) = on then
        connect(50, axishand)     ' 链接到轴 0, 倍率 50, 对步进, 倍率太高会出现丢步
                                   或
                                   ' 长时间才能结束
    elseif mtype = 21 then ' 取消 connect
        cancel
    endif

    if poslast <> dpos then
        poslast = dpos
        trace dpos
    endif

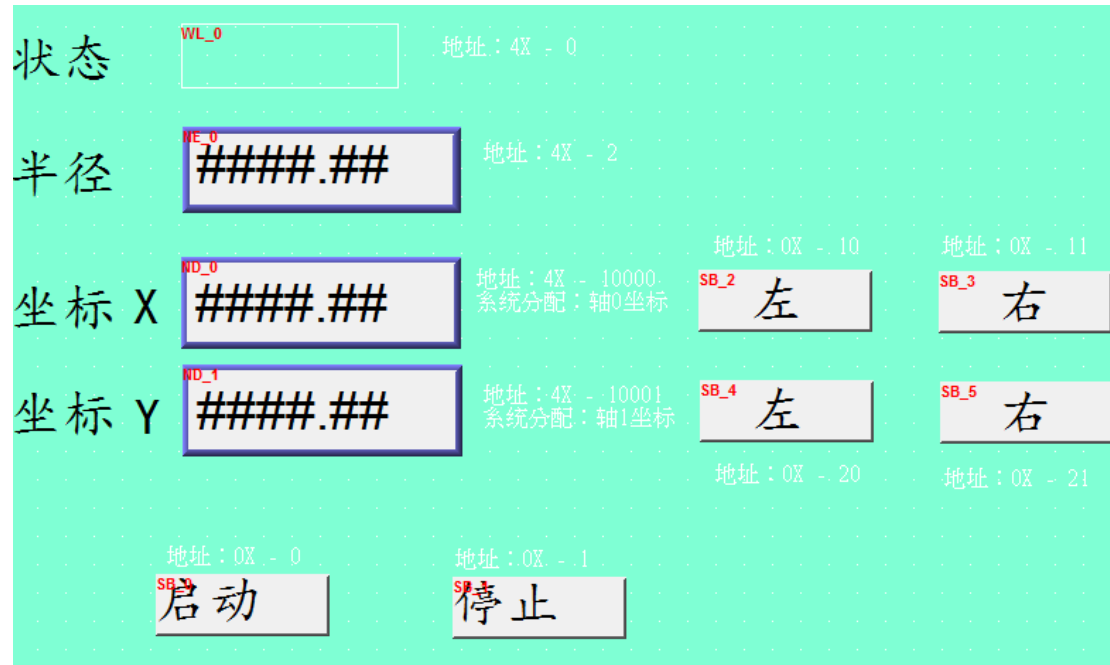
wend

end

```

触摸屏通讯

该例程触摸屏地址从 0 开始，也有部分触摸屏地址是从 1 开始的，地址从 1 开始的触摸屏上的 1 对应程序中的 0。



```
ERRSWITCH = 3      ' 全部信息输出
base(0,1)          ' 选定 X Y
rapidstop
wait idle
units = 100,100
speed = 100,100
accel = 1000,1000
decel = 1000,1000
dim radius         ' 半径
radius = 100       ' 缺省大小
modbus_ieee(2) = radius

dim run_state      ' 运行状态
run_state = 1      ' 1 停止，2，运行
modbus_reg(0) = run_state

dim stop_flag      ' 停止标识

runtask 2, guidetask ' 启动手动扫描任务
modbus_bit(0) = 0    ' 按钮复归
modbus_bit(1) = 0    ' 按钮复归
```

```

while 1      '扫描 modbus 输入

    if modbus_bit(0) = 1 then      '启动按钮按下
        modbus_bit(0) = 0        '按钮复归
        if run_state = 1 then
            trace "move start"
            stoptask 1            '保险
            runtask 1, movetask '启动运行任务
        endif

        elseif modbus_bit(1) = 1 then      '停止按钮按下
            trace "move stop"
            modbus_bit(1) = 0        '按钮复归
            stop_flag = 1
        endif
    endif
wend

end

movetask:      '运行
base(0,1)      '选定 X Y
stop_flag = 0

run_state =2
modbus_reg(0) = run_state
radius = modbus_ieee(2)      '读取半径

movecirc(0,0,radius*cos(pi/4), radius*sin(pi/4), 0) '走一个整圆
wait until idle or (stop_flag = 1)      '等待结束或停止
if (stop_flag = 1) then
    cancel
endif

run_state = 1
modbus_reg(0) = run_state
end

guidetask:      '手动操作
while 1
    if run_state <> 2 then
        base(0)
        if modbus_bit(10) = 1 then      '左

```

```

        vmove(-1)
    elseif modbus_bit(11) = 1 then '右
        vmove(1)
    elseif mtype = 10 or mtype = 11 then '非 vmove 运动
        cancel
    endif

    base(1)
    if modbus_bit(20) = 1 then '左
        vmove(-1)
    elseif modbus_bit(21) = 1 then '右
        vmove(1)
    elseif mtype = 10 or mtype = 11 then '非 vmove 运动
        cancel
    endif

endif

wend
end

```

自定义 G 代码

```

ERRSWITCH = 3 '全部信息输出
base(0,1,2,3) '选择 X Y Z U, G01 里面有指定, 不能随意修改
rapidstop
wait idle

dpos = 0,0,0,0
atype=1,1,1,1 '脉冲方式步进或伺服
units = 100,100,100,100 '脉冲当量, 每 mm100 脉冲
speed = 200,200,200,200
accel = 2000,2000,2000,2000
decel = 2000,2000,2000,2000
merge = on '启动连续插补
CORNER_MODE = 2'启动拐角减速
DECEL_ANGLE = 15 * (PI/180)
STOP_ANGLE = 45 * (PI/180)
g_init() 'G 初始化

while 1 '循环运动

    if in(0) = on then '输入 0 有效启动运动

```

```

        '走一个方框
    G91 '相对
    G01 X100 Y0
    G01 X0 Y100
    G01 X-100 Y0
    G01 X0 Y-100
    wait idle '等待运动停止
    delay(100) '延时
endif

wend

end

global sub g_init()
dim coor_rel
coor_rel = 1 '相对
end sub

end '使得本文件自动运行时退出

global gsub G01(X Y Z U)
trace "G01 entered, distance:" sub_para(0), sub_para(1), sub_para(2), sub_para(3)
' 调试输出
if coor_rel then
    move(sub_para(0), sub_para(1), sub_para(2), sub_para(3)) '相当
else
    local xdis, ydis, zdis, udis
    if sub_ifpara(0) then '是否有参数
        xdis = sub_para(0)
    else
        xdis = dpos(0)
    endif
    if sub_ifpara(1) then
        ydis = sub_para(1)
    else
        ydis = dpos(1)
    endif
    if sub_ifpara(2) then
        zdis = sub_para(2)
    else
        zdis = dpos(2)
    endif
endif

```

```

        if sub_ifpara(3) then
            udis = sub_para(3)
        else
            udis = dpos(3)
        endif
        moveabs(xdis,ydis,zdis,udis)          ' 绝对
    endif
end sub

' 绝对
global gsub G90()
    trace "G90 entered"
    coor_rel = 0
end sub

' 相对
global gsub G91()
    trace "G91 entered"
    coor_rel = 1
end sub

' 延时
global gsub G04(P)
    trace "G04 entered"
    if sub_ifpara(0) then
        delay (sub_para(0))
    else
        ' 绝对
    endif
end sub

global gsub G00(X Y Z U)
    trace "G00 entered, distance:" sub_para(0),sub_para(1),sub_para(2),sub_para(3)
    ' 调试输出
    if coor_rel then
        pmove(sub_para(0),sub_para(1),sub_para(2),sub_para(3))
    else
        local xdis, ydis, zdis, udis
        if sub_ifpara(0) then
            xdis = sub_para(0)
        else
            xdis = dpos(0)
        endif
        if sub_ifpara(1) then

```

```

        ydis = sub_para(1)
    else
        ydis = dpos(1)
    endif
    if sub_ifpara(2) then
        zdis = sub_para(2)
    else
        zdis = dpos(2)
    endif
    if sub_ifpara(3) then
        udis = sub_para(3)
    else
        udis = dpos(3)
    endif
    pmoveabs(xdis,ydis,zdis,udis)
endif
end sub

```

自定义网口通讯

runtask 1,aaa
 Open #11, "TCP_SERVER",1000 ’使用自定义网口通道 2，作为主端，端口号 945

```

global dim NetDataCount
global dim tempchar
global const datamax=20
global dim datanum
        datanum=0
global dim Data(datamax)

```

while 1

```

    tempchar = 0                    ’清除之前的字符
    get #11,tempchar               ’获取单个字符到 tempchar
    print tempchar                 ’打印出字符的 ASCII 码
    Data(datanum) = tempchar       ’保存到数组
    datanum = datanum + 1

```

```

    if datanum = datamax then       ’超过数组空间
        datanum = 0
        for i = 0 to datamax-1     ’清除数组空间内容

```



```

        Data(i) = 0
    next
endif

if tempchar = 59 then    ' ;号终止位

    PRINT #11,"ok1245"

endif

Delay(10)

wend

sub aaa()

    while 1

        tempchar = 0          ' 清除之前的字符
        get #10,tempchar      ' 获取单个字符到 tempchar
        print tempchar        ' 打印出字符的 ASCII 码
        Data(datanum) = tempchar ' 保存到数组
        datanum = datanum + 1

        if datanum = datamax then ' 超过数组空间
            datanum = 0
            for i = 0 to datamax-1 ' 清除数组空间内容
                Data(i) = 0
            next
        endif

        if tempchar = 59 then    ' ;号终止位

            PRINT #10,"aaaaaaaaa"

        endif

    wend

end sub

```

CAN 通讯

控制器间接线

CANL - CANL

CANH - CANH

在控制器主端 CANL 和 CANH 间连接一个 120 欧电阻

主端程序

```

CANIO_ADDRESS = 32          ' 主端
RUNTASK 1, task_canget      ' 启动接受任务

global if_send
if_send = 1

while 1

    if if_send = 1 then
        TABLE(0,0,8,1,2,3,4,5,6,7,8) ' 向 can cob id 为 0 的控制器器发送 8 个字节，依
        依次为 1-8.
        CAN(0,7,0)            ' 0 - CAN 通道号、7 - 发送、0-数据 table 起始地址
        if_send = 0
    endif
wend

end

global sub task_canget()      ' 接受任务
while 1
    CAN(0,6,10)              ' 数据接受，存放在 table(10) 之后，table(10) 表示发送端
    CANID , <0 时表示没有数据
                                ' table(11) 表示接受到到数据个数，table(12)... 表示数据
    if(table(10) >= 0 ) then  ' 有接受到数据
        ?"数据发送端 ID:", table(10)
        ?"数据字节数:", table(11)
        ?"          数          据          :
        "table(12), table(13), table(14), table(15), table(16), table(17), table(18)
    endif                    ' 0 - CAN 通道号、7 - 发送、0-数据 table
    起始地址
wend
end sub

```

从端程序

```

CANIO_ADDRESS = 0      ' 从端
RUNTASK 1, task_canget  ' 启动接受任务

global if_send
if_send = 0

while 1
    wait until if_send = 1 ' 等待有数据发送过来
    TABLE(0, 32, 1, $ff)  ' 向 can cob id 为 32 的控制器发送 1 个字节，依次
    为 FF.
    CAN(0, 7, 0)          ' 0 - CAN 通道号、7 - 发送、0-数据 table 起始地址
    if_send = 0

wend
end

global sub task_canget() ' 接受任务
while 1
    CAN(0, 6, 10)        ' 数据接受，存放在 table(10) 之后，table(10) 表示发送
    端 CANID，<0 时表示没有数据
    ' table(11) 表示接受到数据个数，table(12)... 表示数
    据
    if(table(10) >= 0 ) then ' 有接受到数据
        ?"数据发送端 ID:", table(10)
        ?"数据字节数:", table(11)
        ?"          数          据          :
    "table(12), table(13), table(14), table(15), table(16), table(17), table(18)
        if_send = 1
    endif

wend
end sub

```

EtherCAT 初始化程序

```

RAPIDSTOP(2)
wait idle
for i=0 to 10                                ' 取消原来的总线轴设置
    atype(i)=0
next

slot_scan(0)                                ' 开始扫描
if return then
    ?"总线扫描成功", "连接设备数: "NODE_COUNT(0)
    ?
    ?"开始映射轴号"
    AXIS_ADDRESS(0)=0+1                      ' 映射轴号
    ATYPE(0)=65                              ' EtherCAT 类型, 65 位置控制, 66 速度控制, 67 力矩控制
    DRIVE_PROFILE(0)= -1                     ' 伺服 PDO 功能, 66 时设置为 20 67 时设置为 30
    disable_group(0)                         ' 每轴单独分组
    ?"轴号映射完成"
    delay (100)

    SLOT_START(0)                            ' 总线开启
    if return then
        ?"总线开启成功"
        ?"开始清除驱动器错误(根据驱动器数据字典设置)"
        DRIVE_CONTROLWORD(0)=128            ' 伺服清除错误
        delay (10)
        DRIVE_CONTROLWORD(0)=6              ' 伺服 shutdown
        delay (10)
        DRIVE_CONTROLWORD(0)=15             ' 伺服 switch on
        delay (10)

        datum(0)                            ' 清除控制器错误
        delay (100)
        ?"轴使能准备"
        AXIS_ENABLE(0)=1                    ' 轴 0 使能
        wdog=1                               ' 使能总开关
        ?"轴使能完成"

    else
        ?"总线开启失败"
    endif
endif
else

```

```

    ?"总线扫描失败"
endif

end

```

Rtex 初始化程序

```

RAPIDSTOP(2)
wait idle
for i=0 to 10                ' 取消原来的总线轴设置
    atype(i)=0
next

slot_scan(0)                ' 开始扫描
if return then
    ?"总线扫描成功","连接设备数: "NODE_COUNT(0)
    ?"开始映射轴号"
    AXIS_ADDRESS(0)=0+1      ' 映射轴号
    ATYPE(0)=50              ' Rtex 类型, 50 位置控制, 51 速度控制, 52 力矩控制
    DRIVE_PROFILE(0)=1       ' 伺服带 IO 映射
    disable_group(0)         ' 每轴单独分组
    ?"轴号映射完成"
    delay (100)

    SLOT_START(0)            ' 总线开启
    if return then
        ?"总线开启成功"

        datum(0)             ' 清除控制器错误
        delay (100)
        ?"轴使能准备"
        AXIS_ENABLE(0)=1     ' 轴 0 使能
        wdog=1               ' 使能总开关
        ?"轴使能完成"

    else
        ?"总线开启失败"
    endif
else
    ?"总线扫描失败"
endif

```

End

第十四章 错误与调试

控制器在使用过程中，因为接线不对、程序逻辑问题、指令使用错误等各种原因，会出现电机没有按照设定轨迹运行、控制器报错等各种问题。

如何来排查原因和解决问题，第一原则是关闭其他软件，使用 Zdevelop 软件排查，需要熟练使用以下功能：手动运动调试、断点调试、示波器抓取、寄存器查看、在线命令发送、程序打印信息、I/O 口快速检测。

常见问题列表-解决办法

现象	排查方法
电机不动	手动运动调试
触摸屏寄存器数据值不对	寄存器查看
没有按照程序预期运行	断点调试+打印程序信息
输入输出不起作用	I/O 口快速检测
机台抖动大	示波器抓取

手动运动调试

手动运动可以排查电机接线问题。

关闭所有除 Zdevelop 的软件，同时使用 zdevelop 链接控制器，下载空程序，并在“视图”-“轴参数”中手动设置好脉冲数 UNITS、速度 SPEED、加速度 ACCEL，然后打开“视图”-“手动运动”，手动操作电机。



按住“左”/“右”，电机持续运动，松开停止。

“DPOS”显示当前发出的脉冲（单位为 UNITS）。

填写“距离”参数，点击“运动”，勾选“绝对”时，电机运动到距离参数位置；不勾选“绝对”时，电机按距离参数继续运动。

“左”“右”操作时：

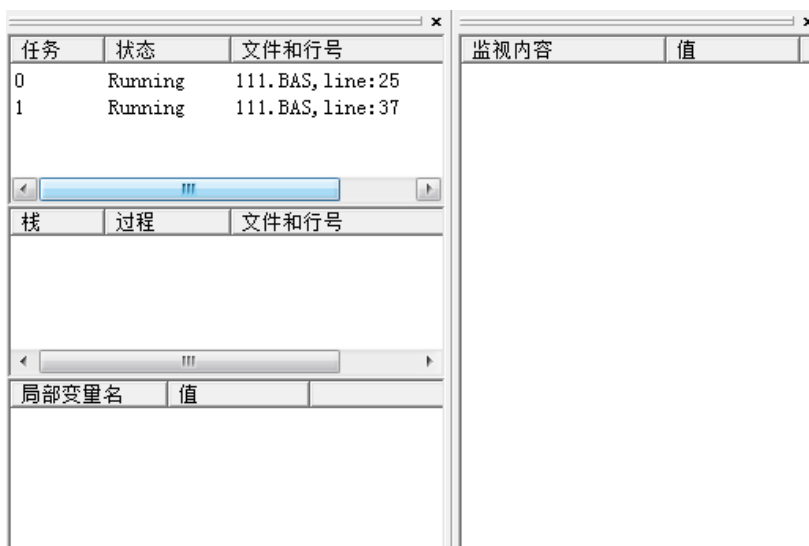
1. 电机不动，但 DPOS 改变。
此时控制器脉冲已发出，检查驱动器有无报警、电机接线。
也可能是 UNITS 设置过小，电机再转动，但是不明显。
2. 电机只往一个方向转动。
查看电机控制方式，目前控制器脉冲轴只能用双脉冲和脉冲+方向两轴控制模式，不可使用正交脉冲控制。
3. 只操作某一边时电机才转动。
检查电机接线。
电机当前控制模式与控制器当前控制模式不同，控制器默认为脉冲+方向控制，使用 INVERT_STEP 指令可以修改。
4. 电机不动，DPOS 也不改变。
检查轴参数 AXISSTATUS 是否报警。

断点调试

断点调试可以查看程序运行的具体过程，主要用于判断程序逻辑错误。配合监视内容可以查看程序每执行一步对寄存器、变量、数组等的影响。**调试程序 and 控制器程序必须一致。**

断点快捷键 F9 添加，可以添加多个。

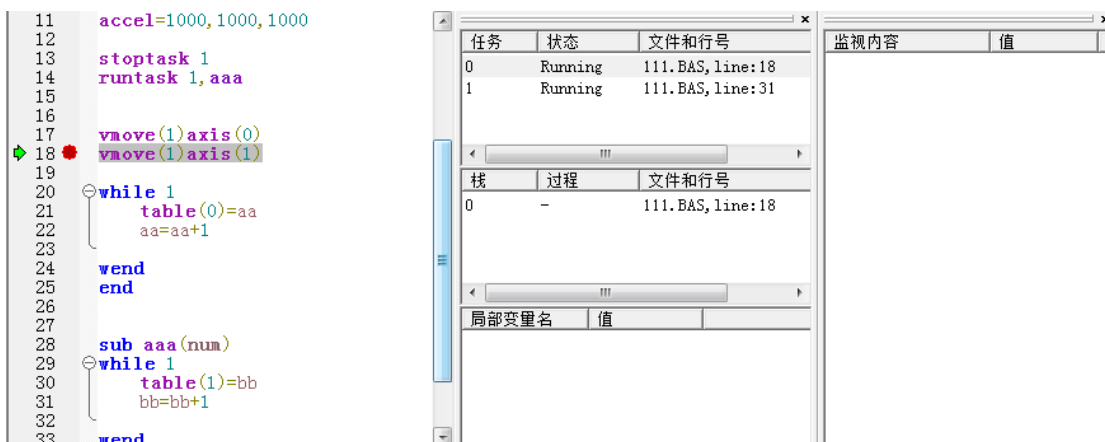
对于使用 zdevelop 软件开发的程序，连接好控制器，点击“调试”-“启动/停止调试”进入调试状态。此时可以查看个任务运行情况、监视内容、子函数调用过程、子函数局部变量值。



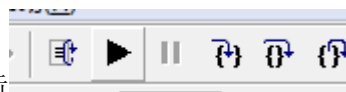
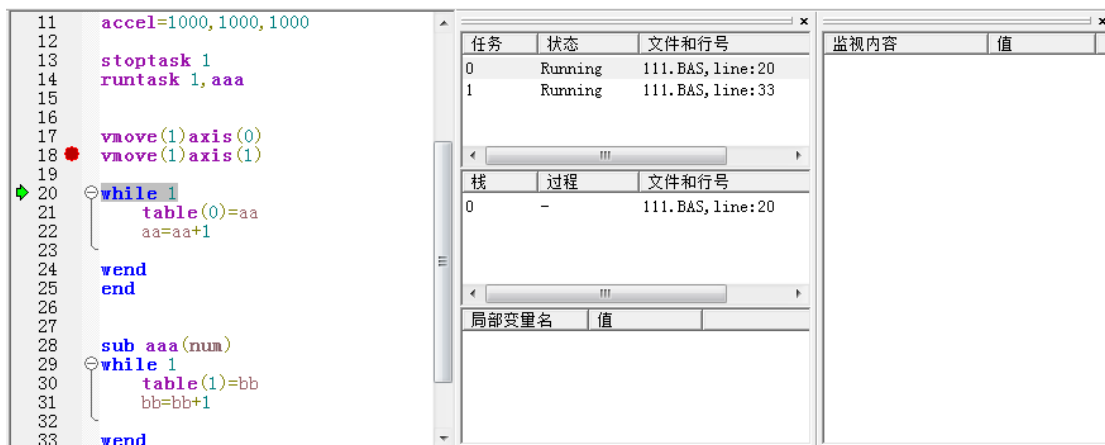
添加断点后，程序会停止在断点处，此时断点处对应的指令还没有执行。如下图，绿色箭头表示当前停止在哪一行，“栈”表示断点停止在哪个任务，“文件和行号”也可以查看停止位置。其他任务此时也同时停止在各自执行到的位置。

前面已经扫描的程序功能不受影响，比如 17 `vmove(1)axis(0)` 已经执行，此时轴 0 正向运动。

后面没有扫描到的程序不执行，比如 18 `vmove(1)axis(1)` 没有执行，此时轴 1 不动。



程序停止在断点处后，就可以进行逐步调试，快捷键 F11，按一次程序向下执行一步。如下图，此时程序执行到行 20，轴 1 开始正向运动。

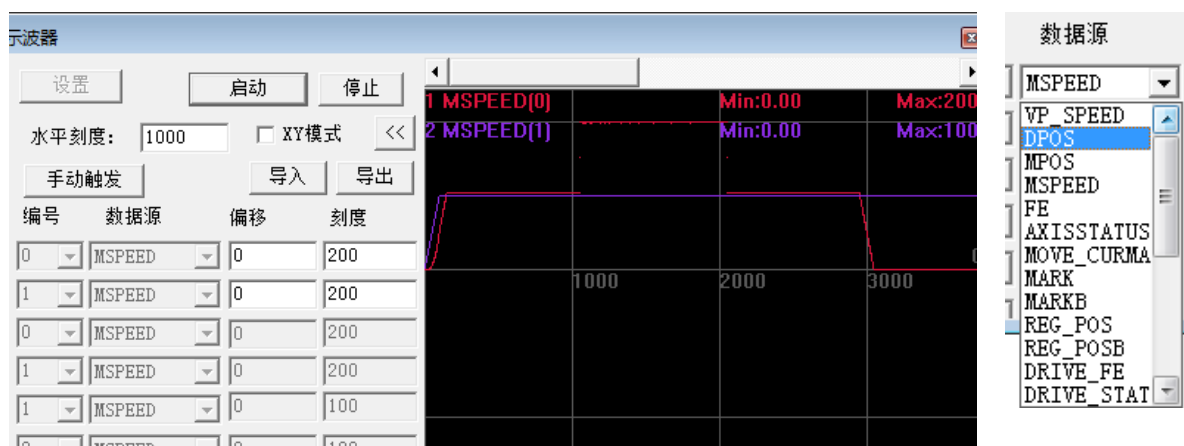


单步调试完一段程序确认无误后，可以点击三角符号恢复程序，也可以使用快捷键 F5。

注意如果断点是设置在循环中，那么下次循环时还是会停止程序。

示波器抓取

示波器可以抓取各类数据，具体种类查看“视图”-“示波器”-“数据源”。示波器抓取一般用来判断电机实际运行速度和实际位置。

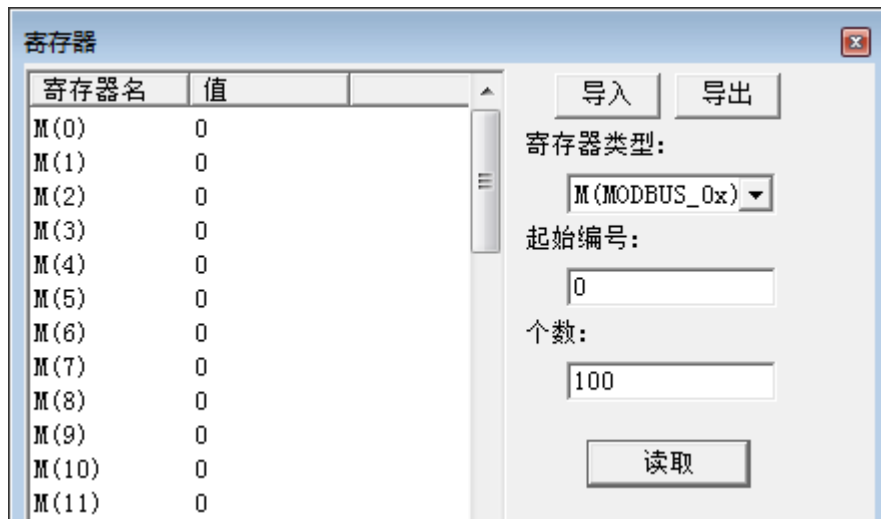


XY 模式可以查看 2 维的路径轨迹，需要将前两个通道的“数据源”设置为 DPOS 或 MPOS。

机台实际运行抖动大时，可以用示波器抓取编码器 mspeed，查看波形是否光滑；如果光滑说明脉冲发送平稳，此时查看速度曲线加减速过程是否过陡，匀速时速度值是否过大。

寄存器查看

使用 Zdevelop 软件可以方便的查看寄存器的数据，点击“视图”-“寄存器”查看。寄存器种类有 IN、OP、MODBUS_4X、MODBUS_0X、TABLE、VR、AIN、AOUT。



必须支持 PLC 功能的控制器才能使用。

比如出现触摸屏某个寄存器参数没有按照预期变化时，此时可直接查看对应寄存器类型及编号，确认控制器中此寄存器数值有无变化。

如果变化，说明控制器与触摸屏通讯出现问题，检查接线和通讯参数设置；

如果无变化，说明程序执行有问题，断点调试检查程序执行是否正确。

在线发送命令

在线命令可以实时执行发送的指令，一般用于直接判断控制器指令功能是否正常。

比如程序已经执行，但实际电机没有按照程序预设功能运行，无法确定是否是程序中其他任务或流程影响，还是控制器功能错误。此时就可以下载空程序到控制器，然后直接在线命令发送功能指令，观察现象即可判断问题原因。

打印程序信息

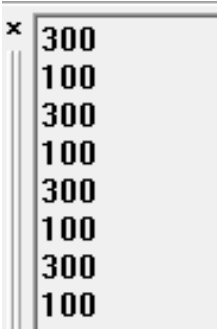
在各段程序间打印信息，可以方便的查看程序段是否执行，执行了几次，执行时对应的参数是什么。

打印指令为 print，也可以使用简写 ? 。

```

1  2  dim ccv(20), aa, bb
3  ccv="ad"
4  aa=100
5  bb=300
6
7  base(0, 1, 2)
8  atype=1, 1, 1
9  units=200, 200, 200
10 speed=100, 100, 100
11 accel=1000, 1000, 1000
12
13 stoptask 1
14 runtask 1, aaa
15
16 while 1
17     ?bb
18     move(bb)
19 wend
20 end
21
22
23 sub aaa(num)
24     while 1
25         ?aa
26         move(aa)
27     wend
28 end sub
29

```



I0 口快速检测

Zdevelop 连接控制器，点击“视图”-“输入口”、“输出口”。同时，将输出口和输入口一对一连接起来（EGND 一定要接），然后操作“输出口”，可以查看到“输入口”中对应状态变化。

某些控制器 I0 口需要另接 24v 电源。

如果检测扩展板 I0，先确认 CANL 和 CANH 接线时是否接入 120 欧电阻；然后确认拨码开关设置是否正确（不能覆盖控制器原本 I0 编号）；再确认主电源与 I0 电源是否接好；最后按照上面所述进行检测。

附录 错误码列表

错误码	意义	可能情况
210	文件过大	
212	状态错误	Resume 时为非暂停状态。
213	文件下载上传出错，丢包。	PC 函数调用返回此错误
214	下载文件的长度校验错误。	
215	缓冲长度不够	发送字符串命令过长时返回此错误。
217	控制器不支持或禁止的功能。	
218	调用传递的参数错误。	
219	下载冲突，同时启动了多个文件下载。	
220	文件名错误，有特殊字符。	

221	文件名错误，超过长度。	
222	文件不存在	
223	密码保护限制。	
224	密码保护限制 2。	
260	硬件错误	
261	磁盘没有格式化	
262	RTC 错误	
263	NORFLASH 错误	
264	RAM 错误	
265	NANDFLASH 错误	
266	U 盘错误	
267	FPGA 错误	
268	以太网硬件错误	
271	备份电源错误	
272	子卡不存在	
273	文件丢失	
274	系统文件错误	
275	无主控，子卡上产生	
276	程序文件校验错误	
277	程序文件错误导致不启动	
278	ZAR 校验 APPPASS 出错	
279	ZAR 校验 ID 出错	
280	BAS 文件超过最大数量	
281	子卡 ID 冲突，或多主冲突。	
282	不支持的功能	
284	zar 与控制器不匹配	
285	图片文件错误	
286	字体文件错误	
1000	运动模块返回错误偏移	
1002	无运动缓冲	
1004	从轴运动中	
1005	不支持的运动功能	

1006	圆弧位置错误	
1007	椭圆 AB 参数错误	
1008	运动模块输入参数错误	
1009	运动中，无法操作	
1010	暂停等重复操作	
1011	IDLE 无法做暂停等操作	
1012	当前运动不支持暂停	
1013	找不到暂停点	
1014	ATYPE 不支持	
1015	ZCAN 的 ATYPE 冲突	
1016	轴不支持的功能	
1017	FRAME 校正数据错误	
1018	FRAME 校正数据过少	
1019	FRAME 校正数据满足条件的数据过少	
1020	FRAME 校正数据辅助参数过少	
1021	FRAME 校正数据间隔过小，小于关节轴数	
1022	FRAME 的输入坐标错误	
1023	FRAME 状态下坐标不能强制修改	
1024	FRAME 逆解异常	
1025	不是 FRAME 状态	
1026	FRAME HAND 错误	
1030	CORNERMODE 7 位设置了但不支持此运动	
1031	CORNERMODE 7 位设置了但不是 FRAME 状态	
2000	ZBASIC 模块偏移	
2021	手动停止	
2022	因其他任务错误导致本任务停止	
2023	试图修改只读状态参数	
2024	数组越界	
2025	变量数超过控制器规格	
2026	数组数超过控制器规格	
2027	数组空间超过控制器规格	
2028	SUB 数超过控制器规格	

2029	标识符命名错误	
2030	标识符命名过长	
2031	没有右括号。	
2032	不认识的字符。	
2033	表达式中碰到不认识的名称。	
2043	不认识的命令标识符，当前行第一个标识名称。	
2044	堆栈溢出	
2045	数学表达式太复杂，不同控制器的规格不一样。	
2046	没有找到结束引用标号	
2047	指令没有返回值，不能用于表达式计算。	
2048	函数必须返回值，不用在一行的开头地方。	
2049	特殊指令必须单独一行	
2050	参数或数组需要索引。	
2051	变量不能使用索引	
2052	数组重定义且长度不一致。	
2053	数组定义长度参数错误，负数或过大。	
2054	标识符已经定义为 SUB 过程，不能再做其他用途。	
2055	标识符已经定义为参数，不能再做其他用途。	
2056	标识符预留，不能使用。	
2057	出现不能识别的字符	
2058	SUB 调用重复出栈。	
2060	语法格式错误	
2062	函数参数范围错误	包括任务号超过范围也返回这个错误。自动运行任务号出错也是这个错误码。
2063	函数参数过多	
2064	函数参数太少	
2065	缺少操作数	
2066	操作符后面缺少操作数	
2067	操作符前面缺少操作数	

2068	不认识的操作符	
2069	需要双目操作符	
2070	CALL 必须调用 SUB	
2072	需要赋值符号	
2073	空文件	
2074	SUB 定义的标识符名称冲突。	
2075	要启动的任务已经运行中。	
2076	多个参数要使用逗号隔开。	
2077	括号不配对，无左括号。	
2078	IF 判断的嵌套太多。	
2079	循环语句嵌套太多。	
2080	插补轴数太少。	
2081	CONST 常量，不能修改。	
2082	命令不能从 PC 在线发送。	
2083	SUB 定义的参数太多。	
2084	SUB 带参数，不能用于 GOTO 语句。	
2085	局部标识符定义太多。	
2086	LOCAL 变量名与文件变量名或其它标识符名称冲突。	
2087	LOCAL 不支持数组定义。	
2088	GSUB 定义的参数字母重复。	
2089	GSUB 定义的参数只能为单字母。	
2090	不能修改只读参数。	
2091	GSUB_IFPARA 函数使用场合错误。	
2092	除数为零	
2093	超过缓冲	
2094	在线命令阻塞时间过长。	
2095	参数重名	
2096	值没有初始化就使用了	
2097	轴号冲突	
2099	内部错误	
2100	SCANEDGE 个数过多	
2101	ZINDEX 类型不匹配	
2901	系统错误，定义的标识符过多。	
	包括变量，数组，过程，过程参数等等。	

3201	超过缓冲	
3202	文件非正常结束	
3204	内部状态错误	
3205	不支持的功能	
3206	内部调用参数错误	
3301	圆弧的三点在一条线上。	
3302	两条直线平行，没有交点。	
3402	消息响应超时	
3407	Modbus 返回参数错误	
3408	Modbus 返回不支持	
3501	ZCAN 返回无子卡	
3502	ZCAN 返回子卡无对应轴	
4000	4000-4500 PLC 模块的错误	
4002	参数错误	
4003	未知类型	
4004	未知函数	
4005	压栈太多 STL	
4006	压栈太多	
4007	程序太复杂，BLOCK 太多	
4008	没有压栈 BLOCK	
4009	没有压栈 STL	
4010	没有压栈	
4014	文件内容错误	

4015	RET 必须在 STL 的后面	
4016	超过范围	
4017	低于范围	
4018	L 没有定义	
4019	不支持 G 代码函数	
4020	不能 GOTO 跨 PLC 与 BASIC	
4021	PLC 主任务只有一个	
4022	语法错误	
4023	FOR NEXT 错误，不匹配	
4024	FOR NEXT 错误，无 NEXT	
4026	FOR MC 混用	
4027	FOR STL 混用	
4030	必须 PLC 主任务中使用	
4031	必须中断中使用	
4032	参数个数少	
4033	参数个数多	
4034	要 8 的倍数	
4035	寄存器标识错误	
4036	寄存器类型错误	
4037	LV 个数超过	
4038	只读	
4500	4500-5000 PLC 上位机端错误	
4503	内存不够	
4504	回流到母线上	
4505	回流	
4506	AND 类型不能直接接母线	
4510	悬空	
4511	最右端必须是输出类型	

5000	5000-5500 HMI 模块的错误	
5000	LCD 号错误	
5002	LCD 号冲突	
5003	不支持对象	
5004	内存不够	
5005	控件层次错误	
5006	窗口号超过	
5007	无效窗口号	
5010	对象属性丢失	
5011	输入窗口有多个显示元件	
5012	ACTION 类型错误	
5013	事件过多	
5014	返回上个窗口失败	
5015	不能关闭基本窗口	
5016	字体中找不到对应字符	
5017	必须在 HMI 任务中使用	
5020	控件 ID 冲突	
5021	LCD 号错误	
5022	找不到可用 LCD	
5023	LCD 没有打开	
5024	LCD 无数据	
5025	程序复位	
5026	lcd 已经打开了	
5027	不是网络 LCD	
5028	不支持的压缩方式	
5029	颜色深度不支持	
5030	不支持的数据类型	
5031	设备号错误	
5032	LCD_SEL 不能使用	
5033	设置 REDRAW 不能再 DRAW 阶段	
5034	DRAW 函数只能在 DRAW 阶段	
5035	操作不能再 DRAW 阶段调用	
5036	内部 LCD 分辨率固定	
5037	LCD 分辨率超过	
5038	库文件名错误	
5039	字符过多	

5501-5599	PC 端 PLC 文件编译的错误	
5503	内存不够	
5504	回流到母线上	
5505	回流	
5506	AND 类型指令不能直接接母线	
5510	右边悬空，没有接输出指令	
5511	最右边不是输出类型指令	
5512	最右边不能连接在一起	
5513	输出类型指令必须在最右边	
5514	不支持的指令类型	
5517	寄存器没有值	
5518	DOT 值超过范围	
5519	索引寄存器超过范围	
5520	字符数过多	
5521	寄存器类型错误	
5522	寄存器值错误	
5523	寄存器个数过多	
5524	寄存器个数过少	
5525	STL 使用错误	
5526	RET 使用错误	
5527	重复 RET	
5528	END 或 LBL 的位置错误	
5529	函数不能直接接母线	
5530	出栈没有压栈	
5531	MPP 太多	
5532	寄存器类型使用错误	
5533	ANB 错误，块数不够	
5534	ORB 错误，块数不够	
5535	ANB 错误，输出操作后不能合并	
5536	ORB 错误，输出操作后不能合并	
5537	AND 直接接母线	
5538	OR 直接接母线	
5539	OR 不能在 OUT 指令的后面	
5540	STL 和 MC 不能共用	
5541	MC 不能直接接母线	

5542	_@寄存器要括号	
5543	注释错误	
5544	梯形图列数过多	
5545	输出类型不能直接接母线	
6000	ECAT 模块错误, slot 编号错误	
6001	内部错误, 功能不支持。	
6005	参数错误	
6006	支持的设备类型数超过限制	
6012	资源不够	
6013	从设备反应超时	
6014	缓冲不够	
6015	应答包 wkc 错误	
6016	SDO 应答超长	
6017	SDO 应答错误	
6018	SDO 应答数据长度错误	
6019	WKC 超时	
6020	state 切换超时	
6021	SDO ABORT	
6023	NODE profile 错误	
6024	轴 profile 错误	
6025	轴数超过	
6029	PDO 列表个数超过	
6031	设备个数超过	
6042	设备不支持	
6045	邮箱超时	
6047	数据类型错误	
20000	PC 端产生错误的偏移	
20002	参数错	
20003	超时	可能是 fifo 缓冲阻塞
20006	操作系统错误	
20007	串口打开失败	

20008	网络打开失败	
20009	句柄错误	
20010	发送错误	
20011	文件错误	
20012	文件长度错误	
20013	文件名过长	
20014	文件不存在	
20015	ZLB 库文件错误	
20016	文件没有编译	一般 PLC 文件
20020	固件文件不匹配	
20021	不支持的功能	
20030	输入缓冲长度不够	